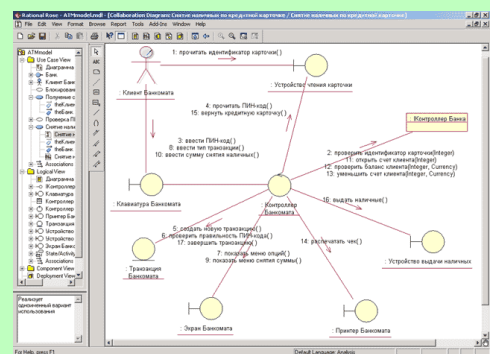
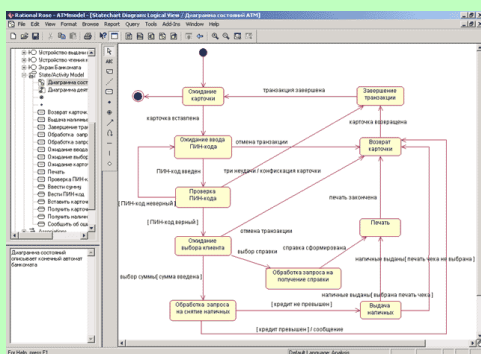


# CASE-ТЕХНОЛОГИИ



Улан-Удэ  
2018

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ  
РОССИЙСКОЙ ФЕДЕРАЦИИ  
БУРЯТСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

## **CASE-ТЕХНОЛОГИИ**

Рекомендовано учебно-методическим советом БГУ  
в качестве учебного пособия для обучающихся по направлениям подготовки:  
01.03.02 Прикладная математика и информатика,  
02.03.01 Математика и компьютерные науки,  
02.03.03 Математическое обеспечение и администрирование информационных систем,  
09.03.03 Прикладная информатика

Улан-Удэ  
Издательство Бурятского госуниверситета  
2018

УДК 004.4'22 (075.8)  
ББК 32.972.13Я73  
С 29

Утверждено к печати редакционно-издательским советом  
Бурятского государственного университета

Рецензенты

***С. Г. Баргуев,***

канд. физ.-мат. наук, доцент, заведующий кафедрой высшей математики и ОПД  
Бурятского института инфокоммуникаций Сибирского  
государственного университета телекоммуникаций и информатики

***Г. И. Занданова***

канд. физ.-мат. наук, доцент кафедры информационных технологий Бурятского  
государственного университета

Текст печатается в авторской редакции

С 29 **CASE-технологии** [Электронный текст] / сост. Т. Г. Дармаев. — Улан-Удэ :  
Издательство Бурятского госуниверситета, 2018. — 141 с. ISBN 978-5-9793-1273-9

В учебном пособии в систематизированном виде приводятся необходимые теоретические сведения и лабораторные работы, направленные на обучение технологии составления диаграмм по стандартам IDEF0, DFD, IDEF3 и UML. Цель пособия — помочь приобрести практические навыки проектирования сложных программных систем с помощью CASE-пакетов AllFusion Process Modeller и Rational Rose. Данное пособие предназначено для проведения практических занятий по дисциплине «Технология разработки программного обеспечения» для бакалавров дневного обучения направлений подготовки: «Прикладная математика и информатика», «Математика и компьютерные науки», «Математическое обеспечение и администрирование информационных систем» и «Прикладная информатика».

ISBN 978-5-9793-1273-9

© Т. Г. Дармаев, составление, 2018  
© Бурятский госуниверситет, 2018

## Содержание

|                                                                          |          |
|--------------------------------------------------------------------------|----------|
| <b>Предисловие .....</b>                                                 | <b>5</b> |
| <b>1. Структурное проектирование</b>                                     |          |
| 1.1 Структурный подход.....                                              | 7        |
| 1.1.1 Лабораторная работа № 1. Интерфейс AllFusion Process Modeller..... | 9        |
| 1.1.2 Задания.....                                                       | 15       |
| 1.1.3 Контрольные вопросы .....                                          | 15       |
| 1.2 Методология IDEF0.....                                               | 16       |
| 1.2.1 Лабораторная работа № 2. Диаграмма IDEF0.....                      | 27       |
| 1.2.2 Задания.....                                                       | 29       |
| 1.2.3 Контрольные вопросы .....                                          | 29       |
| 1.3 Диаграммы потоков данных (Data Flow Diagrams).....                   | 30       |
| 1.3.1 Лабораторная работа № 3. Диаграмма DFD.....                        | 32       |
| 1.3.2 Задания.....                                                       | 33       |
| 1.3.3 Контрольные вопросы .....                                          | 33       |
| 1.4 Диаграммы процессов (IDEF3) .....                                    | 34       |
| 1.4.1 Лабораторная работа № 4. Диаграмма IDEF3.....                      | 37       |
| 1.4.2 Задания.....                                                       | 38       |
| 1.4.3 Контрольные вопросы .....                                          | 38       |
| <b>2. Объектно-ориентированное проектирование</b>                        |          |
| 2.1 Объектно-ориентированный подход .....                                | 39       |
| 2.1.1 Лабораторная работа № 5. Интерфейс IBM Rational Rose.....          | 41       |
| 2.1.2 Задания.....                                                       | 52       |
| 2.1.3 Контрольные вопросы .....                                          | 52       |
| 2.2 Диаграммы прецедентов.....                                           | 53       |
| 2.2.1 Лабораторная работа № 6. Use Case diagram .....                    | 54       |
| 2.2.2 Задания.....                                                       | 61       |
| 2.2.3 Контрольные вопросы .....                                          | 61       |
| 2.3 Диаграммы классов .....                                              | 62       |
| 2.3.1 Лабораторная работа № 7. Class diagram.....                        | 65       |
| 2.3.2 Задания.....                                                       | 82       |
| 2.3.3 Контрольные вопросы .....                                          | 82       |
| 2.4 Диаграммы взаимодействия: последовательности и коопераций .....      | 83       |
| 2.4.1 Лабораторная работа № 8. Sequence diagram .....                    | 84       |
| 2.4.1 Лабораторная работа № 9. Collaboration diagram .....               | 90       |
| 2.4.2 Задания.....                                                       | 98       |
| 2.4.3 Контрольные вопросы .....                                          | 98       |
| 2.5 Диаграммы состояний и деятельности.....                              | 99       |
| 2.5.1 Лабораторная работа № 10. Statechart diagram .....                 | 100      |
| 2.5.2 Лабораторная работа № 11. Activity diagram .....                   | 107      |
| 2.5.3 Лабораторная работа № 12. Bussiness-Activity diagram .....         | 112      |
| 2.5.4 Задания.....                                                       | 119      |
| 2.5.5 Контрольные вопросы .....                                          | 119      |
| 2.6 Диаграммы пакетов, компонентов и размещения .....                    | 120      |
| 2.6.1 Лабораторная работа № 13. Component diagram.....                   | 121      |
| 2.6.2 Лабораторная работа № 14. Deployment diagram.....                  | 127      |
| 2.6.3 Задания.....                                                       | 132      |
| 2.6.4 Контрольные вопросы .....                                          | 132      |
| 2.7 Генерация программного кода .....                                    | 133      |

|                                                                   |            |
|-------------------------------------------------------------------|------------|
| 2.7.1 Лабораторная работа № 15. Генерация программного кода ..... | 133        |
| 2.7.2 Задания.....                                                | 138        |
| 2.7.3 Контрольные вопросы .....                                   | 138        |
| <b>Заключение .....</b>                                           | <b>139</b> |
| <b>Библиографический список .....</b>                             | <b>139</b> |

## ПРЕДИСЛОВИЕ

Настоящее учебное издание представляет собой учебное пособие для дисциплины «Технология разработки программного обеспечения» в рамках реализации образовательной программы высшего образования по направлениям подготовки: 01.03.02 Прикладная математика и информатика, 02.03.03 Математическое обеспечение и администрирование информационных систем, 09.03.03 Прикладная информатика, 02.03.01 Математика и компьютерные науки очной формы обучения и подготовлено в соответствии с требованиями Федерального государственного образовательного стандарта высшего образования.

Дисциплина «Технология разработки программного обеспечения» относится к обязательным дисциплинам базовой части блока Блока 1 в структуре ОП для направлений 01.03.02 Прикладная математика и информатика, 02.03.03 Математическое обеспечение и администрирование информационных систем, 09.03.03 Прикладная информатика; в дисциплину по выбору базовой части блока Блока 1 по направлению подготовки 02.03.01 Математика и компьютерные науки.

Изучение дисциплины направлено на формирование общепрофессиональных / профессиональных компетенций:

для направления 01.03.02 Прикладная математика и информатика:

ОПК-3 — способность к разработке алгоритмических и программных решений в области системного и прикладного программирования, математических, информационных и имитационных моделей, созданию информационных ресурсов глобальных сетей, образовательного контента, прикладных баз данных, тестов и средств тестирования систем и средств на соответствие стандартам и исходным требованиям;

для направления 02.03.03 Математическое обеспечение и администрирование информационных систем:

ОПК-4 — способность применять в профессиональной деятельности основные методы и средства автоматизации; проектирования, производства, испытаний и оценки качества программного обеспечения;

ОПК-8 — способность использовать знания методов проектирования и производства программного продукта, принципов построения, структуры и приемов работы с инструментальными средствами, поддерживающими создание программного обеспечения (далее - ПО);

ОПК-9 — способность использовать знания методов организации работы в коллективах разработчиков ПО, направления развития методов и программных средств коллективной разработки ПО;

ПК-1 — готовность к использованию метода системного моделирования при исследовании и проектировании программных систем.

для направления 09.03.03 Прикладная информатика:

ПК-2 — способность разрабатывать, внедрять и адаптировать прикладное программное обеспечение

ПК-7 — способность проводить описание прикладных процессов и информационного обеспечения решения прикладных задач.

для направления 02.03.01 Математика и компьютерные науки:

ОПК-2 — способность решать стандартные задачи профессиональной деятельности на основе информационной и библиографической культуры с применением информационно-коммуникационных технологий и с учетом основных требований информационной безопасности;

ОПК-4 — способность находить, анализировать, реализовывать программно и использовать на практике математические алгоритмы, в том числе с применением современных вычислительных систем.

В результате освоения дисциплины обучающийся должен:

Знать:

методы анализа прикладной области, информационных потребностей, формирования требований к программному обеспечению (ПО); методологии и технологии проектирования ПО, проектирование обеспечивающих подсистем ПО; методы и средства организации и управления проектом ПО на всех стадиях жизненного цикла, оценка затрат проекта и экономической эффективности ПО.

Уметь:

проводить анализ предметной области, выявлять информационные потребности и разрабатывать требования к ПО; разрабатывать концептуальную модель прикладной области, выбирать инструментальные средства и технологии проектирования ПО; выполнять работы на всех стадиях жизненного цикла ПО.

Владеть:

CASE-средствами моделирования предметной области, прикладных и информационных процессов.

Основной задачей настоящего учебного пособия является изучение в рамках дисциплины «Технология разработки программного обеспечения» проектирования сложных программных систем с помощью CASE-пакетов AllFusion Process Modeller и Rational Rose.

Пособие состоит из предисловия, двух глав, заключения и библиографического списка. Первая глава посвящена структурному проектированию с помощью диаграмм IDEF0, DFD, IDEF3. Вторая глава посвящена объектно-ориентированному проектированию с помощью UML-диаграмм: прецедентов (use case diagrams), классов (class diagrams), взаимодействия (interaction diagrams): последовательностей (sequence diagrams) и кооперации (collaboration diagrams), состояний (statechart diagrams), деятельности (activity diagrams), компонентов (component diagrams) и размещения (deployment diagrams), а также генерации программного кода ANSI C++.

# ГЛАВА 1

## СТРУКТУРНОЕ ПРОЕКТИРОВАНИЕ

### 1.1 Структурный подход

Структурный подход ([6], [13-14], [17-18], [22]) состоит в декомпозиции (разбиении) системы на элементарные функции, т. е. система разбивается на функциональные подсистемы, которые в свою очередь делятся на подфункции, подразделяемые на задачи, и т. д. Процесс разбиения продолжается вплоть до конкретных процедур. При этом создаваемая система сохраняет целостное представление, в котором все составляющие компоненты взаимосвязаны.

Все наиболее распространенные методологии структурного подхода базируются на ряде общих принципов. В качестве двух базовых принципов используются следующие:

- принцип решения сложных проблем путем их разбиения на множество меньших независимых задач, легких для понимания и решения;
- принцип организации составных частей проблемы в иерархические древовидные структуры с добавлением новых деталей на каждом уровне - так называемый принцип иерархического упорядочения.

В структурном анализе используются в основном две группы средств, иллюстрирующих функции, выполняемые системой, и отношения между данными. Каждой группе средств соответствуют определенные виды моделей (диаграмм), наиболее распространенными среди которых являются следующие:

- SADT (Structured Analysis and Design Technique) - модели и соответствующие функциональные диаграммы;
- DFD (Data Flow Diagrams) - диаграммы потоков данных;
- ERD (Entity-Relationship Diagrams) - диаграммы «сущность-связь».

На стадии проектирования системы модели расширяются, уточняются и дополняются диаграммами, отражающими ее структуру.

На современном рынке средств разработки достаточно много систем, в той или иной степени удовлетворяющих перечисленным требованиям. Остановимся на одной из них – пакете программ AllFusion Modeling Suite компании CA ([www.ca.com](http://www.ca.com)).

Рассмотрим методику разработки информационных систем (ИС) с помощью продуктов пакета AllFusion Modeling Suite. Методика уделяет большое внимание разработке требований к ИС организации исходя из текущего состояния бизнеса и его потребностей, а также предоставлению требований в наиболее простой и наглядной форме, обеспечивающей понимание со стороны заказчика на каждом этапе разработки ИС. Методика охватывает следующие этапы разработки ИС: сбор и анализ информации, системное проектирование, реализацию (Рис.1.1.1) и предполагает применение продуктов линейки AllFusion Modeling Suite:

- **AllFusion Process Modeler** (ранее BPwin) - моделирование бизнес-процессов;
- **AllFusion ERwin Data Modeler** (ранее ERwin) - моделирование данных и генерация каталога БД;
- **AllFusion Data Model Validator** (ранее ERwin Examiner) - проверка структуры БД и моделей, созданных в ERwin;
- **AllFusion Model Manager** (ранее ModelMart) - среда командной работы проектировщиков;
- **AllFusion Model Navigator** - просмотр, навигация по моделям данных и бизнес-процессов.

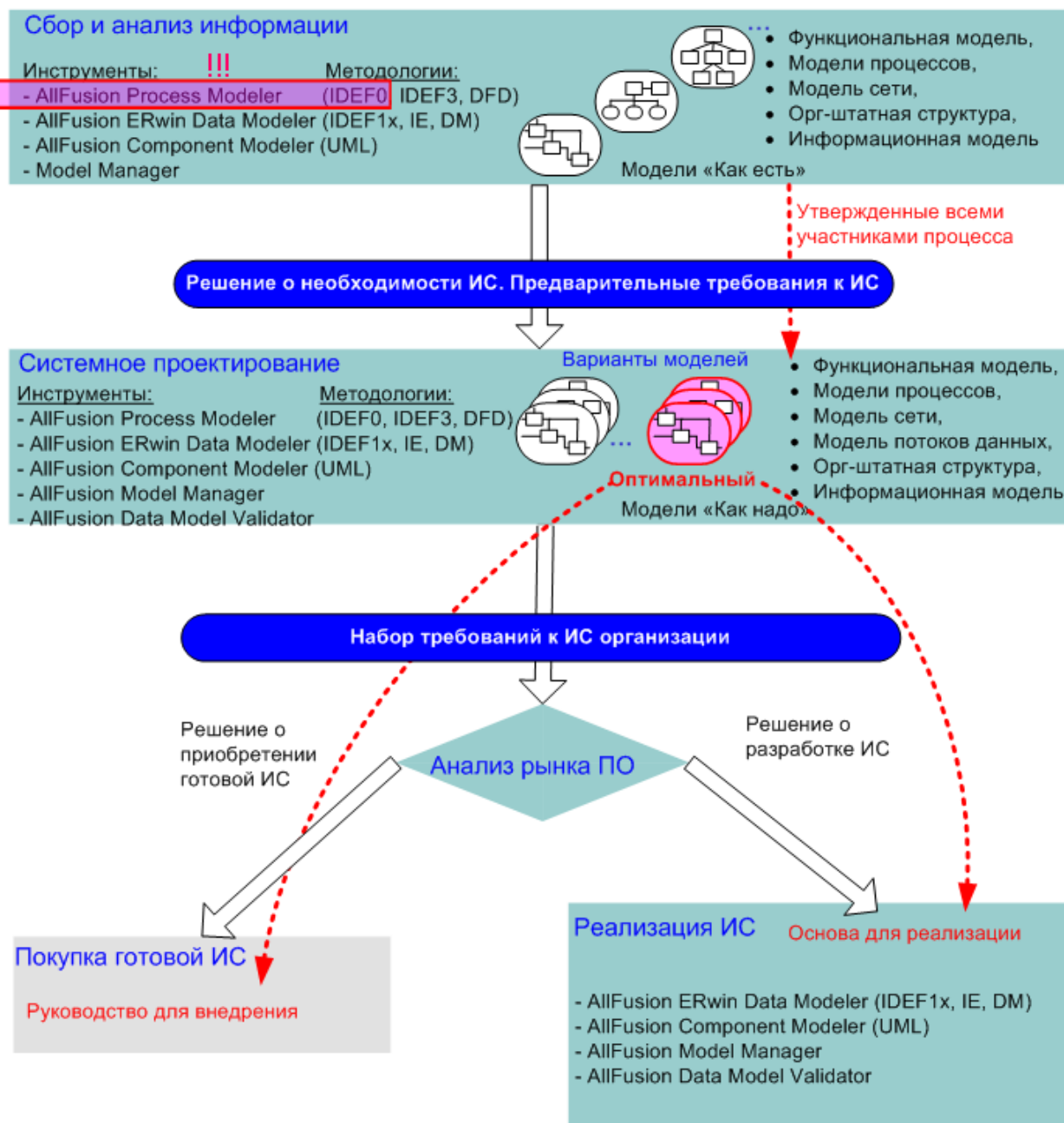


Рис.1.1.1 Методика разработки ИС

Необходимым условием управляемости некоторого объекта является наличие модели объекта управления в системе управления этим объектом. Поэтому результатом этапа сбора и анализа информации является набор моделей типа «как есть»: функциональная модель, модели процессов, модель сети, организационно-штатная структура, информационная модель. Для разработки указанных моделей методика рекомендует использовать следующие инструменты и методологии: AllFusion Process Modeler (IDEF0, IDEF3, DFD), AllFusion ERwin Data Modeler (IDEF1x, IE DM), AllFusion Component Modeler (UML), Model Manager, входящие в пакет AllFusion Modeling Suite. Все модели выполняются в графических нотациях, что упрощает восприятие информации и взаимопонимание с заказчиком.

В данной работе рассматриваются технологии составления диаграмм по стандартам IDEF0, DFD и IDEF3 с помощью CASE-средства структурного анализа AllFusion Process Modeller.

### 1.1.1 Лабораторная работа № 1. Интерфейс AllFusion Process Modeller

#### Цель работы :

- Изучение интерфейса,
- Создание моделей и задание их свойств

#### Общее описание интерфейса

AllFusion Process Modeller имеет достаточно простой и интуитивно понятный интерфейс пользователя, дающий возможность аналитику создавать сложные модели при минимальных усилиях.

При запуске по умолчанию появляется рабочая область, основная панель инструментов, палитра инструментов (вид которой зависит от выбранной нотации) и, в левой части, навигатор модели – Model Explorer (рис. 1.1.2).

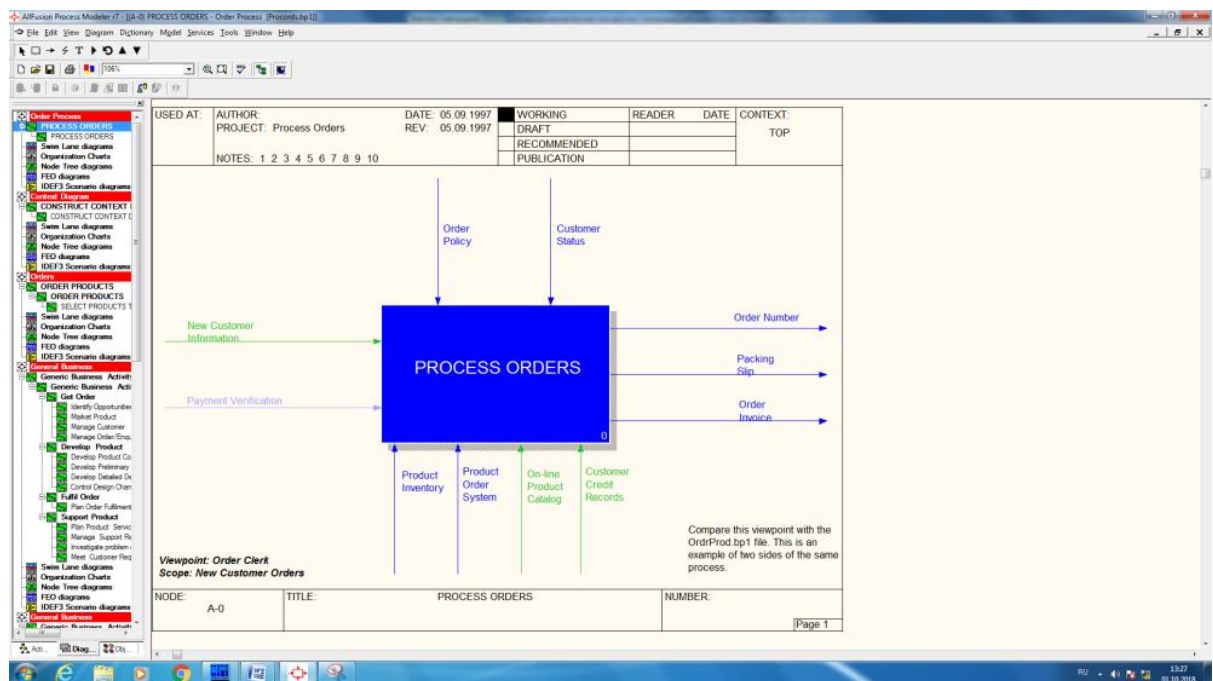


Рис.1.1.2 Интегрированная среда разработки модели AllFusion Process Modeller

Рассмотрим подробнее содержимое каждой из частей программы: Рабочая область – содержит собственно разрабатываемую модель. На каждой странице отображается соответствующий уровень декомпозиции функциональной модели.

Панели инструментов: эти панели содержат практически все используемые при работе элементы. По умолчанию все панели отображаются на экране. При необходимости пользователь может отключить или, наоборот, включить требуемые модели, используя меню «View». Имеются следующие панели инструментов:

- Standard toolbar (Рис.1.1.3) – содержит кнопки для управления файлами (новый, открыть, сохранить, печать), кнопки отображения области свойств модели, кнопки управления масштабом изображения;

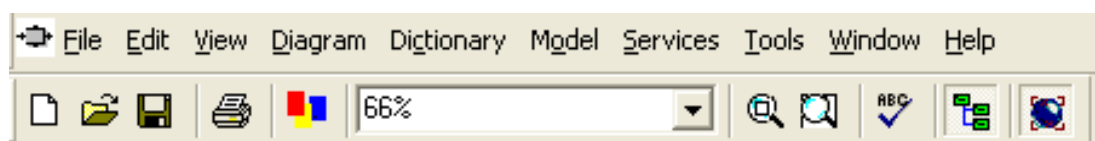


Рис. 1.1.3. Стандартная панель инструментов

- Toolbox for Business Process Diagrams (IDEF0) – инструментальные кнопки создания элементов модели: функциональных блоков и связей (стрелок) (рис. 1.1.4). Содержит кнопки: стрелка – выбор объекта, создание функционального блока, создание стрелки для связи функциональных блоков с внешним миром и между собой, создание текста, редактор модели, переходы к родительской и дочерней моделям (диаграммам).



Рис. 1.1.4. Панель Toolbox for Business Process Diagrams (IDEF0)

- ModelMart – панель кнопок специального инструментального средства, предназначенного для связывания пакета BPWin и пакета ERWin. Область модели содержит название модели, все уровни декомпозиции разрабатываемой функциональной модели, а также названия всех функций, выполняемых на каждом уровне декомпозиции.

Функциональность панели инструментов доступна из основного меню (табл. 1).

Таблица 1

Описание элементов управления основной панели инструментов

| Кнопка | Функция                                                                      | Путь в меню          |
|--------|------------------------------------------------------------------------------|----------------------|
|        | Создать новую модель                                                         | File/New             |
|        | Открыть модель                                                               | File/Open            |
|        | Сохранить модель                                                             | File/Save            |
|        | Напечатать модель                                                            | File/Print           |
|        | Вызвать генератор отчетов Report Builder                                     | Tools/Report Builder |
|        | Выбор масштаба                                                               | View/Zoom            |
|        | Масштабирование                                                              | View/Zoom            |
|        | Проверка правописания                                                        | Tools/Spelling       |
|        | Включение и выключение навигатора модели Model Explorer                      | View/Model Explorer  |
|        | Включение и выключение дополнительной панели инструментов работы с ModelMart | Model/Mart           |

На основной панели инструментов (либо в любом желаемом месте экрана) расположены инструменты редактора (Рис.1.1.5). Нужная панель инструментов подбирается автоматически при выборе методологии моделирования.



Рис.1.1.5. Инструменты редактора

1. Pointer Tool – используется для выбора и определения позиции объектов, добавленных в диаграмму.
2. Activity Box Tool – используется для установки блоков в диаграмме.
3. Arrow Tool – используется, чтобы устанавливать дуги в диаграмме.
4. Squiggle Tool – используется для создания тильды, которая соединяет дугу с ее названием.
5. Text Block Tool – используется для создания текстовых блоков.
6. Diagram Dictionary Editor – открывает диалоговое окно Diagram Dictionary Editor, где

можно перейти на какую-либо диаграмму или создать новую диаграмму.

7. Go to Sibling Diagram – используется для отображения следующей диаграммы того же уровня.

8. Go to Parent Diagram – переход на родительскую диаграмму.

9. Go to Child Diagram – используется, чтобы отобразить диаграмму потомка или разложить выделенный блок на диаграмму потомка.

### Создание новой модели

При создании новой модели возникает диалог (Рис.1.1.6), в котором следует указать, будет ли создана модель заново, или она будет открыта из файла, либо из хранилища **ModelMart**, внести имя модели и выбрать методологию, в которой будет построена модель.

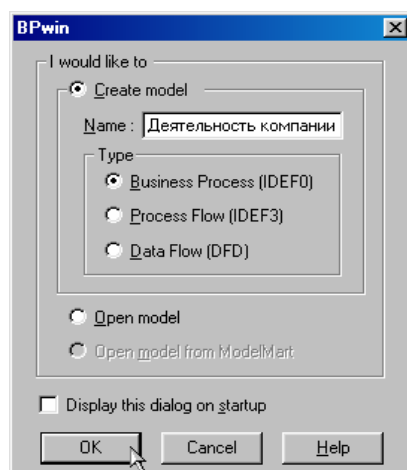


Рис. 1.1.6 Диалог при создании новой модели

Программный продукт **AllFusion Process Modeller** поддерживает три методологии – **IDEF0**, **IDEF3** и **DFD**, каждая из которых решает свои специфические задачи. Возможно построение смешанных моделей, т. е. модель может содержать одновременно как диаграммы **IDEF0**, так и диаграммы **IDEF3** и **DFD**. Состав палитры инструментов изменяется автоматически, когда происходит переключение с одной нотации на другую, поэтому палитра инструментов будет рассмотрена позже.

После щелчка по кнопке **OK** появляется диалог **Properties for New Models** (Рис.1.1.7), в который следует внести свойства модели.

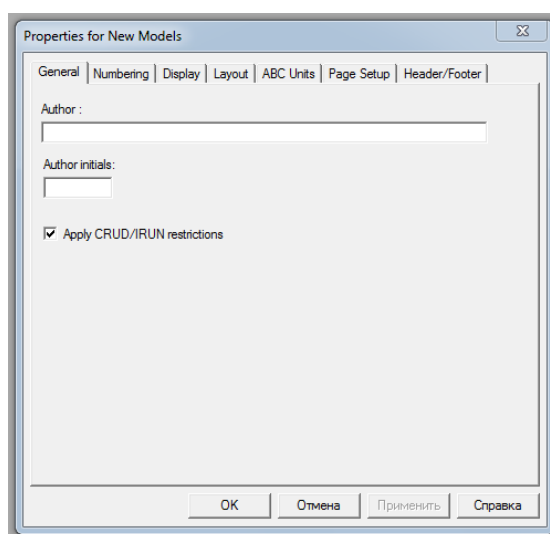


Рис. 1.1.7 Диалог Properties for New Models

Модель рассматривается как совокупность работ, каждая из которых оперирует некоторым набором данных. Работа изображается в виде прямоугольников, данные – в виде стрелок. Если щелкнуть дважды по любому объекту модели левой кнопкой мыши, появляется всплывающее контекстное меню, каждый пункт которого соответствует редактору какого-либо свойства объекта.

### Установка цвета и шрифта объектов

При нажатии правой кнопки мышки на прямоугольнике или стрелке, пункты контекстного меню **Font** и **Color** вызывают диалог **Arrow Properties** или **Activity Properties** (Рис.1.1.8) для установки шрифта (в том числе его размера и стиля) и цвета объекта. В нижней части вкладки **Font** диалогов **Arrow Properties** и **Activity Properties** находятся группа опций **Apply setting to**, позволяющих изменить шрифт для всех работ или стрелок на текущей диаграмме, в модели, и группа **Global**, позволяющая изменить шрифт одновременно для всех объектов модели.

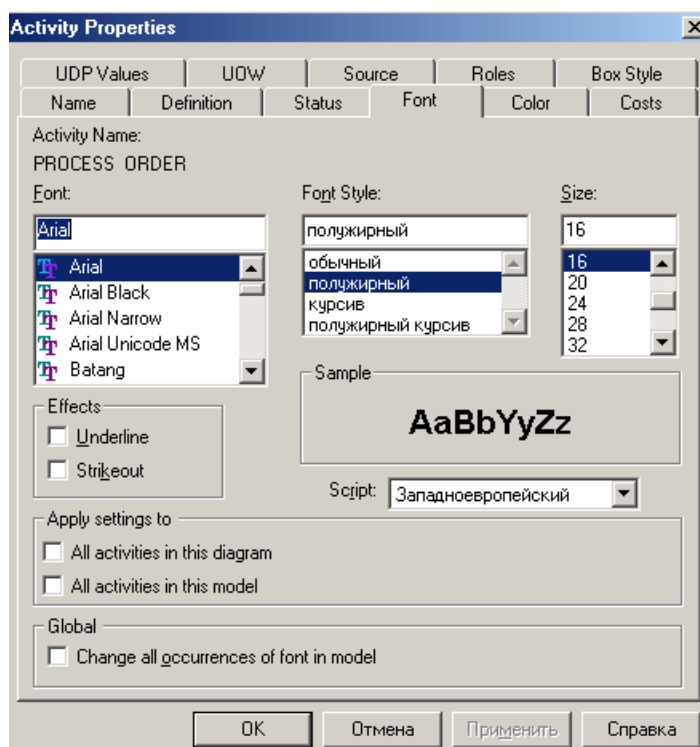


Рис. 1.1.8 Диалог Activity Properties

Кроме того, **AllFusion Process Modeller** позволяет установить **шрифт по умолчанию** для объектов определенного типа на диаграммах и в отчетах. Для этого следует выбрать меню **Model/Default Fonts**, после чего появляется следующее каскадное меню, каждый пункт которого служит для установки шрифтов для определенного типа объектов:

- **Context Activity** – работа на контекстной диаграмме;
- **Context Arrow** – стрелки на контекстной диаграмме;
- **Decomposition Activity** – работа на диаграмме декомпозиции;
- **Decomposition Arrow** – стрелки на диаграмме декомпозиции;
- **Node Tree Text** – текст на диаграмме дерева узлов;
- **Frame User Text** – текст, вносимый пользователем в каркас диаграмм;
- **Frame System Text** – системный текст в каркасе диаграмм;
- **Text Blocks** – текстовые блоки;
- **Parent Diagram Text** – текст родительской диаграммы;
- **Parent Diagram Title Text** – текст заголовка родительской диаграммы;

- **Report Text** – текст отчетов.

### Model Explorer – навигатор модели процессов

Инструмент навигации **Model Explorer** (Рис.1.1.9) имеет три вкладки – **Activities**, **Diagrams** и **Objects**. Вкладка **Activities** показывает в виде раскрывающегося иерархического списка все работы модели. Одновременно могут быть показаны все открытые модели. Работы с диаграммами **IDEF0** показываются зеленым цветом, **IDEF3** – желтым и **DFD** – голубым.

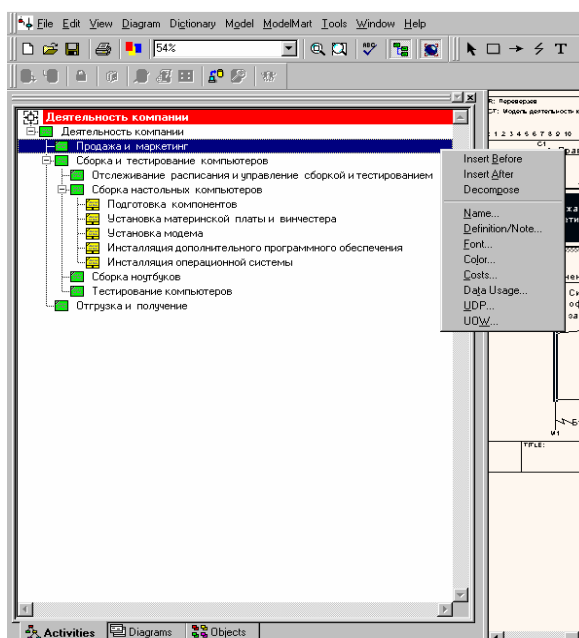


Рис. 1.1.9 Model Explorer

Щелчок по работе во вкладке **Activity** переключает левое окно на диаграмму, на которой эта работа размещена. Для редактирования свойств работы следует щелкнуть по ней правой кнопкой мыши. Появляется контекстное меню. В табл. 2 приведено значение пунктов меню.

Таблица 2

Контекстное меню редактирования свойств работы

| Пункт меню      | Описание                                                                                                  |
|-----------------|-----------------------------------------------------------------------------------------------------------|
| Insert Before   | Создать новую работу на той же самой диаграмме. В списке работ новая работа будет вставлена перед текущей |
| Insert After    | Создать новую работу на той же самой диаграмме. В списке работ новая работа будет вставлена после текущей |
| Decompose       | Декомпозировать работу. В результате будет создана новая диаграмма декомпозиции                           |
| Name            | Вызов редактора имени работы                                                                              |
| Definition/Note | Вызов редактора определения и примечания к работе                                                         |
| Font            | Изменения шрифта работы                                                                                   |
| Color           | Изменения цвета работы                                                                                    |
| Costs           | Задание стоимости работы                                                                                  |
| Data Usage      | Ассоциация работы с данными                                                                               |
| UDP             | Задание свойств, определяемых пользователем                                                               |
| UOW             | Задание свойств для работ IDEF3                                                                           |

Если с помощью вкладки **Activities** можно перейти на стандартные диаграммы

(контекстную и декомпозиции), то вторая вкладка – **Diagrams** – служит для перехода на любую диаграмму модели.

После перехода на вкладку **Objects** на ней показываются все объекты, соответствующие выбранной на вкладке **Diagrams** диаграмме, в том числе работы, хранилища данных, внешние ссылки, объекты ссылок и перекрестки.

### Каркас диаграммы

На рис. 1.1.10 показан типичный пример диаграммы с граничными рамками, которые называются каркасом диаграммы. Каркас содержит заголовок (верхняя часть рамки) и подвал (нижняя часть). Заголовок каркаса используется для отслеживания диаграммы в процессе моделирования. Нижняя часть используется для идентификации и позиционирования в иерархии диаграммы.

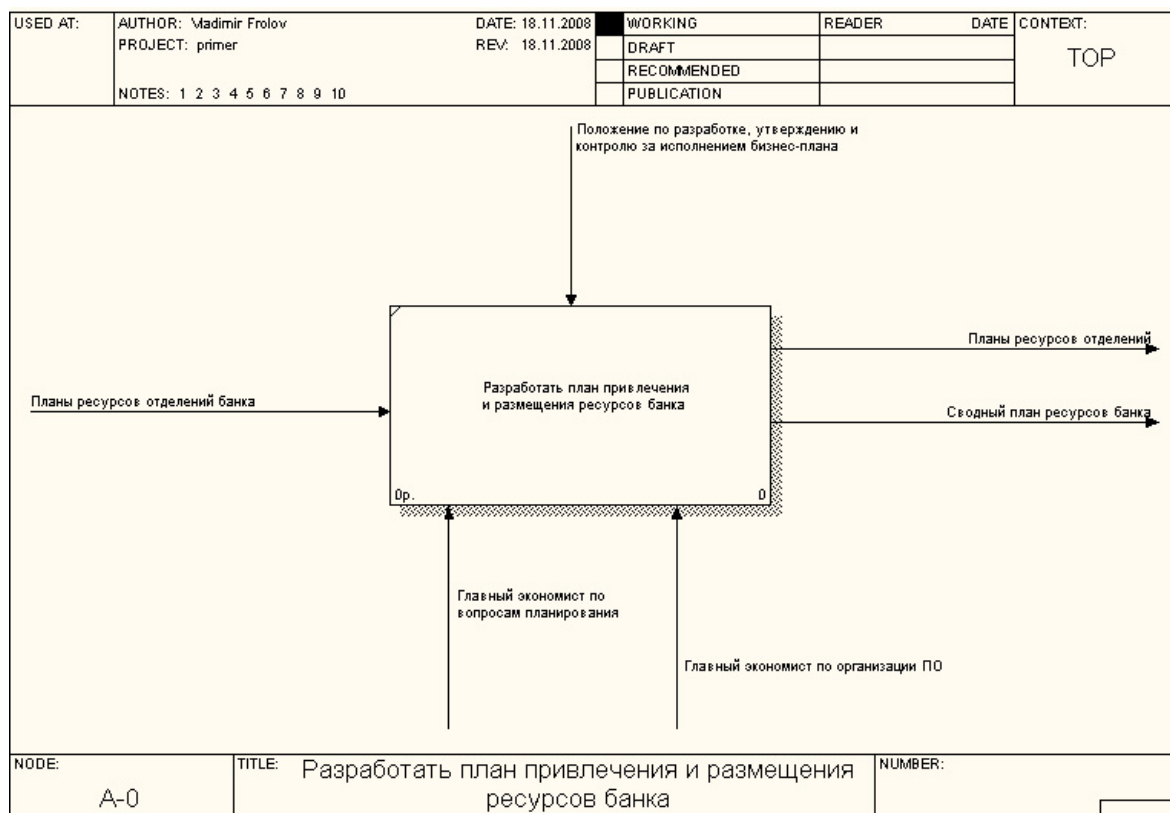


Рис.1.1.10 Каркас диаграммы

Смысл элементов каркаса приведен в табл. 3. Значения полей каркаса задаются в диалоге Diagram Properties (меню Diagram /Diagram Properties).

Таблица 3

Поля заголовка каркаса (слева направо)

| Поле                      | Смысл                                                                                                                                                                                    |
|---------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Used At                   | Используется для указания на родительскую работу в случае, если на текущую диаграмму ссылались посредством стрелки вызова                                                                |
| Autor, Date, Rev, Project | Имя создателя диаграммы, дата создания и имя проекта, в рамках которого была создана диаграмма. REV-дата последнего редактирования диаграммы                                             |
| Notes 123456789 10        | Используется при проведении сеанса экспертизы. Эксперт должен (на бумажной копии диаграммы) указать число замечаний, вычеркивая цифру из списка каждый раз при внесении нового замечания |
| Status                    | Статус отображает стадию создания диаграммы, отображая все этапы публикации                                                                                                              |
| Working                   | Новая диаграмма, кардинально обновленная диаграмма или новый автор диаграммы                                                                                                             |
| Draft                     | Диаграмма прошла первичную экспертизу и готова к дальнейшему обсуждению                                                                                                                  |
| Recommended               | Диаграмма и все ее сопровождающие документы прошли экспертизу. Новых изменений не ожидается                                                                                              |

|             |                                                                                                                                                                                                                                                                          |
|-------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Publication | Диаграмма готова к окончательной печати и публикации                                                                                                                                                                                                                     |
| Reader      | Имя читателя (эксперта)                                                                                                                                                                                                                                                  |
| Date        | Дата прочтения (экспертизы)                                                                                                                                                                                                                                              |
| Context     | Схема расположения работ в диаграмме верхнего уровня. Работа, являющаяся родительской, показана темным прямоугольником, остальные — светлым. На контекстной диаграмме (A-0) показана надпись TOP. В левом нижнем углу показывается номер по узлу родительской диаграммы. |
| Node        | Номер узла диаграммы (номер родительской работы)                                                                                                                                                                                                                         |
| Title       | Имя диаграммы. По умолчанию — имя родительской работы                                                                                                                                                                                                                    |
| Number      | C-Number, уникальный номер версии диаграммы                                                                                                                                                                                                                              |
| Page        | Номер страницы, может использоваться как номер страницы при формировании папки                                                                                                                                                                                           |

### 1.1.2 Задания

1. Создать новый проект.
2. Установить цвета и шрифта объектов.
3. Установить свойства диаграммы.

### 1.1.3 Контрольные вопросы

1. Какие методологии поддерживаются AllFusion Process Modeller?
2. Перечислите основные элементы главного окна AllFusion Process Modeller.
3. Опишите процесс создания новой модели в AllFusion Process Modeller.
4. Может ли модель AllFusion Process Modeller содержать диаграммы нескольких методологий?

## 1.2 Методология IDEF0

Описание системы с помощью IDEF0 называется функциональной моделью. Методология IDEF0 предписывает построение иерархической системы диаграмм - единичных описаний фрагментов системы. Сначала проводится описание системы в целом и ее взаимодействия с окружающим миром (контекстная диаграмма), после чего проводится функциональная декомпозиция - система разбивается на подсистемы и каждая подсистема описывается отдельно (диаграммы декомпозиции). Затем каждая подсистема разбивается на более мелкие и так далее до достижения нужной степени подробности.

Каждая IDEF0-диаграмма содержит блоки (работы) и дуги. Блоки изображают функции моделируемой системы. Дуги связывают блоки вместе и отображают взаимодействия и взаимосвязи между ними.

Процесс моделирования системы в IDEF0 начинается с создания контекстной диаграммы — диаграммы наиболее абстрактного уровня описания системы в целом, содержащей определение субъекта моделирования, цели и точки зрения на модель.

Под субъектом понимается сама система, при этом необходимо точно установить, что входит в систему, а что лежит за ее пределами, другими словами, определить, что будет в дальнейшем рассматриваться как компоненты системы, а что как внешнее воздействие. На определение субъекта системы будут существенно влиять позиция, с которой рассматривается система, и цель моделирования — вопросы, на которые построенная модель должна дать ответ. Другими словами, в начале необходимо определить область моделирования. Описание области как системы в целом, так и ее компонентов является основой построения модели. Хотя предполагается, что в ходе моделирования область может корректироваться, она должна быть в основном сформулирована изначально, поскольку именно область определяет направление моделирования. При формулировании области необходимо учитывать два компонента — широту и глубину. Широта подразумевает определение границ модели — что будет рассматриваться внутри системы, а что снаружи. Глубина определяет, на каком уровне детализации модель является завершенной. При определении глубины системы необходимо помнить об ограничениях времени — трудоемкость построения модели растет в геометрической прогрессии с увеличением глубины декомпозиции. После определения границ модели предполагается, что новые объекты не должны вноситься в моделируемую систему.

**Цель моделирования (Purpose)** определяется из ответов на следующие вопросы:

- Почему этот процесс должен быть смоделирован?
- Что должна показывать модель?
- Что может получить клиент?

**Точка зрения (Viewpoint).** Под точкой зрения понимается перспектива, с которой наблюдалась система при построении модели. Хотя при построении модели учитываются мнения различных людей, все они должны придерживаться единой точки зрения на модель. Точка зрения должна соответствовать цели и границам моделирования. Как правило, выбирается точка зрения человека, ответственного за моделируемую работу в целом.

IDEFO-модель предполагает наличие четко сформулированной цели, единственного субъекта моделирования и одной точки зрения. Для внесения области, цели и точки зрения в модели IDEF0 в AllFusion Process Modeller следует выбрать пункт меню Model/Model Properties, вызывающий диалог Model Properties (рис. 1.2.1). В закладке Purpose следует внести цель и точку зрения, а в закладку Definition — определение модели и описание области.

В закладке Status того же диалога можно описать статус модели (черновой вариант, рабочий, окончательный и т. д.), время создания и последнего редактирования (отслеживается в дальнейшем автоматически по системной дате). В закладке Source описываются источники информации для построения модели (например, «Опрос экспертов предметной области и анализ документации»). Закладка General служит для внесения имени проекта и модели, имени и инициалов автора и временных рамок модели — AS-IS и TO-BE.

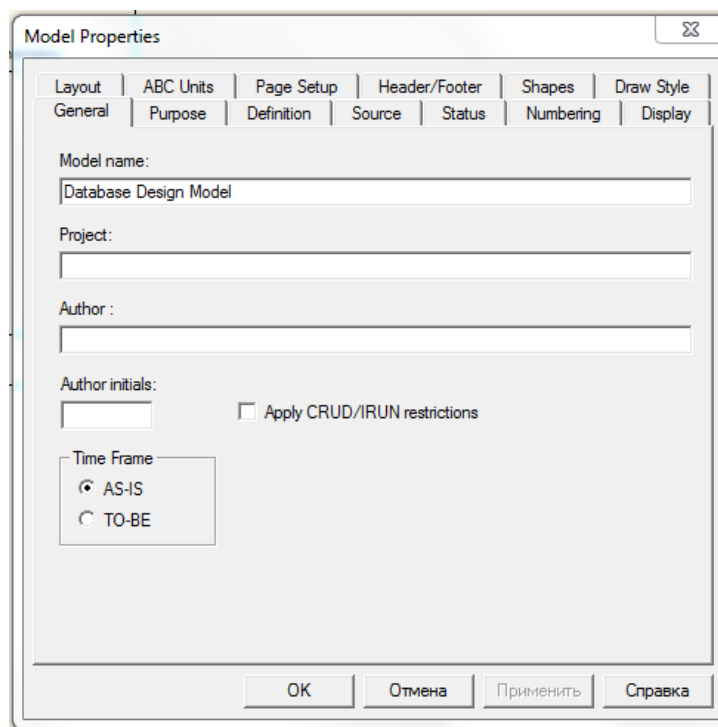


Рис. 1.2.1. Диалог задания свойств модели

**Модели AS-IS и TO-BE.** Обычно сначала строится модель существующей организации работы — AS-IS (как есть). Анализ функциональной модели позволяет понять, где находятся наиболее слабые места, в чем будут состоять преимущества новых бизнес-процессов и насколько глубоким изменениям подвергнется существующая структура организации бизнеса. Детализация бизнес-процессов позволяет выявить недостатки организации даже там, где функциональность на первый взгляд кажется очевидной. Найденные в модели AS-IS недостатки можно исправить при создании модели TO-BE (как будет) — модели новой организации бизнес-процессов.

Технология проектирования ИС подразумевает сначала создание модели AS-IS, ее анализ и улучшение бизнес-процессов, то есть создание модели TO-BE, и только на основе модели TO-BE строится модель данных, прототип и затем окончательный вариант ИС.

Иногда текущая AS-IS и будущая TO-BE модели различаются очень сильно, так что переход от начального к конечному состоянию становится неочевидным. В этом случае необходима третья модель, описывающая процесс перехода от начального к конечному состоянию системы, поскольку такой переход — это тоже бизнес-процесс.

Результат описания модели можно получить в отчете Model Report. Диалог настройки отчета по модели вызывается из пункта меню Tools/Reports/Model Report.

**Модель в нотации IDEFO** представляет собой совокупность иерархически упорядоченных и взаимосвязанных диаграмм. Каждая диаграмма является единицей описания системы и располагается на отдельном листе.

Модель может содержать четыре типа диаграмм:

- контекстную диаграмму (в каждой модели может быть только одна контекстная диаграмма);
- диаграммы декомпозиции;
- диаграммы дерева узлов;
- диаграммы только для экспозиции (FEO).

**Контекстная диаграмма** является вершиной древовидной структуры диаграмм и представляет собой самое общее описание системы и ее взаимодействия с внешней средой. После описания системы в целом проводится разбиение ее на крупные фрагменты. Этот процесс называется функциональной декомпозицией, а диаграммы, которые описывают

каждый фрагмент и взаимодействие фрагментов, называются диаграммами декомпозиции. После декомпозиции контекстной диаграммы проводится декомпозиция каждого большого фрагмента системы на более мелкие и так далее, до достижения нужного уровня подробности описания. После каждого сеанса декомпозиции проводятся сеансы экспертизы — эксперты предметной области указывают на соответствие реальных бизнес-процессов созданным диаграммам. Найденные несоответствия исправляются, и только после прохождения экспертизы без замечаний можно приступить к следующему сеансу декомпозиции. Так достигается соответствие модели реальным бизнес-процессам на любом и каждом уровне модели. Синтаксис описания системы в целом и каждого ее фрагмента одинаков во всей модели.

**Работы/блоки (Activity)** обозначают поименованные процессы, функции или задачи, которые происходят в течение определенного времени и имеют распознаваемые результаты. Работы изображаются в виде прямоугольников. Все работы должны быть названы и определены. Имя работы должно быть выражено отглагольным существительным, обозначающим действие (например, «Деятельность компании», «Прием заказа» и т.д.). Работа «Деятельность компании» может иметь, например, следующее определение: «Это учебная модель, описывающая деятельность компании». При создании новой модели (меню File/New) автоматически создается контекстная диаграмма с единственной работой, изображающей систему в целом.

Для внесения или изменения имени работы следует щелкнуть по работе правой кнопкой мыши, выбрать в меню Name и в появившемся диалоге внести имя работы. Для описания других свойств работы служит диалог Activity Properties (рис. 1.2.2).

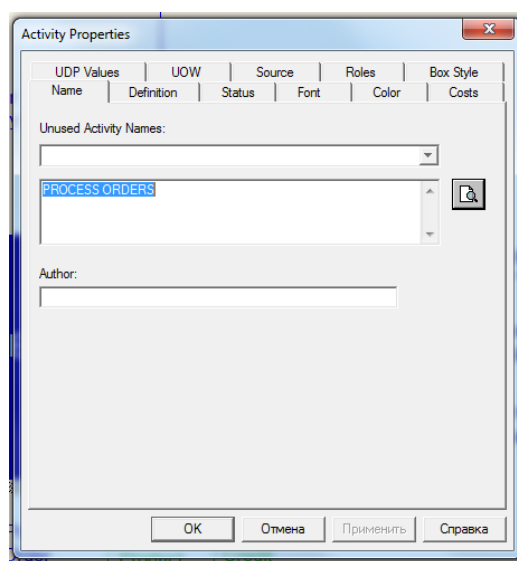


Рис. 1.2.2. Редактор задания свойств работы

**Диаграммы декомпозиции** содержат родственные работы, т. е. дочерние работы, имеющие общую родительскую работу. Для создания диаграммы декомпозиции следует щелкнуть по кнопке Go to Child Diagram ▼ на панели инструментов.

Возникает диалог Activity Box Count (Рис.2.3), в котором следует указать нотацию новой диаграммы и количество работ на ней. Остановимся пока на нотации IDEFO и щелкнем на ОК. Появляется диаграмма декомпозиции (Рис.1.2.4). Допустимый интервал числа работ — 2-8. Декомпозировать работу на одну работу не имеет смысла: диаграммы с количеством работ более восьми получаются перенасыщенными и плохо читаются. Для обеспечения наглядности и лучшего понимания моделируемых процессов рекомендуется использовать от трех до шести блоков на одной диаграмме.



Рис.1.2.3 Диалог Activity Box Count

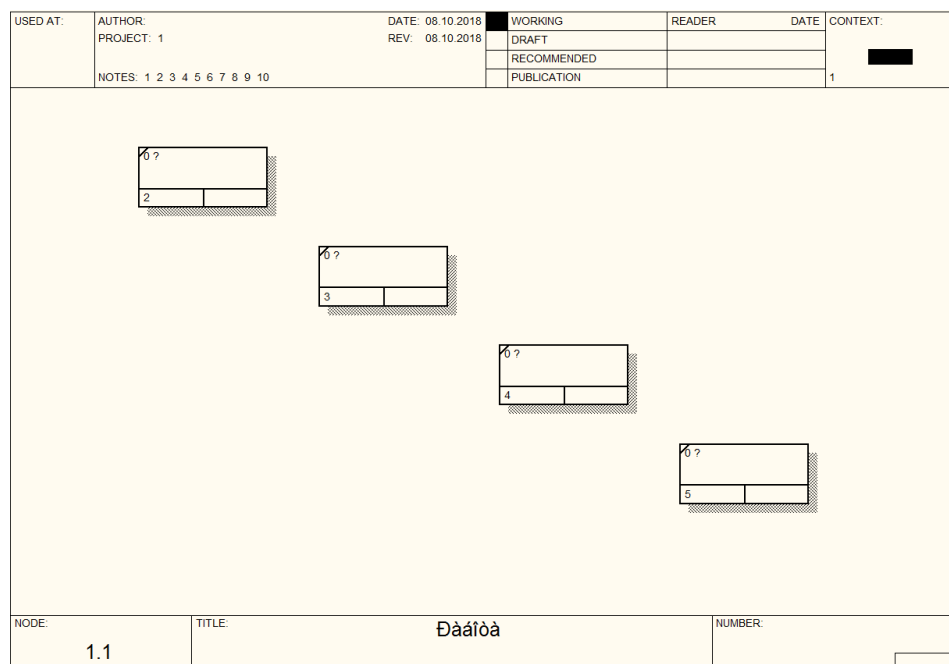


Рис.1.2.4 Диаграмма декомпозиции

Если оказывается, что количество работ недостаточно, то работу можно добавить в диаграмму, щелкнув сначала по кнопке  (Activity Box Tool) на палитре инструментов, а затем по требуемому месту на диаграмме.

Работы на диаграммах декомпозиции обычно располагаются по диагонали от левого верхнего угла к правому нижнему. Блоки в IDEF0 размещаются по степени важности, как ее понимает автор диаграммы. Этот относительный порядок называется доминированием. Доминирование понимается как влияние, которое один блок оказывает на другие блоки диаграммы. Например, самым доминирующим блоком диаграммы может быть либо первый из требуемой последовательности функций, либо планирующая или контролирующая функция, влияющая на все другие. Наиболее доминирующий блок обычно размещается в верхнем левом углу диаграммы, а наименее доминирующий - в правом углу. Таким образом, топология диаграммы показывает, какие функции оказывают большее влияние на остальные. Чтобы подчеркнуть это, аналитик может перенумеровать блоки в соответствии с порядком их доминирования. Порядок доминирования может обозначаться цифрой, размещенной в правом нижнем углу каждого прямоугольника: 1 будет указывать на наибольшее доминирование, 2 - на следующее и т. д.

Каждая из работ на диаграмме декомпозиции может быть в свою очередь декомпозирована. На диаграмме декомпозиции работы нумеруются автоматически слева направо. Номер работы показывается в правом нижнем углу. В левом верхнем углу

изображается небольшая диагональная черта, которая показывает, что данная работа не была декомпозирована.

**Стрелки (Arrow)** описывают взаимодействие работ и представляют собой некую информацию, выраженную существительными. (Например, «Звонки клиентов», «Правила и процедуры», «Бухгалтерская система».) Взаимодействие работ с внешним миром и между собой описывается в виде стрелок, изображаемых одинарными линиями со стрелками на концах. Стрелки представляют собой некую информацию и именуются существительными.

В IDEF0 различают пять типов стрелок. Каждый тип стрелок подходит к определенной стороне прямоугольника, изображающего работу, или выходит из нее. Каждая сторона блока имеет особое, вполне определенное назначение (Рис.1.2.5). Левая сторона блока предназначена для входов, верхняя - для управления, правая - для выходов, нижняя - для механизмов. Такое обозначение отражает определенные системные принципы: входы преобразуются в выходы, управление ограничивает или предписывает условия выполнения преобразований, механизмы показывают, что и как выполняет функция.

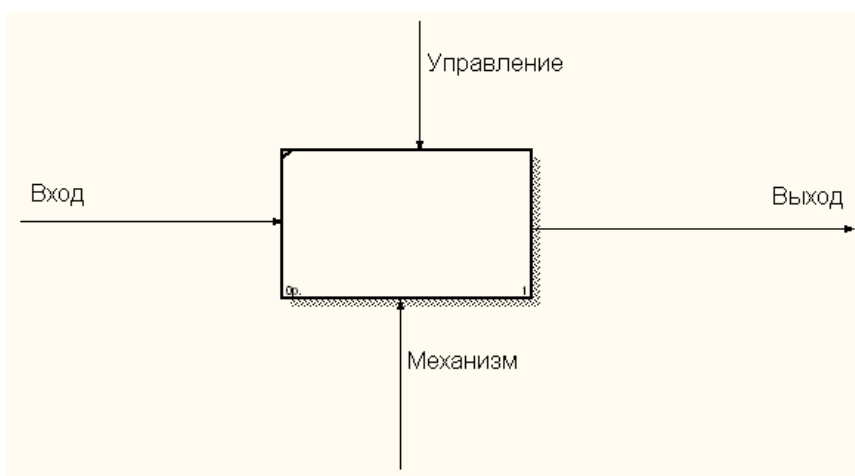


Рис.1.2.5 Типы стрелок

**Вход (Input)** — материал или информация, которые используются или преобразуются работой для получения результата (выхода). Допускается, что работа может не иметь ни одной стрелки входа. Стрелка входа рисуется как входящая в левую грань работы. При описании технологических процессов (для этого и был придуман IDEF0) не возникает проблем определения входов. При моделировании ИС, когда стрелками являются не физические объекты, а данные, не все так очевидно. Например, при «Приеме пациента» карта пациента может быть и на входе и на выходе, между тем качество этих данных меняется. Другими словами, в нашем примере для того, чтобы оправдать свое назначение, стрелки входа и выхода должны быть точно определены с тем, чтобы указать на то, что данные действительно были переработаны (например, на выходе — «Заполненная карта пациента»). Очень часто сложно определить, являются ли данные входом или управлением. В этом случае подсказкой может служить информация о том, перерабатываются/изменяются ли данные в работе или нет. Если изменяются, то, скорее всего, это вход, если нет — управление.

**Управление (Control)** — правила, стратегии, процедуры или стандарты, которыми руководствуется работа. Каждая работа должна иметь хотя бы одну стрелку управления. Стрелка управления рисуется как входящая в верхнюю грань работы. Управление влияет на работу, но не преобразуется работой. Если цель работы — изменить процедуру или стратегию, то такая процедура или стратегия будет для работы входом. В случае возникновения неопределенности в статусе стрелки (управление или вход) рекомендуется рисовать стрелку управления.

**Выход (Output)** — материал или информация, которые производятся работой. Каждая работа должна иметь хотя бы одну стрелку выхода. Работа без результата не имеет смысла и не должна моделироваться. Стрелка выхода рисуется как исходящая из правой грани работы.

**Механизм (Mechanism)** — ресурсы, которые выполняют работу, например персонал предприятия, станки, устройства и т. д. Стрелка механизма рисуется как входящая в нижнюю грань работы. По усмотрению аналитика стрелки механизма могут не изображаться в модели.

**Вызов (Call)** — специальная стрелка, указывающая на другую модель работы. Стрелка вызова рисуется как исходящая из нижней грани работы. Стрелка вызова используется для указания того, что некоторая работа выполняется за пределами моделируемой системы. В AllFusion Process Modeller стрелки вызова используются в механизме слияния и разделения моделей.

**Граничные стрелки.** Стрелки на контекстной диаграмме служат для описания взаимодействия системы с окружающим миром. Они могут начинаться у границы диаграммы и заканчиваться у работы, или наоборот. Такие стрелки называются граничными.

Для внесения граничной стрелки входа следует:

- щелкнуть по кнопке с символом стрелки ➔;
- в палитре инструментов перенести курсор к левой стороне экрана, пока не появится начальная штриховая полоска;
- щелкнуть один раз по полоске (откуда выходит стрелка) и еще раз в левой части работы со стороны входа (где заканчивается стрелка);
- вернуться в палитру инструментов и выбрать опцию редактирования стрелки ↗;
- щелкнуть правой кнопкой мыши на линии стрелки, во всплывающем меню выбрать Name и добавить имя стрелки в закладке Name диалога IDEFO Arrow Properties.

Стрелки управления, входа, механизма и выхода вносятся аналогично. Имена вновь внесенных стрелок автоматически заносятся в словарь Arrow Dictionary.

**Словарь стрелок** редактируется при помощи специального редактора Arrow Dictionary Editor, в котором определяется стрелка и вносится относящийся к ней комментарий. Словарь стрелок решает очень важную задачу. Диаграммы создаются аналитиком для того, чтобы провести сеанс экспертизы, т. е. обсудить диаграмму со специалистом предметной области. В любой предметной области формируется профессиональный жаргон, причем очень часто жаргонные выражения имеют нечеткий смысл и воспринимаются разными специалистами по-разному. В то же время аналитик — автор диаграмм должен употреблять те выражения, которые наиболее понятны экспертам. Поскольку формальные определения часто сложны для восприятия, аналитик вынужден употреблять профессиональный жаргон, а чтобы не возникло неоднозначных трактовок, в словаре стрелок каждому понятию можно дать расширенное и, если это необходимо, формальное определение.

Содержимое словаря стрелок можно распечатать в виде отчета (меню Tools/ Reports /ArrowReport...) и получить толковый словарь терминов предметной области, использующихся в модели.

**ICOM-коды.** Диаграмма декомпозиции предназначена для детализации работы. В отличие от моделей, отображающих структуру организации, работа на диаграмме верхнего уровня в IDEFO — это не элемент управления нижестоящими работами. Работы нижнего уровня — это то же самое, что работы верхнего уровня, но в более детальном изложении. Как следствие этого границы работы верхнего уровня — это то же самое, что границы диаграммы декомпозиции. ICOM (аббревиатура от Input, Control, Output и Mechanism) — коды, предназначенные для идентификации граничных стрелок. Код ICOM содержит префикс, соответствующий типу стрелки (I, C, O или M), и порядковый номер.

AllFusion Process Modeller вносит ICOM-коды автоматически. Для отображения ICOM-кодов следует включить опцию ICOM codes на закладке Display диалога Model Properties (меню Model/Model Properties).

**Несвязанные граничные стрелки (unconnected border arrow).** При декомпозиции работы входящие в нее и исходящие из нее стрелки (кроме стрелки вызова) автоматически

появляются на диаграмме декомпозиции (миграция стрелок), но при этом не касаются работ (Рис.1.2.6). Такие стрелки называются несвязанными и воспринимаются как синтаксическая ошибка.



Рис. 1.2.6. Пример несвязных стрелок

Для связывания стрелок входа, управления или механизма необходимо перейти в режим редактирования стрелок, щелкнуть по наконечнику стрелки и потом по соответствующему сегменту работы. Для связывания стрелки выхода необходимо перейти в режим редактирования стрелок, щелкнуть по сегменту выхода работы и затем по стрелке.

**Внутренние стрелки.** Для связи работ между собой используются внутренние стрелки, то есть стрелки, которые не касаются границы диаграммы, начинаются у одной и кончаются у другой работы.

Для рисования внутренней стрелки необходимо в режиме рисования стрелок щелкнуть по сегменту (например, выхода) одной работы и затем по сегменту (например, входа) другой. В IDEF0 различают пять типов связей работ.

*Связь по входу (output-input)*, когда стрелка выхода вышестоящей работы (далее — просто выход) направляется на вход нижестоящей (Рис.1.2.7).

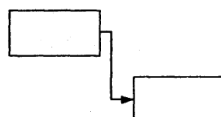


Рис. 1.2.7. Связь по входу

*Связь по управлению (output-control)*, когда выход вышестоящей работы направляется на управление нижестоящей (Рис.2.8). Связь по управлению показывает доминирование вышестоящей работы. Данные или объекты выхода вышестоящей работы не меняются в нижестоящей.

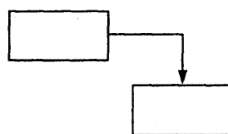


Рис. 1.2.8. Связь по управлению

*Обратная связь по входу (output-input feedback)*, когда выход нижестоящей работы направляется на вход вышестоящей (Рис.1.2.9). Такая связь, как правило, используется для описания циклов.

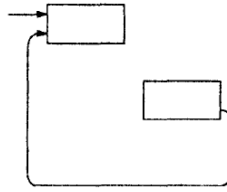


Рис. 1.2.9. Обратная связь по входу

*Обратная связь по управлению (output-control feedback) и обратная связь по входу (output-input feedback)* являются более сложными, поскольку представляют собой итерацию или рекурсию. А именно выходы из одной работы влияют на будущее выполнение других работ, что впоследствии повлияет на исходную работу.

Обратная связь по управлению возникает тогда, когда выход некоторого блока влияет на блок с большим доминированием (Рис.1.2.10).

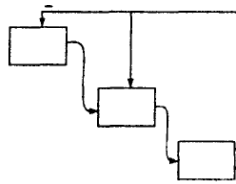


Рис. 1.2.10. Обратная связь по управлению

Связи «выход-механизм» (*output-mechanism*), когда выход одной работы направляется на механизм другой и встречаются нечасто. Они отражают ситуацию, при которой выход одной функции становится средством достижения цели для другой. Связи «выход-механизм» характерны при распределении источников ресурсов (например, требуемые инструменты, обученный персонал, физическое пространство, оборудование, финансирование, материалы).

**Разветвляющиеся и сливающиеся стрелки.** Одни и те же данные или объекты, порожденные одной работой, могут использоваться сразу в нескольких других работах. С другой стороны, стрелки, порожденные в разных работах, могут представлять собой одинаковые или однородные данные или объекты, которые в дальнейшем используются или перерабатываются в одном месте. Для моделирования таких ситуаций в IDEFO используются разветвляющиеся и сливающиеся стрелки. Для разветвления стрелки нужно в режиме редактирования стрелки щелкнуть по фрагменту стрелки и по соответствующему сегменту работы. Для слияния двух стрелок выхода нужно в режиме редактирования стрелки сначала щелкнуть по сегменту выхода работы, а затем по соответствующему фрагменту стрелки.

Смысл разветвляющихся и сливающихся стрелок передается именованием каждой ветви стрелок. Существуют определенные **правила именования** таких стрелок. Рассмотрим их на примере разветвляющихся стрелок. Если стрелка именована до разветвления, а после разветвления ни одна из ветвей не именована, то подразумевается, что каждая ветвь моделирует те же данные или объекты, что и ветвь до разветвления. Если же стрелка именована до разветвления, а после разветвления какая-либо из ветвей тоже именована, то подразумевается, что эти ветви соответствуют именованию. Если при этом какая-либо ветвь после разветвления осталась неименованной, то подразумевается, что она моделирует те же данные или объекты, что и ветвь до разветвления. Недопустима ситуация, когда стрелка до разветвления не именована, а после разветвления не именована какая-либо из ветвей. Такая стрелка будет определена как синтаксическая ошибка. Правила именования сливающихся стрелок полностью аналогичны — ошибкой будет считаться стрелка, которая после слияния не именована, а до слияния не именована какая-либо из ее ветвей. Для именования отдельной ветви разветвляющихся и сливающихся стрелок следует выделить на диаграмме

только одну ветвь, после чего вызвать редактор имени и присвоить имя стрелке. Это имя будет соответствовать только выделенной ветви.

**Туннелирование стрелок.** Вновь внесенные граничные стрелки на диаграмме декомпозиции нижнего уровня изображаются в квадратных скобках и автоматически не появляются на диаграмме верхнего уровня. Для их «перетаскивания» вверх нужно щелкнуть правой кнопкой мыши по квадратным скобкам граничной стрелки и в контекстном меню выбрать команду Arrow Tunnel. Выбор команды из контекстного меню появляется диалог Border Arrow Editor (Рис.1.2.11).

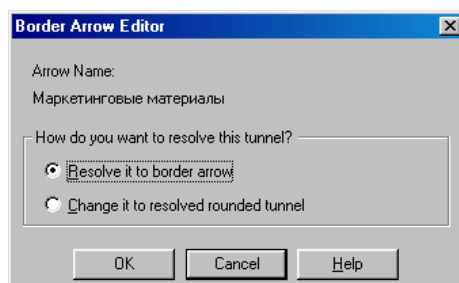


Рис. 1.2.2.7. Диалог Border Arrow Editor

Если выбрать опцию **Resolve it to border arrow**, стрелка мигрирует на диаграмму верхнего уровня, а если выбрать **Change it to resolved rounded tunnel** – стрелка будет затуннелирована и не попадет на другую диаграмму.

Туннельная стрелка изображается с круглыми скобками на конце. Туннелирование может быть применено для изображения малозначимых стрелок. Если на какой-либо диаграмме нижнего уровня необходимо изобразить малозначимые данные или объекты, которые не обрабатываются или не используются работами на текущем уровне, то их необходимо направить на вышестоящий уровень (на родительскую диаграмму). Если эти данные не используются на родительской диаграмме, их нужно направить еще выше, и т. д. В результате малозначимая стрелка будет изображена на всех уровнях и затруднит чтение всех диаграмм, на которых она присутствует. Выходом является туннелирование стрелки на самом нижнем уровне. Такое туннелирование называется «не-в-родительской-диаграмме».

Другим примером туннелирования может быть ситуация, когда стрелка механизма мигрирует с верхнего уровня на нижний, причем на нижнем уровне этот механизм используется одинаково во всех работах без исключения. (Предполагается, что не нужно детализировать стрелку механизма, т. е. стрелка механизма на дочерней работе именована до разветвления, а после разветвления ветви не имеют собственного имени). В этом случае стрелка механизма на нижнем уровне может быть удалена, после чего на родительской диаграмме она может быть туннелирована, а в комментарии к стрелке или в словаре можно указать, что механизм будет использоваться во всех работах дочерней диаграммы декомпозиции. Такое туннелирование называется «не-в-дочерней-работе».

**Нумерация работ и диаграмм.** Все работы модели нумеруются. Номер состоит из префикса и числа. Может быть использован префикс любой длины, но обычно используют префикс А. Контекстная (корневая) работа дерева имеет номер АО. Работы декомпозиции АО имеют номера А1, А2, А3 и т. д. Работы декомпозиции нижнего уровня имеют номер родительской работы и очередной порядковый номер, например работы декомпозиции А3 будут иметь номера А31, А32, А33, А34 и т. д. Работы образуют иерархию, где каждая работа может иметь одну родительскую и несколько дочерних работ, образуя дерево. Такое дерево называют деревом узлов, а вышеописанную нумерацию — нумерацией по узлам. Диаграммы IDEFO имеют двойную нумерацию. Во-первых, диаграммы имеют номера по узлу. Контекстная диаграмма всегда имеет номер А-0, декомпозиция контекстной диаграммы — номер АО, остальные диаграммы декомпозиции — номера по соответствующему узлу (например, А1, А2, А21, А213 и т. д.). Система автоматически поддерживает нумерацию по

узлам, т. е. при проведении декомпозиции создается новая диаграмма и ей автоматически присваивается соответствующий номер. В результате проведения экспертизы диаграммы могут уточняться и изменяться, следовательно, могут быть созданы различные версии одной и той же (с точки зрения ее расположения в дереве узлов) диаграммы декомпозиции. Система позволяет иметь в модели только одну диаграмму декомпозиции в данном узле. Прежние версии диаграммы можно хранить в виде бумажной копии либо как FEO-диаграмму. (К сожалению, при создании FEO-диаграмм отсутствует возможность отката, т. е. из диаграммы можно получить декомпозиции FEO, но не наоборот.) В любом случае следует отличать различные версии одной и той же диаграммы. Для этого существует специальный номер — C-number, который должен присваиваться автором модели вручную. C-number — это произвольная строка, но рекомендуется придерживаться стандарта, когда номер состоит из буквенного префикса и порядкового номера, причем в качестве префикса используются инициалы автора диаграммы, а порядковый номер отслеживается автором вручную, например MCB00021.

**Диаграмма дерева узлов (Node Tree)** показывает иерархию работ в модели и позволяет рассмотреть всю модель целиком, но не показывает взаимосвязи между работами. Диаграмм деревьев узлов может быть в модели сколь угодно много, поскольку дерево может быть построено на произвольную глубину и не обязательно с корня. Процесс создания модели работ является итерационным, следовательно, работы могут менять свое расположение в дереве узлов многократно. Чтобы не запутаться и проверить способ декомпозиции, следует после каждого изменения создавать диаграмму дерева узлов. Впрочем, система имеет мощный инструмент навигации по модели — Model Explorer, который позволяет представить иерархию работ и диаграмм в удобном и компактном виде, однако составляющей стандарта IDEF0.

Для создания диаграммы дерева узлов следует выбрать в меню пункт Diagram/Add Node Tree. В диалоге Node Tree Definition следует указать глубину дерева — Number of Levels (по умолчанию — 3) и корень дерева (по умолчанию — родительская работа текущей диаграммы). По умолчанию нижний уровень декомпозиции показывается в виде списка, остальные работы — в виде прямоугольников. Для отображения всего дерева в виде прямоугольников следует выключить опцию Bullet Last Level. При создании дерева узлов следует указать имя диаграммы, поскольку, если в нескольких диаграммах в качестве корня на дереве узлов использовать одну и ту же работу, все эти диаграммы получат одинаковый номер (номер узла + постфикс N, например AON) и в списке открытых диаграмм (пункт меню Window) их можно будет различить только по имени.

**Диаграммы «только для экспозиции» (FEO)** часто используются в модели для иллюстрации других точек зрения, для отображения отдельных деталей, которые не поддерживаются явно синтаксисом IDEF0. Диаграммы FEO позволяют нарушить любое синтаксическое правило, поскольку по сути являются просто картинками — копиями стандартных диаграмм и не включаются в анализ синтаксиса. Для создания диаграммы FEO следует выбрать пункт меню Diagram/Add FEO Diagram. В возникающем диалоге Add New FEO Diagram следует указать имя диаграммы FEO и тип родительской диаграммы. Новая диаграмма получает номер, который генерируется автоматически (номер родительской диаграммы по узлу + постфикс F, например A1F).

### **Слияние и расщепление моделей**

Возможность слияния и расщепления моделей обеспечивает коллективную работу над проектом. Так, руководитель проекта может создать декомпозицию верхнего уровня и дать задание аналитикам продолжить декомпозицию каждой ветви дерева в виде отдельных моделей. После окончания работы над отдельными ветвями все подмодели могут быть слиты в единую модель. С другой стороны, отдельная ветвь модели может быть отщеплена для использования в качестве независимой модели, для доработки или архивирования.

Система использует для слияния и разветвления моделей стрелки вызова. Для слияния необходимо выполнить следующие условия:

- обе сливаемые модели должны быть открыты;
- имя модели-источника, которое присоединяют к модели-цели, должно совпадать с именем стрелки вызова работы в модели-цели;
- стрелка вызова должна исходить из недекомпозируемой работы (работа должна иметь диагональную черту в левом верхнем углу);
- имена контекстной работы подсоединяемой модели-источника и работы на модели-цели, к которой мы подсоединяем модель-источник, должны совпадать;
- модель-источник должна иметь, по крайней мере, одну диаграмму декомпозиции.

Для слияния моделей нужно щелкнуть правой кнопкой мыши по работе со стрелкой вызова в модели-цели и во всплывающем меню выбрать пункт Merge Model.

Появляется диалог, в котором следует указать опции слияния модели. При слиянии моделей объединяются и словари стрелок и работ. В случае одинаковых определений возможна перезапись определений или принятие определений из модели-источника. То же относится к именам стрелок, хранилищ данных и внешним ссылкам. (Хранилища данных и внешние ссылки — объекты диаграмм потоков данных, DFD, будут рассмотрены ниже.)

После подтверждения слияния (кнопка ОК) модель-источник подсоединяется к модели-цели, стрелка вызова исчезает, а работа, от которой отходила стрелка вызова, становится недекомпозируемой — к ней подсоединяется диаграмма декомпозиции первого уровня модели-источника. Стрелки, касающиеся работы на диаграмме модели-цели, автоматически не мигрируют в декомпозицию, а отображаются как неразрешенные. Их следует туннелировать вручную.

В процессе слияния модель-источник остается неизменной, и к модели-цели подключается фактически ее копия. Не нужно путать слияние моделей с синхронизацией. Если в дальнейшем модель-источник будет редактироваться, эти изменения автоматически не попадут в соответствующую ветвь модели-цели.

Разделение моделей производится аналогично. Для отщепления ветви от модели следует щелкнуть правой кнопкой мыши по декомпозированной работе (работа не должна иметь диагональной черты в левом верхнем углу) и выбрать во всплывающем меню пункт Split Model. В появившемся диалоге Split Options следует указать имя создаваемой модели. После подтверждения расщепления в старой модели работа станет недекомпозированной (признак — диагональная черта в левом верхнем углу), будет создана стрелка вызова, ее имя будет совпадать с именем новой модели, и, наконец, будет создана новая модель, причем имя контекстной работы будет совпадать с именем работы, от которой была «оторвана» декомпозиция.

## 1.2.1 Лабораторная работа № 2. Диаграмма IDEF0

### Цель работы:

- создание нового проекта,
- формирование контекстной диаграммы,
- проведение связей,
- слияние и расщепление моделей.

Начнем построение функциональной модели с описания первоначальной глобальной функции – разработки плана привлечения и размещения ресурсов банка и ее связей с внешним миром (рис. 1.2.12).

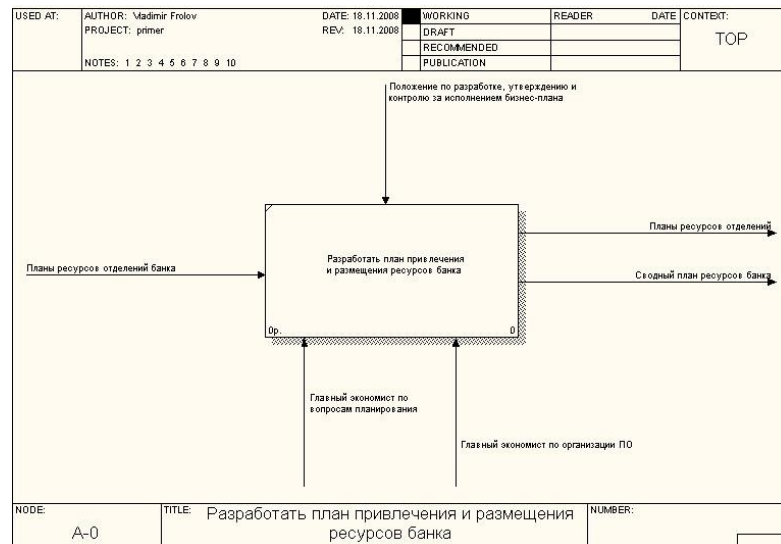


Рис. 1.2.12. Первый уровень функциональной модели

Далее декомпозируем эту функцию на более мелкие функции, описывающие нужный нам процесс. Следующий уровень проектируемой функциональной модели будет состоять из 5 блоков (рис. 1.2.13).

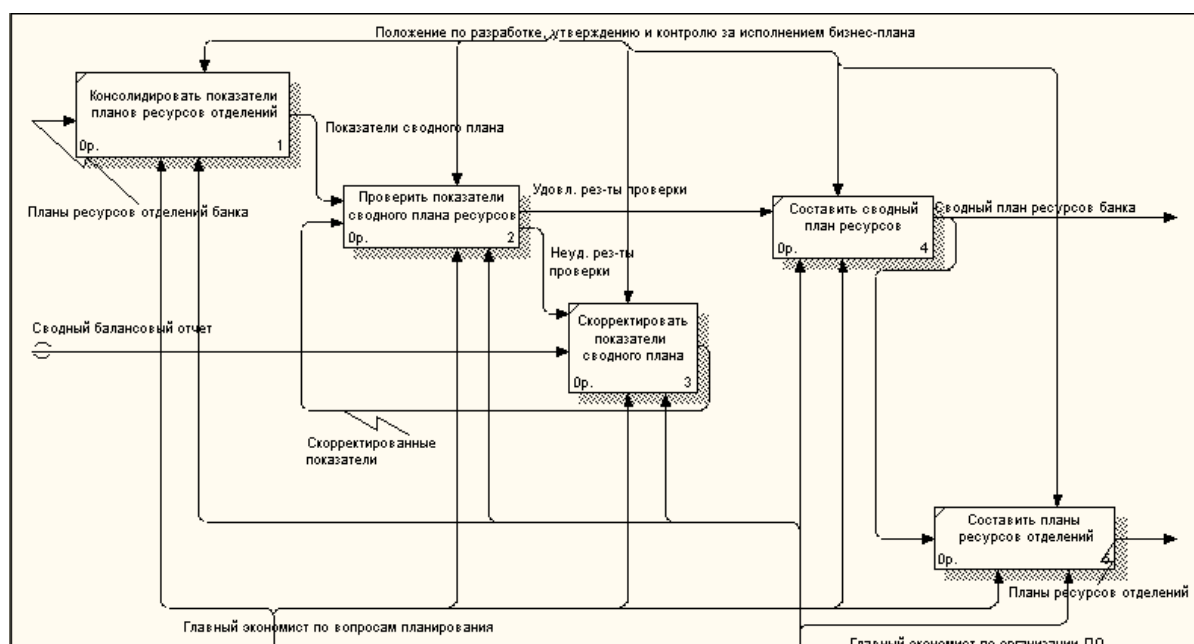


Рис. 1.2.13. Второй уровень функциональной модели

Декомпозируем далее следующий блок функциональной модели – «проверить показатели сводного плана ресурсов». Следующий уровень декомпозиции будет состоять из трех функциональных блоков (рис. 1.2.14).

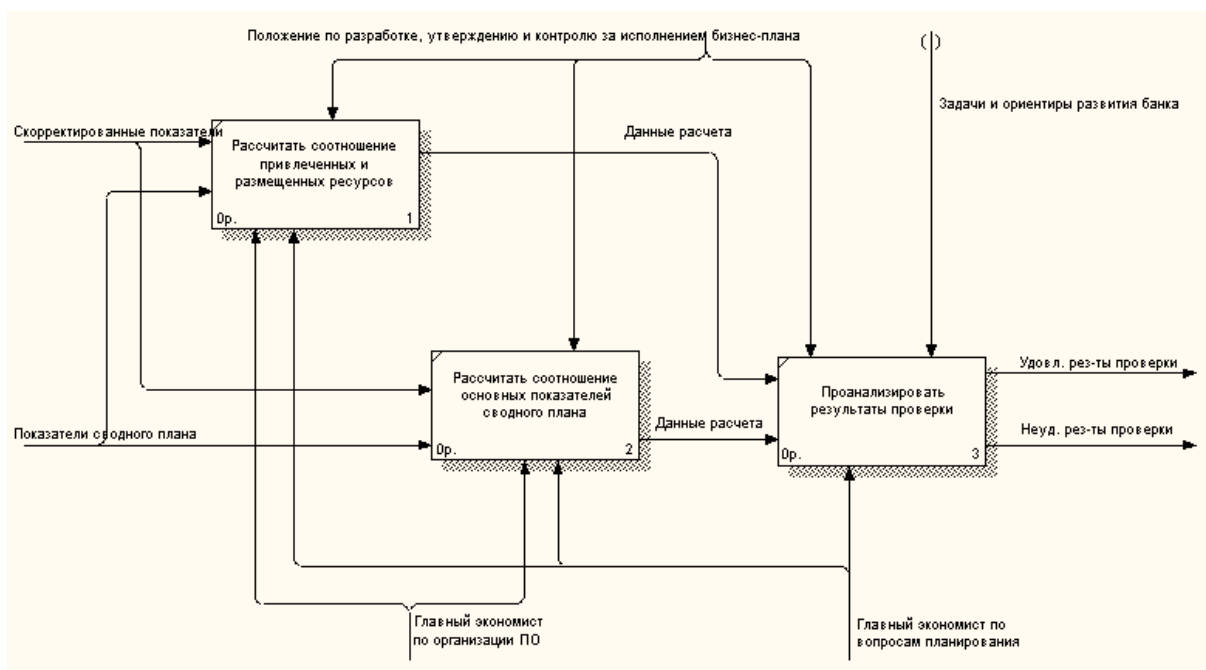


Рис. 1.2.14. Третий уровень функциональной модели

Функциональная модель заданной предметной области построена. Теперь следует проверить синтаксис полученной модели. Программа выдала список синтаксических ошибок (рис. 1.2.15), показывающий, что на уровне декомпозиции диаграммы A0 имеется одна неразрешенная стрелка с названием «сводный балансовый отчет», на уровне декомпозиции диаграммы A2 также имеется неразрешенная стрелка с названием «задачи и ориентиры развития банка».

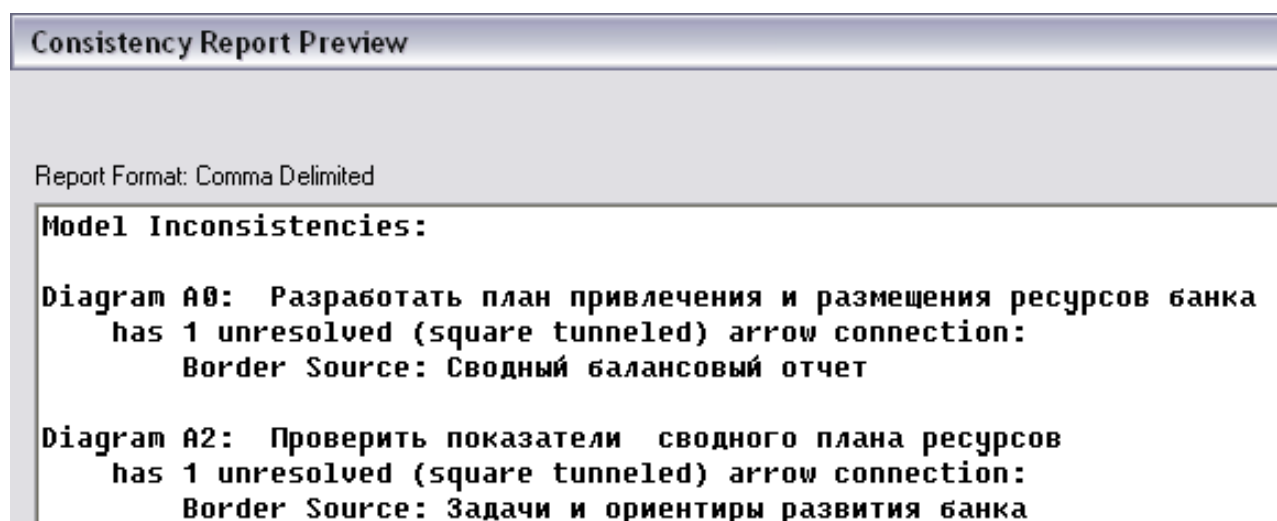


Рис. 1.2.15. Отчет по синтаксическим ошибкам модели

Данные стрелки следует сделать туннельными, так как они свойственны только для указанных уровней диаграммы и не должны появиться на верхних.

И в заключение работы следует сформировать отчет Node Tree (рис. 1.2.16). На сформированном отчете Node Tree наглядно видно количество уровней декомпозиции

построенной функциональной модели и отношение между родительскими и дочерними диаграммами.

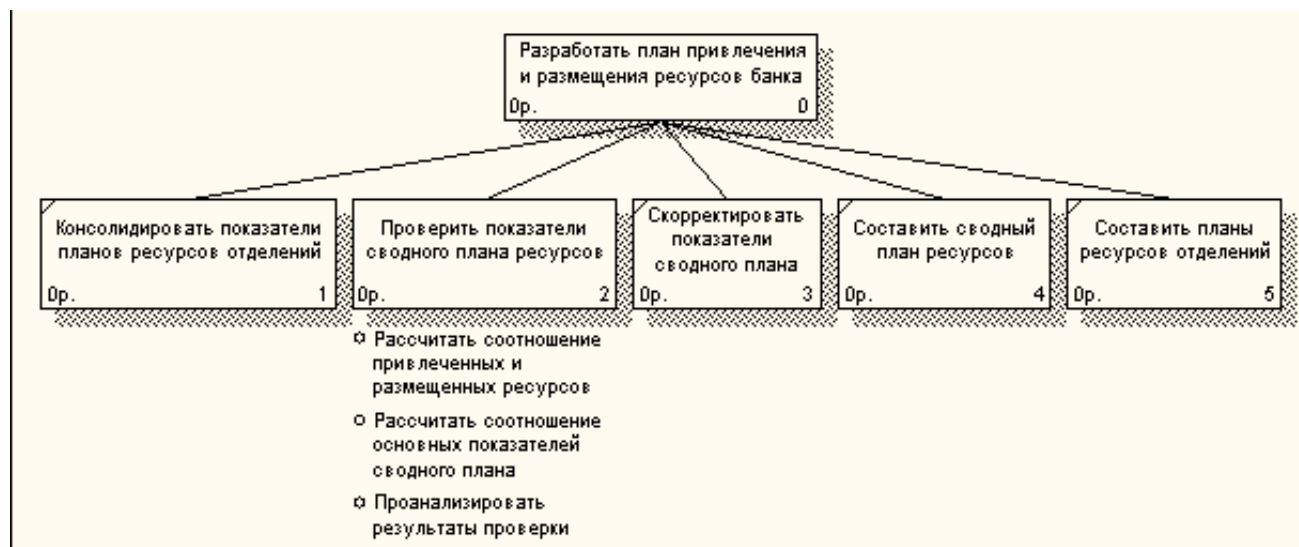


Рис. 1.2.16. Отчет Node Tree

### 1.2.2 Задания

1. Создать новый проект.
2. Сформировать контекстную диаграмму по системе согласно методологии IDEFO.
3. Задать входы, выходы, механизмы и управление.
4. Декомпозировать контекстную диаграмму.
5. Провести связи по выходу, по управлению и по входу.
6. Построить диаграмму дерева узлов.
7. Произвести слияние и расщепление модели.

### 1.2.3 Контрольные вопросы

- 1 Что представляет собой модель в нотации IDEFO?
- 2 Что обозначают работы в IDEFO?
- 3 Назовите порядок наименования работ?
- 4 Какое количество работ должно присутствовать на одной диаграмме?
- 5 Как задать имя работы.
- 6 Как провести связь между работами?
- 7 Как добавить работу на диаграмму?
- 8 Опишите процесс декомпозиции работы.
- 9 Что называется порядком доминирования?
- 10 Как располагаются работы по принципу доминирования?
- 11 Каково назначение сторон прямоугольников работ на диаграммах?
- 12 Перечислите типы стрелок.
- 13 Назовите виды взаимосвязей.
- 14 Что называется граничными стрелками?
- 15 Объясните принцип именования разветвляющихся и сливающихся стрелок.
- 16 Как разрешить туннелированные стрелки?

### 1.3 Диаграммы потоков данных

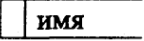
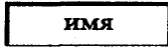
**Диаграммы потоков данных (Data Flow Diagrams)** являются основным средством моделирования функциональных требований к проектируемой системе. Требования представляются в виде иерархии процессов, связанных потоками данных. Диаграммы потоков данных показывают, как каждый процесс преобразует свои входные данные в выходные, и выявляют отношения между этими процессами. DFD-диаграммы успешно используются как дополнение к модели IDEFO для описания документооборота и обработки информации. Подобно IDEFO, DFD представляет моделируемую систему как сеть связанных работ.

Основные компоненты DFD:

1. функции обработки информации (работы);
  2. документы (стрелки, аггов), объекты, сотрудники или отделы, которые участвуют в обработке информации;
  3. внешние ссылки (external reference), которые обеспечивают интерфейс с внешними объектами, находящимися за границами моделируемой системы;
  4. таблицы для хранения документов (хранилища данных, data store).
- Для построения диаграмм DFD используется нотация Гейна – Сарсона (Табл.4).

Таблица 4

Нотация Гейна – Сарсона

| Компонент        | Обозначение                                                                          |
|------------------|--------------------------------------------------------------------------------------|
| Процесс          |    |
| Поток данных     |   |
| Хранилище        |  |
| Внешняя сущность |  |

**Работы (процессы)** представляют собой функции системы, продуцирующие выходные потоки из входных в соответствии с действием, задаваемым именем процесса. Хотя работы изображаются прямоугольниками со скругленными углами, смысл их совпадает со смыслом работ IDEFO и IDEF3. Так же, как процессы IDEF3, они имеют входы и выходы, но не поддерживают управления и механизмы, как IDEFO. Каждый процесс должен иметь уникальный номер для ссылок на него внутри диаграммы. Этот номер может использоваться совместно с номером диаграммы для получения уникального индекса процесса во всей модели.

**Потоки данных** изображаются стрелками и являются механизмами, использующимися для моделирования передачи информации (или физических компонентов) из одной части системы в другую. Поскольку в DFD каждая сторона работы не имеет четкого назначения, как в IDEFO, стрелки могут подходить и выходить из любой грани прямоугольника работы. Потоки изображаются на диаграмме именованными стрелками, ориентация которых указывает направление движения информации. В DFD также применяются двунаправленные стрелки для описания диалогов типа «команда-ответ» между работами, между работой и внешней сущностью и между внешними сущностями.

В отличие от стрелок IDEFO, которые представляют собой жесткие взаимосвязи, стрелки DFD показывают, как объекты (включая данные) двигаются от одной работы к другой. Это представление потоков совместно с хранилищами данных и внешними сущностями делает модели DFD более похожими на физические характеристики системы — движение объектов, хранение объектов, поставка и распространение объектов.

В DFD стрелки могут сливаться и разветвляться, что позволяет описать декомпозицию стрелок. Каждый новый сегмент сливающейся или разветвляющейся стрелки может иметь собственное имя.

**Хранилище данных (data store)** позволяет на определенных участках определять данные, которые будут сохраняться в памяти между процессами. Фактически хранилище представляет «срезы» потоков данных во времени. Информация, которую оно содержит, может использоваться в любое время после ее определения, при этом данные могут выбираться в любом порядке. Имя хранилища должно идентифицировать его содержимое. В случае, когда поток данных входит в хранилище или выходит из него и его структура соответствует структуре хранилища, он должен иметь то же самое имя, которое нет необходимости отражать на диаграмме.

**Внешняя сущность (external reference)** представляет сущность вне контекста системы, являющуюся источником или приемником данных системы. Внешние сущности изображаются в виде прямоугольника с тенью и обычно располагаются по краям диаграммы. Предполагается, что объекты, представленные такими узлами, не должны участвовать ни в какой обработке. Одна внешняя сущность может быть использована многократно на одной или нескольких диаграммах.

Для того чтобы дополнить модель IDEF0 диаграммой DFD, нужно в процессе декомпозиции в диалоге Activity Box Count выбрать радио-кнопку DFD. В палитре инструментов на новой диаграмме DFD появляются новые кнопки:

-  (*External Reference*) — добавить в диаграмму внешнюю ссылку;
-  (*Datastore*) — добавить в диаграмму хранилище данных;
-  *Diagram Dictionary Editor* — ссылка на другую страницу. В отличие от IDEF0 этот инструмент позволяет направить стрелку на любую диаграмму (а не только на верхний уровень).

В отличие от IDEF0, где система рассматривается как взаимосвязанные работы, DFD рассматривает систему как совокупность предметов. Контекстная диаграмма часто включает работы и внешние ссылки. Включение внешних ссылок в контекстную диаграмму не отменяет требования методологии четко определить цель, область и единую точку зрения на моделируемую систему.

Диаграммы DFD могут быть построены с использованием традиционного структурного анализа, подобно тому, как строятся диаграммы IDEF0.

В DFD номер каждой работы может включать префикс, номер родительской работы (A) и номер объекта. Номер объекта — это уникальный номер работы на диаграмме. Например, работа может иметь номер A. 12.4. Уникальный номер имеют хранилища данных и внешние сущности независимо от их расположения на диаграмме. Каждое хранилище данных имеет префикс D и уникальный номер, например D5. Каждая внешняя сущность имеет префикс E и уникальный номер, например E5.

### 1.3.1 Лабораторная работа № 3. Диаграмма DFD

#### Цель работы:

- построение диаграмм потоков данных (DFD)

Для функциональной модели разработки плана привлечения и размещения ресурсов банка и ее связей с внешним миром (с.27-29) построим диаграммы потоков данных. На рис.1.3.1–1.3.3 показаны основные этапы.

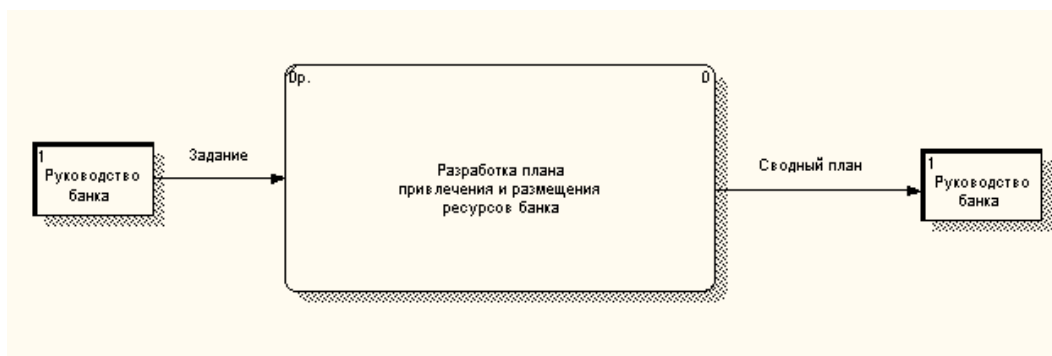


Рис. 1.3.1 Контекстная диаграмма DFD

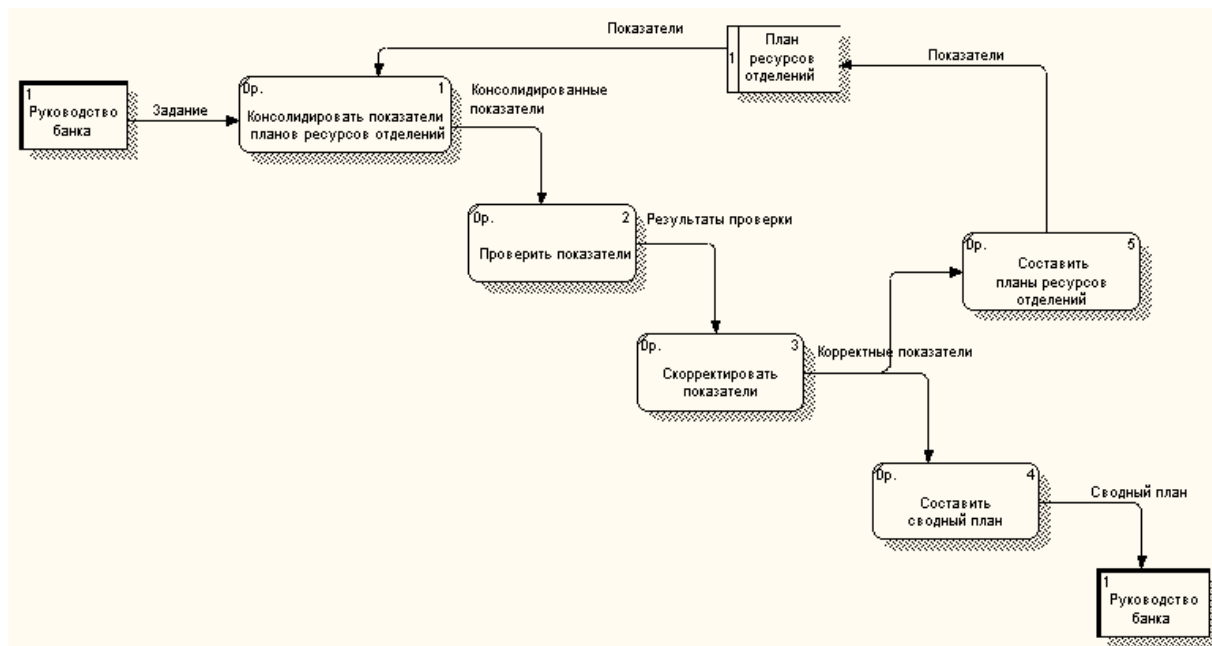


Рис. 1.3.2. Первый уровень декомпозиции

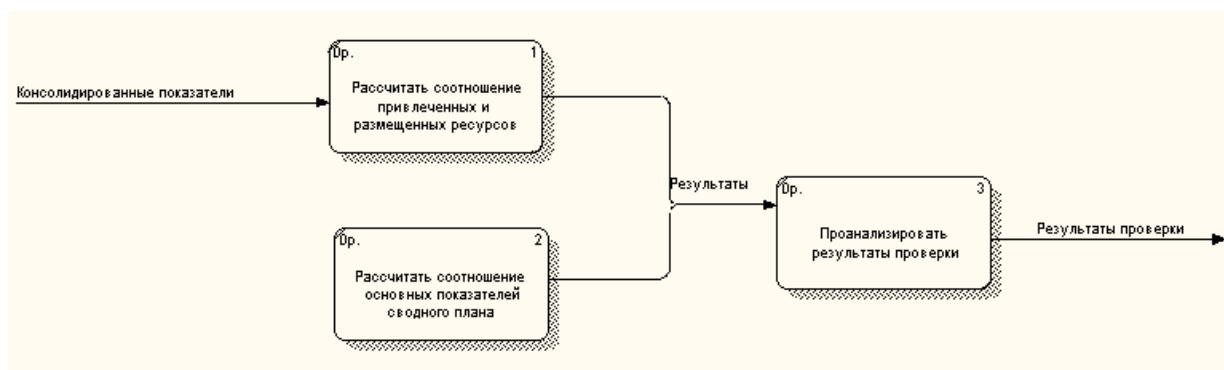


Рис. 1.3.3 Второй уровень декомпозиции

### **1.3.2 Задания**

1. Дополнить созданную на предыдущей работе диаграмму IDEF0 диаграммой DFD.
2. Добавить на диаграмму DFD внешнюю сущность и хранилище данных.
3. Связать диаграмму и внешнюю сущность.
4. Связать диаграмму и хранилище.
5. Определить имя связи с внешней сущностью.

### **1.3.3 Контрольные вопросы**

1. Что описывает диаграмма DFD?
2. Какая нотация используется для построения диаграмм DFD?
3. Перечислите составные части диаграммы DFD.
4. В чем состоит назначение процесса?
5. Что называется внешней сущностью?
6. Что описывают хранилища?
7. Объясните механизм дополнения диаграммы IDEF0 диаграммой DFD.

## 1.4 Диаграммы процессов

Методология моделирования IDEF3 использует графическое описание информационных потоков, взаимоотношений между процессами обработки информации и объектов, являющихся частью этих процессов. Диаграммы Workflow могут быть использованы в моделировании бизнес-процессов для анализа завершенности процедур обработки информации. С их помощью можно описывать сценарии действий сотрудников организации, например последовательность обработки заказа или события, которые необходимо обработать за конечное время. Каждый сценарий сопровождается описанием процесса и может быть использован для документирования каждой функции. IDEF3 дает возможность аналитикам описать ситуацию, когда процессы выполняются в определенной последовательности, а также описать объекты, участвующие совместно в одном процессе.

Техника описания набора данных IDEF3 является частью структурного анализа. В отличие от некоторых методик описаний процессов IDEF3 не ограничивает аналитика чрезмерно жесткими рамками синтаксиса, что может привести к созданию неполных или противоречивых моделей.

IDEF3 может быть также использован как метод создания процессов. IDEF3 дополняет IDEFO и содержит все необходимое для построения моделей, которые в дальнейшем могут быть использованы для имитационного анализа. Каждая работа в IDEF3 описывает какой-либо сценарий бизнес-процесса и может являться составляющей другой работы. Поскольку сценарий описывает цель и рамки модели, важно, чтобы работы именовались отглагольным существительным, обозначающим процесс действия, или фразой, содержащей такое существительное.

Точка зрения на модель должна быть документирована. Обычно это точка зрения человека, ответственного за работу в целом. Также необходимо документировать цель модели — те вопросы, на которые призвана ответить модель.

**Диаграмма** является основной единицей описания в IDEF3. Важно правильно построить диаграммы, поскольку они предназначены для чтения другими людьми (а не только автором).

**Единицы работы (Unit of Work - UOW)** — также называемые процессами (activity), являются центральными компонентами модели. В IDEF3 работы изображаются прямоугольниками с прямыми углами и имеют имя, выраженное отглагольным существительным, обозначающим процесс действия, одиночным или в составе фразы, и номер (идентификатор); другое имя существительное в составе той же фразы обычно отображает основной выход (результат) работы (например, «Изготовление изделия»). Часто имя существительное в имени работы меняется в процессе моделирования, поскольку модель может уточняться и редактироваться. Идентификатор работы присваивается при создании и не меняется никогда. Даже если работа будет удалена, ее идентификатор не будет вновь использоваться для других работ. Обычно номер работы состоит из номера родительской работы и порядкового номера на текущей диаграмме.

**Связи** показывают взаимоотношения работ. Все связи в IDEF3 одно-направлены и могут быть направлены куда угодно, но обычно диаграммы IDEF3 стараются построить так, чтобы связи были направлены слева направо. В IDEF3 различают три типа стрелок, изображающих связи, стиль которых устанавливается через меню Edit/ArrowStyle:

**Старшая (Precedence)** сплошная линия , связывающая единицы работ (UOW). Рисуются слева направо или сверху вниз. Показывает, что работа-источник должна закончиться прежде, чем работа-цель начнется.

**Потоки объектов (Object Flow)** стрелка с двумя наконечниками , применяется для описания того факта, что объект используется в двух или более единицах работы, например, когда объект порождается в одной работе и используется в другой.

**Отношения (Relational Link)** пунктирная линия , используемая для изображения связей между единицами работ (UOW) а также между единицами работ и объектами ссылок. Отношение показывает, что стрелка является альтернативой старшей стрелке или потоку

объектов в смысле задания последовательности выполнения работ — работа-источник не обязательно должна закончиться, прежде чем работа-цель начнется. Более того, работа-цель может закончиться прежде, чем закончится работа-источник.

Окончание одной работы может служить сигналом к началу нескольких работ, или же одна работа для своего запуска может ожидать окончания нескольких работ. Для отображения логики взаимодействия стрелок при слиянии и разветвлении или для отображения множества событий, которые могут или должны быть завершены перед началом следующей работы, используются **перекрестки (Junction)**. Различают перекрестки для слияния (Fan-in Junction) и разветвления (Fan-out Junction) стрелок. Перекресток не может использоваться одновременно для слияния и для разветвления. Для внесения перекрестка служит кнопка  (Junction Tool) в палитре инструментов. В диалоге Select Junction Type необходимо указать тип перекрестка. Смысл каждого типа приведен в таблице 5.

Все перекрестки на диаграмме нумеруются, каждый номер имеет префикс J. Можно редактировать свойства перекрестка при помощи диалога Junction Properties, который вызывается в контекстном меню перекрестка командой Definition/Note. В отличие от IDEF0 и DFD в IDEF3 стрелки могут сливаться и разветвляться только через перекрестки.

Таблица 5

| Типы перекрестков                                                                          |                   |                                                                    |                                                                 |
|--------------------------------------------------------------------------------------------|-------------------|--------------------------------------------------------------------|-----------------------------------------------------------------|
| Обозначение                                                                                | Наименование      | Смысл в случае слияния стрелок                                     | Смысл в случае разветвления стрелок                             |
| <br>AND  | Асинхронное «И»   | Все предшествующие процессы должны быть завершены                  | Все следующие процессы должны быть запущены                     |
| <br>AND | Синхронное «И»    | Все предшествующие процессы завершены одновременно                 | Все следующие процессы запускаются одновременно                 |
| <br>OR  | Асинхронное «ИЛИ» | Один или несколько предшествующих процессов должны быть завершены  | Один или несколько следующих процессов должны быть запущены     |
| <br>OR  | Синхронное «ИЛИ»  | Один или несколько предшествующих процессов завершены одновременно | Один или несколько следующих процессов запускаются одновременно |
| <br>XOR | Исключающее «ИЛИ» | Только один предшествующий процесс завершен                        | Только один следующий процесс запускается                       |

**Объекты-ссылки** - являются специальными символами, которые ссылаются на внешние части описания процесса и выражают некую идею, концепцию или данные, которые нельзя связать со стрелкой, перекрестком или работой. Они добавляются на диаграмму для того, чтобы обратить внимание редактора на что-либо важное, что невозможно связать со стрелкой, работой или перекрестком. Для внесения объекта ссылки служит кнопка  (Referent Tool) в палитре инструментов. Объект ссылки изображается в виде прямоугольника, похожего на прямоугольник работы. Имя объекта ссылки задается в диалоге Referent Properties (пункт Name контекстного меню), в качестве имени можно использовать имя какой-либо стрелки с других диаграмм или имя сущности из модели данных. Объекты ссылки должны быть связаны с единицами работ или перекрестками пунктирными линиями. Официальная спецификация IDEF3 различает *три стиля объектов*

ссылок — безусловные (unconditional), синхронные (synchronous) и асинхронные (asynchronous). AllFusion Process Modeller поддерживает только безусловные объекты ссылок. Синхронные и асинхронные объекты ссылок, используемые в диаграммах переходов состояний объектов, не поддерживаются. При внесении объектов ссылок помимо имени следует указывать тип объекта ссылки. Типы объектов ссылок приведены в таблице 6.

Таблица 6

Типы объектов-ссылок

| Тип объекта-ссылки     | Цель описания                                                                                                                                                                                                                                                                                                                                    |
|------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| OBJECT                 | Описывает участие важного объекта в работе                                                                                                                                                                                                                                                                                                       |
| GOTO                   | Инструмент циклического перехода (в повторяющейся последовательности работ), возможно на текущей диаграмме, но не обязательно. Если все работы цикла присутствуют на текущей диаграмме, цикл может также изображаться стрелкой, возвращающейся на стартовую работу. GOTO может ссылаться на перекресток                                          |
| UOB (Unit of behavior) | Применяется, когда необходимо подчеркнуть множественное использование какой-либо работы, но без цикла. Например, работа «Контроль качества» может быть использована в процессе «Изготовления изделия» несколько раз, после каждой единичной операции. Обычно этот тип ссылки не используется для моделирования автоматически запускающихся работ |
| NOTE                   | Используется для документирования важной информации, относящейся к каким-либо графическим объектам на диаграмме. NOTE является альтернативой внесению текстового объекта в диаграмму                                                                                                                                                             |
| ELAB (Elaboration)     | Используется для усовершенствования графиков или их более детального описания. Обычно употребляется для детального описания разветвления и слияния стрелок на перекрестках                                                                                                                                                                       |

В IDEF3 **декомпозиция** используется для детализации работ. Методология IDEF3 позволяет декомпозировать работу многократно, т. е. работа может иметь множество дочерних работ. Это позволяет в одной модели описать альтернативные потоки. Возможность множественной декомпозиции предъявляет дополнительные требования к нумерации работ. Так, номер работы состоит из номера родительской работы, версии декомпозиции и собственного номера работы на текущей диаграмме.

Рассмотрим процесс декомпозиции диаграмм IDEF3, включающий взаимодействие автора (аналитика) и одного или нескольких экспертов предметной области. Перед проведением сеанса экспертизы у экспертов предметной области должны быть документированные сценарии и рамки модели, для того чтобы понять цели декомпозиции. Обычно эксперт предметной области передает аналитику текстовое описание сценария. В дополнение к этому может существовать документация, описывающая интересующие процессы. Из этой информации аналитик должен составить предварительный список работ (отглагольные существительные, обозначающие процесс) и объектов (существительные, обозначающие результат выполнения работы), которые необходимы для перечисленных работ. В некоторых случаях целесообразно создать графическую модель для представления ее эксперту предметной области.

Поскольку разные фрагменты модели IDEF3 могут быть созданы разными группами аналитиков в разное время, IDEF3 поддерживает простую схему нумерации работ в рамках всей модели. Разные аналитики оперируют разными диапазонами номеров, работая при этом независимо.

В результате дополнения диаграмм IDEFO диаграммами DFD и IDEF3 может быть создана смешанная модель, которая наилучшим образом описывает все стороны деятельности предприятия. Иерархию работ в смешанной модели можно увидеть в окне Model Explorer.

## Лабораторная работа № 4. Диаграмма IDEF3

### Цель работы:

- описание взаимосвязей между процессами при помощи диаграмм IDEF3 (WorkFlow).

Для функциональной модели разработки плана привлечения и размещения ресурсов банка и ее связей с внешним миром (с.27-29) построим диаграммы процессов. На рисунках 1.4.1–1.4.3 показаны основные этапы.

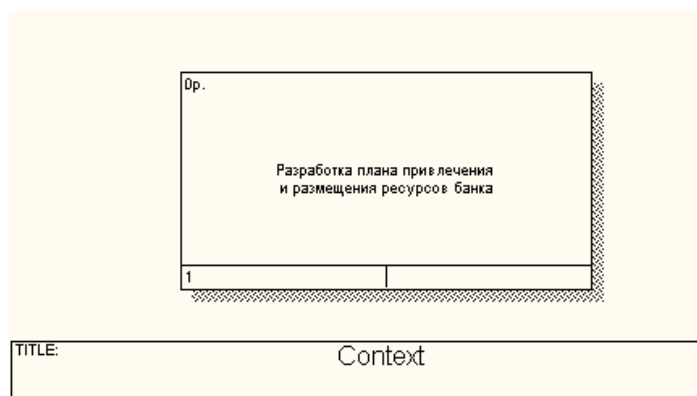


Рис. 1.4.1. Контекстная диаграмма сценария

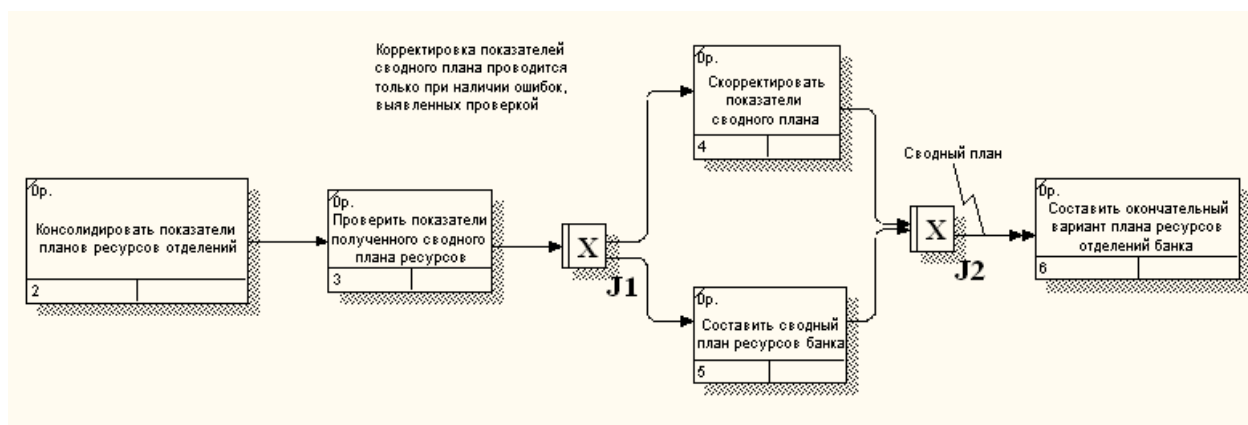


Рис. 1.4.2. Первый уровень декомпозиции

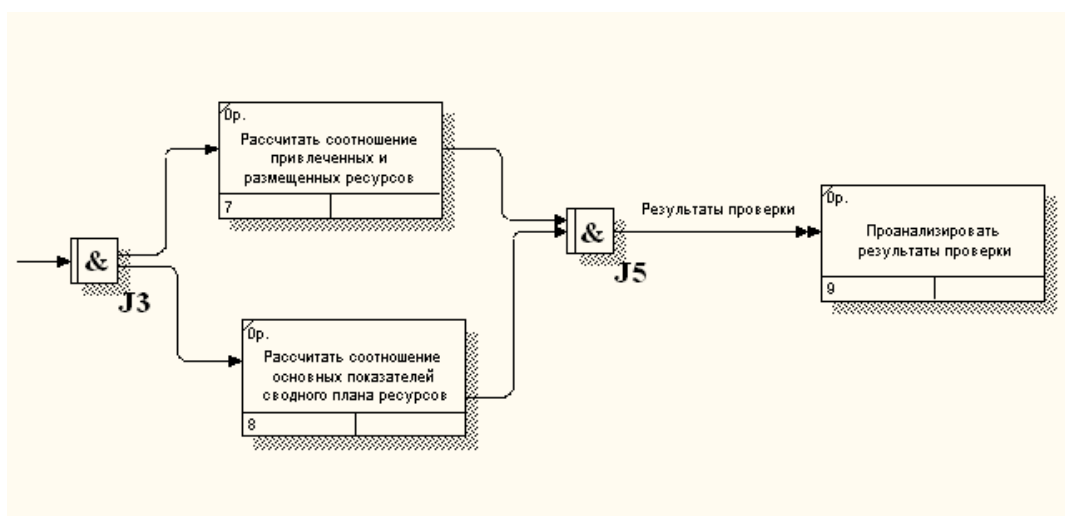


Рис. 1.4.3. Второй уровень декомпозиции

### **1.4.2 Задания**

1. Дополнить созданные на предыдущих работах диаграммы IDEF0 и DFD диаграммой IDEF3.
2. Связать работы на диаграмме.
3. Добавить на диаграмму перекрестки, моделирующие параллельные события.
4. Добавить объект-ссылку и связать его с диаграммой.

### **1.4.3 Контрольные вопросы**

1. Что описывает диаграмма IDEF3?
2. Перечислите составные элементы диаграмм IDEF3.
3. Что показывают связи в диаграммах IDEF3?
4. Перечислите типы стрелок в диаграммах IDEF3.
5. Что называется перекрестком?
6. Назовите типы перекрестков.
7. Что называется объектом-ссылкой?
8. Какие бывают типы объектов-ссылок?
9. Как добавить объект-ссылку?

## ГЛАВА 2 ОБЪЕКТНО-ОРИЕНТИРОВАННОЕ ПРОЕКТИРОВАНИЕ

### 2.1 Объектно-ориентированный подход

Разделение по алгоритмам в структурном подходе концентрирует внимание на порядке происходящих событий, а разделение по объектам в объектно-ориентированном подходе придает особое значение агентам, которые являются либо объектами, либо субъектами действия. Однако эти способы, по сути, ортогональны, поэтому нельзя сконструировать сложную систему одновременно двумя способами. Необходимо начать разделение системы либо по алгоритмам, либо по объектам, а затем, используя полученную структуру, попытаться рассмотреть проблему с другой точки зрения.

При объектно-ориентированной декомпозиции ([1-5], [7-12], [15-16], [19-21], [23-27]) каждый объект обладает своим собственным поведением и каждый из них моделирует некоторый объект реального мира. С этой точки зрения объект является вполне осязаемой вещью, которая демонстрирует вполне определенное поведение. Объекты что-то делают, и мы можем, пошлав им сообщение, попросить их выполнить некоторые операции.

Объектная декомпозиция имеет несколько преимуществ перед алгоритмической:

- объектная декомпозиция уменьшает размер программных систем за счет повторного использования общих механизмов;
  - объектно-ориентированные системы более гибки и проще эволюционируют со временем, потому что их схемы базируются на устойчивых промежуточных формах;
- объектная декомпозиция помогает нам разобраться в сложной программной системе, предлагая нам разумные решения относительно выбора подпространства большого пространства состояний.

В объектно-ориентированном анализе существует четыре основных типа моделей: динамическая, статическая, логическая и физическая. Через них можно выразить результаты анализа и проектирования, выполненные в рамках любого проекта. Эти модели в совокупности семантически достаточно богаты и универсальны, чтобы разработчик мог выразить все заслуживающие внимания стратегические и тактические решения, которые он должен принять при анализе системы и формировании ее архитектуры. Кроме того, эти модели достаточно полны, чтобы служить техническим проектом реализации практически на любом объектно-ориентированном языке программирования.

UML представляет собой объектно-ориентированный язык моделирования, обладающий следующими основными характеристиками:

- является языком визуального моделирования, который обеспечивает разработку репрезентативных моделей для организации взаимодействия заказчика и разработчика ИС, различных групп разработчиков ИС;

- содержит механизмы расширения и специализации базовых концепций языка.

Пользователям языка UML предоставлены возможности:

- строить модели на основе средств ядра, без использования механизмов расширения для большинства типовых приложений;
- добавлять при необходимости новые элементы и условные обозначения, если они не входят в ядро, или специализировать компоненты, систему условных обозначений (нотацию) и ограничения для конкретных предметных областей.

Ниже приводятся определения, и кратко описывается назначение стандартных диаграмм UML и моделей применительно к задачам проектирования ИС (в скобках приведены альтернативные названия диаграмм, используемые в современной литературе).

*Диаграммы прецедентов* (диаграммы вариантов использования, use-case diagrams) – это обобщенная модель функционирования системы в окружающей среде.

*Диаграммы классов* (class diagrams) – логическая модель базовой структуры системы, отражает статическую структуру системы и связи между ее элементами.

*Диаграммы взаимодействия* (interaction diagrams) – модель процесса обмена сообщениями между объектами, представляется в виде диаграмм последовательностей (sequence diagrams) или диаграмм кооперации (collaboration diagrams).

*Диаграммы состояний* (statechart diagrams) – модель динамического поведения системы и ее компонентов при переходе из одного состояния в другое.

*Диаграммы видов деятельности* (диаграммы деятельности, activity diagrams) – другая разновидность диаграммы состояний описывает модель бизнес-процесса или поведение системы в рамках прецедента.

*Диаграммы компонентов* (component diagrams) – модель иерархии подсистем, отражает физическое размещение баз данных, приложений и интерфейсов ИС.

*Диаграммы развертывания* (диаграммы размещения, deployment diagrams) – модель физической архитектуры системы, отображает аппаратную конфигурацию ИС.

### 2.1.1 Лабораторная работа № 5. Интерфейс IBM Rational Rose

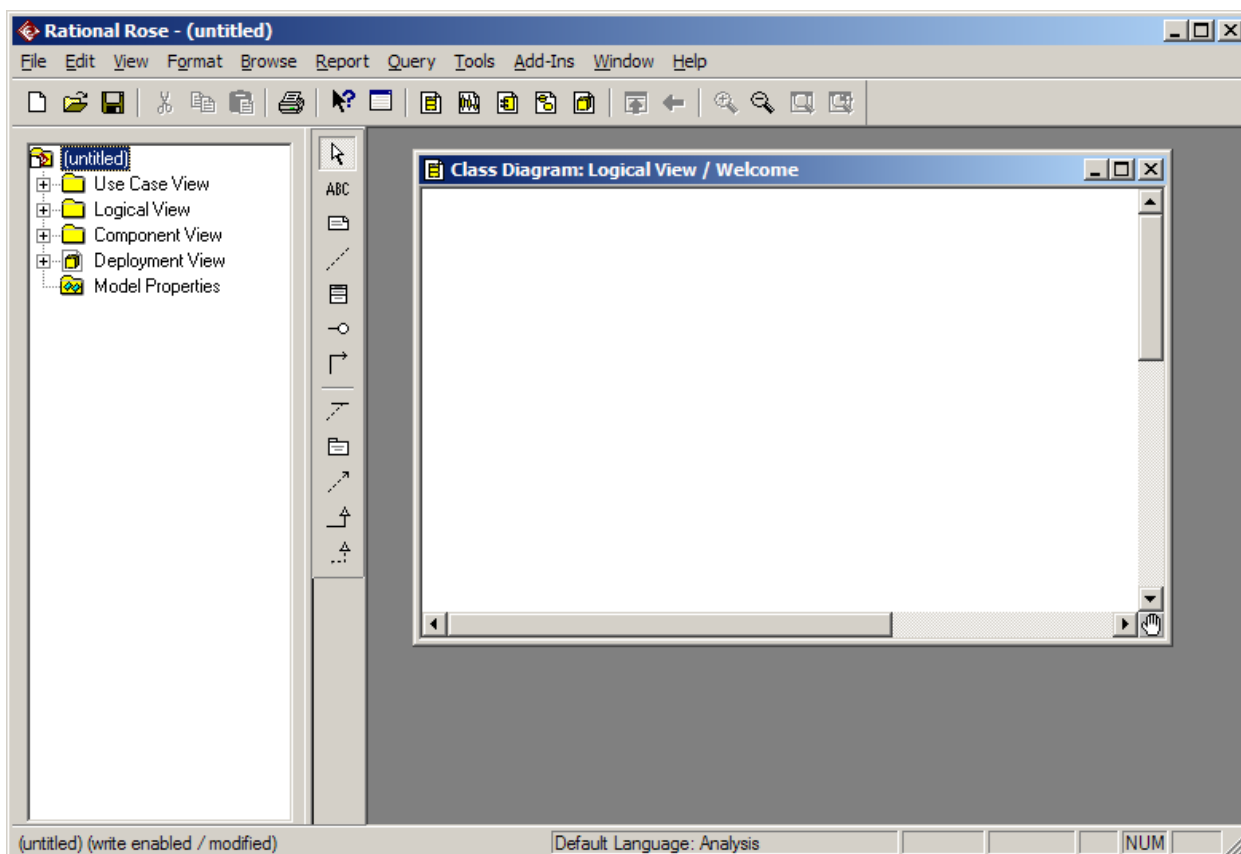


Рис. 2.1.1. «Стандартное» окно системы Rational Rose.

Левая панель диалогового окна является браузером (2.1.1). На правой панели размещаются окна диаграмм. Между этими двумя панелями расположена панель инструментов с пиктограммами для создания диаграммы классов. Этот тип диаграмм предлагается по умолчанию.

Данные, содержащиеся на левой панели, организованы в соответствии с различными представлениями: представлением прецедентов, логическим представлением, а также представлениями компонентов и развертывания. Для того, чтобы создать новую диаграмму, щелкните правой кнопкой мыши на одном из представлений и выберите требуемый тип диаграммы из появившегося контекстного меню. После этого на экране появится новое окно. При этом на панели инструментов будут содержаться пиктограммы, предназначенные для создания диаграмм именно выбранного типа. На рис. 2.1.2 показано окно для создания диаграммы прецедентов и соответствующая панель инструментов.

Если нужно пометить атрибуты и операции класса как открытые, закрытые и защищенные, то рядом с ними можно разместить соответствующие пиктограммы, повышающие ясность диаграммы (Рис. 2.1.3).

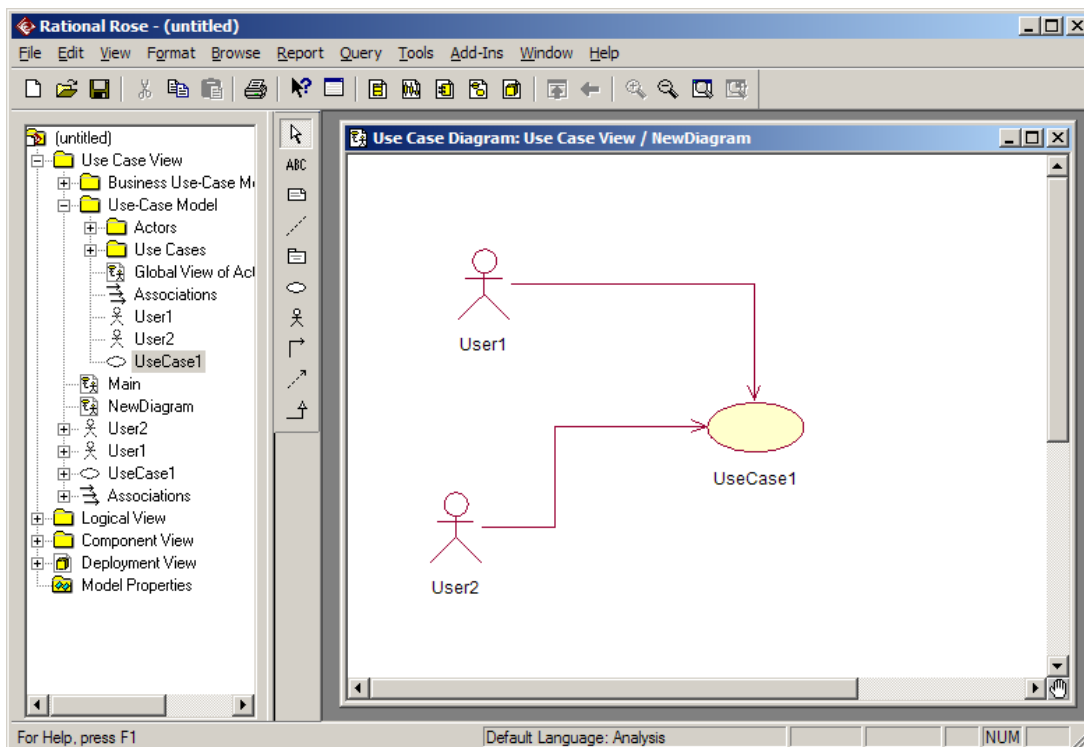


Рис. 2.1.2. Окно для создания диаграммы прецедентов и связанная с ней панель инструментов в Rational Rose.

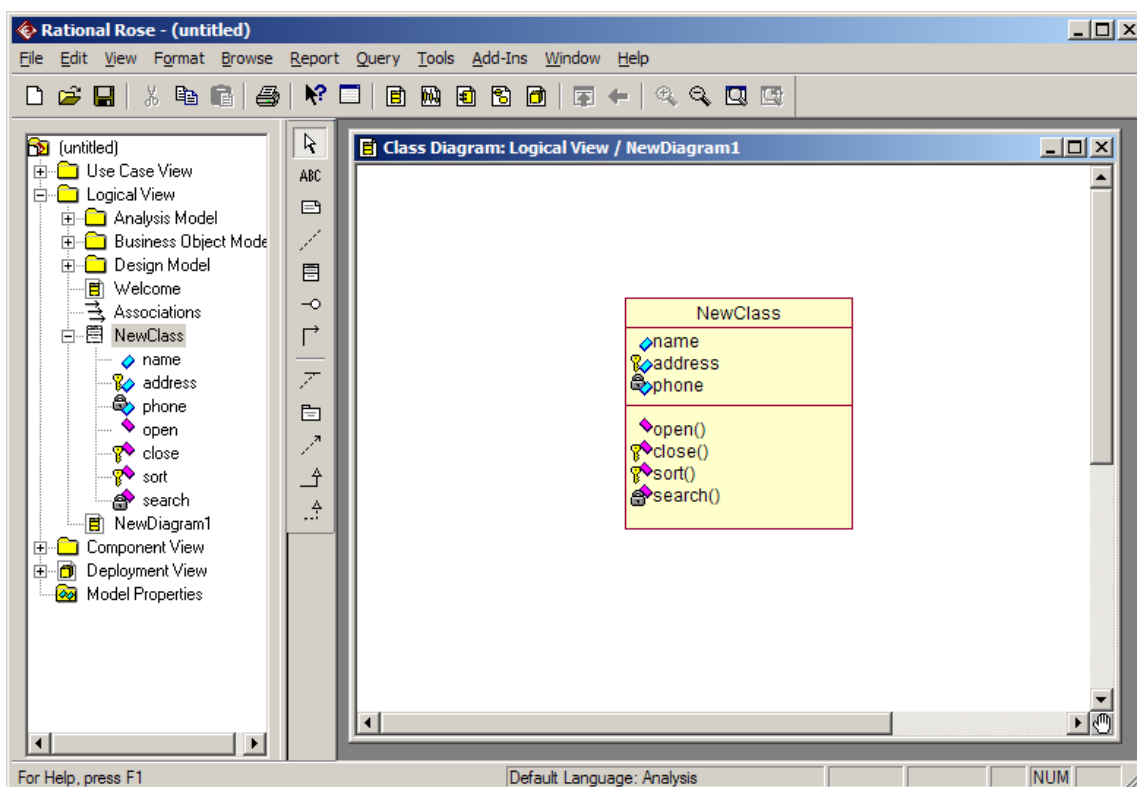


Рис. 2.1.3. В Rational Rose имеются специальные пиктограммы для членов класса.

### Операции главного меню

Операции главного меню **File (Файл)** (Табл.7) позволяют создавать новые модели в нотации языка UML, загружать и сохранять разрабатываемую модель во внешнем файле, распечатывать на принтере разработанные диаграммы.

## Операции пункта главного меню File (Файл)

| Название операции меню  | Назначение операции главного меню                                                                                                                                                                                                                                                               |
|-------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| New                     | Создает новую модель IBM Rational Rose. При этом новая модель по умолчанию имеет имя untitled                                                                                                                                                                                                   |
| Open                    | Вызывает стандартное диалоговое окно открытия внешнего файла с диска. Открыть можно либо файл модели (файл с расширением «mdl»), либо файл подмодели (файл с расширением «ptl»)                                                                                                                 |
| Save                    | Позволяет сохранить разрабатываемую модель во внешнем файле на диске                                                                                                                                                                                                                            |
| Save As                 | Позволяет сохранить разрабатываемую модель под другим именем во внешнем файле на диске. При этом вызывается стандартное диалоговое окно сохранения файла на диске с предложением задать имя соответствующего файла модели или подмодели.                                                        |
| Save Log As             | Позволяет сохранить содержание журнала во внешнем файле на диске с именем error.log. При этом вызывается стандартное диалоговое окно сохранения файла на диске с предложением изменить предлагаемое по умолчанию имя соответствующего файла.                                                    |
| AutoSave Log            | Позволяет автоматически сохранять содержание журнала во внешнем файле на диске с именем error.log. При первом выполнении этого пункта меню также вызывается стандартное диалоговое окно сохранения файла на диске с предложением изменить предлагаемое по умолчанию имя соответствующего файла. |
| Clear Log               | Очищает содержание журнала                                                                                                                                                                                                                                                                      |
| Load Model Workspace    | Позволяет загрузить рабочую область из внешнего файла на диске. Вызывает стандартное диалоговое окно открытия внешнего файла с диска, при этом открыть можно файл с расширением «wsp»                                                                                                           |
| Save Model Workspace    | Позволяет сохранить рабочую область модели во внешнем файле на диске. При выполнении этого пункта меню вызывается стандартное диалоговое окно сохранения файла с расширением «wsp»                                                                                                              |
| Save Model Workspace As | Позволяет сохранить рабочую область модели во внешнем файле на диске. Вызывается стандартное диалоговое окно сохранения файла с предложением изменить предлагаемое по умолчанию имя соответствующего файла                                                                                      |
| Units                   | Позволяет загрузить категорию элементов модели из внешнего файла на диске. Вызывает стандартное диалоговое окно открытия внешнего файла с диска, при этом открыть можно файл с расширением «cat»                                                                                                |
| Import                  | Позволяет импортировать информацию из внешних файлов различных форматов, включая файлы моделей, подмоделей, категорий и подсистем                                                                                                                                                               |
| Export Model            | Позволяет экспортировать информацию о модели во внешний файл. Вид этого пункта меню зависит от выделенного элемента модели                                                                                                                                                                      |
| Update                  | Позволяет вставить информацию обратного проектирования из внешнего файла с расширением «red» в разрабатываемую модель                                                                                                                                                                           |
| Print                   | Позволяет распечатать на принтере отдельные диаграммы и спецификации различных элементов разрабатываемой модели. В этом случае вызывается диалоговое окно выбора диаграмм и спецификаций для печати на подключенном к данному компьютеру принтеру                                               |
| Print Setup             | Вызывается стандартное диалоговое окно макета страницы для настройки свойств печати                                                                                                                                                                                                             |
| Edit Path Map           | Вызывает окно задания путей доступа к файлам системы IBM Rational Rose. Как правило, значения путей, установленные по умолчанию, следует изменять только в случае крайней необходимости                                                                                                         |
| <Имена файлов>          | Секция с именами последних файлов, с которыми осуществлялась работа в IBM Rational Rose                                                                                                                                                                                                         |
| Exit                    | Прекращает работу и закрывает IBM Rational Rose.                                                                                                                                                                                                                                                |

Операции главного меню **Edit (Редактирование)** (Табл.8) позволяют выполнять действия по редактированию элементов модели и их свойств, а также выполнять поиск элементов в рамках разрабатываемого проекта.

Таблица 8.

## Операции пункта главного меню Edit (Редактирование)

| Название операции меню | Назначение операции главного меню                                   |
|------------------------|---------------------------------------------------------------------|
| Undo                   | Отменяет выполнение последнего действия по удалению или перемещению |

|                   |                                                                                                             |
|-------------------|-------------------------------------------------------------------------------------------------------------|
|                   | элементов модели                                                                                            |
| Redo              | Восстанавливает изображение диаграммы после отмены выполнения последней операции перемещения                |
| Cut               | Вырезает выделенный элемент разрабатываемой модели и помещает его в буфер обмена                            |
| Copy              | Копирует выделенный элемент разрабатываемой модели и помещает его в буфер обмена                            |
| Paste             | Вставляет элемент разрабатываемой модели или его копию из буфера обмена в текущую активную диаграмму        |
| Delete            | Удаляет выделенные элементы из текущей диаграммы, но не из разрабатываемой модели                           |
| Select All        | Выделяет все элементы на текущей диаграмме разрабатываемой модели                                           |
| Delete from Model | Удаляет все выделенные элементы из разрабатываемой модели                                                   |
| Relocate          | Позволяет перемещать или отменять перемещение классов, ассоциаций или компонентов из одного пакета в другой |
| Find              | Вызывает диалоговое меню поиска элемента в разрабатываемой модели по его имени                              |
| Reassign          | Позволяет заменить выделенный элемент разрабатываемой модели другим элементом модели                        |
| Compartment       | Позволяет отображать дополнительную информацию об объектах, классах, актерах или пакетах                    |
| Change Info       | Позволяет изменить тип выделенного элемента на текущей диаграмме на другой тип элемента                     |

Операции главного меню **View (Вид)** (табл. 9) позволяют отображать на экране различные элементы рабочего интерфейса и изменять графическое представление диаграмм.

Таблица 9.

#### Операции пункта главного меню View (Вид)

| Название операции меню | Назначение операции главного меню                                                                                                                                                                                                                                                                                                                                                                                                                       |
|------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Toolbars               | Позволяет настроить внешний вид рабочего интерфейса системы IBM Rational Rose 2003 и содержит дополнительные подпункты:<br><i>Standard</i> - делает видимой/невидимой стандартную панель инструментов (рис. 1.3)<br><i>Toolbox</i> - делает видимой/невидимой стандартную панель инструментов текущей активной диаграммы<br><i>Configure</i> - вызывает диалоговое окно настройки параметров модели, открытое на вкладке настройки панелей инструментов |
| Status Bar             | Делает видимой/невидимой строку состояния                                                                                                                                                                                                                                                                                                                                                                                                               |
| Documentation          | Делает видимым/невидимым окно документации                                                                                                                                                                                                                                                                                                                                                                                                              |
| Browser                | Делает видимым/невидимым браузер проекта                                                                                                                                                                                                                                                                                                                                                                                                                |
| Log                    | Делает видимым/невидимым окно журнала                                                                                                                                                                                                                                                                                                                                                                                                                   |
| Editor                 | Делает видимым/невидимым встроенный текстовый редактор                                                                                                                                                                                                                                                                                                                                                                                                  |
| Time Stamp             | Включает/выключает режим отображения времени в записях журнала                                                                                                                                                                                                                                                                                                                                                                                          |
| Zoom to Selection      | Изменяет масштаб изображения выделенных элементов модели, так чтобы они разместились в одном окне                                                                                                                                                                                                                                                                                                                                                       |
| Zoom In                | Увеличивает масштаб изображения                                                                                                                                                                                                                                                                                                                                                                                                                         |
| Zoom Out               | Уменьшает масштаб изображения                                                                                                                                                                                                                                                                                                                                                                                                                           |
| Fit in Window          | Изменяет (уменьшает) масштаб изображения всех элементов текущей диаграммы, так чтобы все они разместились в одном окне                                                                                                                                                                                                                                                                                                                                  |
| Undo Fit in Window     | Отменяет изменение масштаба изображения размещения элементов в одном окне                                                                                                                                                                                                                                                                                                                                                                               |
| Page Breaks            | Разбивает текущую диаграмму на страницы для последующей печати                                                                                                                                                                                                                                                                                                                                                                                          |
| Refresh                | Перерисовывает текущую диаграмму                                                                                                                                                                                                                                                                                                                                                                                                                        |
| As Booch               | Изображает элементы модели в соответствии с нотацией Г. Буча                                                                                                                                                                                                                                                                                                                                                                                            |
| As OMT                 | Изображает элементы модели в соответствии с нотацией OMT                                                                                                                                                                                                                                                                                                                                                                                                |
| As Unified             | Изображает элементы модели в соответствии с нотацией языка UML                                                                                                                                                                                                                                                                                                                                                                                          |

Операции главного меню **Format (Формат)** (Табл.10) позволяют выполнять действия по изменению внешнего вида элементов модели на различных диаграммах.

Таблица 10.

## Операции пункта главного меню Format (Формат)

| Название операции меню       | Назначение операции главного меню                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Font Size                    | Изменяет масштаб используемого шрифта                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| Font                         | Вызывает диалоговое окно выбора шрифта                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| Line Color                   | Вызывает диалоговое окно выбора цвета линий                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| Fill Color                   | Вызывает диалоговое окно выбора цвета для изображения графических элементов диаграмм                                                                                                                                                                                                                                                                                                                                                                                                             |
| Use Fill Color               | Включает/выключает режим отображения цвета для изображения графических элементов диаграмм                                                                                                                                                                                                                                                                                                                                                                                                        |
| Automatic Resize             | Включает/выключает режим автоматического изменения размеров графических элементов диаграмм для отображения текстовой информации об их свойствах                                                                                                                                                                                                                                                                                                                                                  |
| Stereotype Display           | Позволяет выбрать способ изображения стереотипов выделенных элементов диаграммы и содержит дополнительные подпункты:<br><i>None</i> - стереотип не показывается;<br><i>Label</i> - стереотип отображается в форме текста;<br><i>Decoration</i> - стереотип отображается в форме небольшой пиктограммы в правом верхнем углу графического элемента;<br><i>Icon</i> - элемент диаграммы отображается в форме специального графического стереотипа, если данный стереотип предусмотрен в программе. |
| Stereotype Label             | Включает/выключает режим отображения текстовых стереотипов для взаимосвязей (ассоциаций, зависимостей и пр.) диаграммы                                                                                                                                                                                                                                                                                                                                                                           |
| Show Visibility              | Включает/выключает режим отображения кванторов видимости атрибутов и операций выделенных классов                                                                                                                                                                                                                                                                                                                                                                                                 |
| Show Compartment Stereotypes | Включает/выключает режим отображения текстовых стереотипов атрибутов и операций выделенных классов                                                                                                                                                                                                                                                                                                                                                                                               |
| Show Operation Signature     | Включает/выключает режим отображения сигнатуры операций выделенных классов                                                                                                                                                                                                                                                                                                                                                                                                                       |
| Show All Attributes          | Делает видимыми/невидимыми атрибуты выделенных классов                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| Show All Operations          | Делает видимыми/невидимыми операции выделенных классов                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| Suppress Attributes          | Делает видимой/невидимой секцию атрибутов выделенных классов. Скрывает секцию атрибутов даже в том случае, когда выбрана опция Show All Attributes                                                                                                                                                                                                                                                                                                                                               |
| Suppress Operations          | Делает видимой/невидимой секцию операций выделенных классов. Скрывает секцию операций даже в том случае, когда выбрана опция Show All Operations                                                                                                                                                                                                                                                                                                                                                 |
| Line Style                   | Позволяет выбрать способ графического изображения линий взаимосвязей и содержит дополнительные подпункты:<br><i>Rectilinear</i> - линия изображается в форме вертикальных и горизонтальных отрезков;<br><i>Oblique</i> - линия изображается в форме наклонных отрезков;<br><i>Toggle</i> - промежуточный вариант изображения линии.                                                                                                                                                              |
| Layout Diagram               | Позволяет автоматически разместить графические элементы в окне диаграммы с минимальным количеством пересечений и наложений соединительных линий                                                                                                                                                                                                                                                                                                                                                  |
| Autosize All                 | Позволяет автоматически изменить размеры графических элементов текущей диаграммы таким образом, чтобы текстовая информация помещалась внутри изображений соответствующих элементов                                                                                                                                                                                                                                                                                                               |
| Layout Selected Shapes       | Позволяет автоматически разместить выделенные графические элементы в окне диаграммы с минимальным количеством пересечений и наложений соединительных линий                                                                                                                                                                                                                                                                                                                                       |

Операции главного меню **Browse (Обзор)** (Табл.11) позволяют отображать рабочие окна с различными каноническими диаграммами разрабатываемой модели и вызывать диалоговые окна редактирования свойств отдельных элементов модели.

Таблица 11.

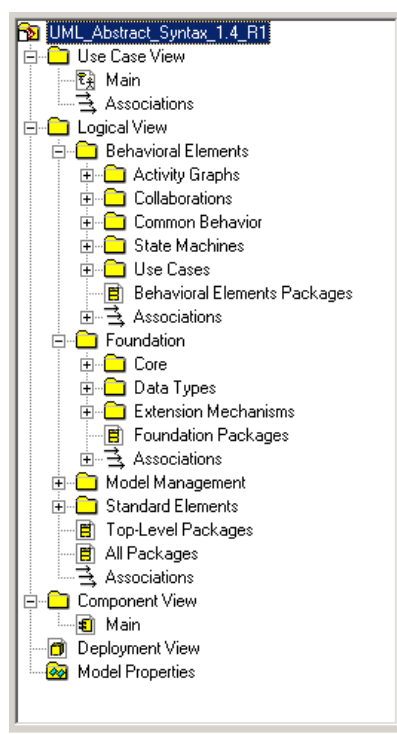
## Операции пункта главного меню Browse (Обзор)

| Название операции меню | Назначение операции главного меню                                                                                                                                                       |
|------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Use Case Diagram       | Вызывает диалоговое окно с предложением выбрать для отображения в рабочем окне одну из существующих диаграмм вариантов использования модели или приступить к разработке новой диаграммы |
| Class Diagram          | Вызывает диалоговое окно с предложением выбрать для отображения                                                                                                                         |

|                              |                                                                                                                                                                                                                |
|------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                              | в рабочем окне одну из существующих диаграмм классов модели или приступить к разработке новой диаграммы                                                                                                        |
| Component Diagram            | Вызывает диалоговое окно с предложением выбрать для отображения в рабочем окне одну из существующих диаграмм компонентов модели или приступить к разработке новой диаграммы                                    |
| Deployment Diagram           | Позволяет отобразить в рабочем окне диаграмму развертывания разрабатываемой модели                                                                                                                             |
| Interaction Diagram          | Вызывает диалоговое окно с предложением выбрать для отображения в рабочем окне одну из существующих диаграмм кооперации или последовательности, а также приступить к разработке новой диаграммы взаимодействия |
| State Machine Diagram        | Вызывает диалоговое окно с предложением выбрать для отображения в рабочем окне одну из существующих диаграмм состояний модели или приступить к разработке новой диаграммы                                      |
| Expand                       | Отображает в рабочем окне первую из диаграмм выделенного пакета модели                                                                                                                                         |
| Parent                       | Отображает в рабочем окне родителя выделенной диаграммы модели                                                                                                                                                 |
| Specification                | Вызывает диалоговое окно свойств выделенного элемента модели                                                                                                                                                   |
| Top Level                    | Отображает в рабочем окне диаграмму самого верхнего уровня для текущей диаграммы модели                                                                                                                        |
| Referenced Item              | Отображает в рабочем окне диаграмму классов, содержащую класс для выделенного объекта модели                                                                                                                   |
| Previous Diagram             | Отображает в рабочем окне предыдущую диаграмму модели                                                                                                                                                          |
| Create Message Trace Diagram | Позволяет создать диаграмму трассировки сообщений                                                                                                                                                              |

### Окно браузера проекта

Окно браузера проекта по умолчанию располагается в левой части рабочего интерфейса ниже стандартной панели инструментов и имеет вид, показанный на рис. 2.1.4.



**Рис. 2.1.4** Внешний вид браузера проекта с иерархическим представлением его структуры

Браузер проекта организует представления модели в виде иерархической структуры, которая упрощает навигацию и позволяет отыскать любой элемент модели в проекте. При этом самая верхняя строка браузера проекта содержит имя разрабатываемого проекта. Любой элемент, который разработчик добавляет в модель, сразу отображается в окне

браузера. Соответственно, выбрав элемент в окне браузера, мы можем его визуализировать в окне диаграммы или изменить его спецификацию.

Браузер проекта позволяет также организовывать элементы модели в пакеты и перемещать элементы между различными представлениями модели. При желании окно браузера можно расположить в другом месте рабочего интерфейса либо скрыть вовсе, используя для этого операцию **Browser** главного меню **View** (Вид). Можно также изменить размеры браузера, переместив мышью границу его внешней рамки.

Иерархическое представление структуры каждого разрабатываемого проекта организовано в форме 4-х представлений:

*Use Case View* - представление вариантов использования, в котором содержатся диаграммы вариантов использования и их реализации в виде вариантов взаимодействия;

*Logical View* - логическое представление, в котором содержатся диаграммы классов, диаграммы состояний и диаграммы деятельности;

*Component View* - представление компонентов, в котором содержатся диаграммы компонентов разрабатываемой модели;

*Deployment View* - представление развертывания, в котором содержится единственная диаграмма развертывания разрабатываемой модели.

При создании нового проекта указанная иерархическая структура формируется программой автоматически.

### Специальная панель инструментов и окно диаграммы

Специальная панель инструментов располагается между окном браузера и окном диаграммы в средней части рабочего интерфейса. По умолчанию предлагается панель инструментов для построения диаграммы классов модели (рис. 2.1.5).

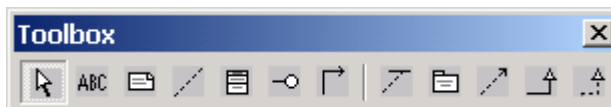


Рис. 2.1.5. Внешний вид специальной панели инструментов для диаграммы классов

Расположение специальной панели инструментов можно изменять, переместив рамку панели в нужное место. Программа IBM Rational Rose позволяет настраивать состав кнопок данной панели, добавляя или удаляя отдельные кнопки, соответствующие тем или иным инструментам. Назначение отдельных кнопок различных панелей инструментов будут рассмотрены далее при изложении особенностей построения различных канонических диаграмм. Названия кнопок данной панели всегда можно узнать из всплывающих подсказок, появляющихся после задержки указателя мыши над соответствующей кнопкой.

Внешний вид специальной панели инструментов зависит не только от выбора типа разрабатываемой диаграммы, но от выбора графической нотации для изображения самих элементов этих диаграмм. В IBM Rational Rose реализованы три таких нотации: UML, OMT и Booch. Хотя последние две нотации практически не используются на практике, разработчики неизменно включают их в очередные версии программы IBM Rational Rose, видимо, отдавая этим дань памяти авторам первых версий языка UML. При использовании отдельной нотации одна и та же диаграмма может быть представлена различным образом, для этого достаточно выбрать желаемое представление через соответствующую операцию главного меню **View** (Вид). При этом никаких дополнительных действий выполнять не требуется - диаграмма преобразуется в выбранную нотацию автоматически. Однако, рассматривая использование программы IBM Rational Rose только в контексте языка UML, можно оставить без внимания особенности двух других нотаций, которые отражают эволюционный аспект этого программного инструмента.

Окно диаграммы является основной графической областью программы IBM Rational Rose, в которой визуализируются различные представления модели проекта. По умолчанию окно

диаграммы располагается в правой части рабочего интерфейса, однако его расположение и размеры также можно изменить. При разработке нового проекта, если не был использован мастер проектов, окно диаграммы представляет собой чистую область, не содержащую никаких элементов модели. По мере разработки отдельных диаграмм в окне диаграммы будут располагаться соответствующие графические элементы модели.

Название диаграммы, которая является активной и располагается в данном окне, которое развернуто на всю область диаграммы, указывается в строке заголовка программы IBM Rational Rose. Если же окно диаграммы не развернуто на всю область диаграммы, то название диаграммы указывается в строке заголовка окна диаграммы. Одновременно в графической области диаграмм могут присутствовать несколько окон диаграмм; при этом активной может быть только одна из них. Переключение между диаграммами можно осуществить выбором нужного представления на стандартной панели инструментов, а также с помощью выделения требуемой диаграммы в браузере проекта или с помощью операций главного меню **Window** (Окно). При активизации отдельного вида диаграммы изменяется внешний вид специальной панели инструментов, которая настраивается под конкретный вид диаграммы.

### Окно документации и окно журнала

Окно документации по умолчанию должно присутствовать на экране после загрузки программы. Если по какой-то причине оно отсутствует, то его можно отобразить через пункт меню **View** → **Documentation** (Вид → Документация), после чего окно документации появится ниже окна браузера проекта (рис. 2.1.6). Окно документации, как следует из его названия, предназначено для документирования элементов разрабатываемой модели. В него можно записывать различную текстовую информацию, и что важно - на русском языке. Эта информация при генерации программного кода преобразуется в комментарии и никак не влияет на логику выполнения программного кода.

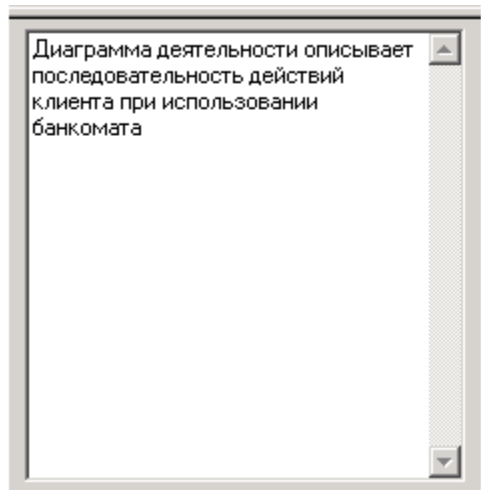


Рис. 2.1.6. Внешний вид окна документации с информацией

В окне документации активизируется та информация, которая относится к выделенному элементу диаграммы или к диаграмме в целом. При этом выделить элемент можно либо в окне браузера, либо непосредственно в окне диаграммы. При добавлении нового элемента на диаграмму, например, класса, документация к нему является пустой (No documentation). В последующем разработчик самостоятельно вносит необходимую пояснительную информацию, которая запоминается программой и может быть изменена в ходе работы над проектом. Так же, как и для других окон рабочего интерфейса программы IBM Rational Rose, разработчик по своему усмотрению может изменять размеры и положение окна документации.

Окно журнала (Log) предназначено для автоматической записи различной служебной информации в ходе работы с программой. В журнале фиксируется время и характер выполняемых разработчиком действий, таких как обновление модели, настройка меню и панелей инструментов, а также сообщений об ошибках, возникающих при генерации программного кода. Окно журнала изображается поверх других окон в нижней области рабочего интерфейса программы (рис. 2.1.7).

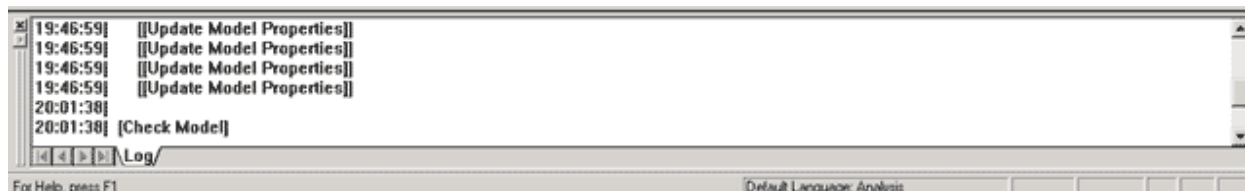


Рис. 2.1.7. Внешний вид окна журнала с информацией о выполненных операциях с моделью

Если окно журнала отсутствует на экране, то отобразить его можно с помощью операции главного меню **View→Log** (Вид→Журнал), для чего следует выставить отметку в соответствующей строке вложенного меню для данной операции. С целью увеличения размеров графической области диаграммы окно журнала чаще всего убирают с экрана, что можно выполнить с помощью кнопки закрытия этого окна в верхнем левом его углу или убрав отметку в соответствующей строке вложенного меню **View→Log**.

### Назначение операций главного меню Report, Query и Tools

Операции главного меню **Report (Отчет)** (Табл.12) позволяют отображать различную информацию об элементах разрабатываемой модели и вызывать диалоговое окно выбора шаблона для генерации отчета о модели.

Таблица 12.

Операции пункта главного меню Report (Отчет)

| Название операции меню  | Назначение операции главного меню                                                                                                                                                             |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Show Usage              | Отображает в диалоговом окне информацию об использовании выделенного элемента модели на различных диаграммах                                                                                  |
| Show Instances          | Отображает в диалоговом окне информацию об использовании объектов выделенного класса модели на различных диаграммах                                                                           |
| Show Access Violations  | Отображает в диалоговом окне информацию о ссылках классов одного пакета на классы другого пакета при отсутствии соответствующей зависимости доступа или импорта между этими пакетами в модели |
| SoDA Report             | Позволяет сгенерировать отчет о разрабатываемой модели в формате MS Word с использованием специального средства IBM Rational SoDA                                                             |
| Show Participants in UC | Отображает в диалоговом окне информацию о классах, компонентах и операциях, которые участвуют в реализации выделенного прецедента модели на различных диаграммах                              |

Операции главного меню **Query (Запрос)** (Табл.13) позволяют добавлять существующие элементы разрабатываемой модели на редактируемую диаграмму, а также настраивать специальный фильтр отображения отношений между отдельными элементами модели.

Таблица 13.

Операции пункта главного меню Query (Запрос)

| Название операции меню   | Назначение операции главного меню                                                                                                                  |
|--------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| Add Classes              | Вызывает диалоговое окно с предложением добавить на текущую диаграмму классы, которые имеются в модели на различных диаграммах                     |
| Add Use Cases            | Вызывает диалоговое окно с предложением добавить на текущую диаграмму прецеденты, которые имеются в модели на различных уровнях                    |
| Expand Selected Elements | Вызывает диалоговое окно с предложением добавить на текущую диаграмму элементы модели, которые связаны с выделенным элементом на других диаграммах |
| Hide Selected Elements   | Вызывает диалоговое окно с предложением удалить с текущей                                                                                          |

|                      |                                                                                                                     |
|----------------------|---------------------------------------------------------------------------------------------------------------------|
|                      | диаграммы элементы модели, которые связаны с выделенным элементом                                                   |
| Filter Relationships | Вызывает диалоговое окно, позволяющее включить/выключить режим отображения различных отношений на текущей диаграмме |

Состав операций пункта главного меню **Tools (Инструменты)** зависит от установленных в программе IBM Rational Rose конкретных расширений. Назначение операций этого пункта главного меню для типовой конфигурации программы представлено в таблице 14.

Таблица 14.

Операции пункта главного меню Tools (Инструменты)

| Название операции меню | Назначение операции главного меню                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Create                 | Создает новый элемент модели из предлагаемого списка, для последующего размещения его на текущей или другой диаграмме, дублируя нажатие соответствующей кнопки на специальной панели инструментов                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| Check Model            | Проверяет разрабатываемую модель на наличие ошибок, информация о которых отображается в окне журнала                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| Model Properties       | Позволяет выполнить настройку свойств языка реализации для выделенного элемента модели и содержит дополнительные подпункты:<br><i>Edit</i> - редактирование набора свойств;<br><i>View</i> - просмотр набора свойств;<br><i>Replace</i> - замена существующего набора свойств на новый набор свойств, загружаемый из внешнего файла с расширением "prp" или "pty";<br><i>Export</i> - сохранение существующего набора свойств во внешнем файле с расширением "prp" или "pty";<br><i>Add</i> - добавление к существующему набору свойств нового набора свойств, загружаемого из внешнего файла с расширением "prp" или "pty";<br><i>Update</i> - обновление существующего набора свойств после его редактирования или дополнения |
| Options                | Вызывает диалоговое окно настройки параметров модели, открытое на вкладке General                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| Open Script            | Вызывает стандартное диалоговое окно для открытия внешнего файла, содержащего текст скрипта (файл с расширением "ebs") для его редактирования в окне встроенного редактора скриптов                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| New Script             | Открывает дополнительное окно встроенного редактора скриптов для создания, отладки, выполнения и сохранения нового скрипта во внешнем файле с расширением "ebs"                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| ANSI C++               | Позволяет выполнить настройку свойств языка программирования ANSI C++, выбранного в качестве языка реализации отдельных элементов модели                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| CORBA                  | Позволяет выполнить настройку свойств и спецификацию модели для генерации объектов CORBA для реализации отдельных элементов модели                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| Java/J2EE              | Позволяет выполнить настройку свойств языка программирования Java/J2EE, выбранного в качестве языка реализации отдельных элементов модели                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| Oracle8                | Позволяет выполнить настройку свойств и спецификацию модели для генерации схем СУБД Oracle8 для отдельных элементов модели                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| Model Integrator       | Открывает окно специального средства интеграции моделей                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| Web Publisher          | Позволяет выполнить настройку свойств модели для ее публикации в гипертекстовом формате                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| TOPLink                | Вызывает мастер преобразования таблиц модели данных в классы языка программирования Java, выбранного в качестве языка реализации отдельных элементов модели                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| COM                    | Позволяет выполнить настройку свойств и спецификацию модели для генерации объектов COM с целью реализации отдельных элементов модели                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| Visual C++             | Позволяет выполнить настройку свойств и спецификацию модели для генерации программного кода MS Visual C++, выбранного в качестве языка реализации отдельных элементов модели                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| Version Control        | Позволяет выполнить настройку свойств модели для установления со специальным средством управления и контроля версий модели                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |

|              |                                                                                                                                                       |
|--------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| Web Modeller | Позволяет выполнить настройку свойств и спецификацию модели для генерации Web кода, выбранного в качестве языка реализации отдельных элементов модели |
| XML_DTD      | Позволяет выполнить настройку свойств и спецификацию модели для ее публикации в формате расширяемого языка разметки XML                               |
| Class Wizard | Вызывает мастер создания нового класса и его размещения на выбранной диаграмме модели                                                                 |

### Назначение операций главного меню Add-Ins, Window и Help

Пункт главного меню **Add-Ins (Расширения)** вызывает специальное диалоговое окно менеджера расширений для добавления их в операции пункта меню Tools (рис. 2.1.8). При этом доступными являются те расширения, которые были установлены при инсталляции программы IBM Rational Rose.

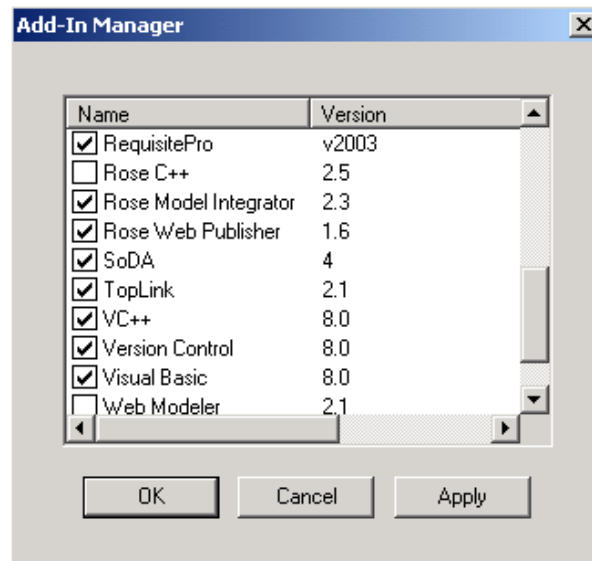


Рис. 2.1.8. Внешний вид диалогового окна менеджера расширений

Операции главного меню **Window (Окно)** (Табл.15) позволяют активизировать окно нужной диаграммы разрабатываемой модели из числа открытых и отображать все открытые окна диаграмм в различном графическом представлении. Назначение операций этого пункта главного меню представлено в таблице 15.

Таблица 15.

Операции пункта главного меню Window (Окно)

| Название операции меню | Назначение операции главного меню                        |
|------------------------|----------------------------------------------------------|
| Cascade                | Размещает окна всех открытых диаграмм модели каскадно    |
| Tile                   | Отображает в окне диаграмм все открытые диаграммы модели |
| Arrange Icons          | Упорядочивает расположение всех открытых диаграмм        |

Секция, содержащая имена всех открытых диаграмм модели для переключения между ними. Если открывается новая диаграмма, то в этой секции появляется новая строка с именем этой диаграммы и ее типом, выбрав которую, можно сразу перейти в нужное окно.

Операции главного меню **Help (Справка)** (Табл.16) позволяют получить справочную информацию о программе IBM Rational Rose и об особенностях разработки графических моделей в соответствующих нотациях.

Таблица 16.

Операции пункта главного меню Help (Справка)

| Название операции меню | Назначение операции главного меню                                               |
|------------------------|---------------------------------------------------------------------------------|
| Contents and Index     | Вызывает программу просмотра справочной системы, открытой на вкладке Содержание |

|                              |                                                                                                                                                                                                                                                                                                                                                                                                                             |
|------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Search for Help on           | Вызывает программу просмотра справочной системы, открытой на вкладке Указатель                                                                                                                                                                                                                                                                                                                                              |
| Using Help                   | Вызывает программу отображения информации об использовании справочной системы                                                                                                                                                                                                                                                                                                                                               |
| Extended Help                | Вызывает специальную программу расширенной справочной системы                                                                                                                                                                                                                                                                                                                                                               |
| Contacting Technical Support | Вызывает установленный в операционной системе по умолчанию браузер Интернет и делает попытку соединиться с web-сайтом технической поддержки компании IBM Rational при наличии доступа в Интернет                                                                                                                                                                                                                            |
| Rational on the Web          | Вызывает установленный в операционной системе по умолчанию браузер Интернет и делает попытку соединиться с web-сайтом компании IBM Rational при наличии доступа в Интернет. Выбор отдельной операции этого пункта меню определяет загрузку той или иной web-страницы компании, предназначенной для выполнения специальных действий по дополнительной поддержке средства IBM Rational Rose или загрузке имеющихся обновлений |
| Rational Developer Network   | Вызывает установленный в операционной системе по умолчанию браузер Интернет и делает попытку соединиться с web-сайтом разработчиков компании IBM Rational при наличии доступа в Интернет                                                                                                                                                                                                                                    |
| About Rational Rose          | Отображает информацию о текущей рабочей версии IBM Rational Rose                                                                                                                                                                                                                                                                                                                                                            |

### 2.1.2 Задания

- 1 Создать новый проект.
- 2 Установить цвета и шрифта объектов.
- 3 Установить свойства диаграммы.

### 2.1.3 Контрольные вопросы

1. Какие методологии поддерживаются Rational Rose?
2. Перечислите основные элементы главного окна Rational Rose.
3. Опишите процесс создания новой модели в Rational Rose.

## 2.2 Диаграммы прецедентов

**Диаграммы прецедентов (use-case diagrams)**, иногда называемые диаграммами вариантов использования описывают функциональность ИС, которая будет видна пользователям системы. Вариант использования или прецедент представляет собой типичное взаимодействие пользователя и проектируемой системы. "Каждая функциональность" изображается в виде "вариантов использования" (use case) или просто прецедентов. Прецедент – это типичное взаимодействие пользователя с системой, которое при этом:

- описывает видимую пользователем функцию,
- может представлять различные уровни детализации,
- обеспечивает достижение конкретной цели, важной для пользователя.

В простейшем случае прецедент создается в процессе обсуждения с пользователями тех вещей, которые они хотели бы получить от системы. При этом каждой отдельной функции, которую они хотели бы реализовать, присваивается некоторое имя и записывается ее краткое текстовое описание. Это все, что необходимо в фазе анализа. Знание некоторых деталей может потребоваться, если предполагается, что данный прецедент содержит важные архитектурные ответвления. Большинство прецедентов может быть детализировано во время конкретной итерации в процессе проектирования.

**Прецедент (UseCase)** или вариант использования обозначается на диаграмме овалом, связанным с пользователями, которых принято называть действующими лицами (актеры, actors). Прецеденты всегда следует анализировать вместе с действующими лицами системы, определяя при этом реальные задачи пользователей и рассматривая альтернативные способы решения этих задач. Действующие лица используют систему (или используются системой) в данном прецеденте. Действующее лицо выполняет некоторую роль в данном прецеденте. На диаграмме изображается только одно действующее лицо, однако реальных пользователей, выступающих в данной роли по отношению к ИС, может быть много. Действующие лица могут играть различные роли по отношению к прецеденту. Они могут применять его результаты или сами непосредственно в нем участвовать.

Список всех прецедентов фактически определяет функциональные требования к ИС, которые лежат в основе разработки технического задания на создание системы. Хорошим источником для идентификации прецедентов служат внешние события. Для этого необходимо перечислить все происходящие во внешнем мире события, на которые система должна реагировать. Какое-либо конкретное событие может повлечь за собой реакцию системы, не требующую вмешательства пользователей, или, наоборот, вызвать чисто пользовательскую реакцию. Идентификация событий, на которые необходимо реагировать, поможет идентифицировать прецеденты.

На диаграммах прецедентов, кроме связей между действующими лицами и прецедентами, возможно использование еще двух видов связей между прецедентами: "*включение*" (<<include>>) и "*расширение*" (<<extend>>) (рис. 2.2.1). Связь типа "расширение" применяется, когда один прецедент подобен другому, но несет несколько большую функциональную нагрузку. Ее следует применять при описании изменений в нормальном поведении системы. Связь типа "включение" позволяет выделить некий фрагмент поведения системы и включать его в различные прецеденты без повторного описания.

На рис. 6.1 показано, что при исполнении прецедента "*формирование заказа*" возможно использование информации из предыдущего заказа, что позволит не вводить все необходимые данные. А при исполнении прецедентов "*оценить риск сделки*" и "*согласовать цену*" необходимо выполнить одно и то же действие — рассчитать стоимость заказа.

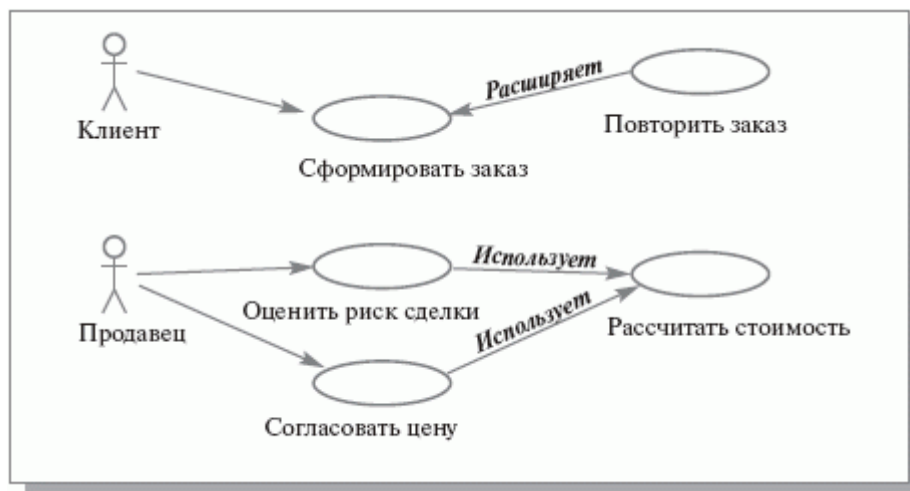


Рис. 2.2.1. Связи на диаграммах прецедентов

## 2.2.1 Лабораторная работа № 6. Диаграмма Use Case

### Цель работы:

- изучение диаграмм прецедентов,
- изучение их применения в процессе постановки задачи.

В качестве проекта далее будет рассматриваться модель системы управления банкоматом. Для изменения имени проекта, предложенного программой по умолчанию, следует сохранить модель во внешнем файле на диске, например, под именем ATMmodel.mdl.

В этом случае изменится имя в строке заголовка и имя проекта в иерархическом представлении модели в браузере проекта.

Для разработки диаграммы прецедентов модели в среде IBM Rational Rose необходимо активизировать соответствующую диаграмму в окне диаграммы. Это можно сделать следующими способами:

- раскрыть представление прецедентов *Use Case View* в браузере проекта и дважды щелкнуть на пиктограмме **Main** (Главная);
- с помощью операции главного меню **Browse** → **Use Case Diagram** (Браузер → Диаграмма вариантов использования).

При этом появляется новое окно с чистым рабочим листом диаграммы прецедентов и специальная панель инструментов, содержащая кнопки с изображением графических элементов, необходимых для разработки диаграммы прецедентов. Назначение отдельных кнопок данной панели указано в таблице 17, а также можно узнать из всплывающих подсказок, которые появляются, если подвести и задержать на некоторое время указатель мыши над той или иной кнопкой.

Таблица 17.

Назначение кнопок специальной панели инструментов для диаграммы прецедентов

| Иконка | Всплывающая подсказка | Назначение кнопки                                                                                |
|--------|-----------------------|--------------------------------------------------------------------------------------------------|
|        | Selection Tool        | Превращает изображение курсора в форму стрелки для последующего выделения элементов на диаграмме |
|        | Text Box              | Добавляет на диаграмму текстовую область                                                         |
|        | Note                  | Добавляет на диаграмму примечание                                                                |
|        | Anchor Note to Item   | Добавляет на диаграмму связь примечания с соответствующим графическим элементом диаграммы        |
|        | Package               | Добавляет на диаграмму пакет                                                                     |

|                                                                                   |  |                            |                                                |
|-----------------------------------------------------------------------------------|--|----------------------------|------------------------------------------------|
|  |  | Use Case                   | Добавляет на диаграмму прецедент               |
|  |  | Actor                      | Добавляет на диаграмму актера                  |
|  |  | Unidirectional Association | Добавляет на диаграмму направленную ассоциацию |
|  |  | Dependency or Instantiates | Добавляет на диаграмму отношение зависимости   |
|  |  | Generalization             | Добавляет на диаграмму отношение обобщения     |

На специальной панели инструментов по умолчанию присутствует только часть кнопок с пиктограммами элементов, которые могут быть использованы для построения диаграммы. Добавить кнопки с пиктограммами других графических элементов, например, таких как бизнес-вариант использования (business use case), бизнес-актер (business actor), сотрудник (business worker), или удалить ненужные кнопки можно с помощью настройки специальной панели инструментов.

Открыть диалоговое окно настройки специальных панелей инструментов для диаграмм в среде IBM Rational Rose можно с помощью операции главного меню: **Tools**→**Options** (Инструменты→Параметры), раскрыв вкладку **Toolbars** (Панели инструментов) и нажав соответствующую кнопку (например, Use Case diagram) в группе опций **Customize Toolbars** (Настройка панелей инструментов). Это окно настройки также можно открыть с помощью операции контекстного меню **Customize** (Настройка) при позиционировании курсора на специальной панели инструментов (рис. 2.2.2).

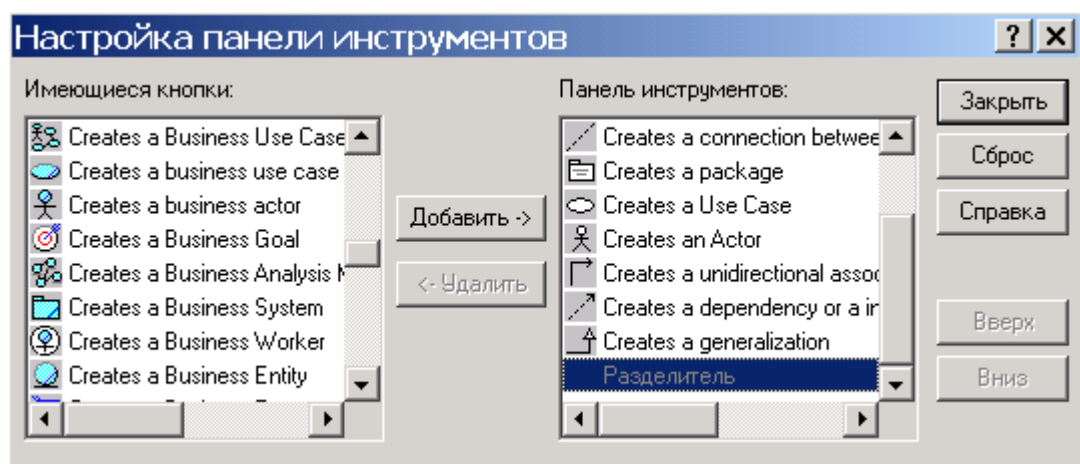


Рис. 2.2.2. Диалоговое окно настройки специальной панели инструментов для диаграммы прецедентов

Для добавления необходимых кнопок на панель следует выделить их в левом окне со списком пиктограмм графических элементов, после чего нажать кнопку **Добавить** в центре диалогового окна. Для удаления ненужных кнопок с панели инструментов следует выделить их в правом окне со списком пиктограмм графических элементов, после чего нажать кнопку **Удалить** в центре диалогового окна. Для восстановления набора пиктограмм по умолчанию можно нажать кнопку **Сброс**. После настройки специальной панели инструментов соответствующее окно следует закрыть нажатием на кнопку **Закреть**.

### Добавление актера

Для добавления актера на диаграмму прецедента нужно с помощью левой кнопки мыши нажать кнопку с изображением пиктограммы актера на специальной панели инструментов, отпустить левую кнопку мыши и щелкнуть левой кнопкой мыши на свободном месте рабочего листа диаграммы. На диаграмме появится изображение актера с маркерами изменения его геометрических размеров и предложенным программой именем по

умолчанию NewClass. Для разрабатываемой модели банкомата предложенное программой имя актера следует изменить на *Клиент Банкомата* (рис. 2.2.3).

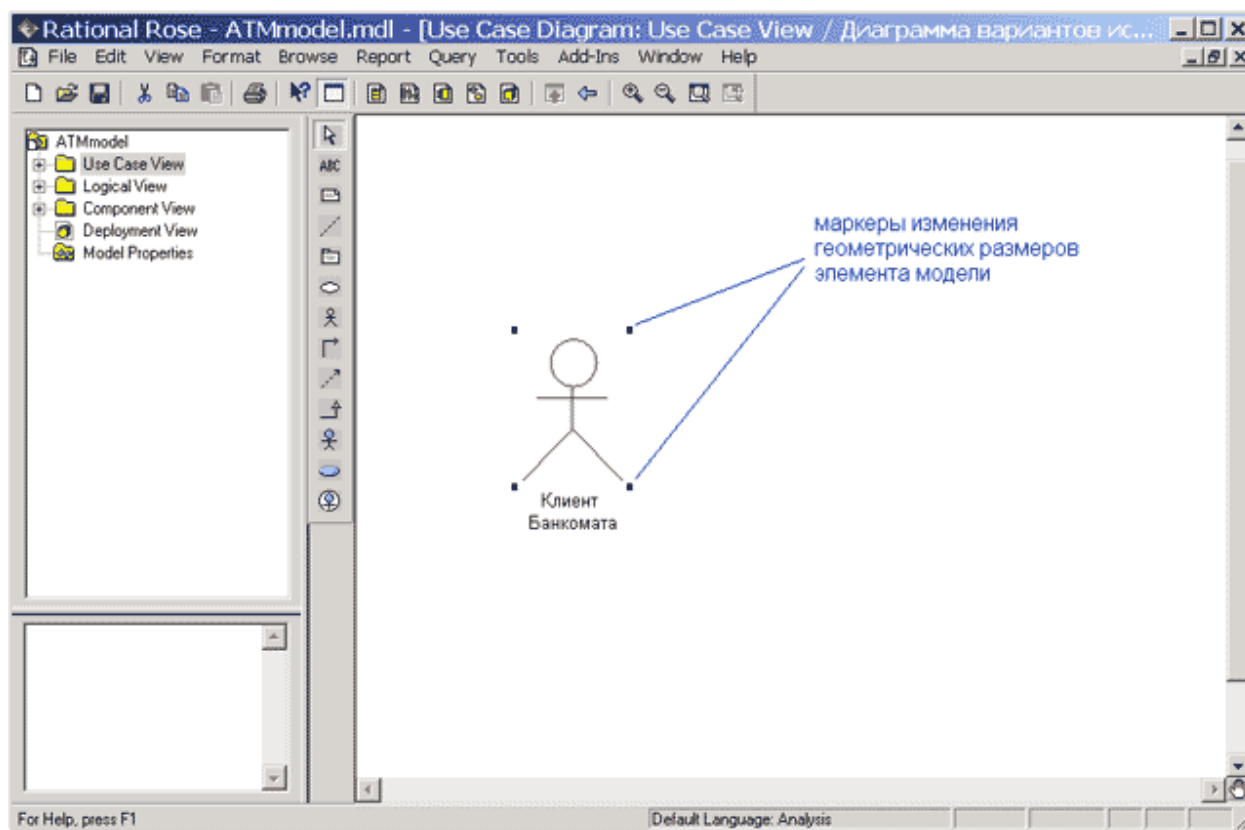


Рис. 2.2.3. Диаграмма прецедентов после добавления на нее актера

Чтобы изменить расположение изображения графического элемента модели, следует щелчком левой кнопки мыши выделить его в рабочей области диаграммы, и, не отпуская левой кнопки, переместить в нужное место диаграммы. При этом выделенный элемент визуально отличается от остальных наличием маркеров изменения его геометрических размеров в форме небольших черных квадратов. Более точное перемещение элемента можно осуществить с помощью стрелок: «←», «↑», «→», «↓» на клавиатуре.

Чтобы изменить графические размеры изображения элемента модели, прежде всего, следует щелчком левой кнопки мыши выделить его в рабочей области диаграммы. Далее необходимо подвести указатель мыши к нужному маркеру геометрических размеров элемента и нажать левую кнопку мыши. В результате этих действий появится пунктирный прямоугольник, изображающий границы выбранного геометрического элемента. После чего, не отпуская левой кнопки мыши, следует диагонально изменить размеры этого прямоугольника нужным образом.

Имя размещенного на диаграмму элемента разработчик может изменить либо сразу после добавления элемента на диаграмму, либо в ходе последующей работы над проектом. Для любого графического элемента модели по щелчку правой кнопкой мыши на выбранном элементе вызывается контекстное меню данного элемента, среди операций которого имеется пункт **Open Specification** (Открыть спецификацию). В этом случае появляется дополнительное диалоговое окно со специальными вкладками, в поля ввода которых можно занести всю информацию по данному элементу. Для добавленного актера *Клиент Банкомата* окно спецификации свойств выглядит следующим образом (рис. 2.2.4).

Следует отметить, что открыть диалоговое окно спецификации свойств любого элемента модели можно также двойным щелчком левой кнопкой мыши на графическом изображении этого элемента на диаграмме.

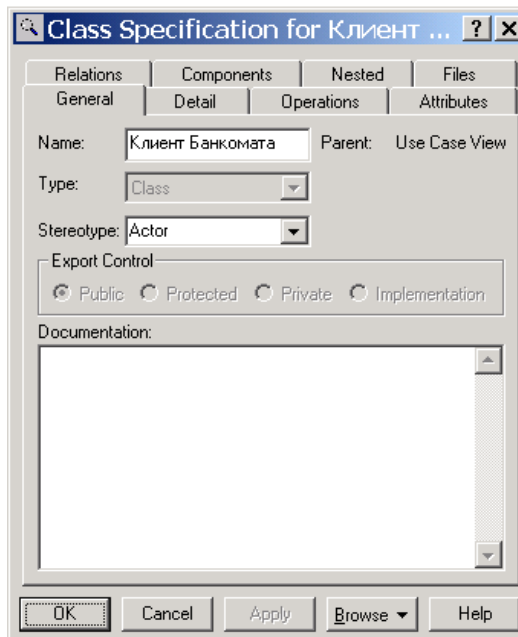


Рис. 2.2.4. Диалоговое окно спецификации свойств актера *Клиент Банкомата*

Для актера *Клиент Банкомата* можно уточнить его назначение в модели. С этой целью следует изменить его стереотип и добавить текст документации. Для изменения стереотипа во вложенном списке **Stereotype** нужно выбрать строку **Business Actor** (бизнес-актер). Для добавления текста документации в секцию **Documentation** следует ввести текст: «Любое физическое лицо, пользующееся услугами банкомата» и нажать кнопку **Apply** (Применить) или **OK**.

### Добавление и редактирование прецедента (Use Case)

Для добавления прецедента на диаграмму нужно с помощью левой кнопки мыши нажать кнопку с изображением прецедента на специальной панели инструментов, отпустить левую кнопку мыши и щелкнуть левой кнопкой мыши на свободном месте диаграммы. На диаграмме появится изображение прецедента с маркерами изменения его геометрических размеров и предложенным программой именем по умолчанию NewUseCase. Для разрабатываемой модели банкомата предложенное программой имя прецедента следует изменить на *Снятие наличных по кредитной карточке* (рис. 2.2.5).

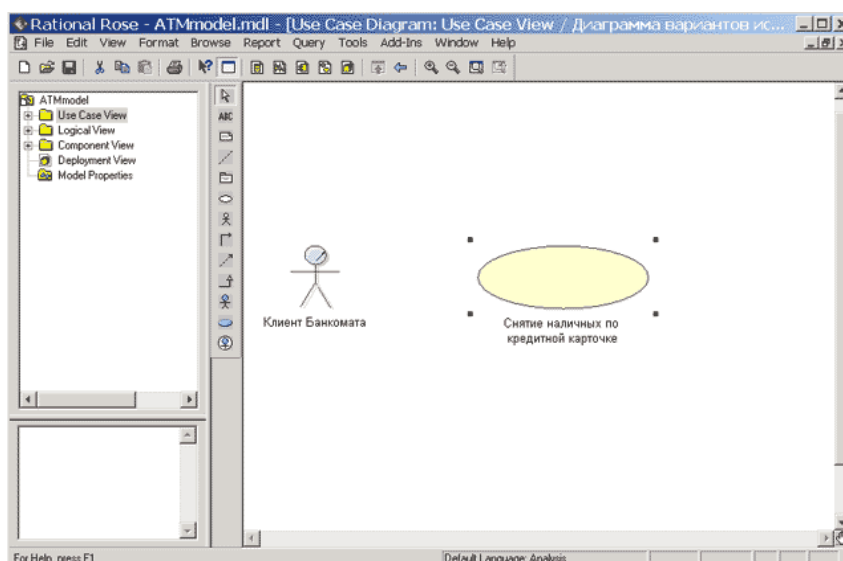


Рис. 2.2.5. Диаграмма прецедентов после добавления на нее прецедента

Для уточнения свойств данного прецедента следует открыть диалоговое окно спецификации его свойств, например, с помощью двойного щелчка левой кнопкой мыши на изображении этого элемента на диаграмме. Для изменения стереотипа во вложенном списке **Stereotype** нужно выбрать строку **Business Use Case**. Для добавления текста документации в секцию **Documentation** следует ввести текст: «Основной прецедент для разрабатываемой модели банкомата» и нажать кнопку **Apply** (Применить) или **OK**. После изменения данных свойств прецедента окно спецификации его свойств будет выглядеть следующим образом (рис. 2.2.6).

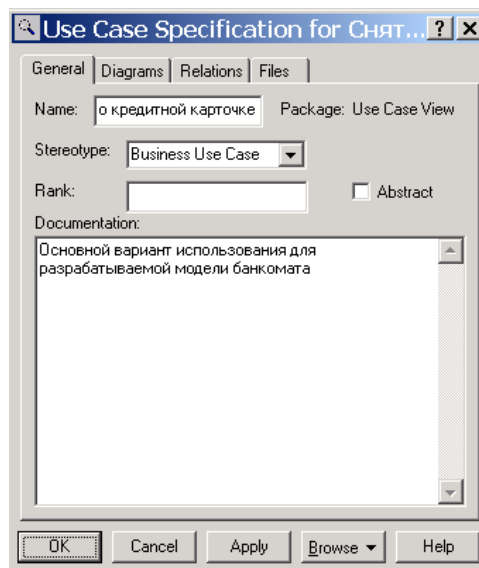


Рис. 2.2.6. Диалоговое окно спецификации свойств прецедента Снятие наличных по кредитной карточке

### Добавление ассоциации

Для добавления ассоциации между актером и прецедентом на диаграмму нужно с помощью левой кнопки мыши нажать на специальной панели инструментов кнопку с изображением пиктограммы направленной ассоциации, отпустить левую кнопку мыши, щелкнуть левой кнопкой мыши на изображении актера на диаграмме и отпустить ее на изображении прецедента. В результате этих действий на диаграмме появится изображение ассоциации, соединяющей актера с прецедентом (рис. 2.2.7).

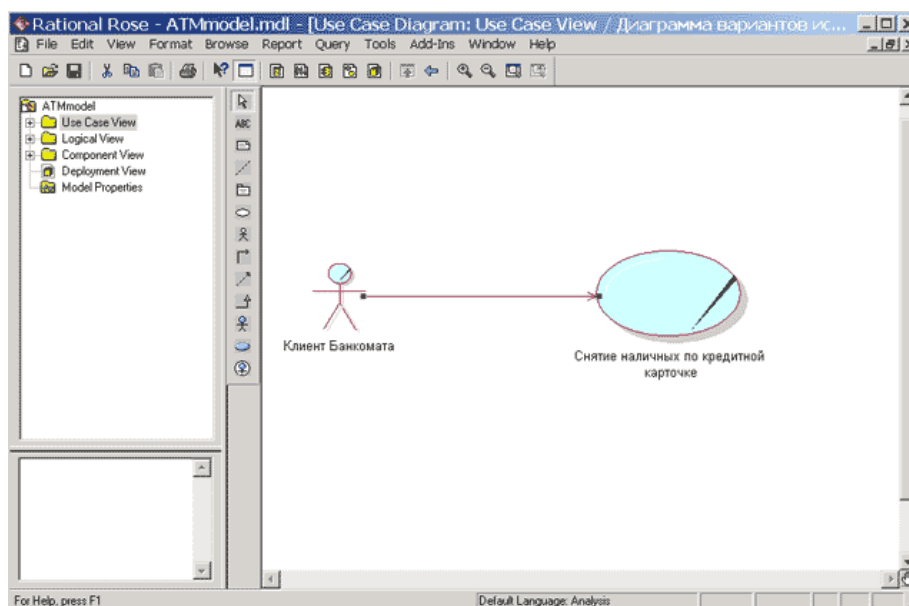


Рис. 2.2.7. Диаграмма прецедентов после добавления на нее направленной ассоциации

При необходимости можно сделать направленную ассоциацию ненаправленной, для чего следует воспользоваться диалоговым окном свойств ассоциации. Открыть это окно можно, например, двойным щелчком на изображении линии ассоциации на диаграмме, после чего убрать отметку строки выбора **Navigable** (Навигация) на вкладке **Role A Detail** (Детальные свойства конечной точки ассоциации A).

### Добавление отношения зависимости

Для добавления отношения зависимости между двумя прецедентами на диаграмму необходимо предварительно рассмотренным выше способом добавить второй прецедент с именем *Проверка ПИН-кода*. После этого с помощью левой кнопки мыши нажать кнопку с изображением пиктограммы зависимости на специальной панели инструментов, отпустить левую кнопку мыши, щелкнуть левой кнопкой мыши на изображении прецедента *Снятие наличных по кредитной карточке* и отпустить ее на изображении прецедента *Проверка ПИН-кода*. В результате этих действий на диаграмме появится изображение отношения зависимости, которое соединяет два выбранных прецедента.

Поскольку прецедент *Проверка ПИН-кода* выполняется всегда, для добавленного отношения зависимости дополнительно следует указать текстовый стереотип «включение» (`<<include>>`). Выполнить это можно уже известным способом с помощью диалогового окна спецификации свойств этого отношения и выбора нужного стереотипа из предлагаемого списка.

После задания для данного отношения зависимости стереотипа `<<include>>` текст этого стереотипа в угловых скобках появится рядом с изображением пунктирной линии зависимости, связывающей соответствующие прецеденты (рис. 2.2.8). С целью лучшей визуализации диаграммы текстовую область стереотипа можно переместить в нужное место диаграммы. Выполнить это можно с помощью общего способа перемещения графических элементов модели, который был рассмотрен ранее в этой лекции применительно к актеру *Клиент Банкомата*.

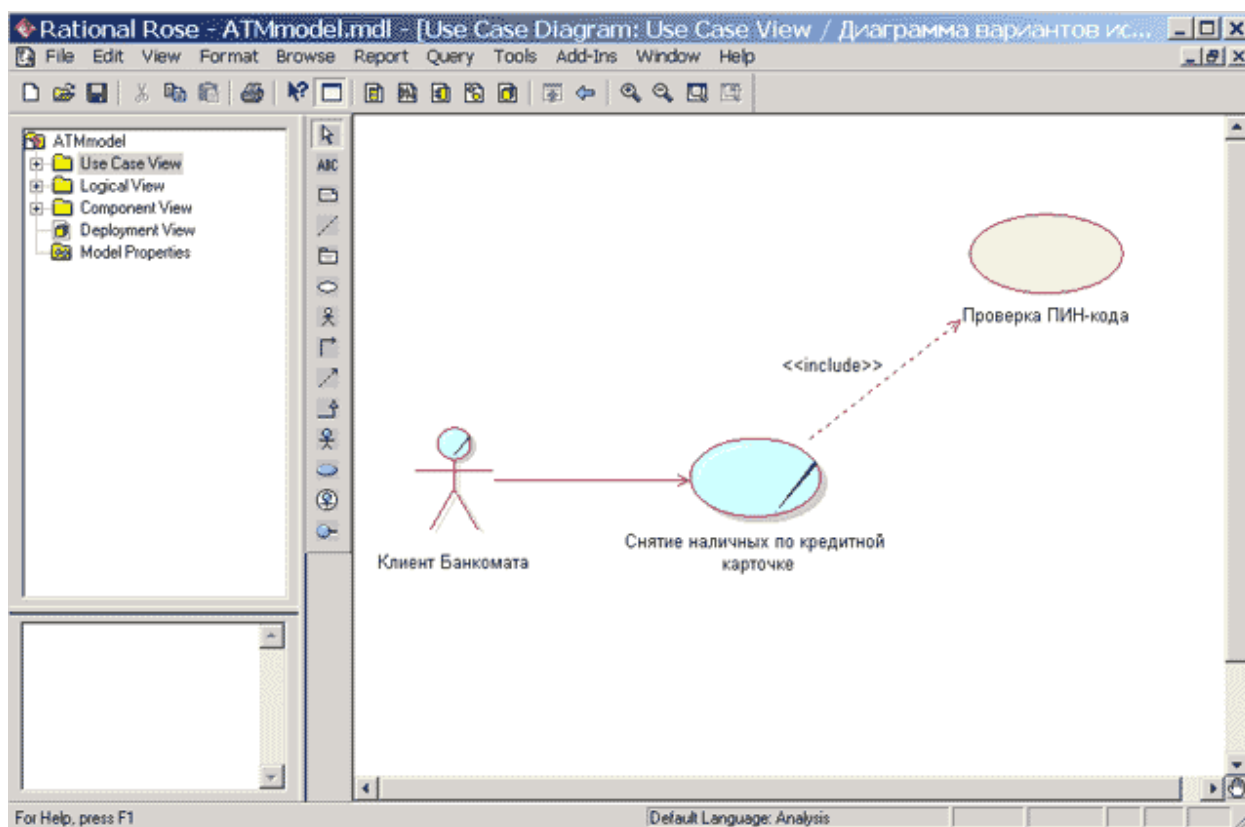


Рис. 2.2.8. Диаграмма прецедентов после добавления на нее отношения зависимости

Аналогичным образом могут быть добавлены на диаграмму прецедентов отношения зависимости со стереотипом «расширение» (<<extend>>), которые применяются для моделирования исключений при выполнении отдельных прецедентов.

### Окончательное построение диаграммы прецедентов

К отдельному прецеденту можно добавить текстовый файл с описанием сценария его выполнения. Для этого необходимо выделить этот прецедент в браузере проекта и выполнить операцию контекстное меню: **New→File** (Новый→Файл). В результате этого будет вызвано стандартное окно открытия файла, в котором необходимо задать имя предварительно созданного с помощью офисной программы MS Word добавляемого файла. После нажатия кнопки **Открыть** пиктограмма добавленного файла появится в браузере проекта ниже соответствующего прецедента. В последующем можно вернуться к редактированию этого файла сценария, выполнив двойной щелчок на этой пиктограмме. При этом файл сценария будет открыт в соответствующем приложении - в текстовом процессоре MS Word.

Для окончательного построения диаграммы прецедента для рассматриваемой модели банкомата следует выполнить следующие действия:

1. Добавить актера с именем *Банк*, для которого выбрать стереотип **Service** (Сервис), означающий, что банкомат использует некоторые услуги Банка в качестве сервиса.
2. Добавить прецедент *Получение справки о состоянии счета*, для которого выбрать стереотип **Business Use Case** (Бизнес-прецедент).
3. Добавить прецедент *Блокирование кредитной карточки*.
4. Добавить направленную ассоциацию от бизнес-актера *Клиент Банкомата* к прецеденту *Получение справки о состоянии счета*.
5. Добавить направленную ассоциацию от прецедента *Снятие наличных* по кредитной карточке к сервису *Банк*.
6. Добавить направленную ассоциацию от прецедента *Получение справки о состоянии счета* к сервису *Банк*.
7. Добавить отношение зависимости со стереотипом <<include>>, направленное от прецедента *Получение справки о состоянии счета* к прецеденту *Проверка ПИН-кода*.
8. Добавить отношение зависимости со стереотипом <<extend>>, направленное от прецедента *Блокирование кредитной карточки* к прецеденту *Проверка ПИН-кода*.

Отношение зависимости со стереотипом <<extend>> на данной диаграмме означает следующее. Прецедент *Блокирование кредитной карточки* будет выполняться только в том случае, если в результате проверки ПИН-кода будет установлено, что соответствующая кредитная карточка утрачена ее владельцем или признана недействительной. Построенная таким образом диаграмма прецедентов будет иметь следующий вид (рис. 2.2.9).

Напомним, что диаграмма прецедентов является высокоуровневым концептуальным представлением модели, поэтому она не должна содержать слишком много прецедентов и актеров. В последующем построенная диаграмма может быть изменена посредством добавления новых элементов, таких как прецеденты и актеры, или их удаления.

Для удаления любого графического элемента с диаграммы его следует выделить на диаграмме и нажать клавишу **Delete** на клавиатуре. При этом выделенный элемент будет удален с активной диаграммы, но не из модели. Для удаления элемента не только из диаграммы, но и из модели проекта необходимо выделить удаляемый элемент на диаграмме и воспользоваться операцией главного меню **Edit→Delete from Model** (Редактирование→Удалить из модели). Для этой же цели служит комбинация клавиш быстрого доступа: **Ctrl+D**.

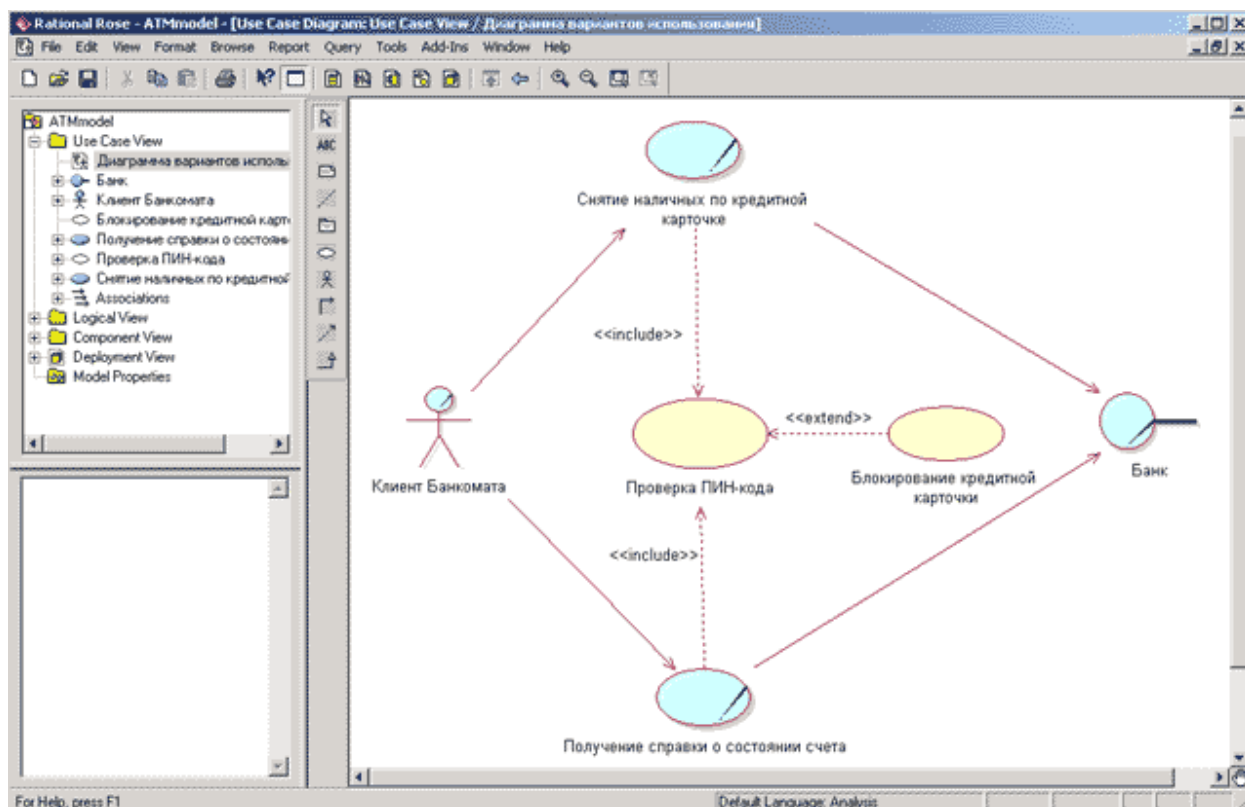


Рис. 2.2.9. Окончательный вид диаграммы прецедентов для разрабатываемой модели банкоматов

При работе с отношениями на диаграмме прецедентов следует помнить о назначении соответствующих отношений в нотации языка UML. Речь идет о том, что если для двух элементов выбранный вид отношения не является допустимым, то в большинстве случаев программа IBM Rational Rose сообщит об этом разработчику, и соответствующая линия связи не будет добавлена на диаграмму.

После окончания сеанса работы над проектом выполненную работу необходимо сохранить в файле проекта с расширением «.MDL». Это можно сделать через меню **File→Save** (Файл→Сохранить) или **File→Save As** (Файл→Сохранить как). При этом вся информация о проекте, включая диаграммы и спецификации элементов, будет сохранена в одном файле.

## 2.2.2 Задания

1. Определить основные функции системы.
2. Для нескольких функций выделить действующие лица, участвующие в них или использующие их результаты.
3. Для каждой выбранной функции построить диаграмму прецедентов.

## 2.2.3 Контрольные вопросы

1. В чем смысл прецедента?
2. Каково назначение диаграмм прецедентов?
3. Назовите основные свойства прецедентов.
4. Назовите основные компоненты диаграмм прецедентов.
5. Что такое «действующее лицо»?
6. Какую роль могут играть действующие лица по отношению к прецеденту?
7. Каким образом анализ внешних событий позволяет определить прецеденты системы?

## 2.3 Диаграммы классов

**Диаграмма классов** показывает классы и их отношения, тем самым представляя логический аспект проекта. Отдельная диаграмма классов представляет определенный ракурс структуры классов. На стадии анализа диаграммы классов используются, чтобы выделить общие роли и обязанности сущностей, обеспечивающих требуемое поведение системы. На стадии проектирования диаграммы классов используются, чтобы передать структуру классов, формирующую архитектуру системы.

Диаграмма классов определяет типы объектов системы и различного рода статические связи, которые существуют между ними. Имеется два основных вида статических связей:

- ассоциации (например, менеджер может вести несколько проектов),
- подтипы (работник является разновидностью личности).

**Классы** — это базовые элементы любой объектно-ориентированной системы. Классы представляют собой описание совокупностей однородных объектов с присущими им свойствами — атрибутами, операциями, отношениями и семантикой (Рис.2.3.1).



Рис. 2.3.1. Изображение класса в UML

В рамках модели каждому классу присваивается уникальное имя, отличающее его от других классов. Если используется составное имя (в начале имени добавляется имя пакета, куда входит класс), то имя класса должно быть уникальным в пакете.

**Атрибут** — это свойство класса, которое может принимать множество значений. Множество допустимых значений атрибута образует домен. Атрибут имеет имя и отражает некоторое свойство моделируемой сущности, общее для всех объектов данного класса. Класс может иметь произвольное количество атрибутов.

**Операция** — реализация функции, которую можно запросить у любого объекта класса. Операция показывает, что можно сделать с объектом. Исполнение операции часто связано с обработкой и изменением значений атрибутов объекта, а также изменением состояния объекта.

**Видимость свойства** указывает на возможность его использования другими классами. Один класс может "видеть" другой, если тот находится в области действия первого и между ними существует явное или неявное отношение. В языке UML определены три уровня видимости:

- **public** (общий) — любой внешний класс, который "видит" данный, может пользоваться его общими свойствами. Обозначаются знаком "+" перед именем атрибута или операции;
- **protected** (защищенный) — только любой потомок данного класса может пользоваться его защищенными свойствами. Обозначаются знаком "#";
- **private** (закрытый) — только данный класс может пользоваться этими свойствами. Обозначаются знаком "-".

Еще одной важной характеристикой атрибутов и операций классов является область действия. Область действия свойства указывает, будет ли оно проявлять себя по-разному в

каждом экземпляре класса, или одно и то же значение свойства будет совместно использоваться всеми экземплярами:

- **instance** (экземпляр) — у каждого экземпляра класса есть собственное значение данного свойства;
- **classifier** (классификатор) — все экземпляры совместно используют общее значение данного свойства (выделяется на диаграммах подчеркиванием).

Возможное количество экземпляров класса называется его **кратностью**. В UML можно определять следующие разновидности классов:

- не содержащие ни одного экземпляра — тогда класс становится служебным (Abstract);
- содержащие ровно один экземпляр (Singleton);
- содержащие заданное число экземпляров;
- содержащие произвольное число экземпляров.

Принципиальное назначение классов характеризуют **стереотипы**. Это, фактически, классификация объектов на высоком уровне, позволяющая определить некоторые основные свойства объекта. Механизм стереотипов является также средством расширения словаря UML за счет создания на основе существующих блоков языка новых, специфичных для решения конкретной проблемы.

Классы в UML изображаются на диаграммах классов, которые позволяют описать систему в статическом состоянии — определить типы объектов системы и различного рода статические связи между ними (Рис.2.3.2).

Между классами возможны различные отношения:

- **зависимости**, которые описывают существующие между классами отношения использования;
- **обобщения**, связывающие обобщенные классы со специализированными;
- **ассоциации**, отражающие структурные отношения между объектами классов.

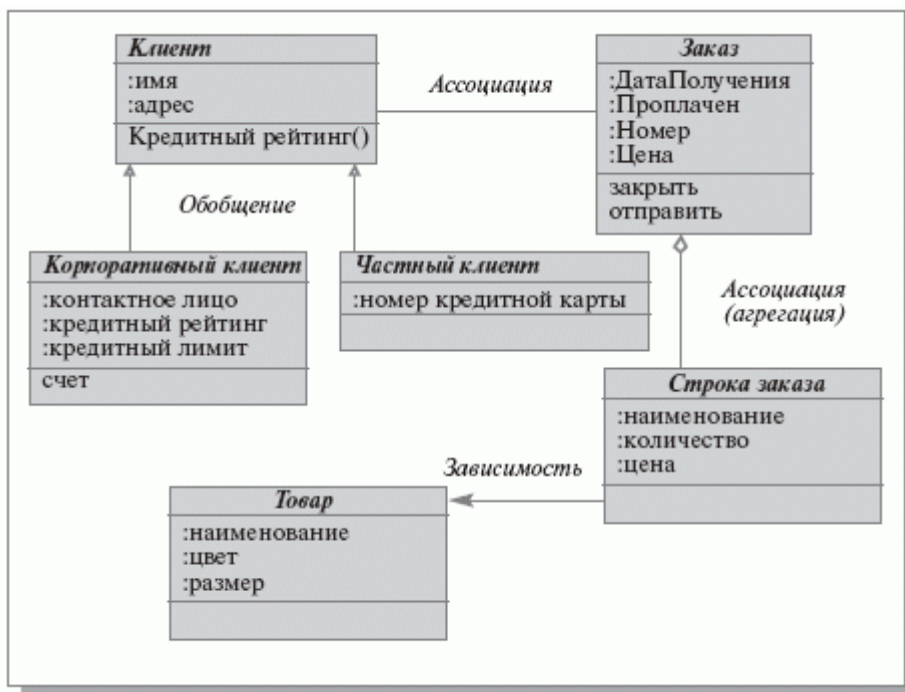


Рис. 2.3.2. Отображение связей между классами

**Зависимостью** называется отношение использования, согласно которому изменение в спецификации одного элемента (например, класса "товар") может повлиять на

использующий его элемент (класс "строка заказа"). Часто зависимости показывают, что один класс использует другой в качестве аргумента.

**Обобщение** — это отношение между общей сущностью (родителем — класс "клиент") и ее конкретным воплощением (потомком — классы "корпоративный клиент" или "частный клиент"). Объекты класса-потомка могут использоваться всюду, где встречаются объекты класса-родителя, но не наоборот. При этом он наследует свойства родителя (его атрибуты и операции). Операция потомка с той же сигнатурой, что и у родителя, замещает операцию родителя; это свойство называют полиморфизмом. Класс, у которого нет родителей, но есть потомки, называется корневым. Класс, у которого нет потомков, называется листовым.

**Ассоциация** — это отношение, показывающее, что объекты одного типа неким образом связаны с объектами другого типа ("клиент" может сделать "заказ"). Если между двумя классами определена ассоциация, то можно перемещаться от объектов одного класса к объектам другого. При необходимости направление навигации может задаваться стрелкой. Допускается задание ассоциаций на одном классе. В этом случае оба конца ассоциации относятся к одному и тому же классу. Это означает, что с объектом некоторого класса можно связать другие объекты из того же класса. Ассоциации может быть присвоено имя, описывающее семантику отношений. Каждая ассоциация имеет две роли, которые могут быть отражены на диаграмме (рис. 2.3.3). Роль ассоциации обладает свойством множественности, которое показывает, сколько соответствующих объектов может участвовать в данной связи.

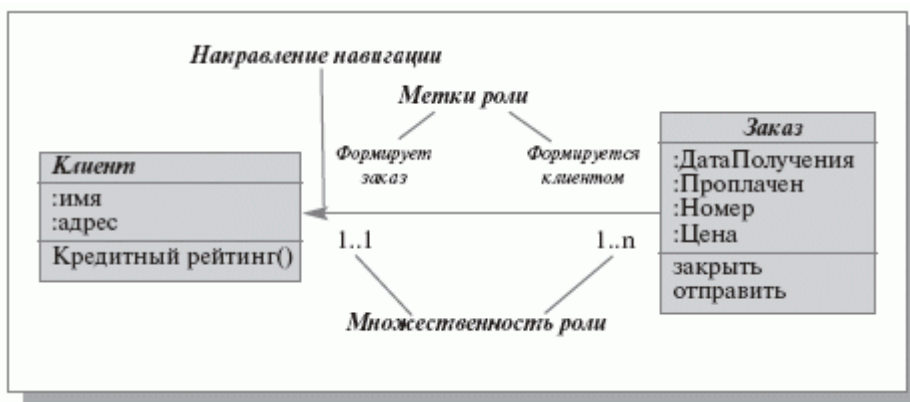


Рис. 2.3.3. Свойства ассоциации

Рис. 2.3.3 иллюстрирует модель формирования заказа. Каждый заказ может быть создан единственным клиентом (множественность роли 1..1). Каждый клиент может создать один и более заказов (множественность роли 1..n). Направление навигации показывает, что каждый заказ должен быть "привязан" к определенному клиенту.

Такого рода ассоциация является простой и отражает отношение между равноправными сущностями, когда оба класса находятся на одном концептуальном уровне и ни один не является более важным, чем другой. Если приходится моделировать отношение типа "часть-целое", то используется специальный тип ассоциации – агрегирование. В такой ассоциации один из классов имеет более высокий ранг (целое – класс "заказ", рис. 2.3.2) и состоит из нескольких меньших по рангу классов (частей – класс "строка заказа"). В UML используется и более сильная разновидность агрегации – композиция, в которой объект-часть может принадлежать только единственному целому. В композиции жизненный цикл частей и целого совпадают, любое удаление целого обязательно захватывает и его части.

### 2.3.1 Лабораторная работа № 7. Class diagram

#### Цель работы:

- изучение диаграмм классов,
- изучение их применения в процессе проектирования.

Диаграмма классов является основным логическим представлением модели и содержит детальную информацию о внутреннем устройстве объектно-ориентированной программной системы или, используя современную терминологию, об архитектуре программной системы. Активизировать рабочее окно диаграммы классов можно несколькими способами:

- окно диаграммы классов появляется по умолчанию в рабочем окне диаграммы после создания нового проекта;
- щелкнуть на кнопке с изображением диаграммы классов на стандартной панели инструментов;
- раскрыть логическое представление (Logical View) в браузере проекта и дважды щелкнуть на пиктограмме **Main** (Главная);
- выполнить операцию главного меню: **Browse** → **Class Diagram** (Обзор → Диаграмма классов).

При этом появляется новое окно с чистым рабочим листом диаграммы классов и специальная панель инструментов, содержащая кнопки с изображением графических примитивов, необходимых для разработки диаграммы классов. Назначение отдельных кнопок панели указано в таблице 18, а также можно узнать из всплывающих подсказок.

Таблица 18.

Назначение кнопок специальной панели инструментов для диаграммы классов

| Кнопка                                                                              | Всплывающая подсказка      | Назначение кнопки                                                                                |
|-------------------------------------------------------------------------------------|----------------------------|--------------------------------------------------------------------------------------------------|
|  | Selection Tool             | Превращает изображение курсора в форму стрелки для последующего выделения элементов на диаграмме |
|  | Text Box                   | Добавляет на диаграмму текстовую область                                                         |
|  | Note                       | Добавляет на диаграмму примечание                                                                |
|  | Anchor Note to Item        | Добавляет на диаграмму связь примечания с соответствующим графическим элементом диаграммы        |
|  | Class                      | Добавляет на диаграмму класс                                                                     |
|  | Interface                  | Добавляет на диаграмму интерфейс                                                                 |
|  | Unidirectional Association | Добавляет на диаграмму направленную ассоциацию                                                   |
|  | Association Class          | Добавляет на диаграмму ассоциацию класс                                                          |
|  | Package                    | Добавляет на диаграмму пакет                                                                     |
|  | Dependency or Instantiates | Добавляет на диаграмму отношение зависимости                                                     |
|  | Generalization             | Добавляет на диаграмму отношение обобщения                                                       |
|  | Realize                    | Добавляет на диаграмму отношение реализации                                                      |

На специальной панели инструментов по умолчанию присутствует только часть пиктограмм элементов, которые могут быть использованы для построения диаграммы классов. Добавить кнопки с пиктограммами других графических элементов таких как, например, отношения агрегации и композиции, шаблон, класс бизнес-сущность, управляющий класс, или удалить ненужные кнопки можно с помощью настройки специальной панели инструментов. Соответствующее диалоговое окно настройки

специальной панели инструментов для диаграммы классов можно вызвать аналогично другим панелям с помощью операции контекстного меню **Customize** (Настройка) при позиционировании курсора на специальной панели инструментов.

### Добавление класса на диаграмму классов

Для добавления класса на диаграмму классов нужно с помощью левой кнопки мыши нажать кнопку с изображением пиктограммы класса на специальной панели инструментов, отпустить левую кнопку мыши и щелкнуть левой кнопкой мыши на свободном месте рабочего листа диаграммы. На диаграмме появится изображение класса с маркерами изменения его геометрических размеров и предложенным средой именем по умолчанию NewClass.

Продолжая разработку модели банкомата в качестве сквозного примера проекта, построим для этой модели следующую каноническую диаграмму - диаграмму классов. С этой целью следует изменить предложенное по умолчанию имя диаграммы Main на Диаграмма классов АТМ, а имя добавленного на диаграмму класса - на *Транзакция Банкомата* (рис. 2.3.4).

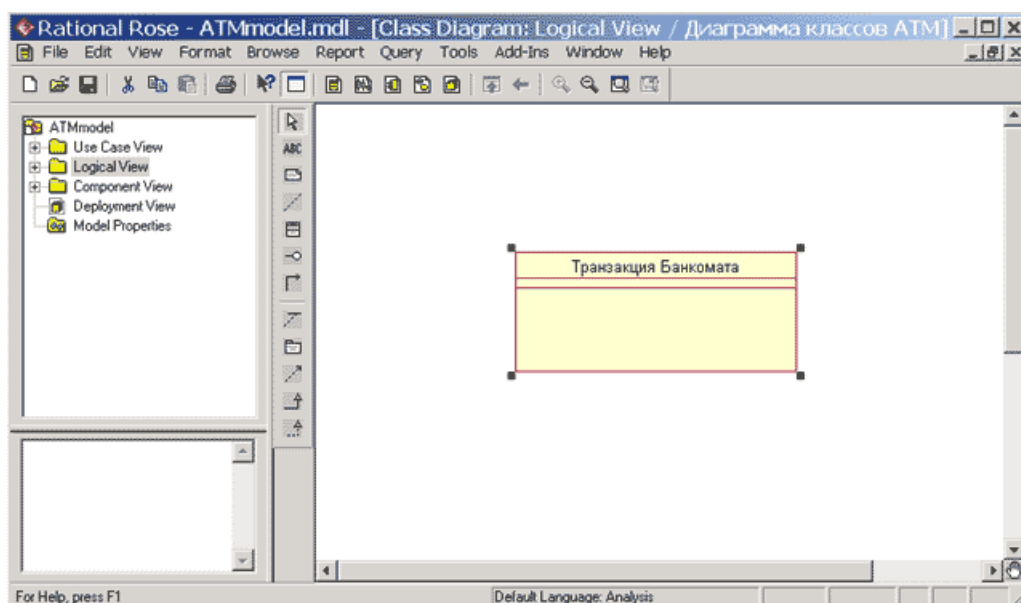


Рис. 2.3.4. Диаграмма классов модели банкомата после добавления на нее класса Транзакция Банкомата

Для класса *Транзакция Банкомата* можно уточнить его назначение в модели с помощью указания стереотипа и пояснительного текста в форме документации. С этой целью двойным щелчком левой кнопкой мыши на изображении этого класса на диаграмме или в браузере проекта следует открыть диалоговое окно спецификации свойств этого класса (рис. 2.3.5) и на вкладке **General** (Общие) выбрать из вложенного списка **Stereotype** стереотип **entity** (сущность).

Выбор данного стереотипа означает, что соответствующий класс предназначен для хранения информации, которая должна сохраняться в системе после уничтожения объектов данного класса. Далее в секцию документации данного класса можно ввести поясняющий текст: "Используется для сохранения информации о выполненных банкоматом транзакциях" и нажать кнопку **Apply** или **OK**, чтобы сохранить результаты редактирования свойств выбранного класса. После назначения стереотипа классу Транзакция банкомата текст данного стереотипа в угловых скобках появится выше имени данного класса.

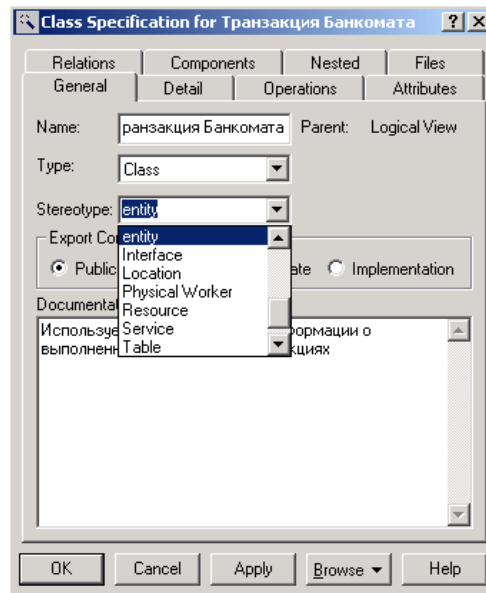


Рис. 2.3.5. Диалоговое окно спецификации свойств класса

Для отдельного класса можно уточнить также и другие его свойства, доступные для редактирования на вкладке **Detail** (Подробно) окна спецификации свойств этого класса (Рис.2.3.6). Например, на этой вкладке с помощью выпадающего списка **Multiplicity** (Кратность) можно задать количество объектов или экземпляров данного класса, для чего следует выбрать строку с буквой **n**. Данное значение означает, что у класса Транзакция банкомата может быть любое конечное число экземпляров. Поле ввода с именем **Space** (Пространство) служит для указания объема абсолютной или относительной памяти, которая требуется, по оценке разработчика, для реализации каждого объекта данного класса. Применительно к рассматриваемой модели это поле можно оставить пустым.

Далее можно задать устойчивость классов в группе выбора **Persistence**. При этом выбор свойства **Persistent** (Устойчивый) означает, что информация об объектах данного класса должна быть сохранена в системе. Выбор свойства **Transient** (Временный) означает, что нет необходимости сохранять информацию об объектах данного класса в системе после завершения работы программного приложения. Применительно к рассматриваемой модели следует выбрать свойство **Persistent**.

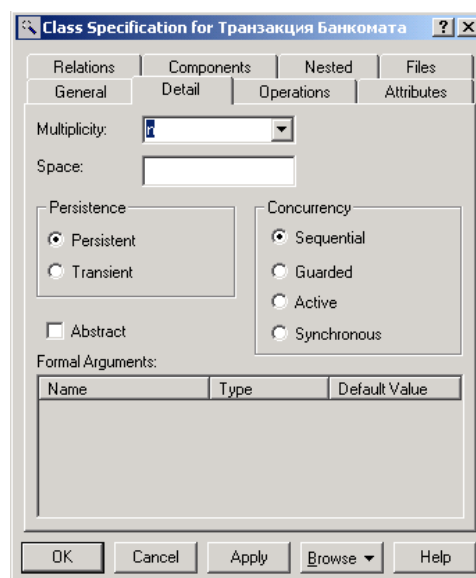


Рис. 2.3.6. Диалоговое окно спецификации свойств класса Транзакция Банкомата, открытое на вкладке **Detail** (Подробно)

В группе выбора **Concurrency** (Параллельность) можно специфицировать условия на возможность реализации объектов данного класса в параллельных потоках управления. Для выбора могут быть использованы следующие свойства:

- *Sequential* (Последовательный) - свойство по умолчанию, которое означает, что объекты класса будут вести себя нормально только при наличии одного потока управления, т. е. соответствующие операции объектов должны выполняться последовательно. В то же время при наличии нескольких потоков управления стабильное поведение объектов класса не гарантируется.

- *Guarded* (Безопасный) - означает, что при наличии нескольких потоков управления объекты класса будут вести себя ожидаемым от них образом. Для этого объекты в различных потоках должны взаимодействовать друг с другом для того, чтобы гарантировать отсутствие конфликта между ними.

- *Active* (Активный) - означает, что класс должен иметь свой собственный поток управления.

- *Synchronous* (Синхронный) - означает, что объекты класса будут вести себя ожидаемым от них образом при наличии нескольких потоков управления. При этом нет необходимости во взаимодействии объектов в различных потоках управления, поскольку объекты данного класса могут самостоятельно разрешать возможные конфликты.

Для того, чтобы специфицировать класс как абстрактный, следует на этой же вкладке выставить отметку в свойстве **Abstract** (Абстрактный). Применительно к рассматриваемой модели для класса Транзакция банкомата следует выбрать свойства **Persistent** и **Sequential**, а отметку для свойства **Abstract** оставить пустой.

### Стереотипы классов и их графическое представление

На разрабатываемой диаграмме классов выбран текстовый способ изображения стереотипов классов, при котором стереотип записывается в угловых кавычках выше имени соответствующего класса. Rational Rose позволяет альтернативно представлять стереотипы в форме специальных графических изображений (как в браузере проекта) или в форме небольших декоративных значков в верхней секции прямоугольника класса на диаграмме, а также вообще отказаться от изображения стереотипов.

Изменить изображение стереотипа для отдельного класса можно, например, с помощью одной из вложенных операций контекстного меню: **Options**→**Stereotype Display** (Параметры→Изображение стереотипа). В качестве примера можно представить изображение класса *Транзакция Банкомата* в форме специальной графической пиктограммы стереотипа. С этой целью следует выполнить операцию контекстного меню: **Options**→**Stereotype Display**→**Icon** (Параметры→Изображение стереотипа→Пиктограмма). Соответствующее графическое изображение стереотипа <<entity>> для класса Транзакция Банкомата в форме пиктограммы будет иметь следующий вид (рис. 2.3.7а).

Для сравнения можно выбрать изображение класса Транзакция Банкомата в форме декоративного графического стереотипа. С этой целью выполним операцию контекстного меню: **Options**→**Stereotype Display**→**Decoration** (Параметры→Изображение стереотипа→Декорация). Соответствующее графическое изображение стереотипа <<entity>> для класса Транзакция Банкомата в форме декорации будет иметь следующий вид (рис. 2.3.7б).

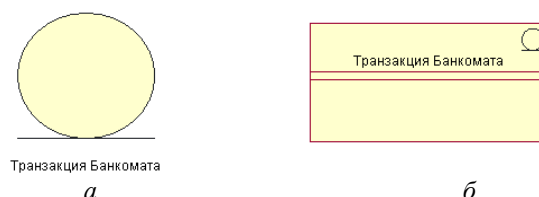


Рис. 2.3.7. Графические способы изображения стереотипа <<entity>>

Изменить изображение стереотипов одновременно для нескольких классов диаграммы можно с помощью одной из вложенных операций главного меню: **Format→Stereoype Display** (Формат→Изображение стереотипов). В этом случае необходимо выделить все классы модели в окне диаграммы классов или в браузере проекта. Для выделения группы классов на диаграмме или в браузере проекта следует, удерживая нажатой клавишу **Ctrl** или **Shift** на клавиатуре, последовательно щелкать на их изображении левой кнопкой мыши.

Выделить все графические элементы на диаграмме классов, также как и на любой другой диаграмме модели, можно с помощью выполнения операции главного меню: **Edit→Select All** (Редактирование→Выделить все) или с помощью комбинации клавиш **Ctrl+A**. Следует отметить, что выбор того или иного способа изображения стереотипов классов на диаграмме классов определяется разработчиком исходя из его личных предпочтений, и не оказывает влияния на содержательный аспект логического представления модели.

Продолжая разработку модели банкомата, добавим на диаграмму второй класс с именем Контроллер Банкомата, для которого в окне спецификации свойств выберем стереотип **control** (управляющий класс), а в качестве документации введем текст: "Реализует логику функционирования банкомата". При этом атрибуты и операции у данного класса будут отсутствовать. Соответствующий фрагмент диаграммы классов после добавления управляющего класса *Контроллер Банкомата* будет иметь следующий вид (рис. 2.3.8).

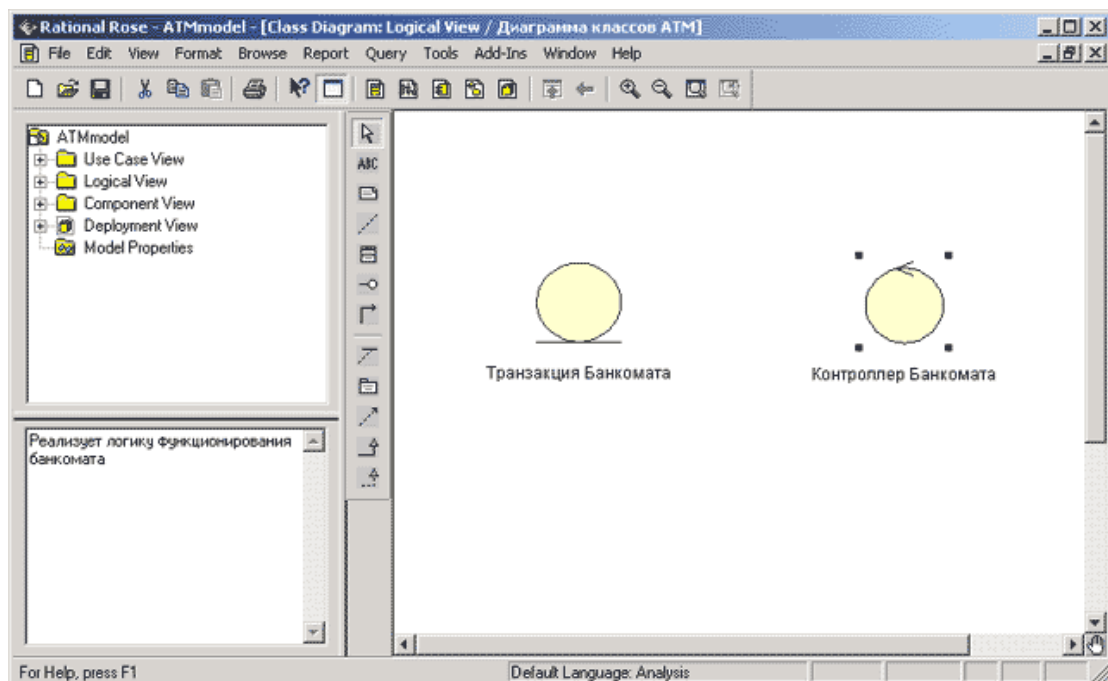


Рис. 2.3.8. Фрагмент диаграммы классов модели банкомата после добавления на нее класса Контроллер Банкомата

Продолжая разработку модели банкомата, добавим на диаграмму третий класс с именем *Устройство чтения карточки*, для которого в окне спецификации свойств выберем стереотип **boundary** (граничный класс). Применение этого стереотипа означает, что данный класс находится на границе моделируемой системы, в качестве которой рассматривается модель банкомата. После этого в секцию документации данного класса можно ввести поясняющий текст: "Устанавливается на банкомате".

Далее следует добавить класс с именем *Интерфейс Банка*, для которого выбрать стереотип **Interface** (Интерфейс), означающий, что банкомат пользуется услугами Банка при обработке своих транзакций. Заметим, что первой буквой в имени этого класса является английское "I", которое служит в языке UML для указания интерфейса. Соответствующий

фрагмент диаграммы классов после добавления на нее классов *Устройство чтения карточки* и *IKонтроллер Банка* будет иметь следующий вид (рис. 2.3.9).

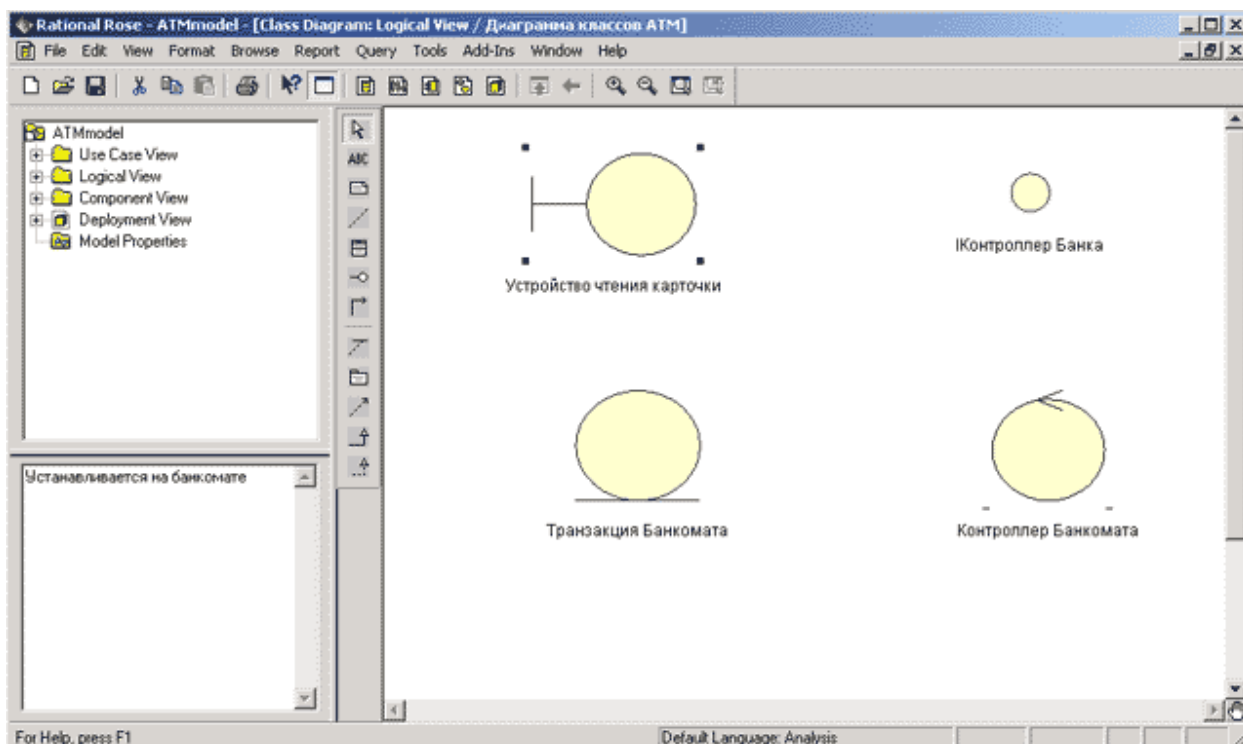


Рис. 2.3.9. Фрагмент диаграммы классов модели банкомата после добавления на нее классов *Устройство чтения карточки* и *Контролер Банкомата*

### Добавление и редактирование атрибутов классов

Из всех графических элементов среды IBM Rational Rose класс обладает максимальным набором свойств, главными из которых являются его атрибуты и операции. Поскольку именно диаграмма классов используется в среде IBM Rational Rose для генерации программного кода, будут подробно рассмотрены соответствующие свойства атрибутов и операций.

Добавить атрибут к созданному ранее классу можно одним из следующих способов:

- С помощью операции контекстного меню **New Attribute** (Новый атрибут) для класса, выделенного на диаграмме классов. В этом случае активизируется курсор ввода текста в области графического изображения класса на диаграмме.
- С помощью операции контекстного меню: **New→Attribute** (Новый→Атрибут) для класса, выделенного в браузере проекта. В этом случае активизируется курсор ввода текста в области иерархического представления класса в браузере проекта под именем соответствующего класса.
- С помощью операции контекстного меню **Insert** (Вставить), вызванного при позиционировании курсора в области открытой вкладки атрибутов в диалоговом окне свойств **Class Specification** соответствующего класса.

После добавления атрибута к классу по умолчанию ему присваивается имя name и некоторый квантор видимости (рис. 2.3.10).

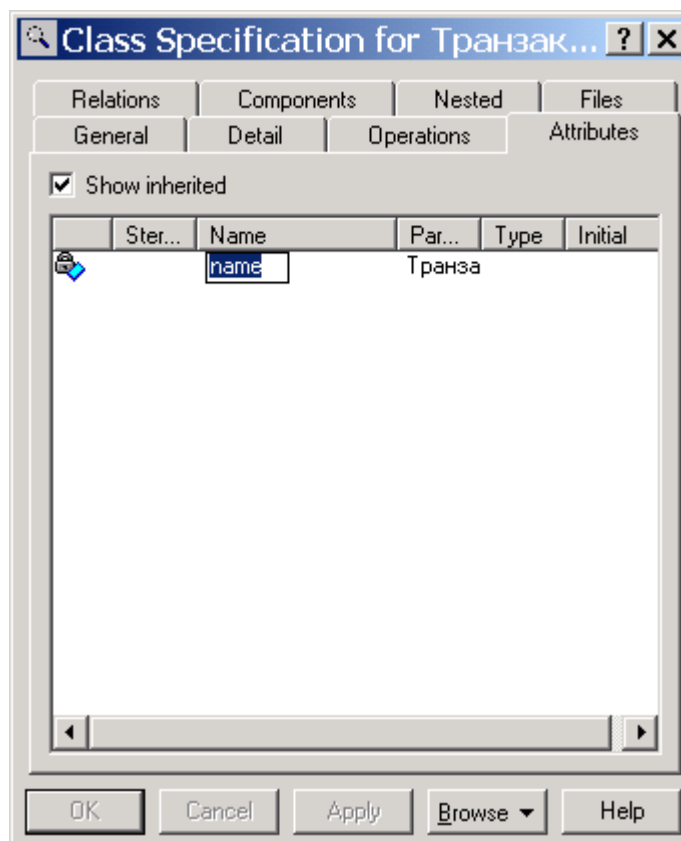


Рис. 2.3.10. Диалоговое окно спецификации свойств класса после добавления нового атрибута

Для рассматриваемой модели банкомата имя добавленного атрибута следует изменить на идентификатор карточки. Напомним, что имена атрибутов и операций классов должны начинаться со строчной буквы. Видимость атрибутов на диаграмме классов изображается в форме специальных пиктограмм или украшений. Используемые пиктограммы видимости изображаются перед именем соответствующего атрибута и имеют следующий смысл (Табл.19).

Таблица 19.

Пиктограммы видимости атрибутов классов

| Иконка | Текстовый аналог | Назначение пиктограммы                                                                 |
|--------|------------------|----------------------------------------------------------------------------------------|
|        | Public           | Общедоступный или открытый. В нотации языка UML такому атрибуту соответствует знак «+» |
|        | Protected        | Защищенный. В нотации языка UML такому атрибуту соответствует знак «#»                 |
|        | Private          | Закрытый. В нотации языка UML такому атрибуту соответствует знак «-»                   |
|        | Implementation   | Реализация. В нотации языка UML такому атрибуту соответствует знак «~»                 |

Для редактирования свойств атрибутов предназначено специальное диалоговое окно спецификации атрибута **Class Attribute Specification**, которое открывается двойным щелчком мыши на строке выбранного атрибута в окне спецификации свойств класса. В окне свойств отдельного атрибута класса можно задать тип данных атрибута и его начальное значение, а также назначить атрибуту стереотип из раскрывающегося списка или изменить его квантор видимости.

Для атрибута идентификатор карточки в качестве типа его допустимых значений из вложенного списка **Type** следует выбрать тип **Integer** (целочисленный), а для задания квантора видимости следует выбрать в группе **Export Control** (Управление экспортом) квантор **Public**. Поскольку начальное значение для данного атрибута не определено, соответствующее поле ввода следует оставить пустым. В секцию документации данного

атрибута класса можно ввести поясняющий текст: «Устройство чтения карточки считывает значение этого атрибута с кредитной карточки клиента» и нажать кнопку **Apply** или **OK**, чтобы сохранить результаты редактирования этих свойств атрибута.

Для отдельного атрибута можно также определить дополнительные свойства, доступные для редактирования на вкладке **Detail** (Подробно) диалогового окна спецификации свойств выбранного атрибута (рис. 2.3.11).

На вкладке **Detail** в группе выбора **Containment** (Локализация) можно специфицировать условия хранения атрибута у объектов выбранного класса. Для выбора могут быть использованы следующие свойства:

- *By value* (По значению) - свойство по умолчанию, которое означает, что значения атрибута хранятся в пределах адресного пространства, выделенного для объекта данного класса. Например, если имеется атрибут типа String, то значение этой строки содержится в пределах определения класса.
- *By reference* (По ссылке) - означает, что значение атрибута хранится вне адресного пространства, выделенного для объекта данного класса, но у объектов класса имеется указатель на этот атрибут.
- *Unspecified* (Не определен) - означает, что метод локализации данного атрибута не определен. В этом случае при генерации программного кода для данного атрибута по умолчанию выбирается значение *By value*.

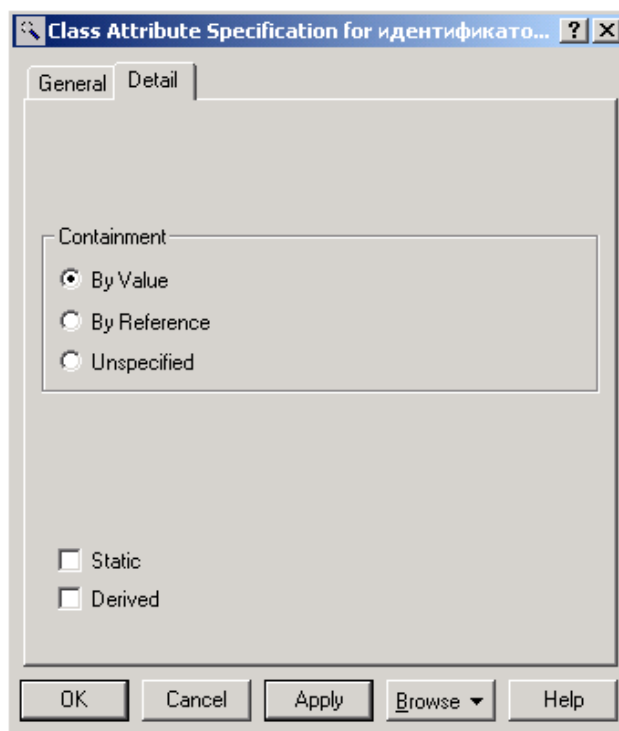


Рис. 2.3.11. Диалоговое окно спецификации свойств атрибута идентификатор карточки, открытое на вкладке **Detail** (Подробно)

Далее можно определить атрибут как статичный, выставив отметку в строке выбора **Static**. Статичный атрибут по определению имеет одно и тоже значение для всех объектов рассматриваемого класса. Наконец, на вкладке **Detail** можно определить атрибут как производный, выставив отметку в строке выбора **Derived**. Значение производного атрибута по определению может быть вычислено на основании значений других атрибутов этого или другого класса.

### Добавление и редактирование операций классов

Функционирование банкомата основано на выполнении отдельными его устройствами тех или иных действий. В модели структуры банкомата все действия представляются с помощью операций классов. Таким образом, следующий этап разработки диаграммы классов связан со спецификацией операций классов.

Добавить операцию к созданному ранее классу можно одним из следующих способов:

- С помощью операции контекстного меню **New Operation** (Новая операция) для класса, выделенного на диаграмме классов. В этом случае активизируется курсор ввода в области графического изображения класса на диаграмме.
- С помощью операции контекстного меню: **New→Operation** (Новая→Операция) для класса, выделенного в браузере проекта. В этом случае активизируется курсор ввода в области иерархического представления класса в браузере под именем соответствующего класса.
- С помощью операции контекстного меню **Insert** (Вставить), вызванного при позиционировании курсора в области открытой вкладки операций в диалоговом окне свойств **Class Specification** соответствующего класса.

После добавления операции к классу по умолчанию ей присваивается имя `opname` и некоторый квантор видимости. Видимость операций на диаграмме классов также изображается в форме специальных пиктограмм или украшений. Используемые пиктограммы видимости изображаются перед именем соответствующей операции и имеют следующий смысл (Табл.20).

Таблица 20.

Пиктограммы видимости операций классов

| Кнопка                                                                              | Текстовый аналог | Назначение пиктограммы                                                                 |
|-------------------------------------------------------------------------------------|------------------|----------------------------------------------------------------------------------------|
|  | Public           | Общедоступный или открытый. В нотации языка UML такому атрибуту соответствует знак «+» |
|  | Protected        | Защищенный. В нотации языка UML такому атрибуту соответствует знак «#»                 |
|  | Private          | Закрытый. В нотации языка UML такому атрибуту соответствует знак «-»                   |
|  | Implementation   | Реализация. В нотации языка UML такому атрибуту соответствует знак «~»                 |

В контексте рассматриваемой модели банкомата в качестве имени первой операции для класса *Транзакция Банкомата* следует задать: создать новую транзакцию. При этом скобки при задании имени операции не записываются, поскольку программа IBM Rational Rose добавляет их автоматически. Однако, следуя правилам именования операций в языке UML, далее в тексте имени операций будут указываться со скобками.

Каждая из операций классов имеет собственное диалоговое окно спецификации свойств **Operation Specification**, которое может быть открыто по двойному щелчку на имени операции на соответствующей вкладке спецификации класса или на имени этой операции в браузере проекта. Для операции **создать новую транзакцию()** в качестве квантора видимости следует выбрать из вложенного списка квантор **public**. В секцию документации данной операции класса можно ввести поясняющий текст: «Вызывается после того, как кредитная карточка вставлена в Устройство чтения карточки» и нажать кнопку **Apply** или **OK**, чтобы сохранить результаты редактирования свойств этой операции.

Для операций классов кроме квантора видимости можно также задать: аргументы и их тип, тип возвращаемого результата, стереотип операции, а также определить протокол и размер, задать исключительные ситуации, специфицировать предусловия и постусловия и целый ряд других свойств. Для отдельной операции эти дополнительные свойства доступны для редактирования на вкладке **Detail** (Подробно) диалогового окна спецификации свойств выбранной операции (рис. 2.3.12).

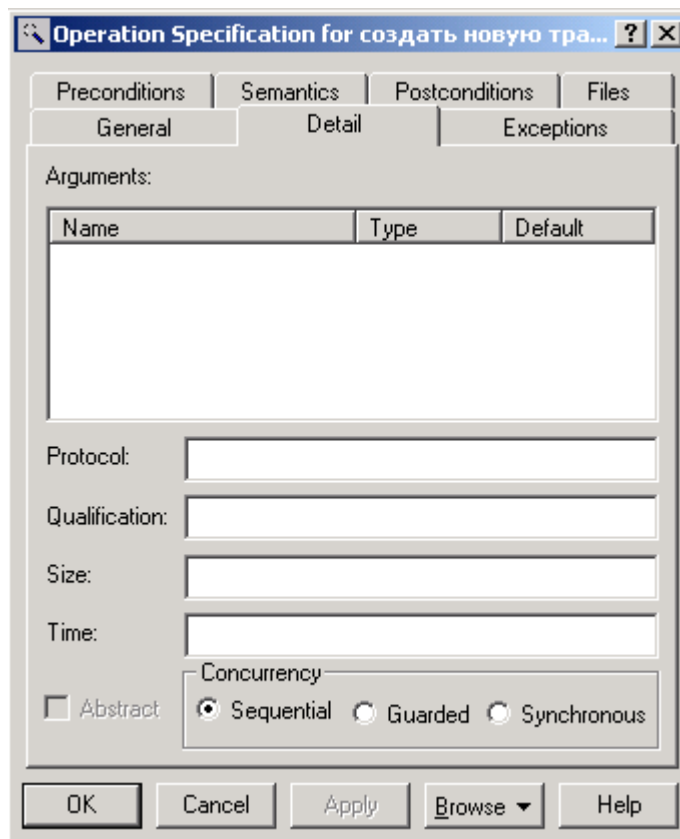


Рис. 2.3.12. Диалоговое окно спецификации свойств операции создать новую транзакцию(), открытое на вкладке **Detail** (Подробно)

На вкладке **Detail** в многостраничном поле **Arguments** (Аргументы) можно определить аргументы редактируемой операции. Для этого следует выполнить операцию контекстного меню **Insert** (Вставить). После этого в этом поле появится аргумент данной операции с именем по умолчанию argname. Для редактирования свойств аргумента предназначено специальное окно свойств аргумента.

На вкладке **Detail** в поле **Protocol** (Протокол) можно специфицировать порядок выполнения операций класса, например, указать, что одна операция не может быть вызвана раньше другой. Соответствующий текст в данное поле вводится с клавиатуры и попадает в генерируемый код в форме комментария. В поле **Qualification** (Квалификация) можно уточнить детали реализации операции, связанные с конкретным языком программирования. Соответствующий текст также вводится в данное поле с клавиатуры и попадает в генерируемый код в форме комментария.

Далее на этой же вкладке в полях **Size** (Размер) и **Time** (Время) можно специфицировать предполагаемый объем памяти и время, необходимое для выполнения операции. Соответствующая информация попадает в генерируемый код в форме комментария.

В группе выбора **Concurrency** (Параллельность) можно специфицировать условия на возможность параллельного выполнения данной операции. Для выбора могут быть использованы следующие свойства:

- *Sequential* (Последовательная) - свойство по умолчанию, которое означает, что данная операция класса может быть выполнена только при наличии одного потока управления, т. е. соответствующая операция класса должна выполняться последовательно. При наличии нескольких потоков управления выполнение данной операции класса не гарантируется.
- *Guarded* (Безопасная) - означает, что при наличии нескольких потоков управления выполнение данной операции класса гарантируется только в том случае, когда обеспечено взаимодействие объектов друг с другом в различных потоках.

- *Synchronous* (Синхронная) - означает, что выполнение данной операции класса гарантируется при наличии нескольких потоков управления. При этом нет необходимости во взаимодействии объектов в различных потоках управления, поскольку данная операция класса будет выполняться в отдельном потоке управления вплоть до своего завершения.

Применительно к рассматриваемой модели для операции **создать новую транзакцию()** следует выбрать свойство **Sequential**, а поля всех других свойств оставить пустыми.

Чтобы закончить спецификацию класса *Транзакция Банкомата* аналогичным способом следует добавить еще 3 атрибута и 2 операции со следующими свойствами:

- *значение ПИН-кода* карточки с квантором видимости **public**. В качестве типа этого атрибута следует выбрать тип **Integer** (целочисленный), а в секцию документации атрибута ввести поясняющий текст: «Устройство чтения карточки считывает значение этого атрибута с кредитной карточки клиента».

- *введенный ПИН-код* с квантором видимости **public**. В качестве типа этого атрибута следует выбрать тип **Integer** (целочисленный), а в секцию документации атрибута ввести поясняющий текст: «Значение этого атрибута вводится клиентом с клавиатуры банкомата».

- *введенная сумма наличных* с квантором видимости **public**. В качестве типа этого атрибута следует выбрать тип **Currency** (Денежный), а в секцию документации атрибута ввести поясняющий текст: «Значение этого атрибута вводится клиентом с клавиатуры банкомата».

- *проверить правильность ПИН-кода()* с квантором видимости **public**. В качестве типа возвращаемого результата для этой операции следует выбрать тип **Boolean** (логический), а в секцию ее документации ввести поясняющий текст: «Вызывается после того, как клиент ввел значение ПИН-кода с клавиатуры банкомата».

- *завершить транзакцию()* с квантором видимости **public**. В секцию ее документации ввести поясняющий текст: «Вызывается после завершения всех действий банкомата по обслуживанию клиента».

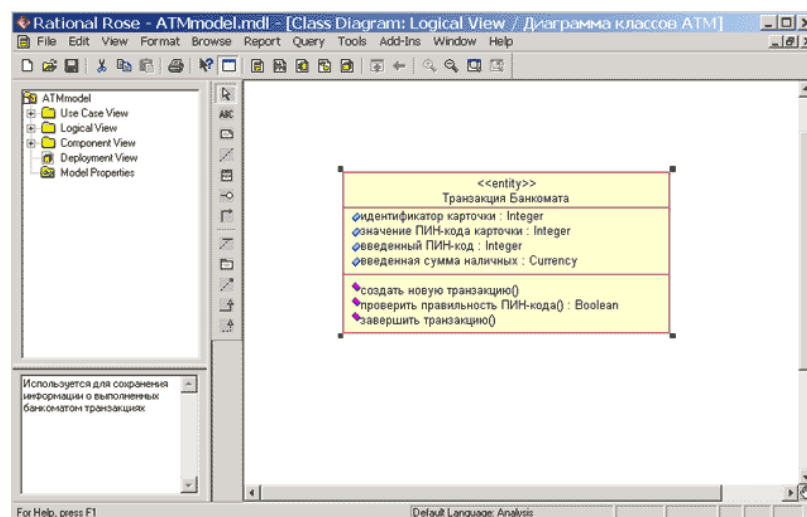


Рис. 2.3.13. Фрагмент диаграммы классов модели банкомата после добавления атрибутов и операций для класса *Транзакция банкомата*

### Добавление ассоциации на диаграмму классов

Добавление на диаграмму ассоциации между двумя классами выполняется следующим образом. На специальной панели инструментов необходимо нажать кнопку с изображением пиктограммы направленной ассоциации и отпустить левую кнопку мыши. Если ассоциация - направленная, то на диаграмме классов надо выделить первый элемент ассоциации или источник, от которого исходит стрелка, и, не отпуская нажатую левую кнопку мыши, переместить ее указатель ко второму элементу отношения или приемнику, к которому направлена стрелка. После перемещения ко второму элементу кнопку мыши следует

отпустить, в результате чего на диаграмму классов будет добавлена направленная ассоциация с именем Untitled между двумя выбранными классами.

Продолжая разработку диаграммы классов модели банкомата, добавим на нее описанным способом направленную ассоциацию между классом *Контроллер Банкомата* и классом *Транзакция Банкомата* (рис. 2.3.14).

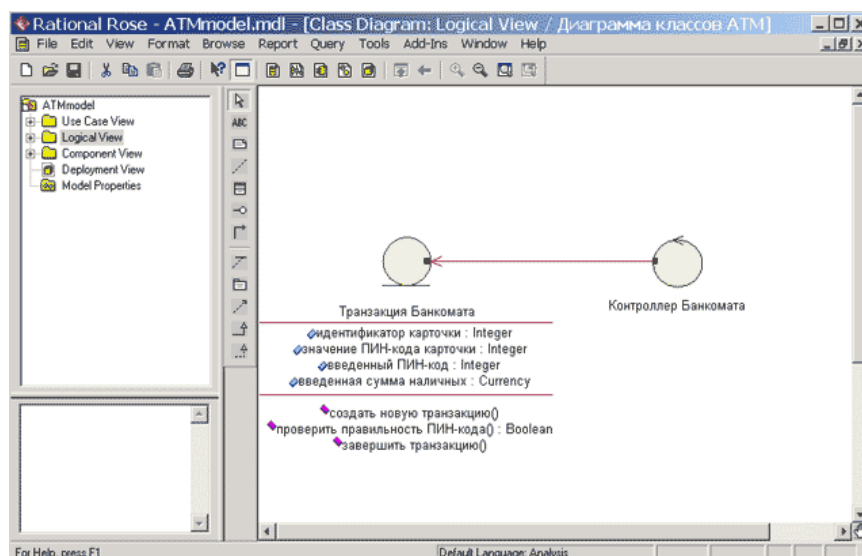


Рис. 2.3.14. Фрагмент диаграммы классов модели банкомата после добавления на неё направленной ассоциации

Изменим имя для данной ассоциации, предложенное средой по умолчанию. Это можно выполнить с помощью окна спецификации свойств ассоциации. Доступ к диалоговому окну спецификации свойств ассоциации **Association Specification** можно получить после выделения линии ассоциации на диаграмме классов или в браузере проекта и двойного щелчка на ней левой кнопки мыши (рис. 2.3.15).

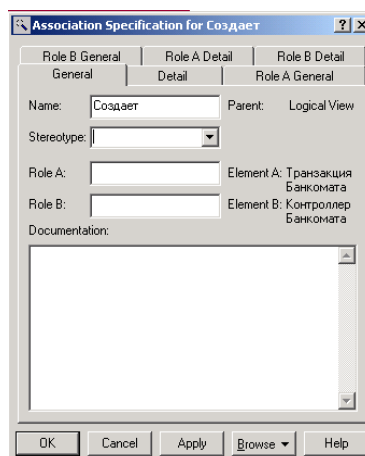


Рис. 2.3.15. Диалоговое окно спецификации свойств ассоциации

Для задания имени ассоциации следует на вкладке **General** (Общие) в поле ввода **Name** (Имя) ввести текст ее имени: *Создает* и нажать кнопку **Apply** или **OK**, чтобы сохранить результаты редактирования имени ассоциации. Для ассоциации можно задать также кратность каждого из концов ассоциации, стереотип, использовать ограничения и роли, а также некоторые другие свойства.

Для добавленной на диаграмму классов ассоциации зададим кратность конца ассоциации у класса *Контроллер Банкомата*, равную 1. Для этого следует в окне спецификации свойств

ассоциации перейти на вкладку **Role B Detail** и выбрать значение 1 из вложенного списка **Multiplicity**. Аналогичным образом следует задать кратность конца ассоциации у класса *Транзакция Банкомата* равную 1..n, для чего на вкладке **Role A Detail** и следует выбрать значение 1..n из вложенного списка **Multiplicity**. Содержательно это будет означать, что каждый объект класса *Контроллер Банкомата* может быть связан с одним или несколькими объектами класса *Транзакция Банкомата*.

Если ассоциация является ненаправленной, то порядок выбора классов может быть произвольный, а после добавления ассоциации на диаграмму классов следует изменить значение соответствующего свойства данной ассоциации. С этой целью необходимо перейти на вкладку **Role A Detail** в окне спецификации свойств ассоциации и убрать отметку у свойства **Navigable** (Навигация).

### Добавление отношений агрегации и композиции

Добавить на диаграмму отношение агрегации между двумя классами можно следующими способами:

- Щелкнуть на кнопке с изображением отношение агрегации на специальной панели инструментов и провести линию агрегации от одного класса к другому.
- Провести линию ассоциации между выбранными классами и изменить ее свойства таким образом, чтобы превратить данную ассоциацию в агрегацию.

В первом случае может оказаться, что по умолчанию на специальной панели инструментов диаграммы классов отсутствует кнопка с пиктограммой агрегации. В этом случае необходимо предварительно добавить ее на панель инструментов одним из описанных ранее способов. Во втором случае следует открыть окно спецификации свойств ассоциации **Association Specification** и на вкладке деталей соответствующего конца ассоциации выставить отметку в строке выбора **Aggregate** (Агрегация).

В качестве примера изменим тип созданной ранее ассоциации и сделаем ее агрегацией. Содержательно это будет означать, что класс *Контроллер Банкомата* будет включать в себя в качестве составной части класс *Транзакция Банкомата*; при этом уничтожение любого объекта класса *Контроллер Банкомата* не должно привести к уничтожению ассоциированных с ним объектов класса *Транзакция Банкомата*. С этой целью на вкладке **Role B Detail** деталей конца ассоциации класса *Контроллер Банкомата* следует выставить отметку в строке выбора **Aggregate** (рис. 2.3.16).

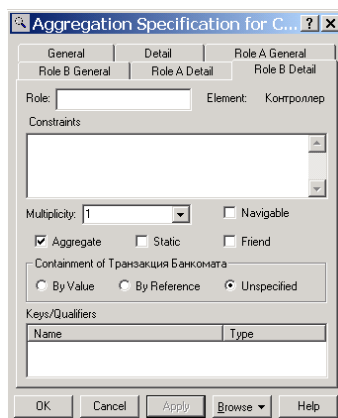


Рис. 2.3.16. Диалоговое окно спецификации свойств ассоциации

Соответствующий фрагмент диаграммы классов после изменения ассоциации между классами *Контроллер Банкомата* и *Транзакция Банкомата* на отношение агрегации будет иметь следующий вид (рис. 2.3.17).

Для изображения отношения композиции можно также вначале изобразить обычную ассоциацию, после чего, открыв окно ее свойств на вкладке деталей соответствующего конца ассоциации (рис. 2.3.17), выставить отметку в строке выбора **Aggregate** (Агрегация) и в

секции **Containment** (Локализация) выбрать опцию **By Value** (По значению). По умолчанию эта опция не специфицирована, т.е. выставлена отметка опции **Unspecified**.

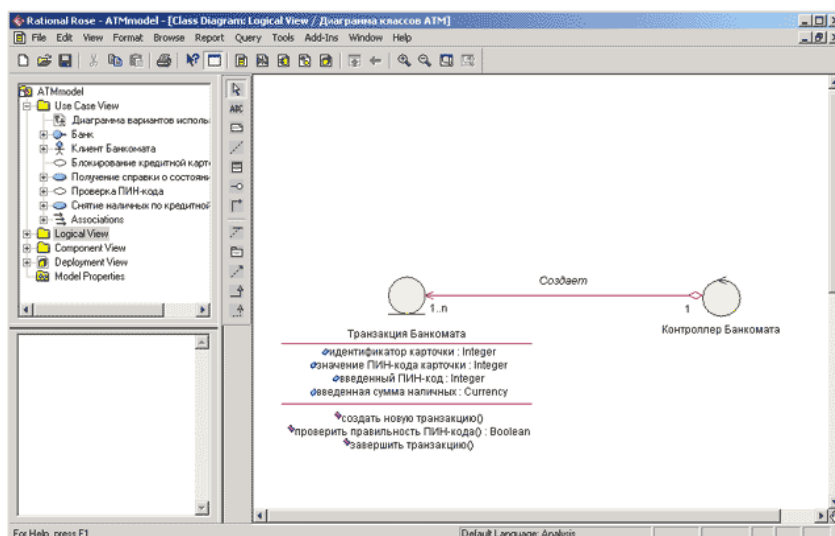


Рис. 2.3.17. Фрагмент диаграммы классов модели банкомата после добавления на нее отношения агрегации

### Добавление отношения обобщения

Добавление на диаграмму отношения обобщения между двумя классами выполняется следующим образом. На специальной панели инструментов необходимо нажать кнопку с изображением пиктограммы обобщения и отпустить левую кнопку мыши. Далее на диаграмме классов надо выделить первый элемент обобщения или потомок, от которого исходит стрелка, и, не отпуская нажатую левую кнопку мыши, переместить ее указатель ко второму элементу отношения или предку, к которому направлена стрелка. После перемещения ко второму элементу кнопку мыши следует отпустить, в результате чего на диаграмму классов будет добавлена линия обобщения с именем *Untitled* между двумя выбранными классами.

Продолжая разработку диаграммы классов модели банкомата, добавим на нее описанным способом направленную ассоциацию между классом *Контроллер Банкомата* и дополнительно созданным абстрактным классом *Контроллер* (рис. 2.3.18). Последний класс может быть предназначен для спецификации системных атрибутов и операций, необходимых при исполнении соответствующей программы. Напомним, что на абстрактный характер класса указывает написание курсивом его имени, а для спецификации данного свойства класса необходимо на вкладке **Detail** (Подробно) окна спецификации свойств класса *Контроллер* выставить отметку в строке выбора **Abstract**.

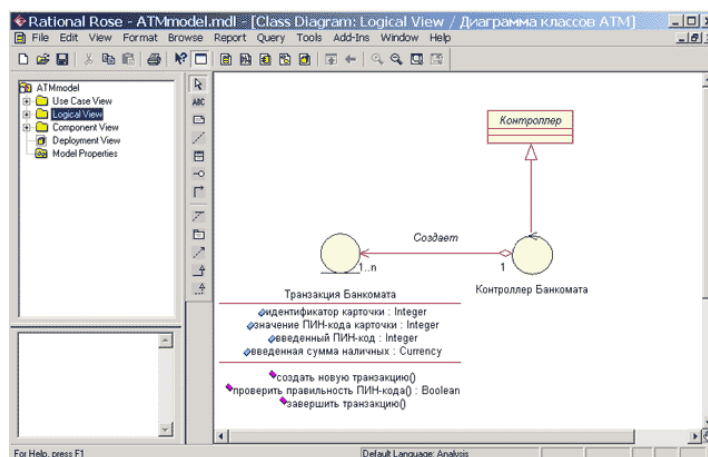


Рис. 2.3.18. Диаграмма классов модели банкомата после добавления на неё отношения обобщения

Изменим имя отношения обобщения, предложенное средой по умолчанию. Это можно выполнить с помощью окна спецификации свойств обобщения. Доступ к диалоговому окну спецификации свойств отношения обобщения **Generalize Specification** можно получить после выделения линии обобщения на диаграмме классов или в браузере проекта и двойного щелчка на ней левой кнопки мыши (рис. 2.3.19).

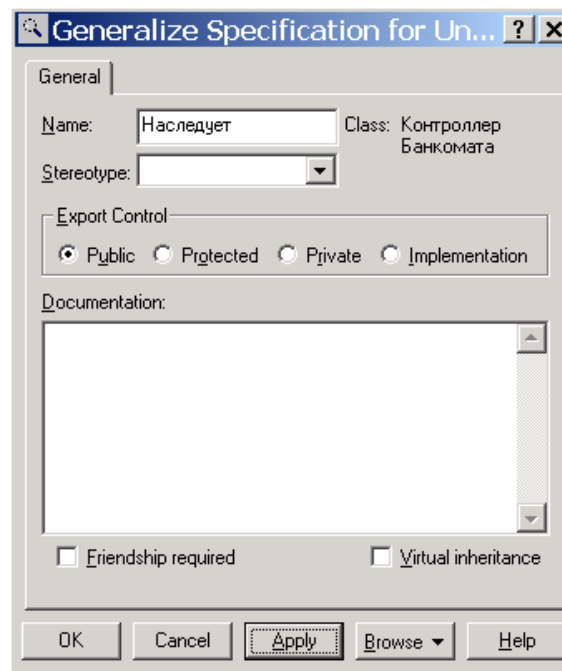


Рис. 2.3.19. Диалоговое окно спецификации свойств отношения обобщения

Для задания имени обобщения следует на единственной вкладке **General** (Общие) в поле ввода **Name** (Имя) ввести текст ее имени: Наследует и нажать кнопку **Apply** или **OK**, чтобы сохранить результаты редактирования имени ассоциации.

#### Завершение диаграммы классов модели банкомата

Для окончательного построения диаграммы классов рассматриваемой модели банкомата следует описанным выше способом добавить оставшиеся классы и ассоциации, а также специфицировать стереотипы, атрибуты и операции этих классов. С этой целью следует выполнить следующие действия:

1. Для класса *Интерфейс Банка* добавить операцию: проверить идентификатор карточки (идентификатор карточки: **Integer**) с квантором видимости **public**. В качестве типа возвращаемого результата для этой операции следует выбрать тип **Boolean** (логический), а в качестве целочисленного аргумента задать идентификатор карточки. Для задания аргумента необходимо перейти на вкладку **Detail** (Подробно) окна спецификации свойств данной операции и после добавления аргумента с помощью операции контекстного меню Insert ввести имя аргумента и его тип **Integer** в соответствующие поля ввода.

2. Для класса *Интерфейс Банка* добавить операцию: открыть счет клиента (идентификатор карточки: *Integer*) с квантором видимости **public**. В качестве целочисленного аргумента этой операции следует задать идентификатор карточки.

3. Для класса *Интерфейс Банка* добавить операцию: проверить баланс клиента (идентификатор карточки: **Integer**, введенная сумма наличных: **Currency**) с квантором видимости **public**. В качестве типа возвращаемого результата для этой операции следует выбрать тип **Boolean** (логический). В качестве первого целочисленного аргумента этой операции следует задать идентификатор карточки, а в качестве второго аргумента - введенная сумма наличных с типом **Currency** (Денежный).

4. Для класса *Интерфейс Банка* добавить операцию: уменьшить счет клиента (идентификатор карточки: **Integer**, введенная сумма наличных: **Currency**) с квантором

видимости **public**. В качестве типа возвращаемого результата для этой операции следует выбрать тип **Boolean** (логический). В качестве первого целочисленного аргумента этой операции следует задать идентификатор карточки, а в качестве второго аргумента - введенная сумма наличных с типом **Currency** (Денежный).

5. Для класса *Устройство чтения карточки* добавить операцию: *прочитать идентификатор карточки()* с квантором видимости **public**. В качестве типа возвращаемого результата для этой операции следует выбрать тип **Integer** (целочисленный), а в секцию документации данной операции следует ввести поясняющий текст: «Вызывается после того, как кредитная карточка вставлена в Устройство чтения карточки».

6. Для класса *Устройство чтения карточки* добавить операцию: *прочитать ПИН-код()* с квантором видимости **public**. В качестве типа возвращаемого результата для этой операции следует выбрать тип **Integer** (целочисленный), а в секцию документации данной операции следует ввести поясняющий текст: «Вызывается после того, как кредитная карточка вставлена в Устройство чтения карточки».

7. Для класса *Устройство чтения карточки* добавить операцию: *вернуть кредитную карточку()* с квантором видимости **public**. В секцию документации данной операции следует ввести поясняющий текст: «Вызывается после завершения транзакции».

8. Для класса *Устройство чтения карточки* добавить операцию: *блокировать кредитную карточку()* с квантором видимости **public**. В секцию документации данной операции следует ввести поясняющий текст: «Вызывается после того, как установлен факт утраты кредитной карточки владельцем».

9. Добавить класс с именем *Экран Банкомата*, для которого выбрать стереотип **boundary**. Данный класс также находится на границе моделируемой системы, на что и указывает этот стереотип. В секцию документации данного класса следует ввести поясняющий текст: «Устанавливается на банкомате».

10. Для класса *Экран Банкомата* добавить операцию: *показать меню опций()* с квантором видимости **public**.

11. Для класса *Экран Банкомата* добавить операцию: *показать меню снятия суммы()* с квантором видимости **public**.

12. Добавить класс с именем *Клавиатура Банкомата*, для которого выбрать стереотип **boundary**. В секцию документации данного класса следует ввести поясняющий текст: «Устанавливается на банкомате».

13. Для класса *Клавиатура Банкомата* добавить операцию: *ввести ПИН-код()* с квантором видимости **public**. В качестве типа возвращаемого результата для этой операции следует выбрать тип **Integer**, а в секцию документации данной операции следует ввести поясняющий текст: «Вызывается после того, как клиент ввел значение ПИН-кода с клавиатуры».

14. Для класса *Клавиатура Банкомата* добавить операцию: *ввести тип транзакции()* с квантором видимости **public**. В качестве типа возвращаемого результата для этой операции следует выбрать тип **Boolean** (логический), а в секцию документации данной операции следует ввести поясняющий текст: «Возвращает значение Истина, если клиент выбирает снятие наличных, и значение Ложь, если клиент выбирает получение справки о состоянии счета».

15. Для класса *Клавиатура Банкомата* добавить операцию: *ввести сумму снятия наличных()* с квантором видимости **public**. В качестве типа возвращаемого результата для этой операции следует выбрать тип **Currency** (Денежный), а в секцию документации данной операции следует ввести поясняющий текст: «Вызывается после того, как клиент ввел значение снимаемой суммы с клавиатуры».

16. Добавить класс с именем *Устройство выдачи наличных*, для которого выбрать стереотип **boundary**. В секцию документации данного класса следует ввести поясняющий текст: «Устанавливается на банкомате».

17. Для класса *Устройство выдачи наличных* добавить операцию: *выдать наличные()* с квантором видимости **public**. В секцию документации данной операции следует ввести

поясняющий текст: «Вызывается после того, как выполнено снятие запрошенной клиентом суммы со счета».

18. Добавить класс с именем *Принтер Банкомата*, для которого выбрать стереотип **boundary**. В секцию документации данного класса следует ввести поясняющий текст: «Устанавливается на банкомате».

19. Для класса *Принтер Банкомата* добавить операцию: *распечатать чек()* с квантором видимости **public**. В секцию документации данной операции следует ввести поясняющий текст: «Вызывается по дополнительному запросу клиента».

20. Добавить направленную ассоциацию от класса *Контроллер Банкомата* к классу *Устройство чтения карточки*. В качестве кратности концов этой ассоциации установить значение 1.

21. Добавить направленную ассоциацию от класса *Контроллер Банкомата* к классу *Принтер Банкомата*. В качестве кратности концов этой ассоциации установить значение 1.

22. Добавить направленную ассоциацию от класса *Контроллер Банкомата* к классу *Клавиатура Банкомата*. В качестве кратности концов этой ассоциации установить значение 1.

23. Добавить направленную ассоциацию от класса *Контроллер Банкомата* к классу *Устройство выдачи наличных*. В качестве кратности концов этой ассоциации установить значение 1.

24. Добавить направленную ассоциацию от класса *Контроллер Банкомата* к классу *Экран Банкомата*. В качестве кратности концов этой ассоциации установить значение 1.

25. Добавить направленную ассоциацию от класса *Контроллер Банкомата* к классу *IKонтроллер Банка*. В качестве кратности конца этой ассоциации для первого класса установить значение 0..n, а кратности конца ассоциации для второго класса установить значение 1. В качестве стереотипа данной ассоциации выбрать из вложенного списка значение <<communicate>>. Применение данного стереотипа означает, что между этими классами должна существовать физическая взаимосвязь.

Построенная в результате указанных действий диаграмма классов будет иметь следующий вид (рис. 2.3.20).

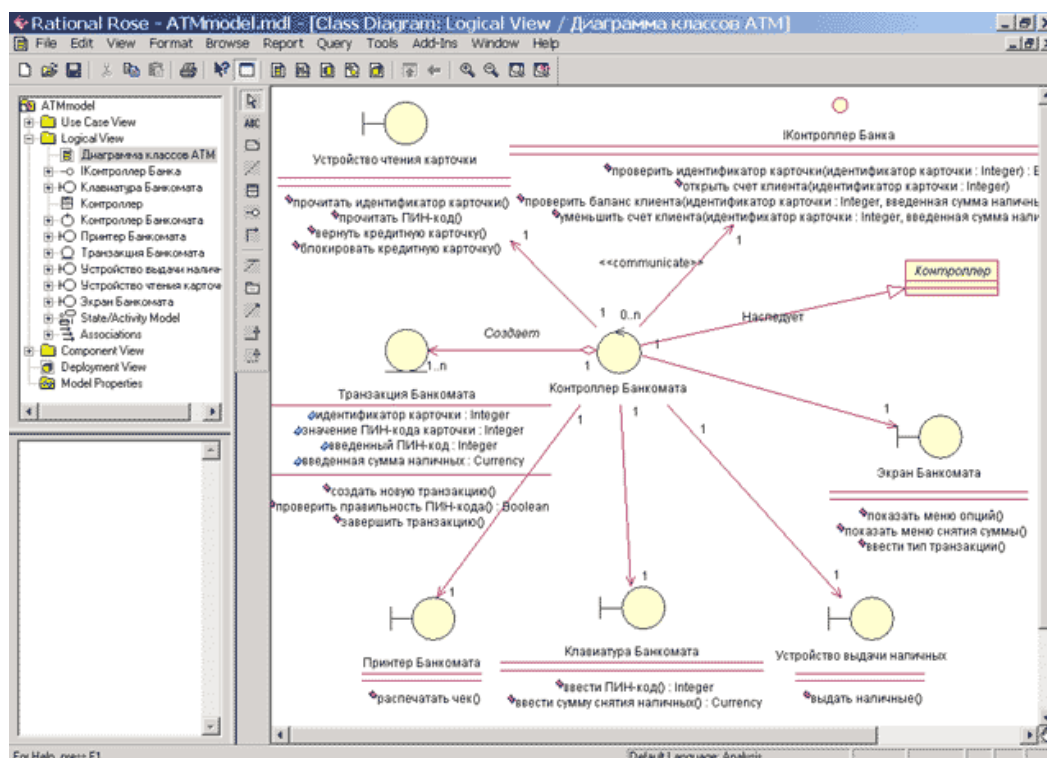


Рис. 2.3.20. Окончательный вид диаграммы классов для разрабатываемой модели банкомата

Следует заметить, что при изображении диаграммы классов все классы представлены в форме графических стереотипов, при этом выбран способ отображения сигнатуры операций классов. Для более компактного представления диаграммы можно убрать отображение атрибутов, операций или сигнатуры операций отдельных классов с помощью соответствующих операций контекстного меню **Options** (Настройка).

### 2.3.2 Задания

1. Выделить основные классы объектов в проектируемой системе.
2. Построить диаграмму классов, в общем виде демонстрирующую архитектуру системы.
3. Построить одну-две диаграммы классов, детализирующие отдельные подсистемы.
4. Указать для классов основные атрибуты и операции, указать вид и направление ассоциаций.

### 2.3.3 Контрольные вопросы

1. Каково назначение диаграмм классов?
2. Для чего используется диаграмма классов на стадии анализа?
3. Для чего используется диаграмма классов на стадии проектирования?
4. Назовите основные компоненты диаграмм классов.
5. Назовите основные типы статических связей между классами.
6. Что представляет собой ассоциация?
7. В чем смысл множественности ассоциаций?
8. В чем отличие атрибутов от ассоциаций?
9. Что такое признак видимости?
10. Что представляет собой операция класса?
11. В чем смысл обобщения?
12. Каково назначение ограничений на диаграммах классов?

## 2.4 Диаграммы взаимодействия: последовательности и кооперации

### Диаграммы взаимодействия (interaction diagrams)

В отличие от некоторых подходов объектного моделирования, когда и состояние, и поведение системы отображаются на диаграммах классов, UML отделяет описание поведения в диаграммы взаимодействия. В UML диаграммы классов не содержат сообщений, которые усложняют их чтение. Поток сообщений между объектами выносится на диаграммы взаимодействия. Как правило, диаграмма взаимодействия охватывает поведение объектов в рамках одного прецедента. На такой диаграмме отображается ряд объектов и те сообщения, которыми они обмениваются между собой в рамках данного прецедента. Существует два вида диаграмм взаимодействия: диаграммы последовательностей (sequence diagrams) и диаграммы кооперации (collaboration diagrams).

### Диаграммы последовательностей (sequence diagrams)

Этот вид диаграмм используется для точного определения логики сценария выполнения прецедента. Диаграммы последовательностей отображают типы объектов, взаимодействующих при исполнении прецедентов, сообщения, которые они посылают друг другу, и любые возвращаемые значения, ассоциированные с этими сообщениями. На диаграмме последовательности объект изображается в виде прямоугольника на вершине пунктирной вертикальной линии. Прямоугольники на вертикальных линиях показывают "время жизни" объекта. Вертикальная линия называется линией жизни (lifeline) объекта. Она представляет собой фрагмент жизненного цикла объекта в процессе взаимодействия. Каждое сообщение представляется в виде стрелки между линиями жизни двух объектов. Сообщения появляются в том порядке, как они показаны на диаграмме (сверху вниз). Каждое сообщение может быть помечено именем, при желании можно показать также аргументы и некоторую управляющую информацию. Также можно показать самоделегирование - сообщение, которое объект посылает самому себе, при этом стрелка сообщения указывает на ту же самую линию жизни.

Из всей возможной управляющей информации два ее вида имеют существенное значение. Во-первых, это условие, показывающее, в каком случае посылается сообщение (например, `[ОтчетУстарел() == true]`). Сообщение посылается только при выполнении данного условия. Другой полезный управляющий маркер - это маркер итерации, показывающий, что сообщение посылается много раз для множества объектов-адресатов (например, `*обновить`).

Активизации - прямоугольники на линиях жизни показывают, когда метод становится активным (во время его выполнения либо при ожидании результата выполнения какой-либо процедуры). Используя механизм активизаций, можно более четко показать смысл самоделегирования. Без них довольно трудно определить, где же выполняются следующие после самоделегирования вызовы - в вызывающем методе или в вызываемом. Активизации вносят ясность в этот вопрос.

Сообщения появляются в той последовательности, как они показаны на диаграмме — сверху вниз. Если предусматривается отправка сообщения объектом самому себе (самоделегирование), то стрелка начинается и заканчивается на одной "линии жизни" (Рис.2.4.1).

На диаграммы может быть добавлена управляющая информация: описание условий, при которых посылается сообщение; признак многократной отправки сообщения (маркер итерации); признак возврата сообщения.

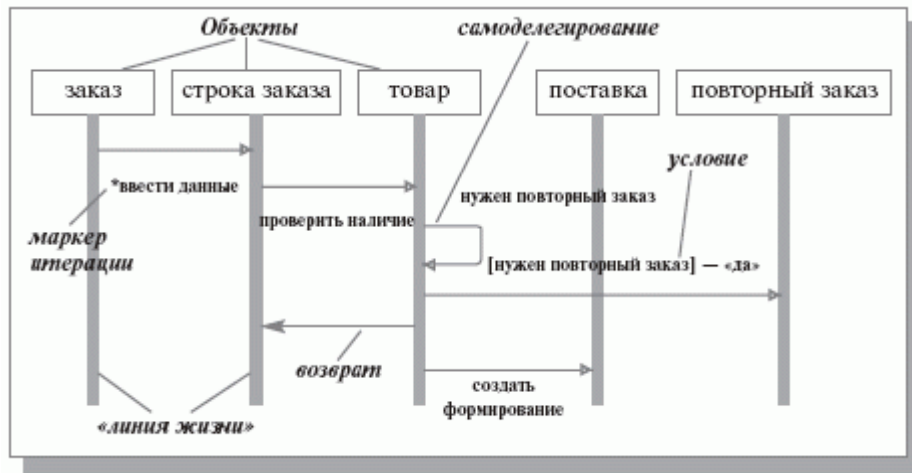


Рис.2.4.1. Диаграмма последовательности обработки заказа

### Диаграммы кооперации (*collaboration diagrams*)

Вторым видом диаграммы взаимодействия является диаграмма кооперации. На этих диаграммах объекты (или классы) показываются в виде прямоугольников, а стрелками обозначаются сообщения, которыми они обмениваются в рамках одного прецедента. Временная последовательность сообщений отражается их нумерацией.

На диаграмме кооперации экземпляры объектов показаны в виде пиктограмм. Линии между ними обозначают сообщения, обмен которыми осуществляется в рамках данного прецедента.

Каждый вид диаграмм взаимодействия имеет свои преимущества, выбор обычно осуществляется исходя из предпочтений разработчика. На диаграммах последовательности делается акцент именно на последовательности сообщений, при этом легче наблюдать порядок, в котором происходят различные события. В случае диаграмм кооперации можно использовать пространственное расположение объектов для того, чтобы показать их статическое взаимодействие.

Одним из главных свойств любой диаграммы взаимодействия является ее простота. Посмотрев на диаграмму, можно легко увидеть все сообщения. Однако при попытке изобразить нечто более сложное, чем единственный последовательный процесс без множества условных переходов или циклов, данный подход может не сработать.

Для отображения условного поведения на диаграммах взаимодействия существует два подхода. Один из них состоит в использовании отдельных диаграмм для каждого сценария. Второй заключается в том, что сообщения сопровождаются условиями, показывающими поведение объектов.

## 2.4.1 Лабораторная работа № 8. Sequence diagrams

### Цель работы:

- изучение диаграмм последовательности,
- изучение их применения в процессе проектирования.

Диаграмма последовательности является другой формой визуализации взаимодействия в модели и, как и диаграмма кооперации, оперирует объектами и сообщениями. Особенность работы в среде IBM Rational Rose заключается в том, что этот вид канонической диаграммы может быть создан автоматически после построения диаграммы кооперации и нажатия клавиши <F5>. С помощью этой же клавиши осуществляется переключение между диаграммами последовательности и кооперации в модели.

Однако в отдельных случаях бывает удобно начать построение диаграмм взаимодействия с диаграммы последовательности. В этом случае активизировать рабочее окно диаграммы последовательности можно несколькими способами:

- Щелкнуть на кнопке с изображением диаграммы взаимодействия на стандартной панели инструментов и выбрать для построения диаграмму последовательности.
- Выполнить операцию главного меню: **Browse→Interaction Diagram** (Браузер→Диаграмма взаимодействия) и выбрать для построения новую диаграмму последовательности.
- Выполнить операцию контекстного меню: **New→Sequence Diagram** (Новая→Диаграмма последовательности) для логического представления или представления прецедентов в браузере проекта.

При этом появляется новое окно с чистым рабочим листом диаграммы классов и специальная панель инструментов, содержащая кнопки с изображением графических примитивов, необходимых для разработки диаграммы последовательности. Назначение отдельных кнопок панели указано в таблице 21, а также можно узнать из всплывающих подсказок.

Таблица 21.

Назначение кнопок специальной панели инструментов диаграммы последовательности

| Кнопка                                                                              | Всплывающая подсказка | Назначение кнопки                                                                                |
|-------------------------------------------------------------------------------------|-----------------------|--------------------------------------------------------------------------------------------------|
|    | Selection Tool        | Превращает изображение курсора в форму стрелки для последующего выделения элементов на диаграмме |
|    | Text Box              | Добавляет на диаграмму текстовую область                                                         |
|  | Note                  | Добавляет на диаграмму примечание                                                                |
|  | Anchor Note to Item   | Добавляет на диаграмму связь примечания с соответствующим графическим элементом диаграммы        |
|  | Object                | Добавляет на диаграмму объект                                                                    |
|  | Object Message        | Добавляет на диаграмму простое сообщение                                                         |
|  | Message To Self       | Добавляет на диаграмму рефлексивное сообщение                                                    |
|  | Return Message        | Добавляет на диаграмму сообщение типа возврата из вызова процедуры                               |
|  | Destruction Marker    | Добавляет на диаграмму символ уничтожения объекта                                                |
|  | Procedure Call        | Добавляет на диаграмму сообщение типа вызова процедуры (по умолчанию отсутствует)                |
|  | Asynchronous Message  | Добавляет на диаграмму асинхронное сообщение (по умолчанию отсутствует)                          |

На специальной панели инструментов по умолчанию присутствует практически все пиктограммы элементов, которые могут быть использованы для построения диаграммы последовательности. Из дополнительных пиктограмм графических элементов на специальную панель инструментов можно добавить лишь сообщение типа вызова процедуры и асинхронное сообщение (последняя строка табл. 21). Относительно изображения асинхронного сообщения в форме полустрелки следует заметить, что хотя в версии языка UML 1.5 этот элемент отсутствует, в среде IBM Rational Rose возможно изобразить этот тип сообщений в форме специального графического стереотипа.

#### Добавление объекта

Добавить объект на диаграмму последовательности можно как стандартным образом с помощью соответствующей кнопки на специальной панели инструментов, так и более удобным способом - с помощью перетаскивания изображения пиктограммы класса из браузера на свободное место рабочего листа диаграммы последовательности.

В результате этих действий на диаграмме последовательности появится изображение объекта с именем класса, маркерами изменения его геометрических размеров и вертикальной пунктирной линией, означающей линию жизни этого объекта (рис. 2.4.2).

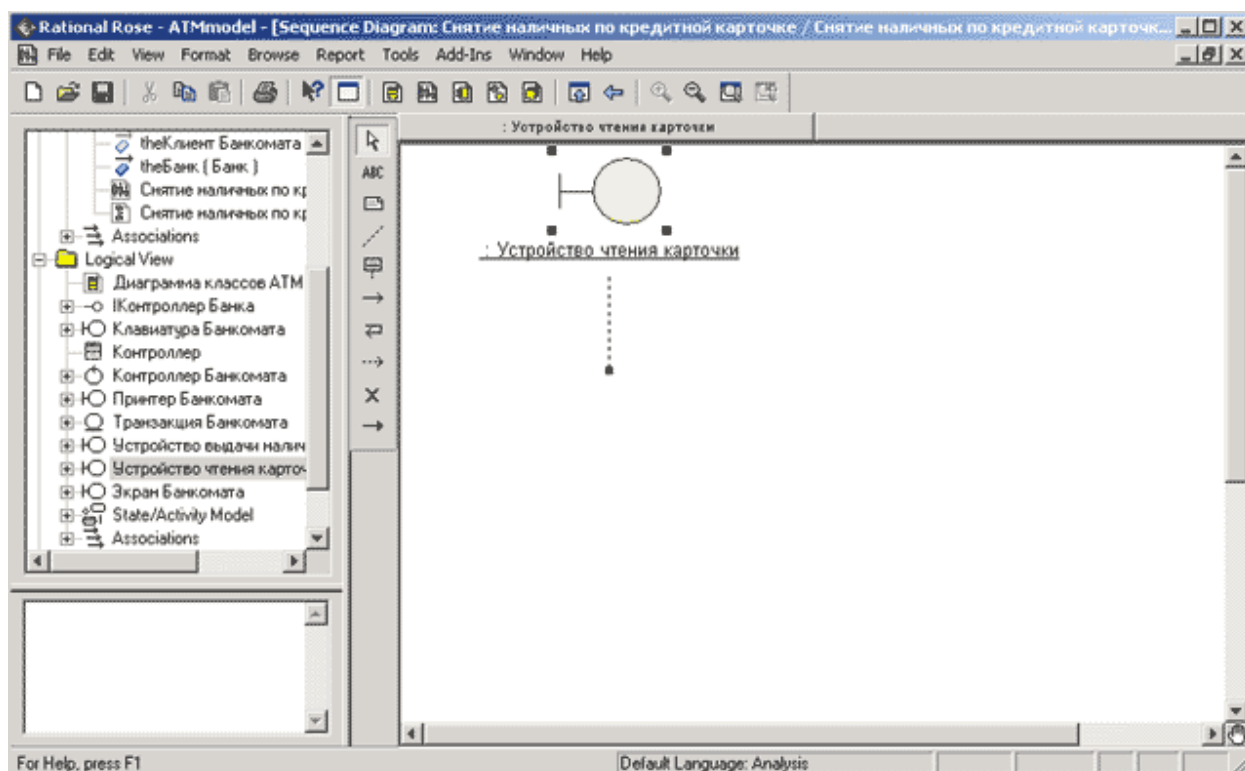


Рис. 2.4.2. Диаграмма последовательности после добавления анонимного объекта класса Устройство чтения карточки

Так же как и для диаграммы кооперации, для диаграммы последовательности каждый добавляемый объект по умолчанию считается анонимным. При необходимости можно задать собственное имя объекта, для чего следует вызвать диалоговое окно свойств объекта (например, двойным щелчком на изображении объекта на диаграмме).

### Добавление сообщения

Для добавления сообщения между предварительно размещенными на диаграмме объектами нужно с помощью левой кнопки мыши нажать кнопку с изображением сообщения на специальной панели инструментов, отпустить левую кнопку мыши, щелкнуть левой кнопкой мыши на изображении линии жизни одного объекта на диаграмме и отпустить ее на изображении линии жизни второго объекта.

В результате этих действий на диаграмме появится изображение сообщения, передаваемого, например, от экземпляра актера *Клиент Банкомата* объекту класса *Устройство чтения карточки*. Поскольку кнопка с изображением актера отсутствует на специальной панели инструментов диаграммы последовательности, соответствующий объект следует предварительно поместить на диаграмму способом перетаскивания пиктограммы актера из браузера проекта. При этом изображение линии жизни у соответствующей пары объектов изменится на изображение фокуса управления (рис. 2.4.3).

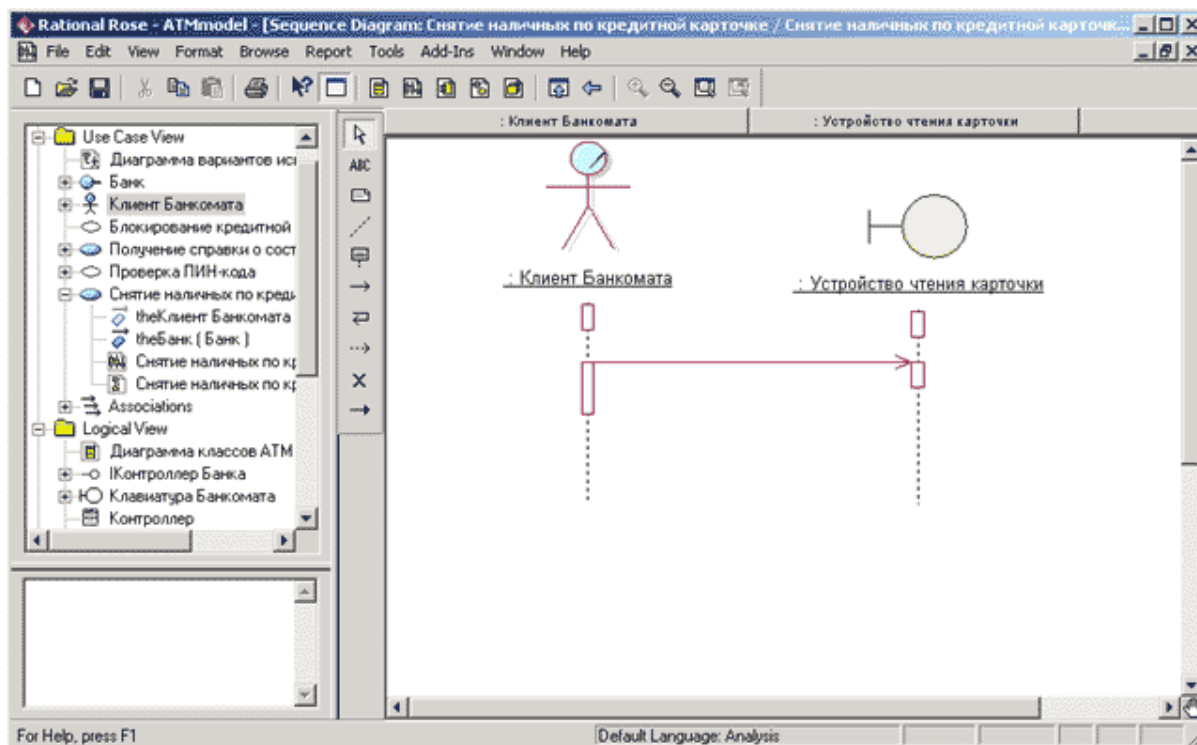


Рис. 2.4.3. Диаграмма последовательности после добавления сообщения от экземпляра актера Клиент Банкомата к объекту класса Устройства чтения карточки

Для спецификации свойств добавленного сообщения предназначено специальное окно, которое можно открыть двойным щелчком на изображении сообщения на диаграмме последовательности. Имя сообщения можно выбрать на вкладке **General** (Общие) из выпадающего списка операций соответствующего класса-приемника (рис. 2.4.4).

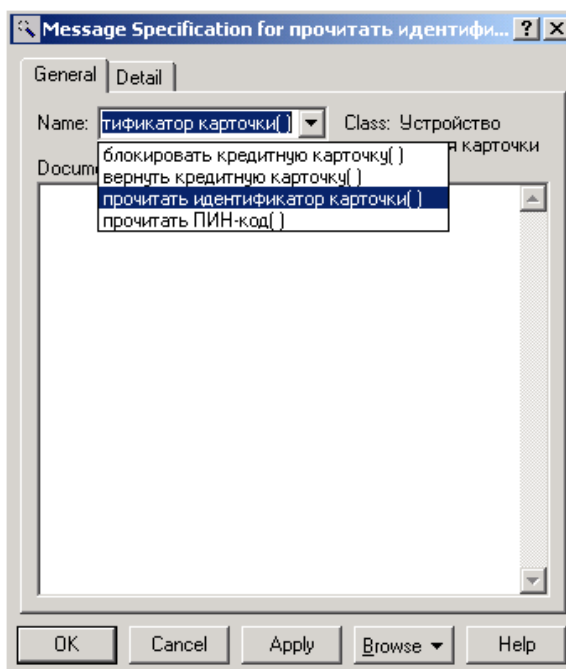


Рис. 2.4.4. Диалоговое окно спецификации свойств сообщения

Имя сообщения можно выбрать также из контекстного меню сообщения, в котором перечислены все операции класса-приемника данного сообщения (рис.2.4.5). При необходимости в контекстном меню можно задать новую операцию, в этом случае следует

выбрать строку <new operation>. При этом откроется диалоговое окно спецификации свойств новой операции класса-приемника, особенности редактирования которых были рассмотрены ранее.

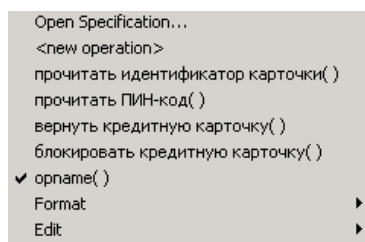


Рис. 2.4.5. Контекстное меню сообщения на диаграмме последовательности

Для рассматриваемой модели банкомата в качестве имени первого сообщения следует выбрать операцию *прочитать идентификатор карточки()*. После выбора операции для данного сообщения следует нажать кнопку **Apply** или **OK**, в результате чего имя сообщения будет изображено на диаграмме последовательности рядом с линией сообщения.

Построение диаграммы последовательности сводится к добавлению и редактированию свойств отдельных объектов и сообщений. Доступ к окну спецификации свойств соответствующих элементов возможен также либо через контекстное меню, либо с помощью операции главного меню **Browse→Specification** (Обзор→Спецификация). При добавлении сообщений на диаграмму последовательности они получают по умолчанию свой номер в общей последовательности сообщений.

Следует заметить, что по умолчанию нумерация сообщений на диаграмме последовательности может быть отключена. При необходимости показать номера сообщений следует выполнить операцию главного меню: **Tools→Options** (Инструменты→Параметры), открыть вкладку **Diagram** (Диаграмма) и выставить отметку выбора строки **Sequence numbering** (Нумерация сообщений на диаграмме последовательности) в группе свойств **Display** (рис. 2.4.6).

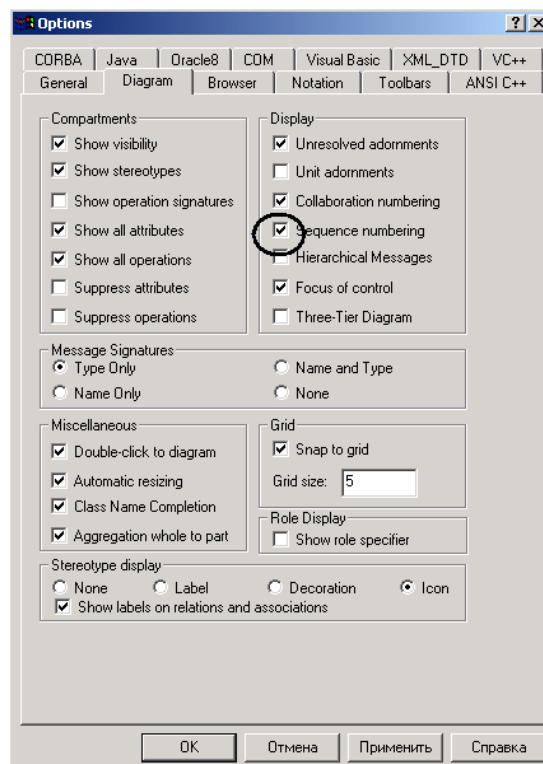


Рис. 2.4.6. Диалоговое окно спецификации свойств модели

Это же окно спецификации свойств модели можно открыть с помощью операции главного меню: **View**→**Toolbars**→**Configure** (Вид→Панели инструментов→Настроить),

Для детальной спецификации свойств сообщений на диаграмме последовательности можно использовать также группу свойств **Synchronization** (Синхронизация) и **Frequency** (Частота), доступные для выбора на вкладке **Detail** (Подробно) окна спецификации сообщения. При изменении способа синхронизации передаваемого сообщения изменяется графическое изображение стрелки соответствующего сообщения.

### Завершение диаграммы последовательности модели банкомата

Для завершения построения диаграммы последовательности рассматриваемого примера следует описанным выше способом добавить оставшиеся объекты и сообщения. С этой целью следует выполнить следующие действия:

1. Добавить объекты классов с именами: *Контроллер Банкомата*, *Транзакция Банкомата*, *Клавиатура Банкомата*, *Экран Банкомата*, *Принтер Банкомата*, *Устройство выдачи наличных* и *Интерфейс Банка*.

2. Добавить сообщение: *проверить идентификатор карточки (Integer)*, направленное от объекта класса *Контроллер Банкомата* к объекту класса *Интерфейс Банка*.

3. Добавить сообщение: *ввести ПИН-код()*, направленное от объекта класса-актера *Клиент Банкомата* к объекту класса *Клавиатура Банкомата*.

4. Добавить сообщение: *прочитать ПИН-код()*, направленное от объекта класса *Контроллер Банкомата* к объекту класса *Устройство чтения карточки*.

5. Добавить сообщение: *создать новую транзакцию()*, направленное от объекта класса *Контроллер Банкомата* к изображению объекта класса *Транзакция Банкомата*. При этом изображение объекта класса *Транзакция Банкомата* следует переместить вниз на уровень этого сообщения, что будет визуально означать создание данного объекта в более поздний момент времени, чем начало функционирования моделируемой программной системы.

6. Добавить сообщение: *проверить правильность ПИН-кода()*, направленное от объекта класса *Контроллер Банкомата* к объекту класса *Транзакция Банкомата*.

7. Добавить сообщение: *показать меню опций()*, направленное от объекта класса *Контроллер Банкомата* к объекту класса *Экран Банкомата*.

8. Добавить сообщение: *ввести тип транзакции()*, направленное от объекта класса-актера *Клиент Банкомата* к объекту класса *Клавиатура Банкомата*.

9. Добавить сообщение: *показать меню снятия суммы()*, направленное от объекта класса *Контроллер Банкомата* к объекту класса *Экран Банкомата*.

10. Добавить сообщение: *ввести сумму снятия наличных()*, направленное от объекта класса-актера *Клиент Банкомата* к объекту класса *Клавиатура Банкомата*.

11. Последовательно добавить 3 сообщения: *открыть счет клиента (Integer)*, *проверить баланс клиента (Integer, Currency)* и *уменьшить счет клиента (Integer, Currency)*, направленные от объекта класса *Контроллер Банкомата* к объекту класса *Интерфейс Банка*.

12. Добавить сообщение: *распечатать чек()*, направленное от объекта класса *Контроллер Банкомата* к объекту класса *Принтер Банкомата*.

13. Добавить сообщение: *вернуть кредитную карточку()*, направленное от объекта класса *Контроллер Банкомата* к объекту класса *Устройство чтения карточки*.

14. Добавить сообщение: *выдать наличные()*, направленное от объекта класса *Контроллер Банкомата* к объекту класса *Устройство выдачи наличных*.

15. Добавить сообщение: *завершить транзакцию()*, направленное от объекта класса *Контроллер Банкомата* к объекту класса *Транзакция Банкомата*.

16. После добавления сообщения *завершить транзакцию()* поместить на линию жизни объекта класса *Транзакция Банкомата* символ уничтожения этого объекта.

Фрагмент диаграммы последовательности, описывающая реализацию типичного хода событий прецедента *Снятие наличных по кредитной карточке* для проекта системы управления банкоматом, показан на рис. 2.4.7.

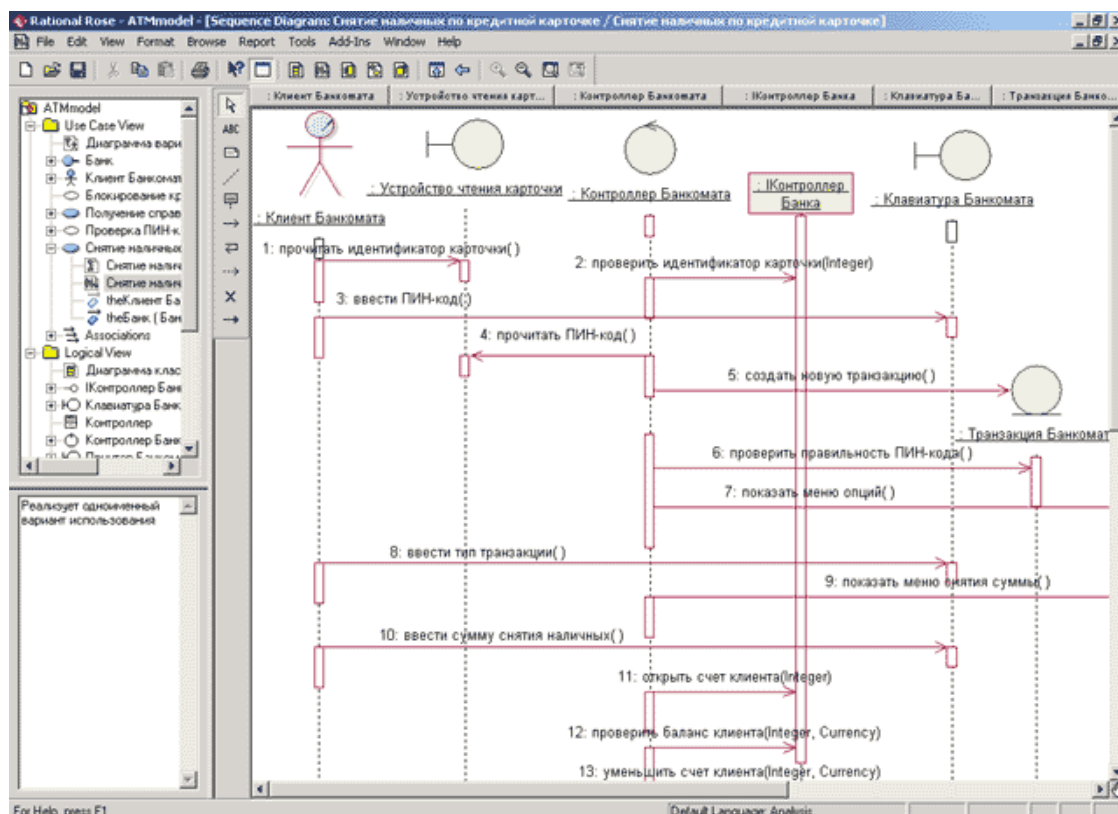


Рис. 2.4.7. Фрагмент окончательного вида диаграммы последовательности

Если необходимо изменить порядок следования сообщений, то из двух диаграмм взаимодействия данное действие удобнее выполнить на диаграмме последовательности, чем на диаграмме кооперации. В этом случае достаточно нажать левую кнопку мыши на стрелке соответствующего сообщения и, не отпуская ее, перетащить вертикально вверх или вниз данное сообщение. Дополнительно можно добавить потоки данных и определить устойчивость объектов на основе активизации соответствующих спецификаций.

## 2.4.2 Лабораторная работа № 9. Collaboration diagrams

### Цель работы:

- изучение диаграмм кооперации,
- изучение их применения в процессе проектирования.

Диаграмма кооперации является разновидностью диаграммы взаимодействия, и в контексте языка UML описывает динамический аспект взаимодействия объектов при реализации отдельных прецедентов. Активизировать рабочее окно диаграммы кооперации в программе IBM Rational Rose можно несколькими способами:

- Щелкнуть на кнопке с изображением диаграммы взаимодействия на стандартной панели инструментов и выбрать для построения новую диаграмму кооперации.
- Выполнить операцию главного меню: **Browse**→**Interaction Diagram** (Браузер→Диаграмма взаимодействия) и выбрать для построения новую диаграмму кооперации.

- Выполнить операцию контекстного меню: **New→Collaboration Diagram** (Новая→Диаграмма кооперации) для логического представления или представления прецедентов в браузере проекта.

При этом появляется новое окно с чистым рабочим листом диаграммы кооперации и специальная панель инструментов, содержащая кнопки с изображением графических примитивов, необходимых для разработки диаграммы кооперации. Назначение отдельных кнопок панели указано в таблице 22, а также можно узнать из всплывающих подсказок.

Таблица 22.

Назначение кнопок специальной панели инструментов диаграммы кооперации

| Кнопка                                                                              | Всплывающая подсказка | Назначение кнопки                                                                                |
|-------------------------------------------------------------------------------------|-----------------------|--------------------------------------------------------------------------------------------------|
|    | Selection Tool        | Превращает изображение курсора в форму стрелки для последующего выделения элементов на диаграмме |
|    | Text Box              | Добавляет на диаграмму текстовую область                                                         |
|    | Note                  | Добавляет на диаграмму примечание                                                                |
|    | Anchor Note to Item   | Добавляет на диаграмму связь примечания с соответствующим графическим элементом диаграммы        |
|    | Object                | Добавляет на диаграмму объект                                                                    |
|    | Class Instance        | Добавляет на диаграмму экземпляр класса                                                          |
|    | Object Link           | Добавляет на диаграмму связь                                                                     |
|   | Link To Self          | Добавляет на диаграмму рефлексивную связь                                                        |
|  | Link Message          | Добавляет на связь диаграммы прямое сообщение                                                    |
|  | Reverse Link Message  | Добавляет на связь диаграммы обратное сообщение                                                  |
|  | Data Token            | Добавляет на связь диаграммы элемент прямого потока данных                                       |
|  | Reverse Data Token    | Добавляет на связь диаграммы элемент обратного потока данных                                     |

На специальной панели инструментов по умолчанию присутствуют практически все кнопки с пиктограммами элементов, которые могут быть использованы для построения диаграммы. В данном разделе в качестве примера рассматривается процесс построения диаграммы кооперации, которая представляет собой реализацию прецедента *Снятие наличных по кредитной карточке* применительно к разрабатываемому проекту системы управления банкоматом. В модели данная диаграмма кооперации соответствует этому прецеденту и может быть размещена в представлении прецедентов (Use Case View). После активизации новой диаграммы кооперации одним из описанных выше способов следует в качестве имени данной диаграммы задать: *Снятие наличных по кредитной карточке*.

В общем случае работа с диаграммой кооперации состоит в добавлении объектов, связей и сообщений, а также редактировании их свойств. При этом изменения, вносимые в диаграмму кооперации, автоматически вносятся в диаграмму последовательности, что можно увидеть в любой момент, активизировав последнюю нажатием клавиши <F5>.

#### Добавление объекта на диаграмму кооперации

Добавить объект на диаграмму кооперации можно стандартным образом с помощью соответствующей кнопки на специальной панели инструментов. Однако, в случае наличия построенной ранее диаграммы классов, в браузере проекта выделить необходимый класс и, удерживая нажатой левую кнопку мыши, перетащить изображение пиктограммы класса из браузера на свободное место рабочего листа диаграммы кооперации. В результате этих действий на диаграмме кооперации появится изображение объекта с именем класса и маркерами изменения его геометрических размеров (рис. 2.4.8).

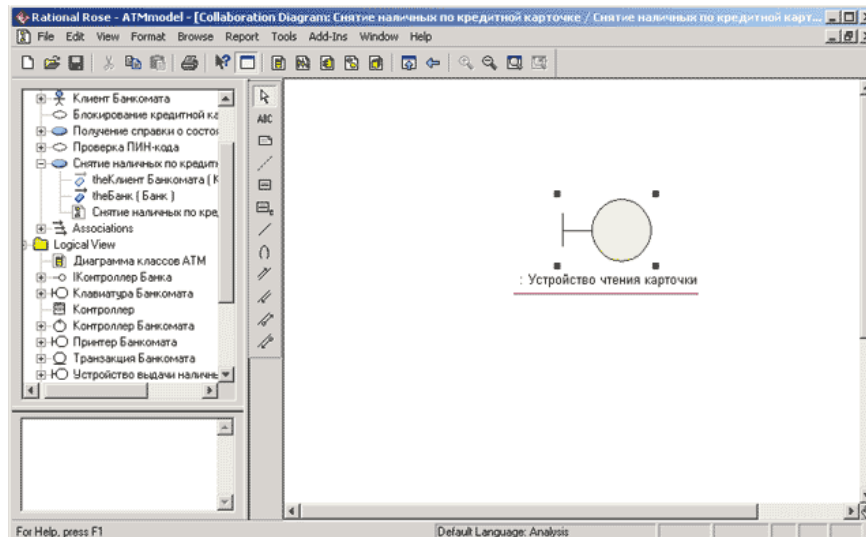


Рис. 2.4.8. Диаграмма кооперации после добавления на нее анонимного объекта класса *Устройство чтения карточки*

По умолчанию каждый добавляемый объект считается анонимным. При необходимости можно задать собственное имя объекта, для чего двойным щелчком на изображении объекта на диаграмме кооперации следует вызвать диалоговое окно свойств этого объекта (рис. 2.4.9).

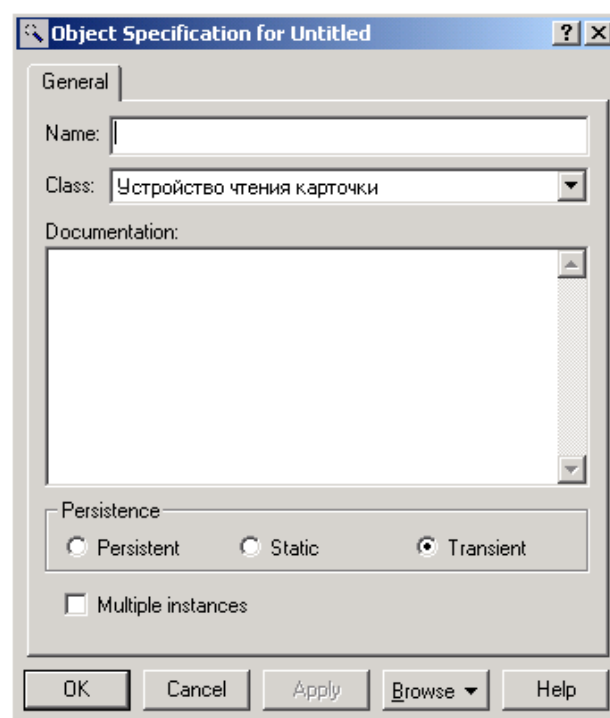


Рис. 2.4.9. Диалоговое окно спецификации свойств объекта класса *Устройство чтения карточки*

Как видно из рассмотрения этого окна свойств, для объекта выбранного класса можно задавать: собственное имя объекта, особенности его реализации и множественность экземпляров.

Группа свойств **Persistence** (Устойчивость) предназначена для спецификации устойчивости объектов соответствующего класса. При этом свойство **Persistent** (Устойчивый) означает, что информация об объектах данного класса должна быть сохранена в системе некоторым подходящим способом. Свойство **Static** (Статический) означает, что

соответствующий объект сохраняется в памяти компьютера в течение всего времени работы программного приложения. Свойство **Transient** (Временный) соответствующий объект хранится в памяти компьютера в течение короткого времени, необходимого только для выполнения его операций. Применительно к рассматриваемой для объекта класса *Устройство чтения карточки* модели следует выбрать свойство **Persistent**.

При необходимости можно представить объект в форме мультиобъекта. Для этого следует выбрать отметку у свойства **Multiple instances** (Несколько экземпляров). Однако для объекта класса *Устройство чтения карточки* это свойство следует оставить пустым, поскольку данный объект присутствует в модели в единственном экземпляре.

### Добавление связи

Для добавления связи между предварительно размещенными на диаграмме объектами нужно с помощью левой кнопки мыши нажать кнопку с изображением связи на специальной панели инструментов, отпустить левую кнопку мыши, щелкнуть левой кнопкой мыши на изображении одного объекта на диаграмме и отпустить ее на изображении другого объекта. В результате этих действий на диаграмме появится изображение связи, например, соединяющей объект класса *Клиент Банкомата* (актера) с объектом класса *Устройство чтения карточки* (рис. 2.4.10). Поскольку кнопка с изображением актера отсутствует на специальной панели инструментов диаграммы кооперации, соответствующий объект следует предварительно поместить на диаграмму способом перетаскивания пиктограммы актера из браузера проекта.

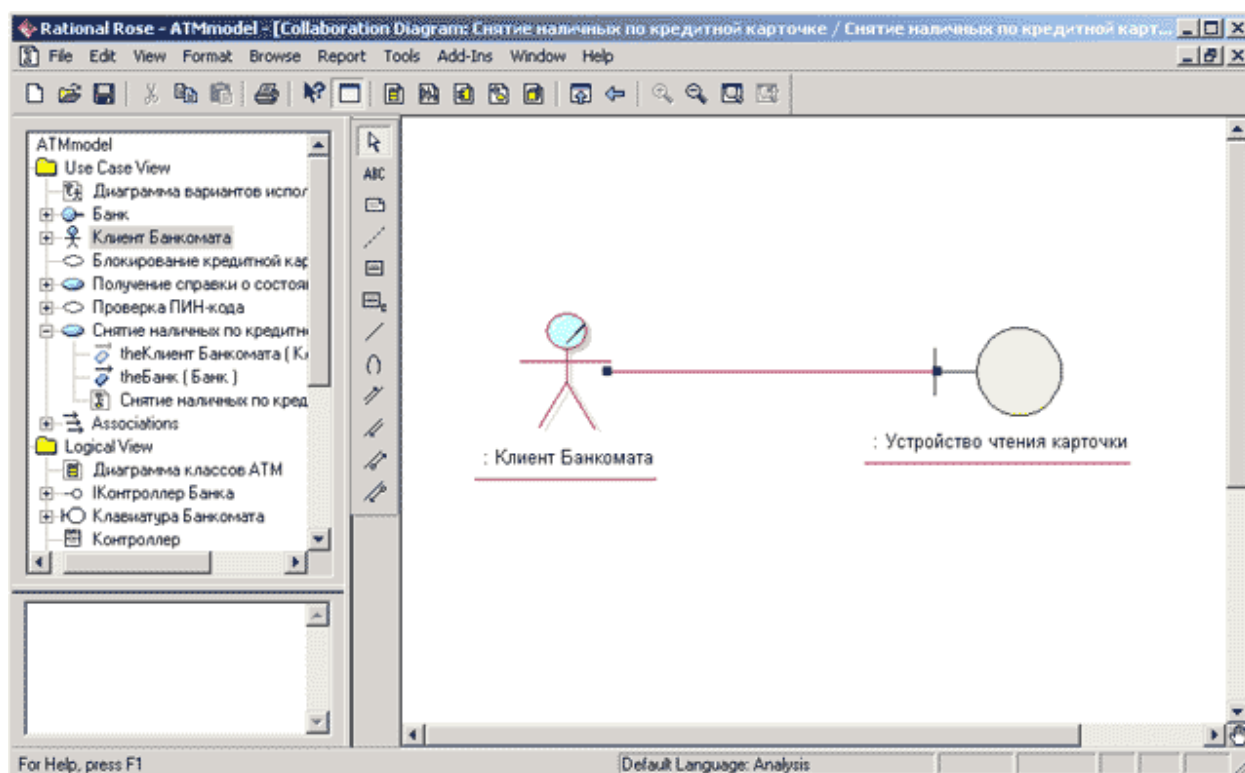


Рис. 2.4.10. Диаграмма кооперации после добавления связи между объектом класса Клиент Банкомата (актером) и объектом класса Устройство чтения карточки

По умолчанию каждая добавляемая связь считается анонимной. При необходимости можно задать имя связи с помощью диалогового окна спецификации свойств данной связи (рис. 2.4.11).

Кроме имени связи можно также задать: имя ассоциации, видимость соответствующей пары объектов и наличие общих ролей. Однако более важной представляется следующая

вкладка **Messages** (сообщения), служащая для спецификации сообщений, передаваемых между соответствующей парой объектов.

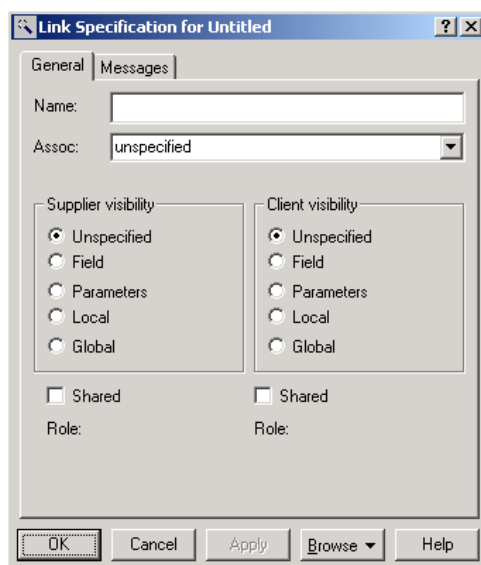


Рис. 2.4.11 Диалоговое окно редактирования свойств связи

### Добавление сообщения

Добавить сообщения на диаграмму кооперации можно несколькими способами. Стандартный способ заключается в использовании кнопки с пиктограммой сообщения на специальной панели инструментов. В этом случае необходимо левой кнопкой мыши нажать кнопку с изображением прямого или обратного сообщения на специальной панели инструментов, отпустить левую кнопку мыши, щелкнуть левой кнопкой мыши на изображении линии связи на диаграмме и отпустить ее. В результате этих действий на диаграмме рядом с линией связи появится изображение стрелки сообщения.

Однако более удобным представляется способ добавления сообщений с помощью диалогового окна свойств связей. Для этого двойным щелчком на линии связи вызывается окно ее свойств и раскрывается вкладка **Messages** (сообщения). После этого следует выполнить операцию контекстного меню **Insert To** (Вставить в направлении), в результате чего появляется вложенный список с предложением выбрать одну из операций целевого класса для спецификации имени сообщения (рис. 2.4.12).

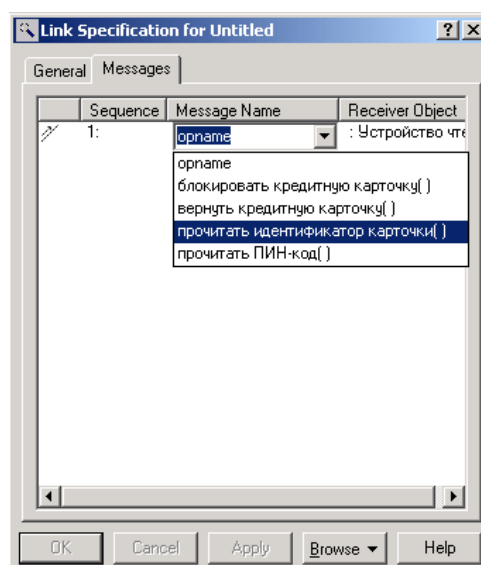


Рис. 2.4.12. Диалоговое окно добавления сообщения для выбранной связи

Для рассматриваемой модели банкомата для первого сообщения следует выбрать операцию прочитать идентификатор карточки(). После выбора операции для данного сообщения оно добавляется в список сообщений данной связи, а рядом с линией связи на диаграмме кооперации появится стрелка с номером и именем этого сообщения (рис. 2.4.13).

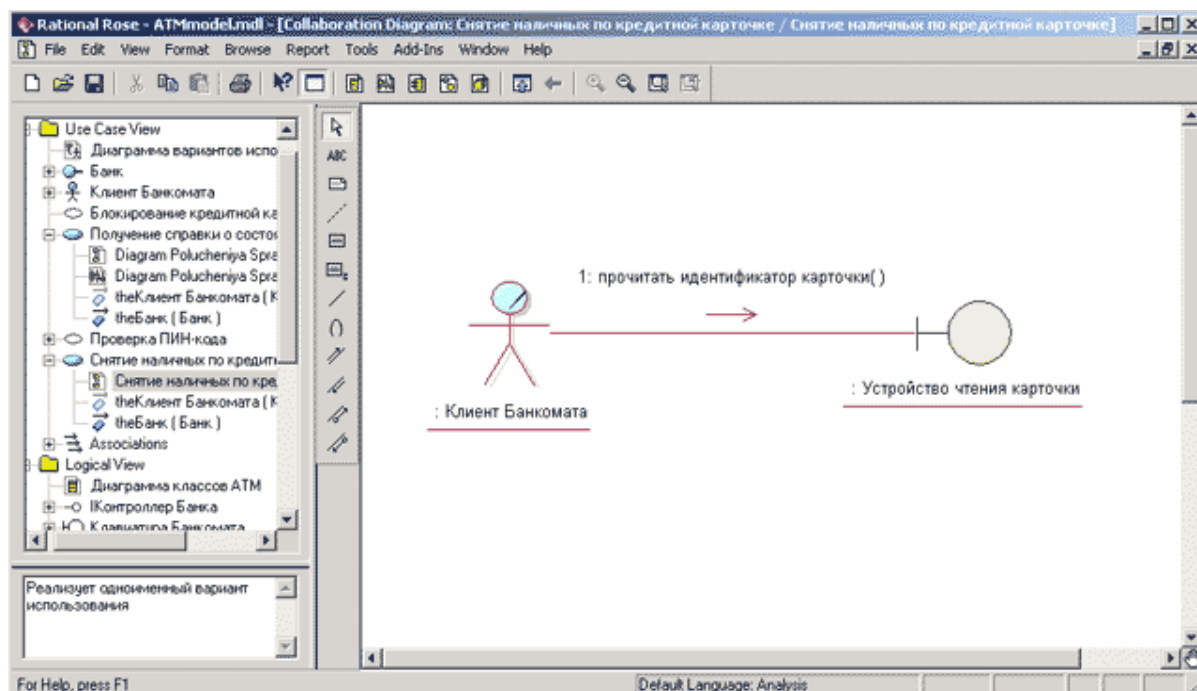


Рис. 2.4.13. Диаграмма кооперации после добавления связи

Кроме имени сообщения можно также задать стереотип синхронизации и частоту передачи. Для этой цели следует воспользоваться диалоговым окном спецификации свойств сообщений (рис. 2.4.14), которое можно открыть двойным щелчком на имени сообщения в списке рассматриваемой вкладки **Messages** окна спецификации свойств связи.

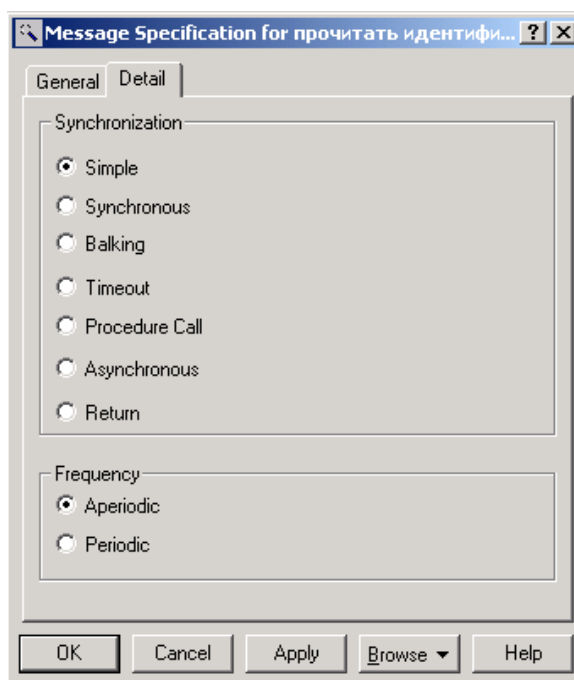


Рис. 2.4.14. Диалоговое окно спецификации свойств сообщения

Группа свойств **Synchronization** (Синхронизация) предназначена для определения способа синхронизации передаваемого сообщения. При изменении этого свойства изменяется графическое изображение стрелки соответствующего сообщения. Характеристика отдельных свойств синхронизации сообщений и графическое изображение соответствующих стрелок сообщений приводится в следующей таблице 23.

Таблица 23.

Характеристика свойств синхронизации сообщений

| Название свойства                | Иконка                                                                              | Назначение свойства                                                                                                                                                                                                 |
|----------------------------------|-------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Simple (Простое)                 |    | Данное сообщение выполняется в одном потоке управления. Это свойство задается добавляемому на диаграмму сообщению по умолчанию                                                                                      |
| Synchronous (Синхронное)         |    | После передачи данного сообщения клиент ожидает ответа от объекта-приемника о результате выполнения соответствующей операции                                                                                        |
| Balking (С отказом)              |    | После передачи данного сообщения объект-приемник отказывает клиенту в выполнении соответствующей операции, если он занят выполнением других операций                                                                |
| Timeout (С ожиданием)            |    | После передачи данного сообщения объект-приемник может поместить данное сообщение в очередь с ограниченным временем ожидания, если он занят выполнением других операций                                             |
| Procedure Call (Вызов процедуры) |    | Клиент посылает данное сообщение объекту-приемнику и, чтобы продолжить свою работу ожидает, пока вся дальнейшая вложенная последовательность сообщений не будет обработана приемником                               |
| Asynchronous (Асинхронное)       |    | Клиент посылает данное сообщение и продолжает свою работу, не ожидая подтверждения от объекта-приемника о получении этого сообщения. При этом соответствующая операция может быть как выполнена, так и не выполнена |
| Return (Возврат)                 |  | Данное сообщение посылается клиенту после окончания выполнения вызова процедуры                                                                                                                                     |

Группа свойств **Frequency** (Частота) предназначена для указания периодического характера передачи сообщения. При изменении этого свойства графическое изображение стрелки соответствующего сообщения не изменяется. Свойство **Aperiodic** (Апериодическое) означает, что сообщение посылается клиентом нерегулярно. При этом сообщение может быть отправлено один или несколько раз через различные промежутки времени. Это свойство задается для сообщения по умолчанию. Свойство **Periodic** (Периодическое) означает, что сообщение регулярно посылается клиентом через определенные промежутки времени.

Применительно для модели банкомата можно оставить рассмотренные свойства сообщений без изменения, в том виде, в каком они определены по умолчанию программой IBM Rational Rose.

#### Завершение диаграммы кооперации для модели банкомата

Для завершения построения диаграммы кооперации рассматриваемого примера следует описанным выше способом добавить оставшиеся объекты, связи и сообщения. С этой целью следует выполнить следующие действия:

1. Добавить объекты классов с именами: *Контроллер Банкомата*, *Транзакция Банкомата*, *Клавиатура Банкомата*, *Экран Банкомата*, *Принтер Банкомата*, *Устройство выдачи наличных* и *Интерфейс Банка*.

2. Добавить связи, соединяющие объекты классов с именами: *Контроллер Банкомата* с *Устройством чтения карточки*, *Контроллер Банкомата* с *Транзакцией Банкомата*, *Контроллер Банкомата* с *Клавиатурой Банкомата*, *Контроллер Банкомата* с *Экраном Банкомата*, *Контроллер Банкомата* с *Принтером Банкомата*, *Контроллер Банкомата* с *Устройством выдачи наличных* и *Контроллер Банкомата* с *Интерфейсом Банка*.

3. Добавить сообщение: *проверить идентификатор карточки (Integer)*, направленное от объекта класса *Контроллер Банкомата* к объекту класса *Интерфейс Банка*.
4. Добавить сообщение: *ввести ПИН-код()*, направленное от объекта класса-актера *Клиент Банкомата* к объекту класса *Клавиатура Банкомата*.
5. Добавить сообщение: *прочитать ПИН-код()*, направленное от объекта класса *Контроллер Банкомата* к объекту класса *Устройство чтения карточки*.
6. Добавить сообщение: *создать новую транзакцию()*, направленное от объекта класса *Контроллер Банкомата* к объекту класса *Транзакция Банкомата*.
7. Добавить сообщение: *проверить правильность ПИН-кода()*, направленное от объекта класса *Контроллер Банкомата* к объекту класса *Транзакция Банкомата*.
8. Добавить сообщение: *показать меню опций()*, направленное от объекта класса *Контроллер Банкомата* к объекту класса *Экран Банкомата*.
9. Добавить сообщение: *ввести тип транзакции()*, направленное от объекта класса-актера *Клиент Банкомата* к объекту класса *Клавиатура Банкомата*.
10. Добавить сообщение: *показать меню снятия суммы()*, направленное от объекта класса *Контроллер Банкомата* к объекту класса *Экран Банкомата*.
11. Добавить сообщение: *ввести сумму снятия наличных()*, направленное от объекта класса-актера *Клиент Банкомата* к объекту класса *Клавиатура Банкомата*.
12. Последовательно добавить 3 сообщения: *открыть счет клиента (Integer)* , *проверить баланс клиента (Integer, Currency)* и *уменьшить счет клиента (Integer, Currency)*, направленные от объекта класса *Контроллер Банкомата* к объекту класса *Интерфейс Банка*.
13. Добавить сообщение: *распечатать чек()*, направленное от объекта класса *Контроллер Банкомата* к объекту класса *Принтер Банкомата*.
14. Добавить сообщение: *вернуть кредитную карточку()*, направленное от объекта класса *Контроллер Банкомата* к объекту класса *Устройство чтения карточки*.
15. Добавить сообщение: *выдать наличные()*, направленное от объекта класса *Контроллер Банкомата* к объекту класса *Устройство выдачи наличных*.
16. Добавить сообщение: *завершить транзакцию()*, направленное от объекта класса *Контроллер Банкомата* к объекту класса *Транзакция Банкомата*.

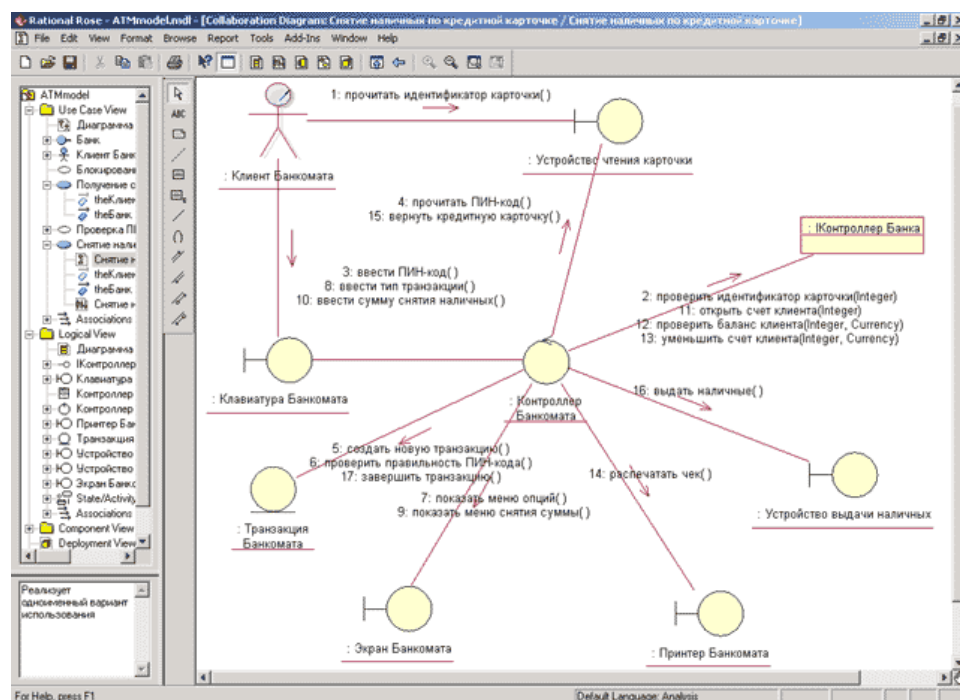


Рис. 2.4.15. Окончательный вариант диаграммы кооперации

Диаграмма кооперации, описывающая реализацию типичного хода событий прецедента Снятие наличных по кредитной карточке для проекта системы управления банкоматом, показана на рис. 2.4.15.

При необходимости можно изменить порядок следования сообщений и их спецификацию, а также установить дополнительную синхронизацию сообщений и связать с сообщениями примечания.

### **2.4.3 Задания**

1. Выбрать в моделируемой системе прецедент, для которого будут строиться диаграммы взаимодействия.
2. Построить для выбранного прецедента диаграмму последовательности.
3. Построить для того же прецедента диаграмму кооперации.
4. Сформулировать достоинства и недостатки каждого вида диаграмм при моделировании данного прецедента.

### **2.4.4 Контрольные вопросы**

1. Каково назначение диаграмм взаимодействия?
2. Назовите два вида диаграмм взаимодействия.
3. Как относятся между собой диаграммы прецедентов и диаграммы взаимодействия?
4. Что такое «жизненная линия» на диаграмме последовательности?
5. Как на диаграмме последовательности представляются сообщения?
6. Что такое самоделегирование?
7. Что показывает активизация объекта?
8. В чем отличие диаграмм кооперации от диаграмм последовательности?
9. Каковы преимущества и недостатки каждого вида взаимодействия?
10. Как отображается условное поведение на диаграммах взаимодействия?

## 2.5. Диаграммы состояний и деятельности

**Диаграммы состояний (statechart diagrams)** используются для описания поведения сложных систем. Они определяют все возможные состояния, в которых может находиться объект, а также процесс смены состояний объекта в результате некоторых событий. Эти диаграммы обычно используются для описания поведения одного объекта в нескольких прецедентах.

Прямоугольниками представляются состояния, через которые проходит объект во время своего поведения. Состояниям соответствуют определенные значения атрибутов объектов. Стрелки представляют переходы от одного состояния к другому, которые вызываются выполнением некоторых функций объекта. Имеется также два вида псевдосостояний: начальное состояние , в котором находится только что созданный объект, и конечное состояние , которое объект не покидает, как только туда перешел.

На диаграммах также отображаются функции, которые выполняются объектом в определенном состоянии (см. рис. 2.5.1). Синтаксис метки деятельности: *выполнить* / *деятельность* >.

Переходы имеют метки, которые синтаксически состоят из трех необязательных частей: <Событие>[Условие]< / Действие>.

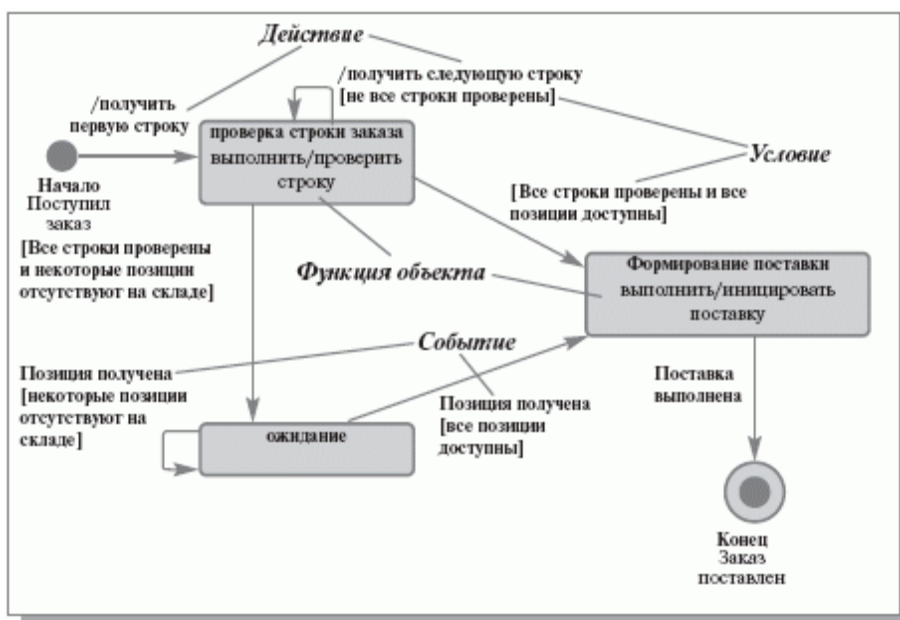


Рис. 2.5.1. Диаграмма состояний объекта «заказ»

### Диаграммы деятельности (activity diagrams)

Диаграмма деятельности – это частный случай диаграммы состояний. На диаграмме деятельности представлены переходы потока управления от одной деятельности к другой внутри системы. Этот вид диаграмм обычно используется для описания поведения, включающего в себя множество параллельных процессов.

Основными элементами диаграмм деятельности являются (рис. 2.5.2):

- овалы, изображающие действия объекта;
- линейки синхронизации, указывающие на необходимость завершить или начать несколько действий (модель логического условия "И");
- ромбы, отражающие принятие решений по выбору одного из маршрутов выполнения процесса (модель логического условия "ИЛИ");
- стрелки — отражают последовательность действий, могут иметь метки условий.

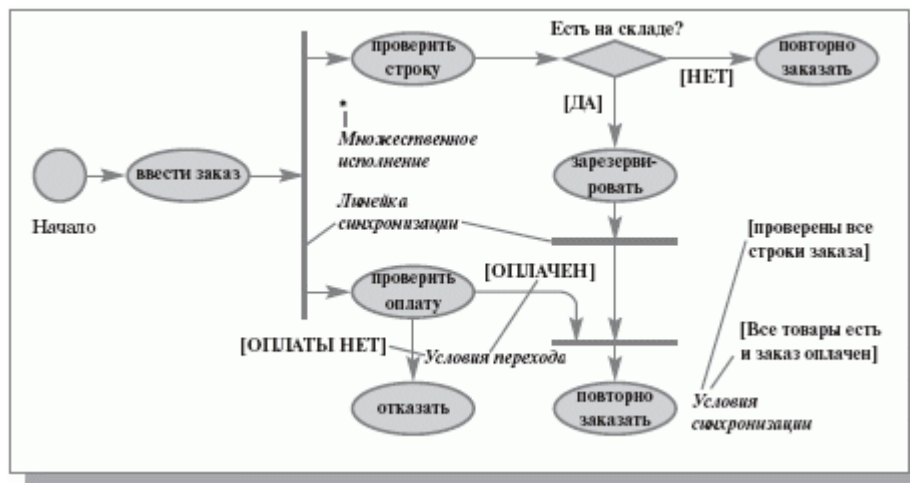


Рис. 2.5.2. Диаграмма деятельности — обработка заказа

На диаграмме деятельности могут быть представлены действия, соответствующие нескольким прецедентам. На таких диаграммах появляется множество начальных точек, поскольку они отражают теперь реакцию системы на множество внешних событий. Таким образом, диаграммы деятельности позволяют получить полную картину поведения системы и легко оценивать влияние изменений в отдельных прецедентах на конечное поведение системы.

Любая деятельность может быть подвергнута дальнейшей декомпозиции и представлена в виде отдельной диаграммы деятельности или спецификации (словесного описания).

### 2.5.1 Лабораторная работа № 10. Statechart diagrams

#### Цель работы:

- изучение диаграмм состояний,
- изучение их применения в процессе проектирования.

Переходя к рассмотрению диаграммы состояний, следует отметить, что в среде IBM Rational Rose этот тип диаграмм может относиться к отдельному классу, операции класса, варианту использования, пакету или представлению.

Начать построение диаграммы состояний для выбранного элемента модели или моделируемой системы в целом можно одним из следующих способов:

- Щелкнуть на кнопке с изображением диаграммы состояний на стандартной панели инструментов, после чего следует выбрать представление и тип разрабатываемой диаграммы - новая диаграмма состояний.
- Выделить логическое представление (Logical View) или представление прецедентов (Use Case View) в браузере проекта и выполнить операцию контекстного меню: **New→Statechart Diagram** (Новая→Диаграмма состояний).
- Раскрыть логическое представление (Logical View) в браузере проекта и выделить рассматриваемый класс, операцию класса, пакет, или раскрыть представление прецедентов (Use Case View) и выбрать прецедент, после чего выполнить операцию контекстного меню: **New→Statechart Diagram** (Новая→Диаграмма состояний).
- Выполнить операцию главного меню: **Browse→State Machine Diagram** (Обзор→Диаграмма состояний), после чего следует выбрать представление и тип разрабатываемой диаграммы.

В результате выполнения этих действий появляется новое окно с чистым рабочим листом диаграммы состояний и специальная панель инструментов, содержащая кнопки с изображением графических элементов модели, необходимых для разработки диаграммы

состояний. Назначение отдельных кнопок панели указано в таблице 24, а также можно узнать из всплывающих подсказок.

Таблица 24.

Назначение кнопок специальной панели инструментов диаграммы состояний

| Кнопка                                                                             | Всплывающая подсказка      | Назначение кнопки                                                                                      |
|------------------------------------------------------------------------------------|----------------------------|--------------------------------------------------------------------------------------------------------|
|   | Selection Tool             | Превращает изображение курсора в форму стрелки для последующего выделения элементов на диаграмме       |
|   | Text Box                   | Добавляет на диаграмму текстовую область                                                               |
|   | Note                       | Добавляет на диаграмму примечание                                                                      |
|   | Anchor Note to Item        | Добавляет на диаграмму связь примечания с соответствующим графическим элементом диаграммы              |
|   | State                      | Добавляет на диаграмму состояние                                                                       |
|   | Start State                | Добавляет на диаграмму начальное состояние                                                             |
|   | End State                  | Добавляет на диаграмму конечное состояние                                                              |
|   | State Transition           | Добавляет на диаграмму переход                                                                         |
|   | Transition to Self         | Добавляет на диаграмму рефлексивный переход                                                            |
|   | Horizontal Synchronization | Добавляет на диаграмму горизонтально расположенный символ синхронизации (по умолчанию отсутствует)     |
|   | Vertical Synchronization   | Добавляет на диаграмму вертикально расположенный символ синхронизации (по умолчанию отсутствует)       |
|  | Decision                   | Добавляет на диаграмму символ принятия решения для альтернативных переходов (по умолчанию отсутствует) |

По умолчанию на специальной панели инструментов могут отсутствовать кнопки с тремя последними графическими элементами. При необходимости их можно добавить на специальную панель диаграммы состояний аналогично способу, рассмотренному ранее. Продолжая разработку проекта по моделированию системы управления банкоматом, можно приступить к разработке новой диаграммы состояний. С этой целью для диаграммы состояний модели банкомата зададим имя *Диаграмма состояний АТМ*, а в секцию ее документации введем текст «Диаграмма состояний описывает конечный автомат банкомата».

### Добавление состояния на диаграмму состояний

Для добавления состояния на диаграмму состояний необходимо с помощью левой кнопки мыши нажать кнопку с изображением пиктограммы состояния на специальной панели инструментов, отпустить левую кнопку мыши и щелкнуть левой кнопкой мыши на свободном месте рабочего листа диаграммы. На диаграмме появится изображение состояния с маркерами изменения его геометрических размеров и предложенным средой именем по умолчанию, которое разработчику следует изменить.

Для диаграммы состояний модели банкомата в качестве имени первого добавленного состояния изменим предложенное программой по умолчанию имя NewState на *Ожидание карточки* (рис. 2.5.3). Задать имя состояния можно либо непосредственно при добавлении нового состояния на диаграмму состояний, либо открыв окно спецификации свойств нового состояния.

Для добавленного состояния можно открыть диалоговое окно его свойств двойным щелчком левой кнопкой мыши на изображении этого элемента на диаграмме. В этом случае активизируется диалоговое окно со специальными вкладками, в поля которых можно занести всю информацию по данному состоянию (рис. 2.5.4).

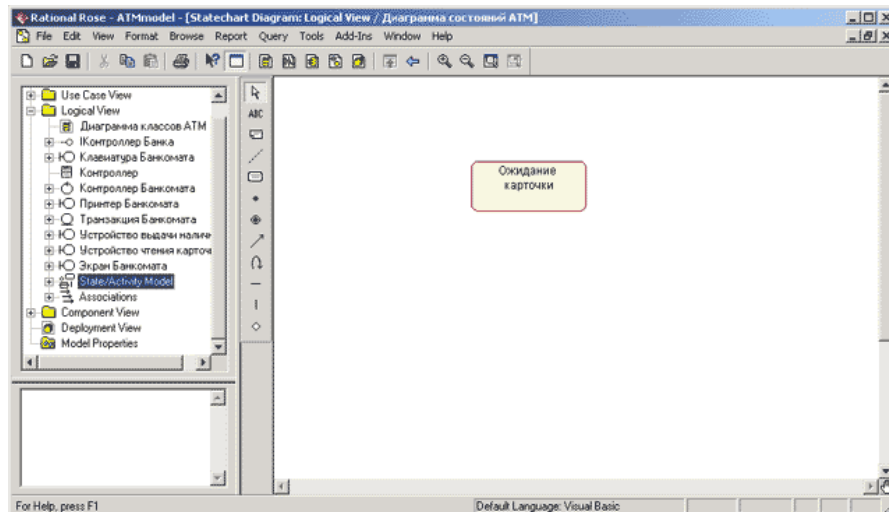


Рис. 2.5.3 Диаграмма состояний после добавления на нее состояния *Ожидание карточки*

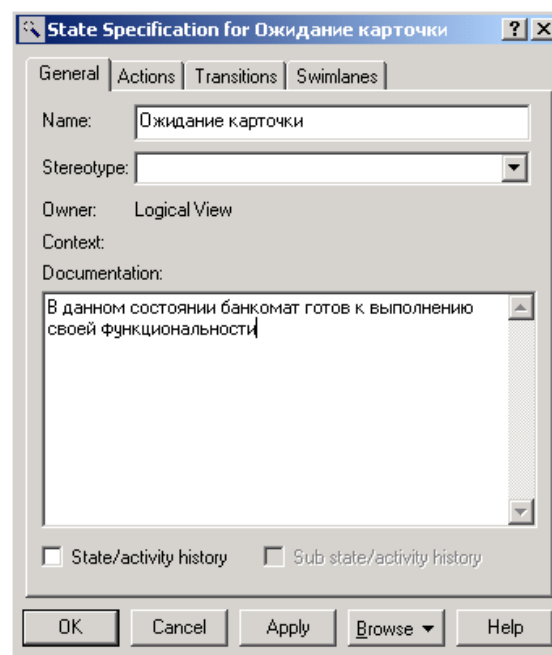


Рис. 2.5.4. Диалоговое окно спецификации свойств состояния

При необходимости в диалоговом окне спецификации свойств выбранного состояния можно задать вложенное историческое состояние. Для этого следует выставить отметку у свойства **State/activity history** (Историческое состояние/деятельность) и нажать кнопку **Apply**. В результате внутри исходного состояния появится вложенное историческое состояние (рис. 2.5.5а).



Рис. 2.5.5. Добавление вложенного исторического состояния (а) и состояния глубокой истории (б) для состояния *Ожидание карточки*

Чтобы обычное историческое состояние превратить в состояние глубокой истории, следует дополнительно выставить отметку у свойства **Sub state/activity history** (Историческое под-состояние/деятельность), которое становится доступным для редактирования после выбора первого свойства, и нажать кнопку **Apply**. В результате внутри исходного состояния появится вложенное состояние глубокой истории (рис. 2.5.5б).

Чтобы обычное состояние превратить в композит, следует при добавлении нового состояния поместить его внутри границы того состояния, которое необходимо сделать композитным. В результате внутри исходного состояния появится новое вложенное состояние с именем **NewState**, которое при перемещении композита в области диаграммы состояний всегда будет находиться внутри своего композита (рис. 2.5.6).

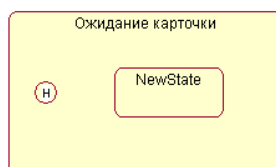


Рис. 2.5.6. Превращение состояния Ожидание карточки в композитное состояние

Рассмотренные выше действия приведены только с целью иллюстрации особенностей спецификации исторических и вложенных подсостояний и не относятся к разрабатываемой модели банкомата.

Дополнительно можно определить следующие свойства состояний: задать текстовый стереотип состояния, определить внутренние действия на входе и выходе, а также внутреннюю деятельность. Эти свойства доступны для редактирования на вкладке **General** (Общие) и **Actions** (Действия). На вкладке **Transitions** (Переходы) можно определять и редактировать переходы, которые входят и выходят из рассматриваемого состояния. Последняя вкладка **Swimlanes** (Дорожки) служит для спецификации дорожек, которые, в контексте языка UML, определяются для диаграммы деятельности.

### Добавление перехода

Для добавления перехода между двумя состояниями нужно с помощью левой кнопки мыши нажать кнопку с изображением перехода на специальной панели инструментов, отпустить левую кнопку мыши, щелкнуть левой кнопкой мыши на изображении исходного состояния на диаграмме и отпустить ее на изображении целевого состояния. В результате этих действий на диаграмме появится изображение перехода, соединяющего два выбранных состояния. Продолжая разработку модели системы управления банкоматом, добавим на диаграмму состояний начальное состояние (*Start State*) и соединим его переходом с состоянием *Ожидание карточки* (рис. 2.5.7).

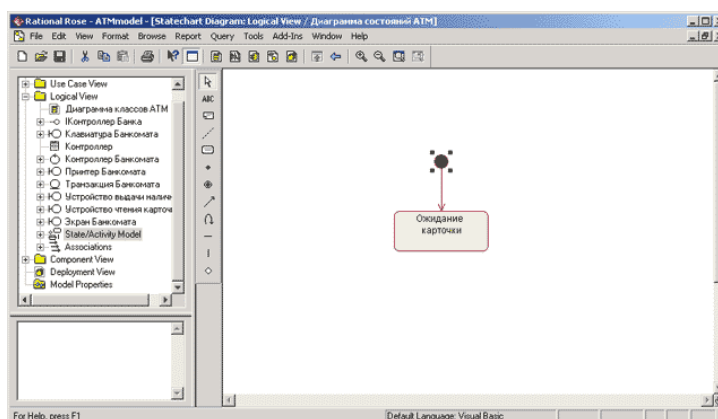


Рис. 2.5.7. Диаграмма состояний после добавления на нее перехода из начального состояния в состояние Ожидание карточки

После добавления перехода на диаграмму состояний можно открыть диалоговое окно его свойств и специфицировать дополнительные свойства, доступные на соответствующих вкладках (рис. 2.5.8). Следует обратить внимание на две первые строки вкладки **Detail** (Подробно), которые представляются наиболее важными из свойств перехода. Первое поле ввода **Guard Condition** служит для задания сторожевого условия, которое определяет правило срабатывания соответствующего перехода. Во втором поле ввода **Action** можно специфицировать действие, которое происходит при срабатывании перехода до того, как моделируемая система попадет в целевое состояние.

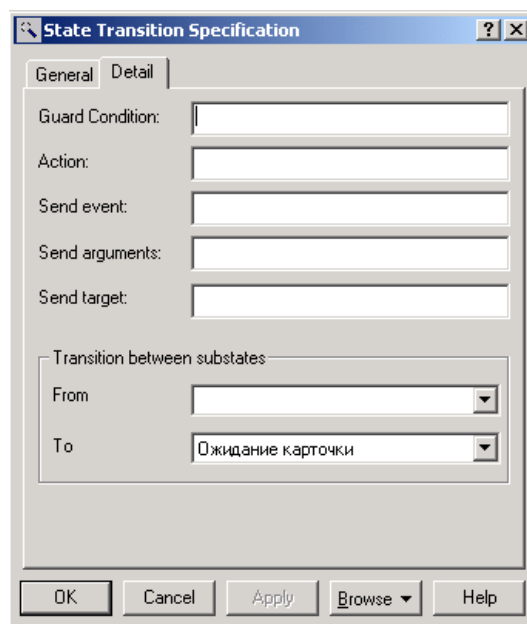


Рис. 2.5.8. Диалоговое окно спецификации свойств перехода, открытое на вкладке **Detail**

При необходимости можно определить сообщение о событии, происходящем при срабатывании перехода, а также визуализировать вложенность состояний и подключить историю отдельных состояний.

#### **Завершение диаграммы состояний модели банкомата**

Для завершения построения диаграммы состояний рассматриваемого примера следует описанным выше способом добавить оставшиеся состояния и переходы. С этой целью следует выполнить следующие действия:

1. Добавить состояния с именами: *Ожидание ввода ПИН-кода*, *Проверка ПИН-кода*, *Ожидание выбора клиента*, *Обработка запроса на снятие наличных*, *Обработка запроса на получение справки*, *Выдача наличных*, *Печать*, *Возврат карточки*, *Завершение транзакции* и финальное состояние.
2. Добавить переход: *карточка вставлена*, направленный от состояния *Ожидание карточки* к состоянию *Ожидание ввода ПИН-кода*.
3. Добавить переход: *ПИН-код введен*, направленный от состояния *Ожидание ввода ПИН-кода* к состоянию *Проверка ПИН-кода*.
4. Добавить переход: *отмена транзакции*, направленный от состояния *Ожидание ввода ПИН-кода* к состоянию *Возврат карточки*.
5. Добавить переход со сторожевым условием: [ПИН-код верный], направленный от состояния *Проверка ПИН-кода* к состоянию *Ожидание выбора клиента*.
6. Добавить переход со сторожевым условием: [ПИН-код неверный], направленный от состояния *Проверка ПИН-кода* к состоянию *Ожидание ввода ПИН-кода*.
7. Добавить переход: *три неудачи* с действием на переходе *конфискация карточки*, направленный от состояния *Проверка ПИН-кода* к состоянию *Завершение транзакции*. Для задания действия на данном переходе следует ввести текст *конфискация карточки* в поле

ввода **Action** (Действие) на вкладке **Detail** (Подробно) окна спецификации свойств данного перехода (рис.2.5.9).

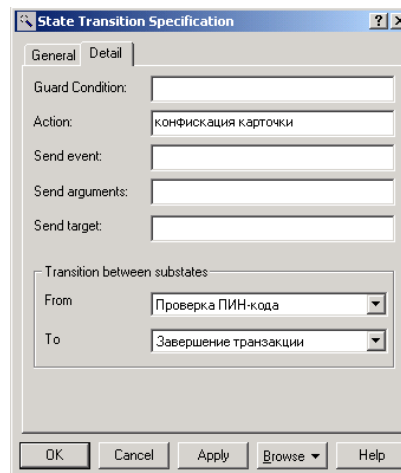


Рис. 2.5.9. Диалоговое окно спецификации свойств перехода при неудаче при задании действия на переходе

Для продолжения построения диаграммы состояний следует добавить следующие переходы:

8. *выбор суммы со сторожевым условием: [сумма введена], направленный от состояния Ожидание выбора клиента к состоянию Обработка запроса на снятие наличных.*

9. *выбор справки, направленный от состояния Ожидание выбора клиента к состоянию Обработка запроса на получение справки.*

10. *отмена транзакции, направленный от состояния Ожидание выбора клиента к состоянию Возврат карточки.*

11. *со сторожевым условием: [кредит не превышен], направленный от состояния Обработка запроса на снятие наличных к состоянию Выдача наличных.*

12. *со сторожевым условием: [кредит превышен] с действием на переходе сообщение, направленный от состояния Обработка запроса на снятие наличных к состоянию Возврат карточки.*

13. *наличные выданы со сторожевым условием: [выбрана печать чека], направленный от состояния Выдача наличных к состоянию Печать.*

14. *наличные выданы со сторожевым условием: [печать чека не выбрана], направленный от состояния Выдача наличных к состоянию Возврат карточки.*

15. *справка сформирована, направленный от состояния Обработка запроса на получение справки к состоянию Печать.*

16. *печать закончена, направленный от состояния Печать к состоянию Возврат карточки.*

17. *карточка возвращена, направленный от состояния Возврат карточки к состоянию Завершение транзакции.*

18. *транзакция завершена, направленный от состояния Завершение транзакции к состоянию Ожидание карточки.*

19. *направленный от состояния Ожидание карточки к финальному состоянию.*

Диаграмма состояний для рассматриваемой модели банкомата будет иметь следующий вид (рис. 2.5.10).

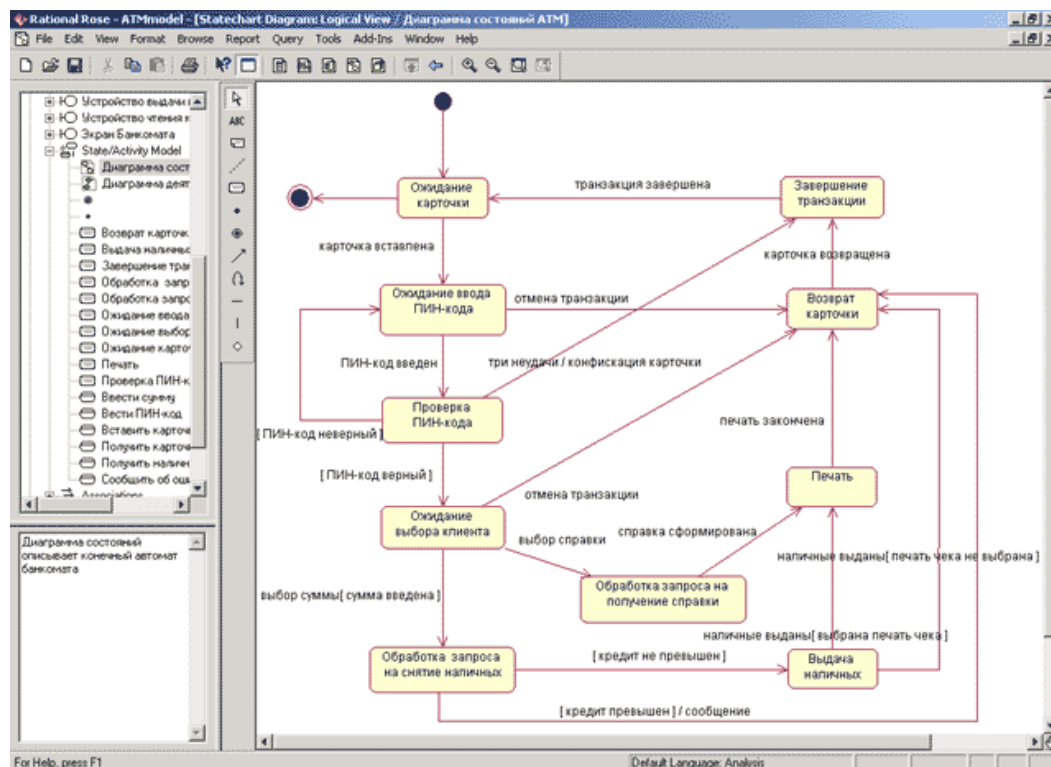


Рис. 2.5.10. Окончательный вид диаграммы состояний для моделирования поведения банкомата

Следует заметить, что в разрабатываемой модели диаграмма состояний является единственной и описывает поведение системы управления банкоматом в целом. Главное достоинство данной диаграммы состояний - возможность моделировать условный характер реализации всех прецедентов в форме изменения отдельных состояний разрабатываемой системы. В то же время в среде IBM Rational Rose данная диаграмма не является необходимой для генерации программного кода. Поэтому в случае дублирования информации, представленной на диаграммах кооперации и последовательности, разработку диаграммы состояний, особенно в условиях дефицита времени, отпущенного на выполнение проекта, иногда опускают.

## 2.5.2 Лабораторная работа № 10. Activity diagrams

### Цель работы:

- изучение диаграмм деятельности,
- изучение их применения в процессе проектирования.

Диаграмма деятельности в среде IBM Rational Rose, так же как и диаграмма состояний, может относиться к отдельному классу, операции класса, прецеденту, пакету или представлению. Для того чтобы построить диаграмму деятельности, ее вначале необходимо создать и активизировать.

Начать построение диаграммы деятельности для выбранного элемента модели или моделируемой системы в целом можно одним из следующих способов:

- Щелкнуть на кнопке с изображением диаграммы состояний на стандартной панели инструментов, после чего следует выбрать представление и тип разрабатываемой диаграммы - диаграмма деятельности.

- Выделить логическое представление (Logical View) или представление прецедентов (Use Case View) в браузере проекта и выполнить операцию контекстного меню: **New→Activity Diagram** (Новая→Диаграмма деятельности).

- Раскрыть логическое представление (Logical View) в браузере проекта и выделить рассматриваемый класс, операцию класса, пакет, или раскрыть представление прецедентов (Use Case View) и выбрать прецедент, после чего выполнить операцию контекстного меню: **NewActivity Diagram** (НоваяДиаграмма деятельности).

- Выполнить операцию главного меню: **Browse→State Machine Diagram** (Обзор→Диаграмма состояний), после следует чего выбрать представление и тип разрабатываемой диаграммы - диаграмма деятельности.

В результате выполнения этих действий появляется новое окно с чистым рабочим листом диаграммы деятельности и специальная панель инструментов, содержащая кнопки с изображением графических элементов, необходимых для разработки диаграммы деятельности. Назначение отдельных кнопок панели указано в таблице 25, а также можно узнать из всплывающих подсказок.

Таблица 25.

Назначение кнопок специальной панели инструментов диаграммы деятельности

| Кнопка                                                                              | Всплывающая подсказка      | Назначение кнопки                                                                                |
|-------------------------------------------------------------------------------------|----------------------------|--------------------------------------------------------------------------------------------------|
|  | Selection Tool             | Превращает изображение курсора в форму стрелки для последующего выделения элементов на диаграмме |
|  | Text Box                   | Добавляет на диаграмму текстовую область                                                         |
|  | Note                       | Добавляет на диаграмму примечание                                                                |
|  | Anchor Note to Item        | Добавляет на диаграмму связь примечания с соответствующим графическим элементом диаграммы        |
|  | State                      | Добавляет на диаграмму состояние                                                                 |
|  | Activity                   | Добавляет на диаграмму деятельность                                                              |
|  | Start State                | Добавляет на диаграмму начальное состояние                                                       |
|  | End State                  | Добавляет на диаграмму конечное состояние                                                        |
|  | State Transition           | Добавляет на диаграмму переход                                                                   |
|  | Transition to Self         | Добавляет на диаграмму рефлексивный переход                                                      |
|  | Horizontal Synchronization | Добавляет на диаграмму горизонтально расположенный символ синхронизации                          |
|  | Vertical Synchronization   | Добавляет на диаграмму вертикально расположенный символ синхронизации                            |
|  | Decision                   | Добавляет на диаграмму символ принятия решения для альтернативных переходов                      |

|                                                                                   |                      |                                                                           |
|-----------------------------------------------------------------------------------|----------------------|---------------------------------------------------------------------------|
|  | Swimlane             | Добавляет на диаграмму дорожку                                            |
|  | Object               | Добавляет на диаграмму объект (по умолчанию отсутствует)                  |
|  | Object Flow          | Добавляет на диаграмму стрелку потока объектов (по умолчанию отсутствует) |
|  | Business Activity    | Добавляет на диаграмму бизнес-деятельность (по умолчанию отсутствует)     |
|  | Business Transaction | Добавляет на диаграмму бизнес-транзакцию (по умолчанию отсутствует)       |

Как видно из этой таблицы, по умолчанию на панели инструментов отсутствуют некоторые графические элементы, а именно - кнопки с пиктограммами объекта и потока объектов. При необходимости их можно добавить на специальную панель диаграммы деятельности стандартным способом, который был описан ранее.

Для разрабатываемого проекта системы управления банкоматом диаграмма деятельности описывает последовательность действий клиента при использовании банкомата. Для удобства можно включить эту диаграмму в логическое представление, для чего необходимо в браузере проекта выделить логическое представление (Logical View) и выполнить операцию контекстного меню: **New→Activity Diagram** (Новая→Диаграмма деятельности). Продолжая разработку проекта по моделированию системы управления банкоматом, можно приступить к разработке новой диаграммы деятельности. С этой целью для диаграммы деятельности модели банкомата зададим имя *Диаграмма деятельности АТМ*, а в секцию ее документации введем текст «Диаграмма деятельности описывает последовательность действий клиента при использовании банкомата».

### Добавление и редактирование деятельности

Для добавления деятельности на диаграмму деятельности нужно с помощью левой кнопки мыши нажать кнопку с изображением пиктограммы деятельности на специальной панели инструментов, отпустить левую кнопку мыши и щелкнуть левой кнопкой мыши на свободном месте рабочего листа диаграммы. На диаграмме появится изображение деятельности с маркерами изменения его геометрических размеров и предложенным средой именем по умолчанию, которое разработчику следует изменить. Добавить деятельность на диаграмму можно также с помощью операции главного меню: **Tools→Create→Activity** или с помощью операции контекстного меню: **New→Activity**, предварительно выделив диаграмму деятельности в браузере проекта.

В результате этих действий на диаграмме появится изображение деятельности с именем NewActivity, предложенное программой по умолчанию. Начиная построение диаграммы деятельности модели банкомата, для первой добавленной деятельности зададим имя *Вставить карточку* (Рис.9.11).

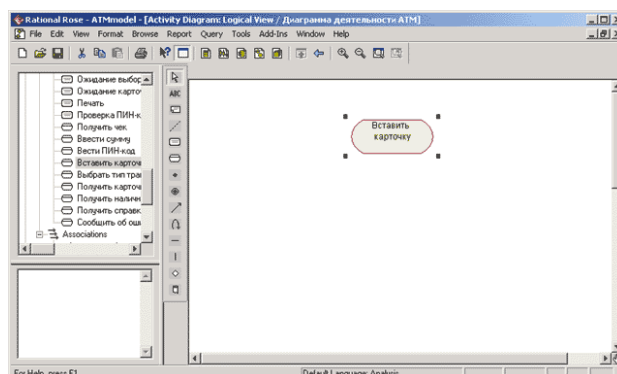


Рис. 2.5.11. Диаграмма деятельности после добавления на нее деятельности Вставить карточку

После добавления деятельности на диаграмму деятельности можно открыть диалоговое окно спецификации ее свойств и определить дополнительные свойства деятельности, доступные на соответствующих вкладках.

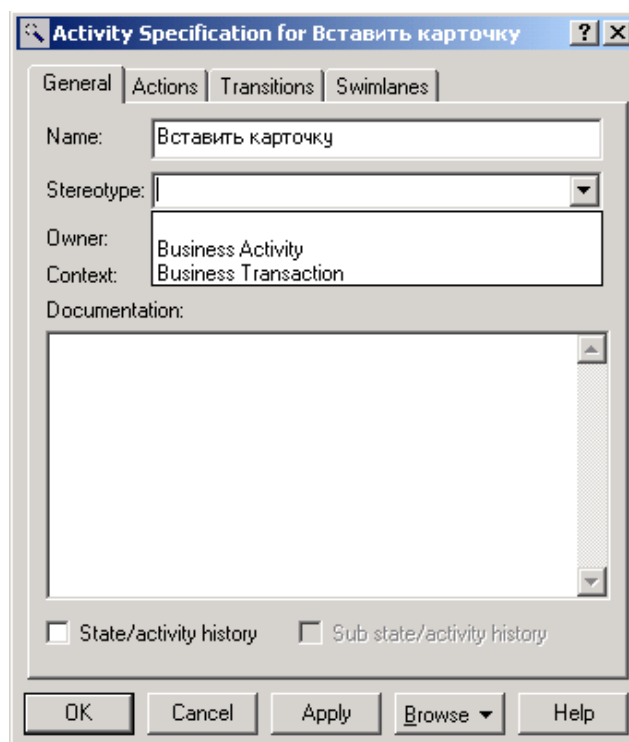


Рис. 2.5.12. Диалоговое окно спецификации свойств деятельности

При этом для деятельности становятся доступными для выбора два стереотипа: **BusinessActivity** (Бизнес-деятельность) и **BusinessTransaction** (Бизнес-транзакция), которые имеют собственное графическое изображение. На вкладке **Transitions** (Переходы) окна спецификации свойств деятельности можно определять и редактировать переходы, которые входят и выходят из рассматриваемой деятельности. Последняя вкладка **Swimlanes** (Дорожки) служит для спецификации дорожки, на которую помещается рассматриваемая деятельность.

Хотя программа IBM Rational Rose позволяет определить свойства деятельности, доступные на вкладке **Actions** (Действия), следует помнить, что внутренние действия являются свойствами общего понятия состояния, а внутренняя деятельность служит именем собственно деятельности, помещаемой на диаграмму деятельности. Поэтому для деятельности во избежание недоразумений лучше оставить эту вкладку пустой.

### Добавление перехода

Добавление перехода на диаграмму деятельности полностью аналогично диаграмме состояний. А именно, для добавления перехода между двумя деятельностями нужно с помощью левой кнопки мыши нажать кнопку с изображением перехода на специальной панели инструментов, отпустить левую кнопку мыши, щелкнуть левой кнопкой мыши на изображении исходной деятельности на диаграмме и отпустить ее на изображении целевой деятельности. В результате этих действий на диаграмме появится изображение перехода, соединяющего две выбранных деятельности. Если в качестве одной из деятельностей является символ ветвления или соединения, то порядок добавления перехода сохраняется прежним.

Следует заметить, что при наличии в проекте законченной диаграммы состояний попытка добавить начальное состояние на диаграмму деятельности с помощью кнопки специальной панели инструментов окажется безуспешной. В этом случае программа IBM Rational Rose

фиксирует наличие в модели начального состояния и не позволит добавить его с помощью соответствующей кнопки на разрабатываемые диаграммы состояний или деятельности. Решить данную проблему можно посредством перетаскивания с помощью мыши начального состояния из браузера проекта на любую из вновь разрабатываемых диаграмм.

После добавления перехода на диаграмму деятельности становятся доступными для редактирования его свойства в специальном диалоговом окне (рис. 2.5.13), которое можно открыть по двойному щелчку левой кнопкой мыши на изображении перехода.

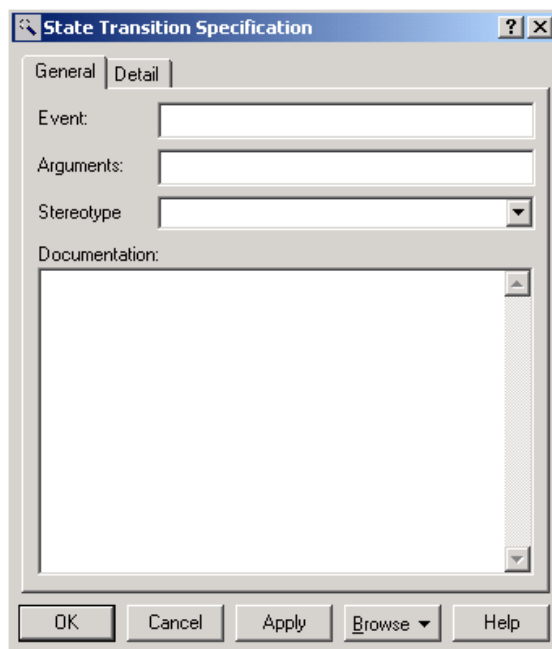


Рис. 2.5.13. Диалоговое окно спецификации свойств перехода

При спецификации свойств переходов следует помнить, что все переходы на диаграмме деятельности являются нетриггерными, т.е. не имеют имен событий. По этой причине поле ввода с именем **Event** (Событие) для всех переходов должно оставаться пустым. Но все переходы, выходящие из символов ветвления (решения), должны иметь сторожевые условия, которые специфицируются на вкладке **Detail** (Подробно) диалогового окна спецификации свойств перехода.

#### Завершение диаграммы деятельности модели банкомата

Для завершения построения диаграммы деятельности рассматриваемого примера следует выполнить следующие действия:

1. Добавить деятельности с именами: *Ввести ПИН-код*, *Выбрать тип транзакции*, *Ввести сумму*, *Получить справку о состоянии счета*, *Получить наличные*, *Получить чек*, *Получить карточку* и *финальное состояние*.

2. Добавить символы ветвления (решения), расположив их между деятельностями с именами: *Ввести ПИН-код* и *Выбрать тип транзакции*, *Выбрать тип транзакции* и *Ввести сумму*, *Ввести сумму* и *Получить справку о состоянии счета*, *Получить наличные* и *Получить чек*, *Получить чек* и *Получить карточку*. При этом последний символ решения будет использоваться в качестве символа соединения.

3. Добавить переход, направленный от деятельности *Ввести ПИН-код* к символу решения.

4. Добавить переход со сторожевым условием: [ПИН-код верный], направленный от символа решения к деятельности *Выбрать тип транзакции*. Для задания сторожевого условия данного перехода следует ввести текст *ПИН-код верный* в поле ввода **GuardCondition** (Сторожевое условие) на вкладке **Detail** окна спецификации свойств

данного перехода (рис. 2.5.14). При этом текст сторожевого условия следует вводить без скобок.

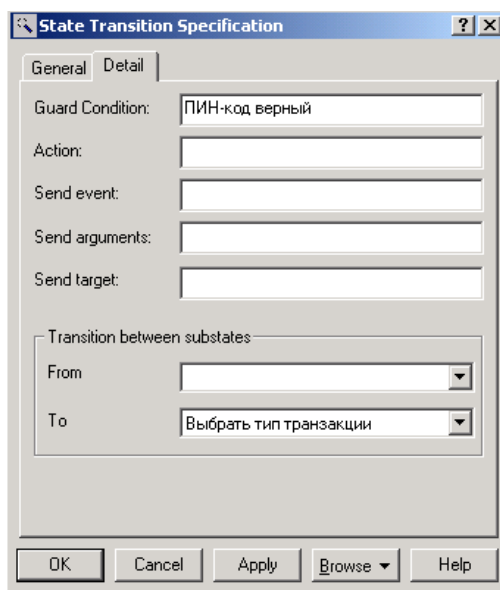


Рис. 2.5.14. Диалоговое окно спецификации свойств перехода при задании сторожевого условия

Для продолжения построения диаграммы деятельности следует добавить следующие переходы:

5. со сторожевым условием: [ПИН-код неверный], направленный от символа решения к символу соединения.

6. направленный от деятельности *Выбрать тип транзакции* к символу решения.

7. со сторожевым условием: [выбор снятия суммы], направленный от символа решения к деятельности *Ввести сумму*.

8. со сторожевым условием: [выбор получения справки], направленный от символа решения к деятельности *Получить справку о состоянии счета*.

9. направленный от деятельности *Ввести сумму* к символу решения.

10. со сторожевым условием: [сумма не превышает кредит], направленный от символа решения к деятельности *Получить наличные*.

11. со сторожевым условием: [сумма превышает кредит], направленный от символа решения к символу соединения.

12. направленный от деятельности *Получить наличные* к символу решения.

13. со сторожевым условием: [выбрана печать чека], направленный от символа решения к деятельности *Получить чек*.

14. со сторожевым условием: [печать чека не выбрана], направленный от символа решения к символу соединения.

15. направленный от деятельности *Получить чек* к символу соединения.

16. направленный от деятельности *Получить справку о состоянии счета* к символу соединения.

17. направленный от символа соединения к деятельности *Получить карточку*.

18. направленный от деятельности *Получить карточку* к финальному состоянию.

Построенная таким образом диаграмма деятельности будет иметь следующий вид (рис. 2.5.15).

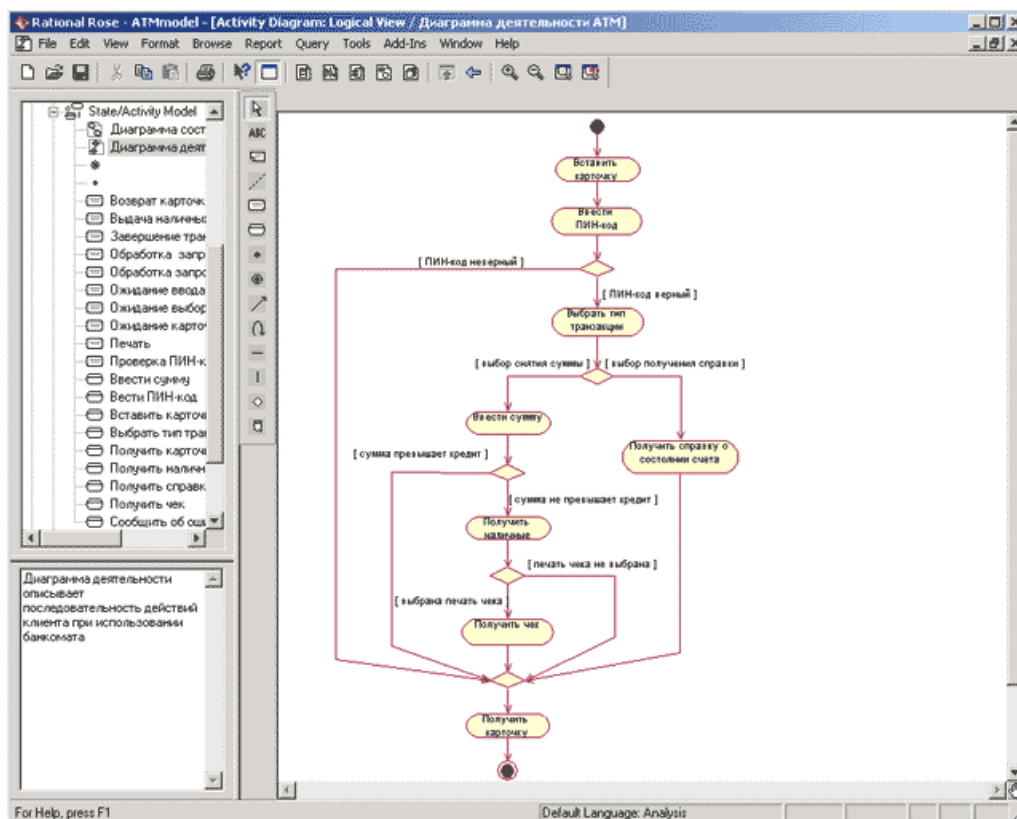


Рис. 2.5.15. Окончательный вид диаграммы деятельности для модели банкомата

Следует заметить, что в разрабатываемой модели диаграмма деятельности не описывает ситуацию блокирования карточки при трижды неверно введенном ПИН-коде. Дополнить данную диаграмму деятельности, которая учитывает данное условие в форме проверки отдельного условия, предлагается читателям самостоятельно в качестве упражнения.

Следует помнить, что в среде IBM Rational Rose диаграмма деятельности не является необходимой для генерации программного кода. Поэтому разработку диаграмм этого типа, особенно в условиях дефицита времени, отпущенного на выполнение проекта, иногда опускают. В то же время следует отметить, что в проектах реинжиниринга и документирования бизнес-процессов диаграмма деятельности является основным средством визуализации бизнес-процессов в контексте языка UML.

### 2.5.3 Лабораторная работа № 12. Bussiness-Activity diagram

Продолжая рассмотрение особенностей разработки диаграмм деятельности, следует отметить, что программа IBM Rational Rose может быть успешно использована для выполнения проектов по моделированию бизнес-процессов. Наиболее подходящим типом диаграмм для визуального представления схем выполнения бизнес-процессов являются диаграммы деятельности, на которых дополнительно размещаются так называемые дорожки (**Swimlane**). Назначение дорожек состоит в том, чтобы указать зоны ответственности за выполнения отдельных деятельностей в рамках моделируемого бизнес-процесса. В качестве имен дорожек используются либо названия подразделений (департаментов) рассматриваемой компании, либо названия отдельных должностей сотрудников тех или иных подразделений.

Проекты по моделированию бизнес-процессов могут выполняться либо с целью реорганизации или реинжиниринга компании, либо с целью собственно документирования бизнес-процессов. Особенности данных проектов заключаются в том, что в обоих случаях необходимо построить модели бизнес-процессов некоторой существующей компании. Чтобы акцентировать внимание на подобных проектах, их часто называют проектами типа «As-is»

(«Как есть»). Соответственно проекты по разработке новых продуктов или моделей новых систем называют проектами типа «*To-be*» («Как должно быть»).

В данном контексте рассматриваемый ранее проект по разработке системы управления банкоматом следует отнести к проектам типа «*Как есть*», поскольку при построении диаграмм предполагалась известная существующая технология использования банкоматов для обслуживания клиентов. С другой стороны, если бы стояла цель разработки новой модели банкомата с некоторой дополнительной функциональностью или, например, разработки нового Интернет-магазина, то подобные проекты можно было бы отнести к проектам типа «*Как должно быть*». Именно этот тип проектов служит базовым для принятой в курсе лекций последовательности разработки канонических диаграмм в нотации UML, начиная от представления диаграмм прецедентов и заканчивая диаграммами физического представления.

Выполнение проектов типа «*Как есть*» по моделированию бизнес-процессов в большинстве случаев начинают с построения диаграмм деятельности, которые служат для графического представления схем выполнения бизнес-процессов и документооборота рассматриваемой компании. После этого, исходя из требований проекта, разрабатывается модель диаграммы прецедентов и выполняется реорганизация бизнес-процессов. Наконец, в случае необходимости разработки или внедрения корпоративной информационной системы, строятся диаграмма классов, диаграммы взаимодействия и компонентов, которые служат основой для программной реализации соответствующего проекта.

Таким образом, первый этап выполнения проектов типа «*Как есть*» связан с построением моделей существующих бизнес-процессов компании в форме диаграмм деятельности. В качестве примера проекта этого типа в данной лекции рассматривается модель бизнес-процесса по оптовой продаже товаров со склада торговой компании. Хотя данный пример имеет упрощенный характер, он позволяет наглядно представить основные особенности моделирования бизнес-процессов в нотации языка UML с использованием средства IBM Rational Rose.

Для вновь разрабатываемого проекта по моделированию бизнес-процессов торговой компании в среде IBM Rational Rose создадим новый проект с именем: *МодельБП*. В качестве первой диаграммы проекта будет служить диаграмма деятельности, которая описывает отдельный бизнес-процесс в виде последовательности выполнения действий подразделениями компании при оптовой продаже товаров клиентам. Для удобства можно включить эту диаграмму в логическое представление, для чего необходимо в браузере проекта выделить логическое представление (LogicalView) и выполнить операцию контекстного меню: **New→ActivityDiagram** (Новая→Диаграмма деятельности).

#### Добавление дорожек на диаграмму деятельности

Для представления модели бизнес-процесса в форме диаграммы деятельности первоначально необходимо добавить на нее дорожки. Для добавления дорожки на диаграмму деятельности нужно с помощью левой кнопки мыши нажать кнопку с изображением пиктограммы дорожки на специальной панели инструментов, отпустить левую кнопку мыши и щелкнуть левой кнопкой мыши на свободном месте рабочего листа диаграммы. Добавить дорожку на диаграмму можно также с помощью операции главного меню: **Tools→Create→Swimlane** или с помощью операции контекстного меню: **New→Swimlane**, предварительно выделив диаграмму деятельности в браузере проекта.

В результате этих действий на диаграмме в области диаграммы появится изображение дорожки с вертикальной линией и именем дорожки NewSwimlane в верхней части, предложенное программой по умолчанию. Для задания имени дорожки следует открыть диалоговое окно спецификации ее свойств и ввести ее имя в поле ввода **Name** (рис. 2.5.16).

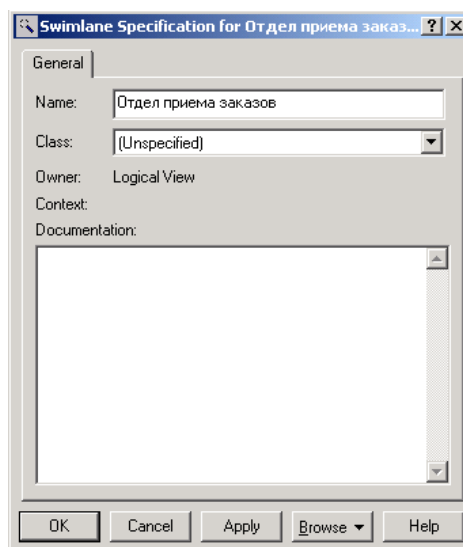


Рис. 2.5.16. Диалоговое окно спецификации свойств дорожки

Начиная практическую разработку модели бизнес-процесса оптовой продажи товаров со склада компании, последовательно добавим на диаграмму деятельности дорожки с именами отдельных подразделений компании: *Отдел приема заказов*, *Бухгалтерия*, *Склад* и *Отдел доставки* (рис. 2.5.17).

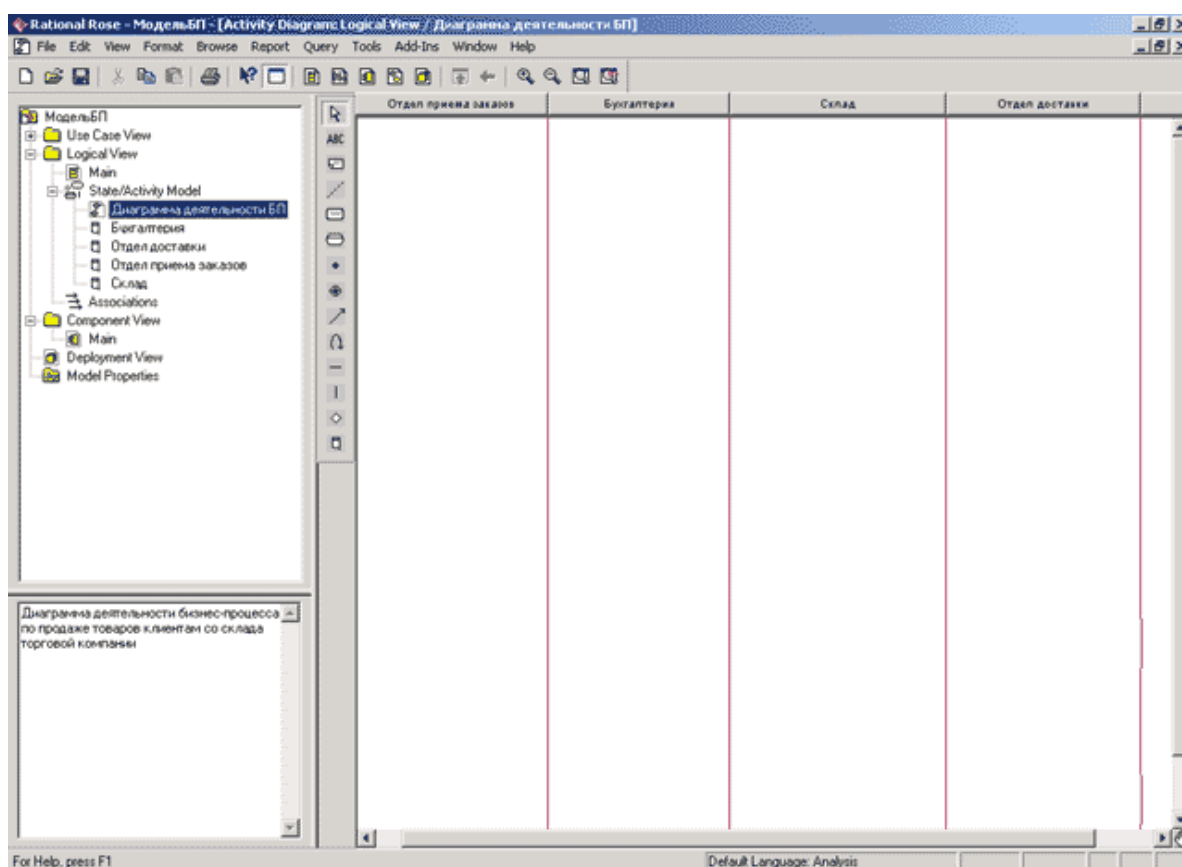


Рис. 2.5.17. Диаграмма деятельности после добавления на нее дорожек

После добавления дорожек на диаграмму состояний можно перейти к добавлению деятельности и переходов. В качестве первой деятельности добавим деятельность с именем *Принять заказ по факсу*, которую разместим в первой дорожке с именем *Отдел приема заказов*. Этот факт будет означать, что деятельность *Принять заказ по факсу* выполняется в

Отделе приема заказов или, другими словами, сотрудники этого отдела несут ответственность за выполнение данной деятельности.

Деятельности *Принять заказ по факсу* должно предшествовать начальное состояние, которое также следует добавить в эту же дорожку и соединить переходом с этой деятельностью. После добавления начального состояния и перехода диаграмма деятельности будет иметь следующий вид (рис. 2.5.18).

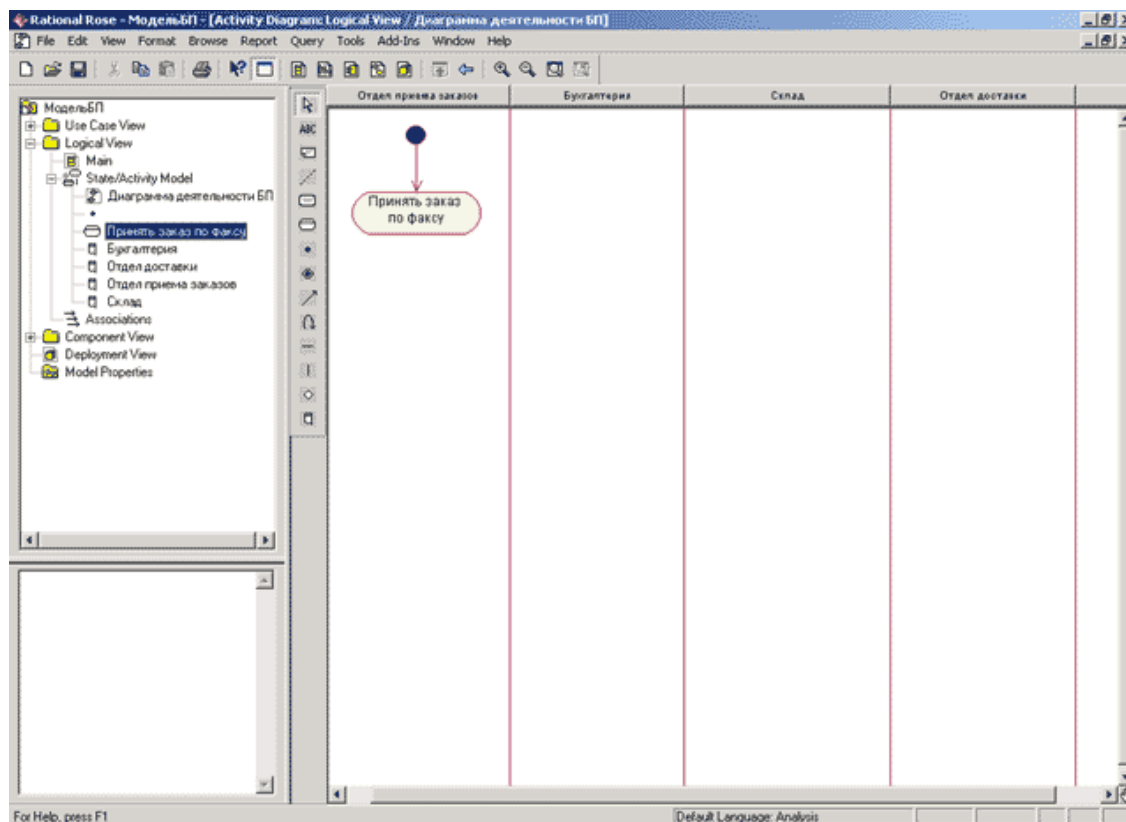


Рис. 2.5.18. Диаграмма деятельности после добавления на нее перехода из изначального состояния в деятельность *Принять заказ по факсу*

Далее следует добавить оставшиеся деятельности и переходы. С этой целью следует выполнить следующие действия:

1. Добавить деятельность с именем: *Заказать товар на складе* в дорожку *Отдел приема заказов*.
2. Добавить деятельности с именами: *Выставить счет к оплате* и *Получить оплату за товар* в дорожку *Бухгалтерия*.
3. Добавить деятельности с именами: *Подобрать товар* и *Подготовить товар к отправке* в дорожку *Склад*.
4. Добавить деятельность с именем: *Отправить товар клиенту* в дорожку *Отдел доставки*.
5. Добавить символ горизонтальной синхронизации в дорожки *Отдел приема заказов* и *Склад*. Следует заметить, что первый символ будет использован для разделения параллельных потоков деятельностей, а второй - для слияния этих потоков.
6. Добавить переход, направленный от деятельности *Принять заказ по факсу* к деятельности *Заказать товар на складе*.
7. Добавить переход, направленный от деятельности *Заказать товар на складе* к символу горизонтальной синхронизации.
8. Добавить переход, направленный от символа горизонтальной синхронизации к деятельности *Выставить счет к оплате*.

9. Добавить переход, направленный от символа горизонтальной синхронизации к деятельности *Подобрать товар*.
10. Добавить переход, направленный от деятельности *Выставить счет к оплате* к деятельности *Получить оплату за товар*.
11. Добавить переход, направленный от деятельности *Подобрать товар* к деятельности *Подготовить товар к отправке*.
12. Добавить переход, направленный от деятельности *Получить оплату за товар* к символу горизонтальной синхронизации.
13. Добавить переход, направленный от деятельности *Подготовить товар к отправке* к символу горизонтальной синхронизации.
14. Добавить переход, направленный от символа горизонтальной синхронизации к деятельности *Отправить товар клиенту*.
15. Добавить переход, направленный от деятельности *Отправить товар клиенту* к финальному состоянию.

Построенная таким образом диаграмма деятельности с дорожками будет иметь следующий вид (рис. 2.5.19).

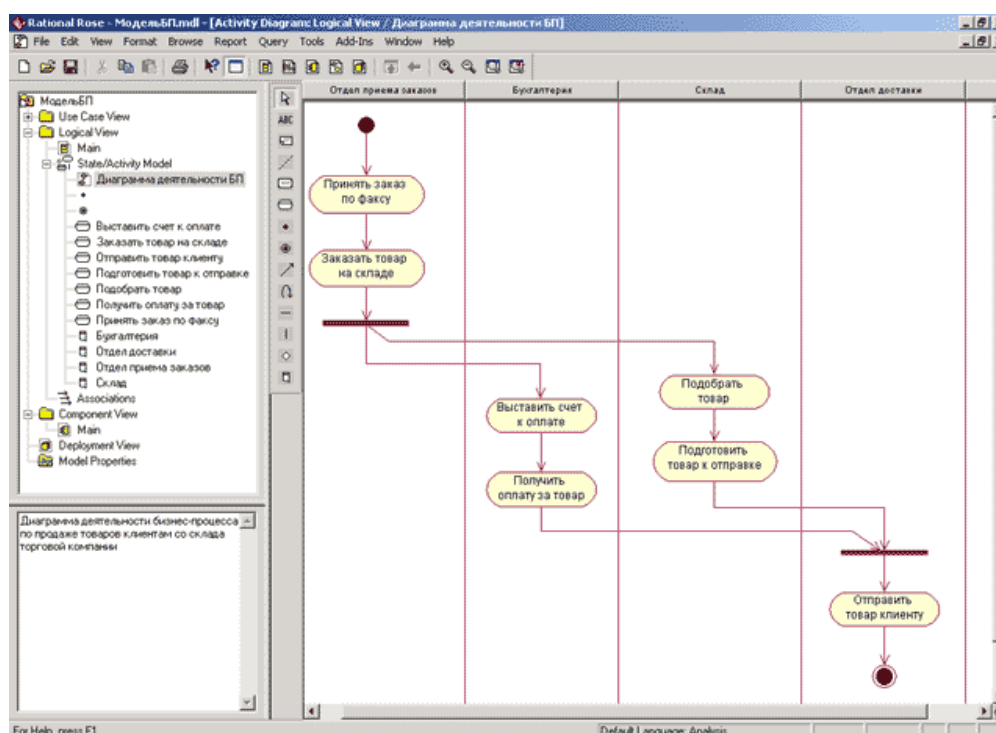


Рис. 2.5.19. Диаграмма деятельности с дорожками для модели бизнес-процесса

Следует заметить, что в разрабатываемой модели диаграмма деятельности не описывает ситуацию, когда заказанного клиентом товара не окажется на складе.

### Добавление потоков объектов

Для построения диаграммы деятельности с дорожками и потоком объектов для рассматриваемой модели бизнес-процесса следует добавить на диаграмму объекты и стрелки потоков объектов. Объекты на диаграмме деятельности могут обозначать отдельные документы, которые необходимы для выполнения моделируемого бизнес-процесса. Соответственно поток объектов служит моделью документооборота рассматриваемой компании. Для добавления на диаграмму объекта следует воспользоваться соответствующей кнопкой на специальной панели инструментов. При этом данную кнопку предварительно следует на нее добавить, поскольку по умолчанию на панели она отсутствует.

В качестве первого объекта добавим на диаграмму деятельности объект с именем заказ, для которого зададим состояние: *получен*. Для задания состояния добавленного объекта следует открыть диалоговое окно свойств данного объекта, во вложенном списке **State** (Состояние) выбрать нужное состояние или задать новое (рис. 2.5.20). При этом будет открыто дополнительное окно свойств состояния, в которое можно занести всю информацию по данному состоянию.

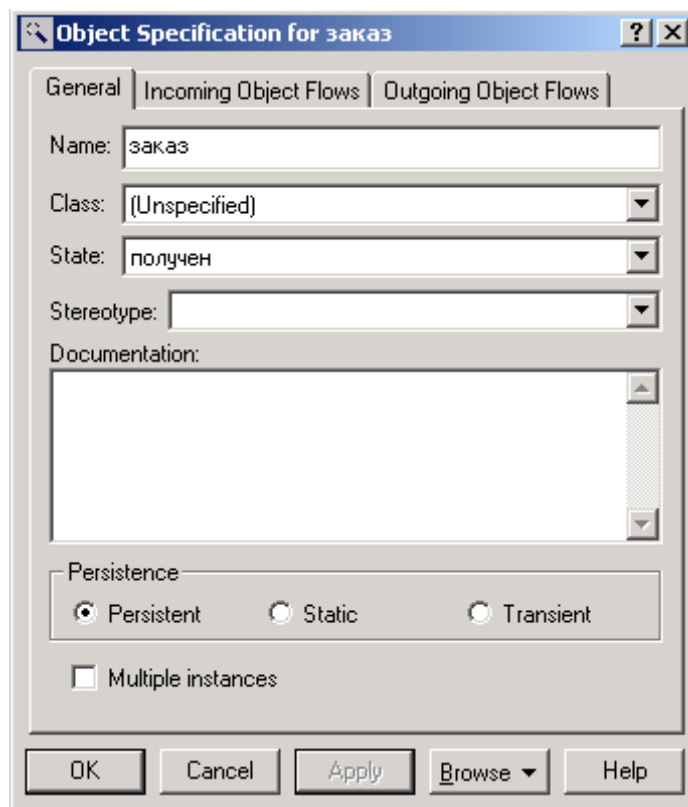


Рис. 2.5.20. Диалоговое окно спецификации свойств объекта

Для завершения построения диаграммы деятельности рассматриваемого примера следует описанным выше способом добавить оставшиеся объекты и стрелки потоков объектов. С этой целью следует выполнить следующие действия:

1. Добавить стрелку потока объектов, направленную от деятельности *Принять заказ по факсу* к объекту *заказ* в состоянии *получен*.
2. Добавить стрелку потока объектов, направленную от объекта *заказ* в состоянии *получен* к деятельности *Заказать товар на складе*.
3. Добавим объект с именем *заказ*, для которого зададим состояние: *оформлен*. Следует заметить, что для добавления на диаграмму деятельности уже существующего в модели объекта его следует просто перетащить из браузера проекта на диаграмму и задать ему новое состояние.
4. Добавить стрелку потока объектов, направленную от деятельности *Заказать товар на складе* к объекту *заказ* в состоянии *оформлен*.
5. Добавить стрелку потока объектов, направленную от объекта *заказ* в состоянии *оформлен* к деятельности *Выставить счет к оплате*.
6. Добавим объект с именем *Счет*, для которого зададим состояние: *выставлен*.
7. Добавить стрелку потока объектов, направленную от деятельности *Выставить счет к оплате* к объекту *Счет* в состоянии *выставлен*.
8. Добавить стрелку потока объектов, направленную от объекта *Счет* в состоянии *выставлен* к деятельности *Получить оплату за товар*.
9. Добавим объект с именем *Счет*, для которого зададим состояние: *оплачен*.

10. Добавить стрелку потока объектов, направленную от деятельности *Получить оплату за товар* к объекту *Счет* в состоянии *оплачен*.
11. Добавить стрелку потока объектов, направленную от объекта *Счет* в состоянии *оплачен* к деятельности *Отправить товар клиенту*.
12. Добавим объект с именем *накладная*, для которого зададим состояние: *выписана*.
13. Добавить стрелку потока объектов, направленную от деятельности *Заказать товар на складе* к объекту *накладная* в состоянии *выписана*.
14. Добавить стрелку потока объектов, направленную от объекта *накладная* в состоянии *выписана* к деятельности *Подобрать товар*.
15. Добавим объект с именем *накладная*, для которого зададим состояние: *оформлена*.
16. Добавить стрелку потока объектов, направленную от деятельности *Подготовить товар к отправке* к объекту *накладная* в состоянии *оформлена*.
17. Добавить стрелку потока объектов, направленную от объекта *накладная* в состоянии *оформлена* к деятельности *Отправить товар клиенту*.

Построенная таким образом диаграмма деятельности с дорожками и потоком объектов будет иметь следующий вид (рис. 2.5.21).

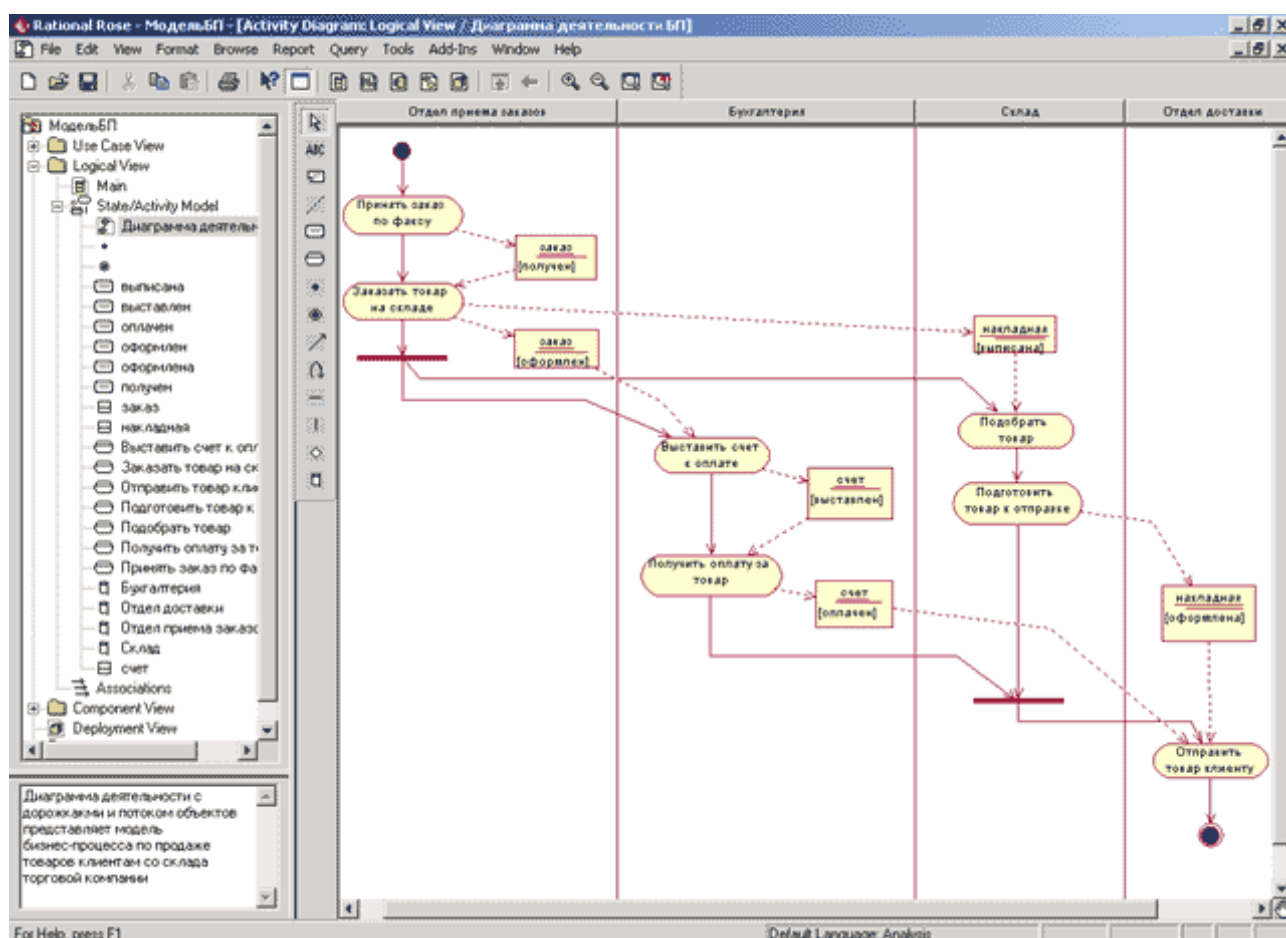


Рис. 2.5.21. Окончательный вид диаграммы деятельности для модели бизнес-процесса

Для большей наглядности представления данной модели можно задать для всех деятельности стереотип **BusinessActivity** (Бизнес-деятельность) с помощью выбора нужного варианта стереотипа в окне спецификации свойств деятельности. Соответствующий вариант изображения диаграммы деятельности представлен на рис. 2.5.22.

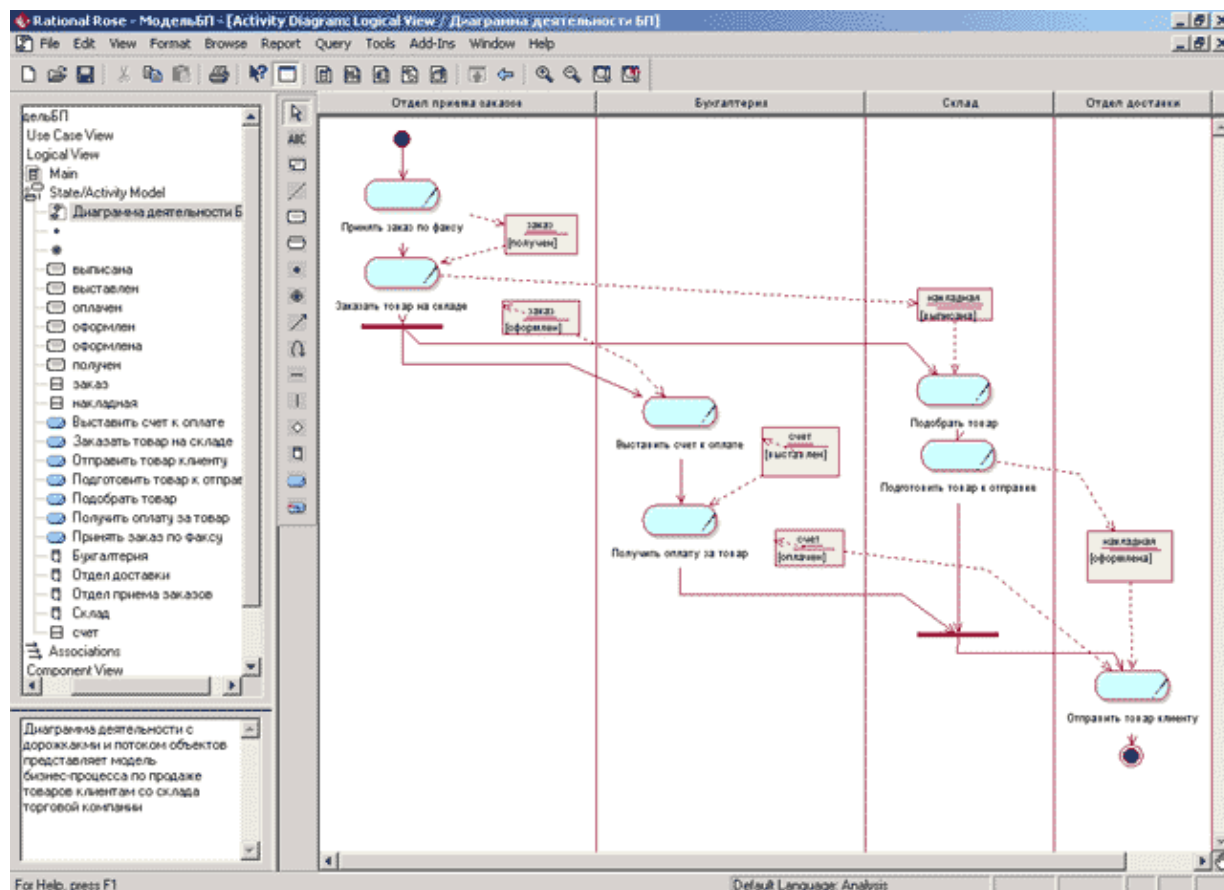


Рис. 2.5.22. Окончательный вид диаграммы деятельности для модели бизнес-процесса со стереотипами деятельности

Следует заметить, что в разрабатываемой модели диаграмма деятельности не описывает ситуацию, когда клиент отказался от оплаты товара после выставления ему счета.

Хотя в среде IBM Rational Rose диаграмма деятельности не является необходимой для генерации программного кода, диаграммы данного типа имеют большое значение для документирования бизнес-процессов и их последующей сертификации по международному стандарту ISO 9000. Поэтому разработка диаграмм этого типа занимает центральное место при выполнении проектов по реинжинирингу и оптимизации бизнес-процессов с использованием нотации UML.

#### 2.5.4 Задания

1. Выбрать в моделируемой системе классы, для объектов которых будут строиться диаграммы состояний.
2. Построить для каждого выбранного класса диаграмму состояний, характеризующую поведение его объектов в нескольких прецедентах.

#### 2.5.5 Контрольные вопросы

1. Каково назначение диаграмм состояния?
2. Как отображаются действия и деятельности на диаграммах состояния?
3. Что такое условный переход и как он описывается на диаграмме?
4. Какие особые состояния объекта отображаются на диаграмме?
5. Каковы преимущества и недостатки диаграмм состояния?

## 2.6 Диаграммы пакетов, компонентов и размещения

### Диаграммы пакетов (package diagrams)

Один из важнейших вопросов методологии создания программного обеспечения - как разбить большую систему на небольшие подсистемы? Именно с этой точки зрения изменения, связанные с переходом от структурного подхода к объектно-ориентированному, являются наиболее заметными. Одна из идей заключается в группировке классов в компоненты более высокого уровня. В UML такой механизм группировки носит название пакетов (package).

Диаграммой пакетов является диаграмма, содержащая пакеты классов и зависимости между ними. Строго говоря, пакеты и зависимости являются элементами диаграммы классов, т. е. диаграмма пакетов - это всего лишь форма диаграммы классов. Однако на практике причины построения таких диаграмм различны.

Зависимость между двумя элементами имеет место в том случае, если изменения в определении одного элемента могут повлечь за собой изменение в другом. Что касается классов, то причины зависимостей могут быть самыми разными: один класс посылает сообщение другому; один класс включает часть данных другого класса; один класс ссылается на другой как на параметр операции. Если класс меняет свой интерфейс, то любое сообщение, которое он посылает, может стать неправильным.

В идеальном случае только изменения в интерфейсе класса должны воздействовать на другие классы. Искусство проектирования больших систем включает в себя минимизацию зависимостей, которая снижает воздействие изменений и требует меньше усилий на их внесение.

### Пакеты UML

Пакеты представляют собой универсальный механизм организации элементов в группы. В пакет можно поместить диаграммы различного типа и назначения. В отличие от компонентов, существующих во время работы программы, пакеты носят чисто концептуальный характер, то есть существуют только во время разработки. Изображается пакет в виде папки с закладкой, содержащей, как правило, только имя и иногда — описание содержимого.

Диаграмма пакетов содержит пакеты классов и зависимости между ними. Зависимость между двумя пакетами имеет место в том случае, если изменения в определении одного элемента влекут за собой изменения в другом. По отношению к пакетам можно использовать механизм обобщения.

### Диаграммы компонентов

Диаграммы компонентов позволяют изобразить модель системы на физическом уровне.

Элементами диаграммы являются компоненты — физические замещаемые модули системы. Каждый компонент является полностью независимым элементом системы. Разновидностью компонентов являются узлы. Узел — это элемент реальной (физической) системы, который существует во время функционирования программного комплекса и представляет собой вычислительный ресурс, обычно обладающий как минимум некоторым объемом памяти, а часто еще и способностью обработки. Узлы делятся на два типа:

*устройства* — узлы системы, в которых данные не обрабатываются.

*процессоры* — узлы системы, осуществляющие обработку данных.

Компонентом может быть любой достаточно крупный модульный объект, такой как таблица или экстенд базы данных, подсистема, бинарный исполняемый файл, готовая к использованию система или приложение. Таким образом, диаграмму компонентов можно рассматривать как диаграмму классов в более крупном (менее детальном) масштабе. Компонент, как правило, представляет собой физическую упаковку логических элементов, таких как *классы, интерфейсы и кооперации*.

Основное назначение диаграмм компонентов — разделение системы на элементы, которые имеют стабильный интерфейс и образуют единое целое. Это позволяет создать ядро системы, которое не будет меняться в ответ на изменения, происходящие на уровне подсистем.

На рис. 2.6.1 показана упрощенная схема элементов фрагмента корпоративной системы. "Коробки" представляют собой компоненты — приложения или внутренние подсистемы. Пунктирные линии отражают зависимости между компонентами.

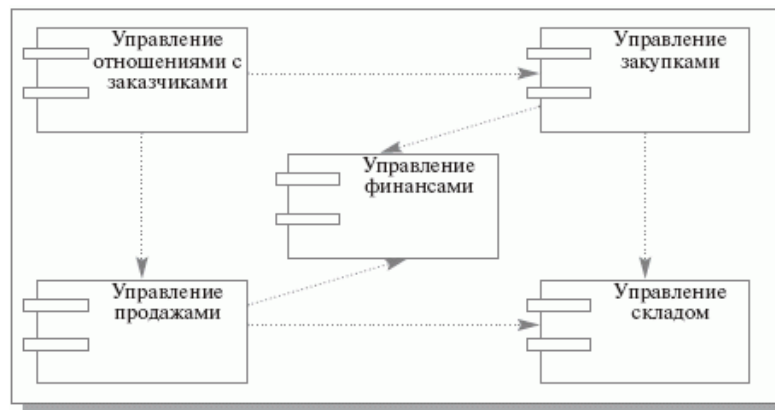


Рис. 2.6.1. Диаграмма компонентов фрагмента КИС

Каждый компонент диаграммы при необходимости документируется с помощью более детальной диаграммы компонентов, диаграммы сценариев или диаграммы классов.

### 2.6.1 Лабораторная работа № 13. Component diagram

#### Цель работы:

- изучение диаграмм пакетов, диаграммы компонентов и диаграммы размещения,
- изучение их применения в процессе проектирования.

Диаграмма компонентов служит частью физического представления модели и является необходимой для генерации программного кода. Для разработки диаграмм компонентов в браузере проекта предназначено отдельное представление компонентов (Component View), в котором уже содержится диаграмма компонентов с пустым содержанием и именем по умолчанию Main (Главная).

Активизация диаграммы компонентов может быть выполнена одним из следующих способов:

- Щелкнуть на кнопке с изображением диаграммы компонентов на стандартной панели инструментов.
- Раскрыть представление компонентов в браузере (Component View) и дважды щелкнуть на пиктограмме **Main** (Главная).
- Через пункт меню **Browse** → **Component Diagram** (Браузер → Диаграмма компонентов).

В результате выполнения этих действий появляется новое окно с чистым рабочим листом диаграммы компонентов и специальная панель инструментов, содержащая кнопки с изображением графических примитивов, необходимых для разработки диаграммы компонентов (Табл.26).

Таблица 26.

Назначение кнопок специальной панели инструментов диаграммы компонентов

| Кнопка | Всплывающая подсказка | Назначение кнопки                                                                                |
|--------|-----------------------|--------------------------------------------------------------------------------------------------|
|        | Selection Tool        | Превращает изображение курсора в форму стрелки для последующего выделения элементов на диаграмме |
|        | TextBox               | Добавляет на диаграмму текстовую область                                                         |

|                                                                                   |                         |                                                                                           |
|-----------------------------------------------------------------------------------|-------------------------|-------------------------------------------------------------------------------------------|
|  | Note                    | Добавляет на диаграмму примечание                                                         |
|  | AnchorNoteToItem        | Добавляет на диаграмму связь примечания с соответствующим графическим элементом диаграммы |
|  | Component               | Добавляет на диаграмму компонент                                                          |
|  | Package                 | Добавляет на диаграмму пакет                                                              |
|  | Dependency              | Добавляет на диаграмму отношение зависимости                                              |
|  | SubprogramSpecification | Добавляет на диаграмму спецификацию подпрограммы                                          |
|  | SubprogramBody          | Добавляет на диаграмму тело подпрограммы                                                  |
|  | MainProgram             | Добавляет на диаграмму главную программу                                                  |
|  | PackageSpecification    | Добавляет на диаграмму спецификацию пакета                                                |
|  | PackageBody             | Добавляет на диаграмму тело пакета                                                        |
|  | TaskSpecification       | Добавляет на диаграмму спецификацию задачи                                                |
|  | TaskBody                | Добавляет на диаграмму тело задачи                                                        |
|  | GenericSubprogram       | Добавляет на диаграмму типовую подпрограммы(по умолчанию отсутствует)                     |
|  | GenericPackage          | Добавляет на диаграмму типовой пакет (по умолчанию отсутствует)                           |
|  | Database                | Добавляет на диаграмму базу данных (по умолчанию отсутствует)                             |

Как видно из этой таблицы, по умолчанию на панели инструментов отсутствуют только три графических элемента из рассмотренных ранее элементов диаграммы компонентов, а именно - кнопки с пиктограммами типовой подпрограммы, типового пакета и базы данных. При необходимости их можно добавить на специальную панель диаграммы компонента стандартным способом.

Программа IBM Rational Rose не поддерживает графические стереотипы, и предлагает целый ряд собственных стереотипов. Графическое изображение этих стереотипов и их краткая характеристика приводятся в следующей таблице (табл. 27). При этом каждому из компонентов, как правило, соответствует отдельный файл исходной сборки программного приложения.

Таблица 27.

Графическое изображение стереотипов компонентов и их характеристика

| Кнопка                                                                              | Название стереотипа     | Характеристика стереотипа компонента                                                                                                                                          |
|-------------------------------------------------------------------------------------|-------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | SubprogramSpecification | Спецификация подпрограммы. Содержит описание переменных, процедур и функций и не содержит определений классов                                                                 |
|  | SubprogramBody          | Тело подпрограммы. Содержит реализацию процедур и функций, не относящихся к каким-то классам, при этом не содержит определений классов или реализаций операций других классов |
|  | MainProgram             | Главная программа. Реализует базовую логику работы программного приложения и содержит ссылки на другие компоненты модели                                                      |
|  | PackageSpecification    | Спецификация пакета. Содержит определение класса, его атрибутов и операций. В языке программирования C++ спецификации пакета соответствует отдельный файл с расширением «h»   |
|  | PackageBody             | Тело пакета. Содержит код реализации операций класса. В языке программирования C++ спецификации пакета соответствует                                                          |

|                                                                                                     |                   |                                                                                                                                                                                                             |
|-----------------------------------------------------------------------------------------------------|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                                                     |                   | отдельный файл с расширением «сpp»                                                                                                                                                                          |
|                    | TaskSpecification | Спецификация задачи. Может содержать определение класса, его атрибутов и операций, которые предполагается использовать в независимом потоке управления                                                      |
|                    | TaskBody          | Тело задачи. Может содержать реализацию операций класса, которые имеют независимый поток управления.                                                                                                        |
|                    | GenericSubprogram | Типовая подпрограмма. Содержит описание переменных, процедур и функций, которые могут быть использованы в нескольких программных приложениях. При этом типовая подпрограмма не содержит определений классов |
|                    | GenericPackage    | Типовой пакет. Содержит определение класса, его атрибутов и операций, которое может быть использовано в нескольких программных приложениях                                                                  |
| <br>NewSubprogSpec | Database          | База данных. Содержит определение одного или нескольких классов, их атрибутов и, возможно, операций. При этом соответствующие классы могут быть реализованы в форме одной или нескольких таблиц базы данных |

Использование рассмотренных стереотипов существенно увеличивают наглядность графического представления диаграммы компонентов и позволяют архитектору уточнить характер реализации модели программистом на выбранном языке программирования.

### Добавление компонента

Для добавления компонента на диаграмму компонентов нужно с помощью левой кнопки мыши нажать кнопку с изображением пиктограммы компонента на специальной панели инструментов, отпустить левую кнопку мыши и щелкнуть левой кнопкой мыши на свободном месте рабочего листа диаграммы. Добавить компонент на диаграмму можно также с помощью операции главного меню: **Tools→Create→Component** или с помощью операции контекстного меню: **New→Component**, предварительно выделив представление компонентов в браузере проекта.

В результате этих действий на диаграмме появится изображение компонента с маркерами изменения его геометрических размеров и предложенным средой именем по умолчанию, которое разработчику следует изменить. Продолжая разработку модели системы управления банкоматом, построим для нее диаграмму компонентов. С этой целью изменим имя диаграммы, предложенное по умолчанию Main, на *Диаграмма компонентов ATM*, а для первого добавленного компонента зададим имя MainATM.exe (рис. 2.6.2).

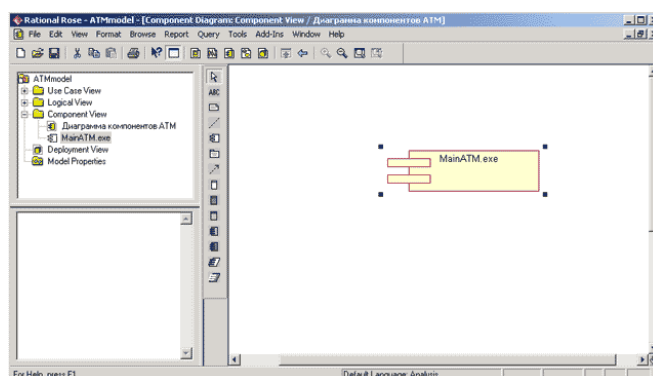


Рис. 2.6.2. Диаграмма компонентов после добавления компонента MainATM.exe

Для каждого компонента можно определить различные свойства, такие как стереотип, язык программирования, декларации, реализуемые классы. Редактирование этих свойств для произвольного компонента осуществляется с помощью диалогового окна спецификации свойств (рис. 2.6.3).

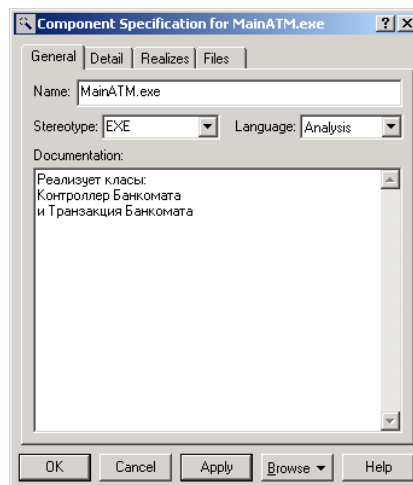


Рис. 2.6.3. Диалоговое окно спецификации свойств компонента MainATM.exe

В частности, для компонента MainATM.exe можно выбрать стереотип <<EXE>> из предлагаемого вложенного списка, поскольку применительно к разрабатываемой модели предполагается реализация этого компонента в форме исполнимого файла. При этом на вкладке **Realizes** (Реализует) содержатся все классы, включая и актеров, которые на данный момент присутствуют в модели (рис. 2.6.4). Следует заметить, что классы будут показаны в этом окне только при выбранном свойстве **Show all classes** (Показать все классы).

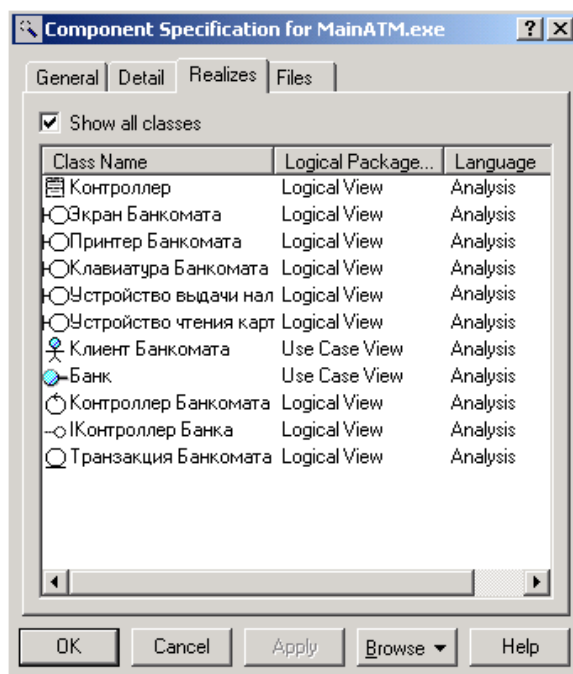


Рис. 2.6.4. Диалоговое окно спецификации свойств компонента MainATM.exe, открытое на вкладке **Realizes** (Реализует)

По умолчанию в среде IBM Rational Rose для всех добавляемых на диаграмму компонентов в качестве языка реализации используется язык анализа, который в последствии следует изменить на тот язык программирования, который предполагается

использовать для написания программного кода. В дальнейшем при генерации программного кода необходимо будет дополнительно выбрать те классы, которые реализует тот или иной компонент модели.

### Добавление отношения зависимости

Добавление отношения зависимости на диаграмму компонентов аналогично добавлению соответствующего отношения на диаграмму прецедентов. Продолжая разработку модели банкомата, на диаграмму компонентов предварительно следует добавить второй компонент с именем MainBank, для которого выбрать стереотип **Main Program**. Для добавления зависимости между двумя компонентами нужно с помощью левой кнопки мыши нажать кнопку с изображением зависимости на специальной панели инструментов, отпустить левую кнопку мыши, щелкнуть левой кнопкой мыши на изображении исходного компонента на диаграмме и отпустить ее на изображении целевого компонента. В результате этих действий на диаграмме появится изображение отношения зависимости в форме пунктирной линии со стрелкой, соединяющей два выбранных компонента.

Применительно к диаграмме компонентов модели банкомата рассмотренным способом следует добавить отношение зависимости от компонента с именем MainATM.exe к компоненту с именем MainBank. В дополнение к этому для наглядности можно указать в форме примечаний те классы модели, которые предполагается реализовать в данных компонентах (рис. 2.6.5).

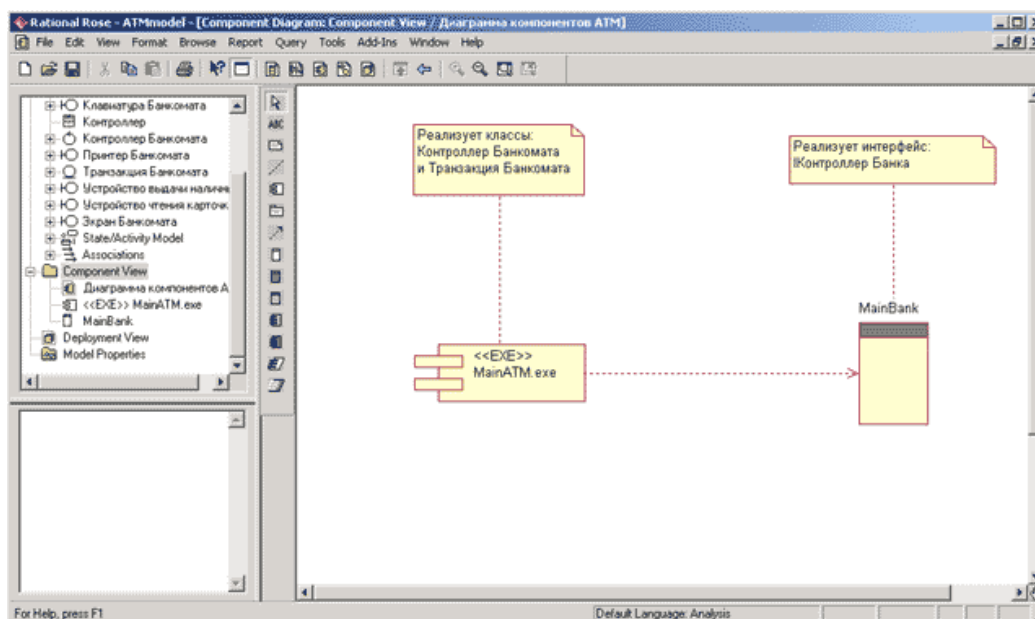


Рис. 2.6.5. Диаграмма компонентов после добавления отношения зависимости между компонентами MainATM.exe и MainBank

Следует заметить, что отношение зависимости в среде IBM Rational Rose не имеет собственного окна спецификации свойств. Именно по этой причине специфицировать свойства данного отношения, такие как имя и стереотип, можно только с помощью текстовой области, что нельзя признать удобным с практической точки зрения.

### Завершение диаграммы компонентов модели банкомата

Для завершения построения диаграммы компонентов рассматриваемого примера следует описанным выше способом добавить оставшиеся компоненты и зависимости. С этой целью следует выполнить следующие действия:

1. Добавить компонент с именем: *Устройства Банкомата*, для которого задать стереотип Task Specification.

2. Добавить компоненты с именами: *Устройство чтения карточки*, *Клавиатура Банкомата*, *Принтер Банкомата*, *Экран Банкомата*, *Устройство выдачи наличных*, для которых задать стереотип Task Body.

3. Добавить зависимость от компонента с именем MainATM.exe к компоненту с именем *Устройства Банкомата*.

4. Добавить зависимость от компонента с именем *Устройство чтения карточки* к компоненту с именем *Устройства Банкомата*.

5. Добавить зависимость от компонента с именем *Клавиатура Банкомата* к компоненту с именем *Устройства Банкомата*.

6. Добавить зависимость от компонента с именем *Принтер Банкомата* к компоненту с именем *Устройства Банкомата*.

7. Добавить зависимость от компонента с именем *Экран Банкомата* к компоненту с именем *Устройства Банкомата*.

8. Добавить зависимость от компонента с именем *Устройство выдачи наличных* к компоненту с именем *Устройства Банкомата*.

Построенная таким образом диаграмма компонентов будет иметь следующий вид (рис. 2.6.6).

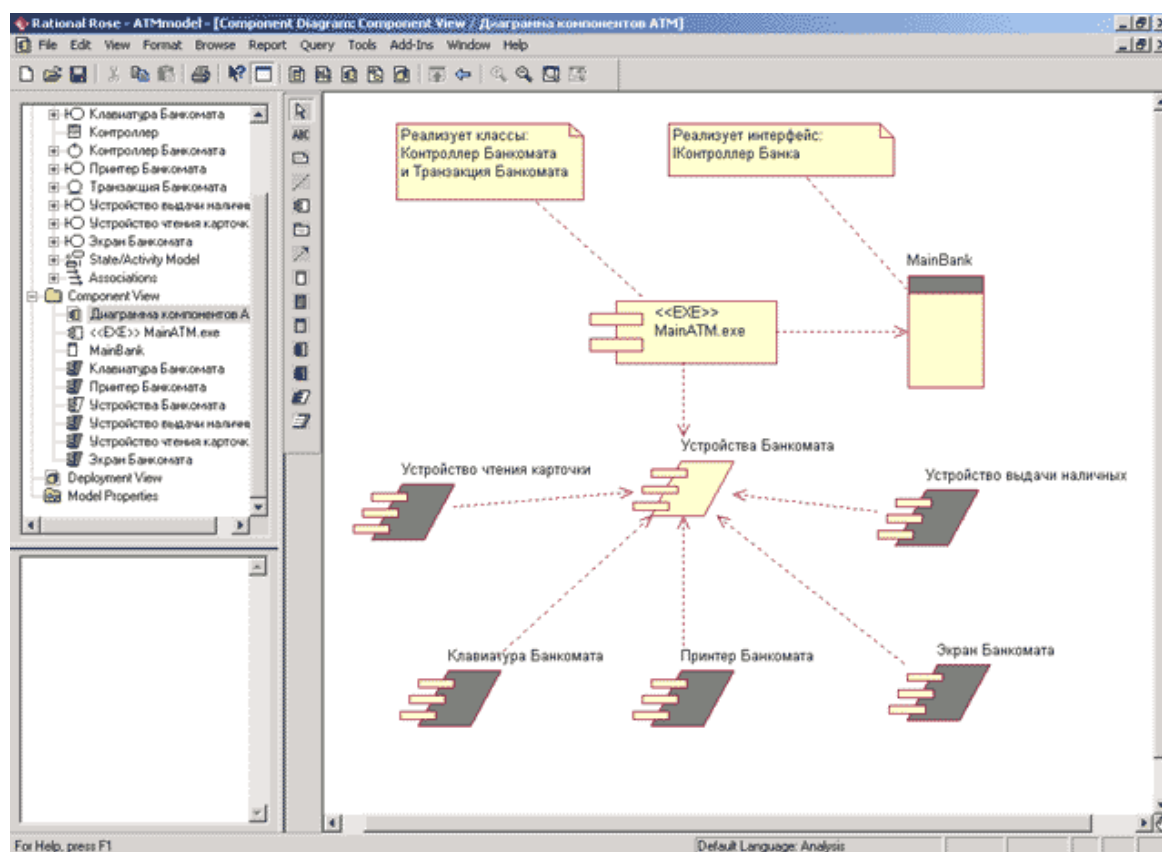


Рис. 2.6.6. Окончательный вид диаграммы компонентов разрабатываемой модели управления

Следует заметить, что различные графические стереотипы компонентов не оказывают влияния на особенности генерации программного кода. Поэтому при разработке диаграммы компонентов присутствует некоторая неоднозначность выбора соответствующих стереотипов, связанная с особенностями предполагаемой реализации программного приложения. При работе с диаграммой компонентов можно также создавать пакеты и размещать в них компоненты, изменять их спецификацию и отношения зависимости между различными элементами диаграммы.

## 2.6.2 Лабораторная работа № 14. Deployment diagram

### Цель работы:

- изучение диаграммы размещения,
- изучение их применения в процессе проектирования.

Диаграмма развертывания является второй составной частью физического представления модели и разрабатывается, как правило, для территориально распределенных систем. Для разработки диаграмм компонентов в браузере проекта предназначено отдельное представление развертывания (Deployment View), в котором уже содержится диаграмма развертывания с пустым содержанием и без собственного имени.

Активизация диаграммы развертывания может быть выполнена одним из следующих способов:

- Щелкнуть на кнопке с изображением диаграммы развертывания на стандартной панели инструментов.
- Дважды щелкнуть на пиктограмме представления развертывания (Deployment View) в браузере проекта.
- Выполнить операцию главного меню: **Browse→Deployment Diagram** (Обзор→Диаграмма развертывания).

В результате выполнения этих действий появляется новое окно с чистым рабочим листом диаграммы развертывания и специальная панель инструментов, содержащая кнопки с изображением графических примитивов, необходимых для разработки диаграммы развертывания (Табл.28).

Таблица 28.

Назначение кнопок специальной панели инструментов диаграммы развертывания

| Кнопка                                                                              | Всплывающая подсказка | Назначение кнопки                                                                                |
|-------------------------------------------------------------------------------------|-----------------------|--------------------------------------------------------------------------------------------------|
|  | Selection Tool        | Превращает изображение курсора в форму стрелки для последующего выделения элементов на диаграмме |
|  | TextBox               | Добавляет на диаграмму текстовую область                                                         |
|  | Note                  | Добавляет на диаграмму примечание                                                                |
|  | AnchorNoteToItem      | Добавляет на диаграмму связь примечания с соответствующим графическим элементом диаграммы        |
|  | Processor             | Добавляет на диаграмму процессор                                                                 |
|  | Connection            | Добавляет на диаграмму отношение соединения                                                      |
|  | Device                | Добавляет на диаграмму устройство                                                                |

Работа с диаграммой развертывания состоит в создании процессоров и устройств, их спецификации, установлении связей между ними, а также добавлении и спецификации процессов.

### Добавление узла на диаграмму развертывания

Для добавления узла на диаграмму развертывания нужно с помощью левой кнопки мыши нажать кнопку с изображением пиктограммы требуемого узла (процессора или устройства) на специальной панели инструментов, отпустить левую кнопку мыши и щелкнуть левой кнопкой мыши на свободном месте рабочего листа диаграммы. Добавить процессор на диаграмму развертывания можно также с помощью операции главного меню: **Tools→Create→Processor** или с помощью операции контекстного меню: **New→Processor**, предварительно выделив представление развертывания в браузере проекта. Аналогично добавить устройство на диаграмму можно также с помощью операции главного меню:

**Tools**→**Create**→**Device** или с помощью операции контекстного меню: **New**→**Device**, предварительно выделив представление развертывания в браузере проекта.

В результате этих действий на диаграмме развертывания появится изображение узла требуемого типа с маркерами изменения его геометрических размеров и предложенным средой именем по умолчанию, которое разработчику следует изменить. При этом следует иметь в виду, что в среде IBM Rational Rose под процессором понимается ресурсоемкий узел, а под устройством - нересурсоемкий узел.

Продолжая разработку модели системы управления банкоматом, построим для нее диаграмму развертывания. С этой целью в качестве первого узла выберем тип процессор и зададим ему имя *Банкомат №1*, для которого в форме примечания укажем помеченное значение: {адрес = ул. Садовая, д.5}. Это значение служит для спецификации конкретного адреса одного из банкоматов системы (рис. 2.6.7).

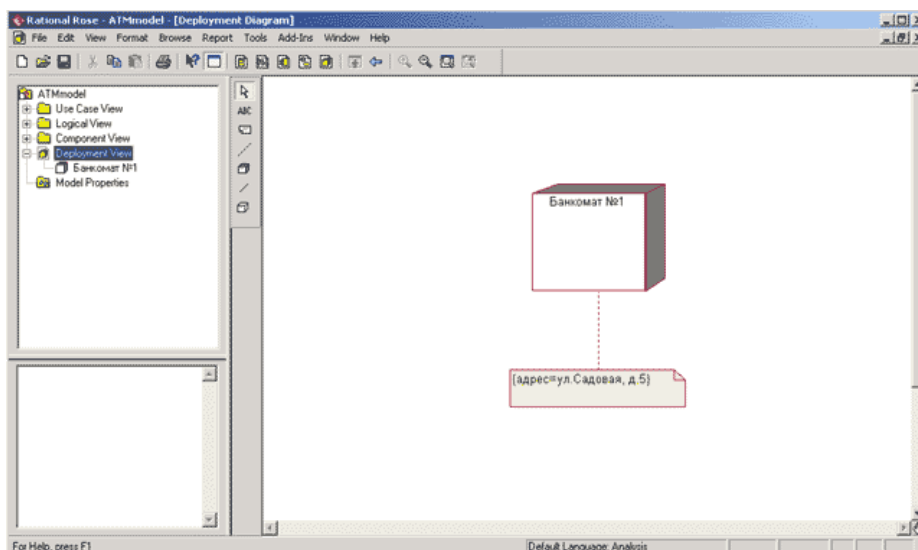


Рис. 2.6.7. Диаграмма развертывания после добавления узла Банкомат № 1

Для каждого процессора можно специфицировать различные свойства, такие как стереотип, характеристику, процессы и их приоритет. Спецификация этих свойств осуществляется с помощью диалогового окна спецификации свойств процессора (рис. 2.6.8).

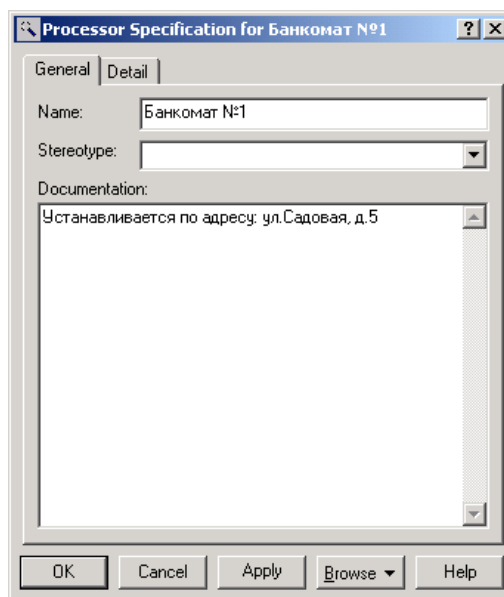


Рис. 2.6.8. Диалоговое окно спецификации свойств узла Банкомат № 1

При этом на вкладке **General** (Общие) можно только изменить имя процессора, ввести текст стереотипа, предложенный самим разработчиком, и текст документации, поясняющий особенности физического размещения данного компонента. На вкладке **Detail** (Подробно) окна спецификации свойств процессора можно определить его характеристики, выбрать процессы и вариант планирования его работы (рис. 2.6.9).

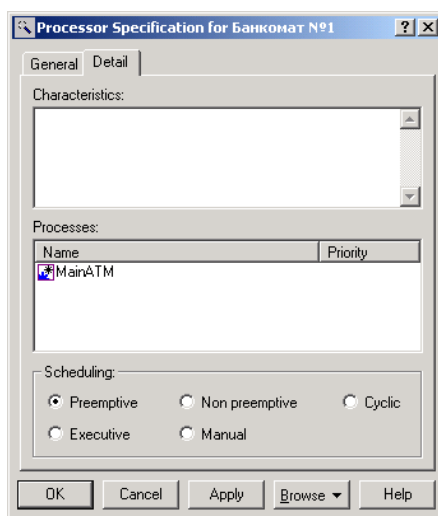


Рис. 2.6.9. Диалоговое окно спецификации свойств узла Банкомат № 1, открытое на вкладке **Detail** (Подробно)

Характеристики процессора, такие как его быстродействие и объем оперативной памяти, могут быть записаны в форме текста в многостраничное поле с именем **Characteristics**. В поле **Processes** (Процессы) можно задать некоторый процесс, который предполагается реализовать на данном процессоре. С этой целью необходимо выполнить операцию контекстного меню **Insert** (Вставить) и ввести текст имени процесса. Далее можно задать приоритет процесса, введя некоторое число в соответствующее поле ввода.

При наличии у процессора нескольких процессов может быть дополнительно определена процедура планирования их выполнения. Для спецификации процедуры планирования процессора могут быть использованы следующие варианты выбора в группе **Scheduling**:

- **Preemptive** (С приоритетом) - определяет процедуру планирования, при которой процесс с большим приоритетом будет иметь преимущество при использовании ресурсов процессора по сравнению с менее приоритетными процессами.
- **Non preemptive** (Без приоритета) - определяет процедуру планирования, при которой все приоритеты процессов игнорируются. При этом текущий процесс выполняется до своего завершения, после чего может быть начато выполнение следующего процесса.
- **Cyclic** (Циклический) - определяет процедуру планирования, при которой приоритеты процессов также игнорируются. Все процессы выполняются циклически по кругу, при этом каждому из них выделяется фиксированное время на выполнение, попросту говоря управление передается следующему процессу.
- **Executive** (Исполнительный) - определяет процедуру планирования, для которой существует некоторый алгоритм, предназначенный для управления отдельными процессами.
- **Manual** (Вручную) - определяет процедуру планирования, при которой планирование выполнения процессов осуществляется пользователем.

Для отображения информации о процессах, выполняемых на отдельных процессорах, представленных на диаграмме развертывания, следует выполнить операцию контекстного меню **Show Processes** (Показать процессы). Для отображения информации о процедуре планирования отдельных процессов на выбранном процессоре следует выполнить операцию контекстного меню **Show Scheduling** (Показать планирование).

Продолжая разработку диаграммы развертывания для модели банкомата, следует добавить второй узел типа устройство (Device) с именем Сеть, для которого задать стереотип <<закрытая сеть>>. При этом для задания стереотипа следует ввести его текст без угловых кавычек в строку с именем Stereotype.

Для устройства набор редактируемых свойств меньше, поэтому для него с помощью соответствующего окна спецификации свойств можно определить: имя, стереотип, документацию и характеристику (рис. 2.6.10). Этот факт согласуется с определением устройства как нересурсоемкого узла, на котором отсутствует процессор.



Рис. 2.6.10. Диалоговое окно спецификации свойств устройства Сеть, открытое на вкладке **Detail** (Подробно)

### Добавление соединения и редактирование его свойств

Для добавления соединения между двумя узлами нужно с помощью левой кнопки мыши нажать кнопку с изображением соединения на специальной панели инструментов, отпустить левую кнопку мыши, щелкнуть левой кнопкой мыши на изображении одного из узлов на диаграмме и отпустить ее на изображении другого узла. Добавить соединения на диаграмму развертывания можно также с помощью операции главного меню: **Tools→Create→Connection**.

В результате этих действий на диаграмме появится изображение соединения в форме линии без стрелок, соединяющей два выбранных узла. Применительно к диаграмме развертывания модели банкомата одним из рассмотренных способов следует добавить соединение для узлов с именами Банкомат №1 и Сеть (рис. 2.6.11).

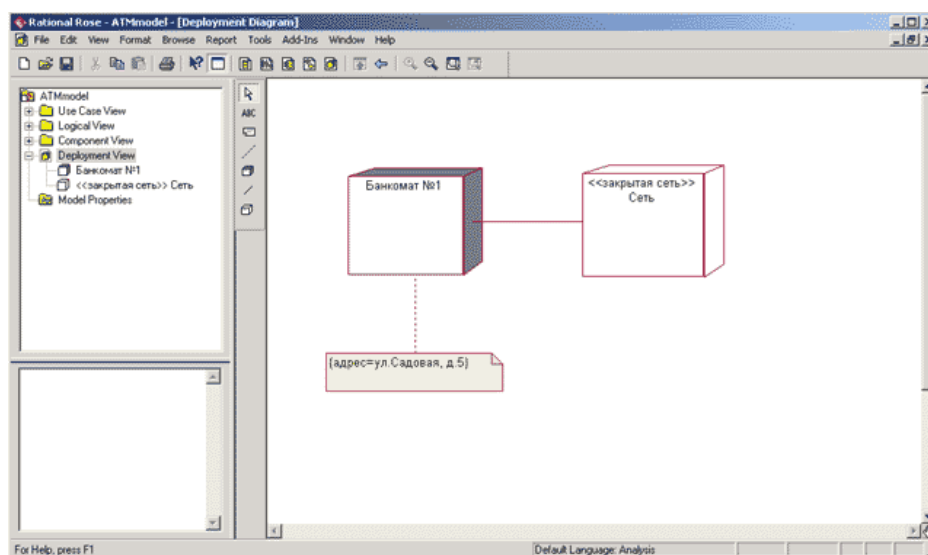


Рис. 2.6.11. Диаграмма развертывания после добавления соединения между узлами Банкомат № 1 и Сеть

Для соединения набор редактируемых свойств аналогичен набору свойств устройства, поэтому для него с помощью соответствующего окна спецификации свойств можно определить только имя, стереотип, документацию и характеристику (рис. 2.6.10).

### Завершение диаграммы развертывания модели банкомата

Для завершения построения диаграммы развертывания рассматриваемого примера следует описанным выше способом добавить оставшиеся узлы и соединения. С этой целью следует выполнить следующие действия:

1. Добавить процессор с именем: *Банкомат №2*, для которого задать помеченное значение в форме примечания: {адрес = ул. Парковая, д.7}, а на вкладке свойств Detail определить новый процесс и выбрать для него имя MainATM из вложенного списка.
2. Добавить процессор с именем: *Банкомат №3*, для которого задать помеченное значение в форме примечания: {адрес = ул. Лесная, д.9}, а на вкладке свойств Detail определить новый процесс и выбрать для него имя MainATM из вложенного списка.
3. Добавить процессор с именем: *Сервер Банка*, для которого на вкладке свойств Detail определить новый процесс с именем MainBank.
4. Добавить соединение для узлов с именами *Банкомат №2* и *Сеть*.
5. Добавить соединение для узлов с именами *Банкомат №3* и *Сеть*.
6. Добавить соединение для узлов с именами *Сервер Банка* и *Сеть*.

Построенная таким образом диаграмма развертывания будет иметь следующий вид (рис. 2.6.11).

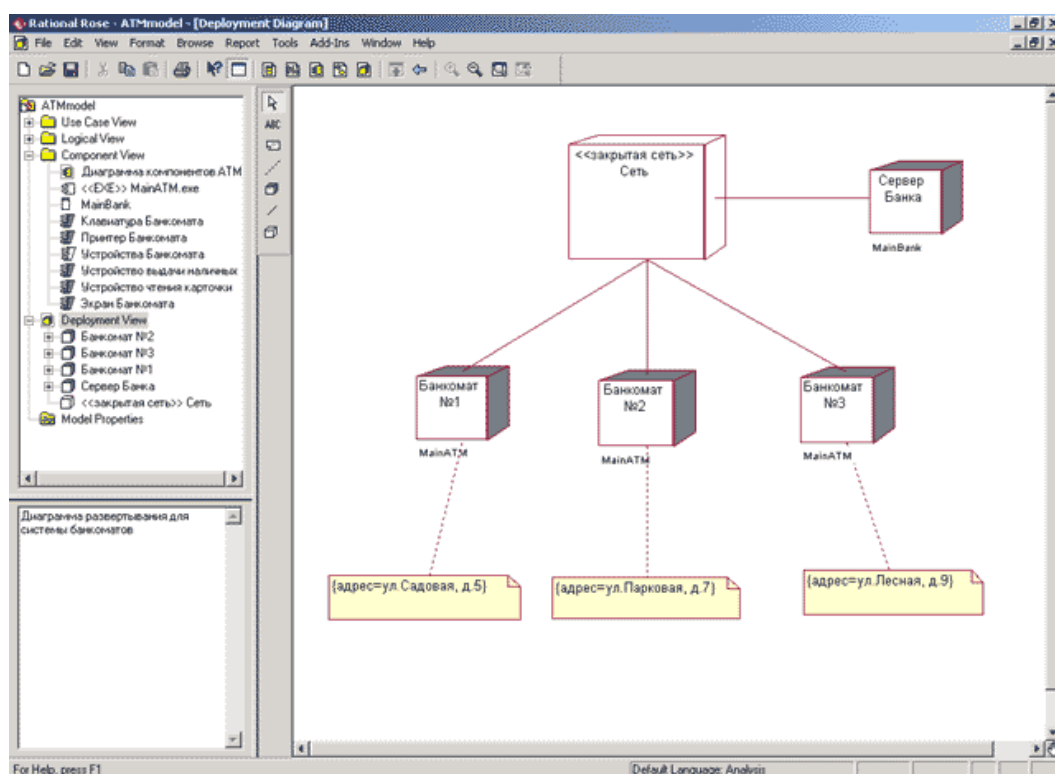


Рис. 2.6.11. Окончательный вид диаграммы развертывания разрабатываемой модели управления банкоматом

Следует отметить, что программа IBM Rational Rose не поддерживает возможности графического размещения внутри узлов развертываемых на них компонентов. Указать размещение компонентов модели в узлах диаграммы развертывания можно с помощью документации соответствующих узлов. Выполнить эти действия предлагается читателям самостоятельно в качестве упражнения. После построения диаграммы развертывания разработка визуальной модели системы управления банкоматом в нотации UML может считаться завершенной.

### **2.6.3 Задания**

1. Построить для моделируемой системы общую диаграмму пакетов, отметить на ней пакеты с необходимыми системными библиотеками, отобразить зависимости между пакетами.
2. Построить для данной системы диаграмму компонентов, соответствующую построенной диаграмме пакетов, системные пакеты изобразить в виде спецификаций пакетов.
3. Построить для проектируемой системы несколько вариантов диаграммы размещения (для архитектуры «клиент-сервер», трехуровневой архитектуры и т. д.), обосновать каждый вариант, предложить наиболее оптимальный.

### **2.6.4 Контрольные вопросы**

1. Какую проблему проектирования призваны решить диаграммы пакетов?
2. В чем отличие диаграмм пакетов от диаграмм классов?
3. В чем смысл зависимости между элементами диаграммы пакетов?
4. Что такое интерфейс класса?
5. По каким признакам классы группируются в пакеты?
6. Какие виды элементов модели представлены на диаграмме компонентов?
7. Как связаны между собой диаграммы пакетов и диаграммы компонентов?
8. Что показывает диаграмма размещения?
9. Какие сущности отображаются на диаграммах размещения?
10. В каких случаях необходимо применение диаграмм размещения?

## 2.7 Генерация программного кода

Одним из наиболее важных свойств программы IBM Rational Rose является возможность генерации программного кода на нескольких языках программирования, которая может быть использована разработчиком после построения модели. Для этой цели в среде IBM Rational Rose присутствует достаточно большой выбор языков программирования и схем баз данных. Однако возможность генерации текста программы на том или ином языке программирования зависит от установленной версии IBM Rational Rose.

Общая последовательность действий, которые необходимо выполнить для генерации программного кода в среде IBM Rational Rose, состоит из следующих этапов:

- Проверка модели на отсутствие ошибок.
- Создание компонентов для реализации классов.
- Отображение классов на компоненты.
- Выбор языка программирования для генерации текста программного кода.
- Установка свойств генерации программного кода.
- Выбор класса, компонента или пакета.
- Генерация программного кода.

Особенности выполнения каждого из этапов могут изменяться в зависимости от выбора языка программирования или схемы базы данных.

В среде IBM Rational Rose предусмотрено задание достаточно большого числа свойств, характеризующих как отдельные классы, так и проект в целом. Для определенности в качестве языка реализации проекта целесообразно выбрать язык программирования ANSI C++, который не требует инсталляции дополнительных программ и поставляется практически во всех конфигурациях IBM Rational Rose. Рассмотрим особенности выполнения каждого из указанных выше этапов для языка реализации модели ANSI C++.

### Проверка модели независимо от выбора языка генерации кода

В общем случае проверка модели может выполняться на любом этапе работы над проектом. Однако после завершения разработки графических диаграмм она является обязательной, поскольку позволяет выявить целый ряд ошибок разработчика. К числу таких ошибок и предупреждений относятся, например, не используемые ассоциации и классы, оставшиеся после удаления отдельных графических элементов с диаграмм, а также операции, не являющиеся именами сообщений на диаграммах взаимодействия.

Для проверки модели следует выполнить операцию главного меню: **Tools**→**Check Model** (Инструменты→Проверить модель). Результаты проверки разработанной модели на наличие ошибок отображаются в окне журнала. Прежде чем приступить к генерации текста программного кода разработчику следует добиться устранения всех ошибок и предупреждений, о чем должно свидетельствовать чистое окно журнала (рис. 2.7.1).



Рис. 2.7.1. Вид журнала при отсутствии ошибок по результатам проверки модели

## Лабораторная работа №15. Генерация программного кода

### Цель работы:

Подготовка модели и генерация программного кода

По существу данные этапы выполняются в ходе разработки диаграммы компонентов. Хотя программа IBM Rational Rose позволяет генерировать программный код на языке ANSI C++ для каждого класса модели без предварительного построения диаграммы компонентов, имеет смысл воспользоваться разработанной ранее диаграммой компонентов. Применительно к разрабатываемому проекту желательно переименовать компоненты, задав им англоязычные имена. В этом случае соответствующая диаграмма компонентов модели банкомата будет иметь вид, представленный на рис. 2.7.2.

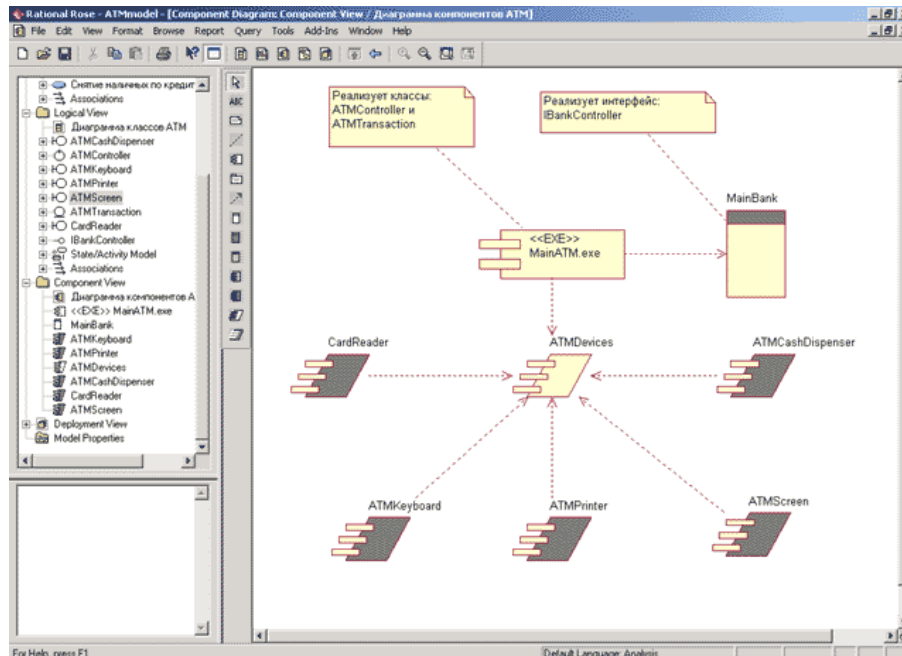


Рис. 2.7.2. Диаграмма компонентов модели банкомата после преобразования имен компонентов

Для отображения классов на компоненты можно воспользоваться окном спецификации свойств компонента, открытого на вкладке **Realizes** (Реализует). Для включения реализации класса в данный компонент следует выделить требуемый класс на этой вкладке и выполнить для него операцию контекстного меню **Assign** (Назначить). В результате перед именем класса на этой вкладке появится специальная отметка.

Применительно к модели банкомата для компонента MainATM.exe выберем для генерации программного кода классы ATMTransaction и ATMController (рис. 2.7.3).

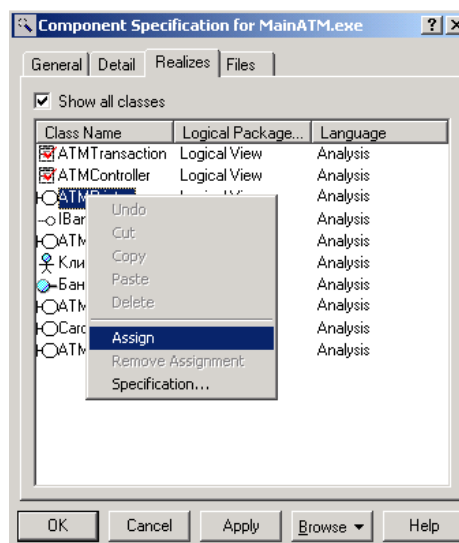


Рис. 2.7.3. Диалоговое окно настройки свойств реализации классов в компоненте MainATM.exe

Подобная операция должна быть выполнена для всех классов модели, которые предполагается реализовывать на выбранном языке программирования. Имеется и другой способ установления реализации классов на компоненте. А именно, можно просто выделить класс в браузере проекта и перетащить его на нужный компонент диаграммы компонентов.

### **Выбор языка программирования и редактирование свойств генерации программного кода**

Для выбора языка ANSI C++ в качестве языка реализации модели следует выполнить операцию главного меню: **Tools**→**Options** (Инструменты→Параметры), в результате чего будет вызвано диалоговое окно настройки параметров модели. Далее на вкладке Notation (Нотация) в строке Default Language (Язык по умолчанию) из вложенного списка следует выбрать язык - ANSI C++.

Если по какой-то причине языка ANSI C++ не оказалось во вложенном списке, то следует убедиться в том, что этот язык программирования установлен в качестве расширения IBM Rational Rose. Для этого следует открыть окно установленных расширений, выполнив операцию главного меню: **Add-Ins**→**Add-In Manager** (Расширения→Менеджер расширений), и убедиться в том, что выставлена отметка в строке с именем языка ANSI C++. Если ее нет, то ее следует добавить, после чего появится группа доступных операций ANSI C++ в главном меню **Tools**.

После выбора языка программирования по умолчанию следует изменить язык реализации каждого из компонентов модели. С этой целью следует изменить язык в строке **Language** (Язык) на вкладке **General** (Общие) окна спецификации свойств компонента, для чего из вложенного списка следует выбрать язык - ANSI C++ (рис. 2.7.4).

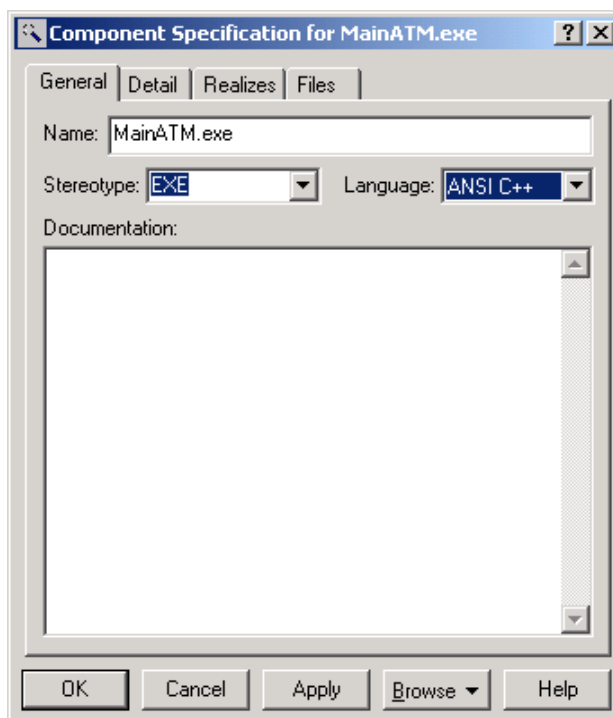


Рис. 2.7.4. Окно спецификации свойств компонента MainATM.exe при выборе языка его реализации

Следует заметить, что после выбора языка программирования следует привести в соответствие типы атрибутов, типы аргументов и возвращаемых значений операций. С этой целью нужно просмотреть все классы диаграммы классов и изменить те типы данных, которые не являются синтаксически допустимыми в выбранном языке программирования. Применительно к языку ANSI C++ следует заменить тип **Integer** на **int**, **Boolean** на **bool**,

**Currency** на **float**. В противном случае соответствующие исправления придется выполнять вручную после генерации программного кода.

Редактирование общих свойств генерации программного кода возможно в специальном диалоговом окне, которое может быть открыто в результате выполнения операции главного меню: **Tools**→**ANSI C++**→**Open ANSI C++ Specification** (Инструменты→Язык ANSI C++→Открыть спецификацию языка ANSI C++). Дополнительные свойства генерации программного кода отдельного класса можно специфицировать в диалоговом окне, которое может быть открыто в результате выполнения операции контекстного меню: **ANSI C++**→**Class Customization** (Язык ANSI C++→Настройка свойств класса). При этом соответствующий класс должен быть выделен в браузере проекта.

При генерации программного кода на языке ANSI C++ для модели банкомата значения свойств, предлагаемых средой IBM Rational Rose по умолчанию, первоначально можно оставить без изменения.

### Выбор класса или компонента и генерация для него программного кода

Выбор класса или компонента для генерации программного кода означает выделение соответствующего элемента модели в браузере проекта. Применительно к рассматриваемой модели системы управления банкоматом для генерации программного кода на языке ANSI C++ выберем компонент с именем MainATM.exe.

Генерация программного кода в среде IBM Rational Rose возможна для отдельного класса или компонента. Для этого нужный элемент модели предварительно следует выделить в браузере проекта и выполнить операцию контекстного меню: **ANSI C++**→**Generate Code\_** (Язык ANSI C++→Генерировать код). В результате этого будет открыто диалоговое окно с предложением выбора классов для генерации программного кода на выбранном языке программирования (рис. 2.7.5). После выбора соответствующих классов и нажатия кнопки **ОК** программа IBM Rational Rose выполняет кодогенерацию.

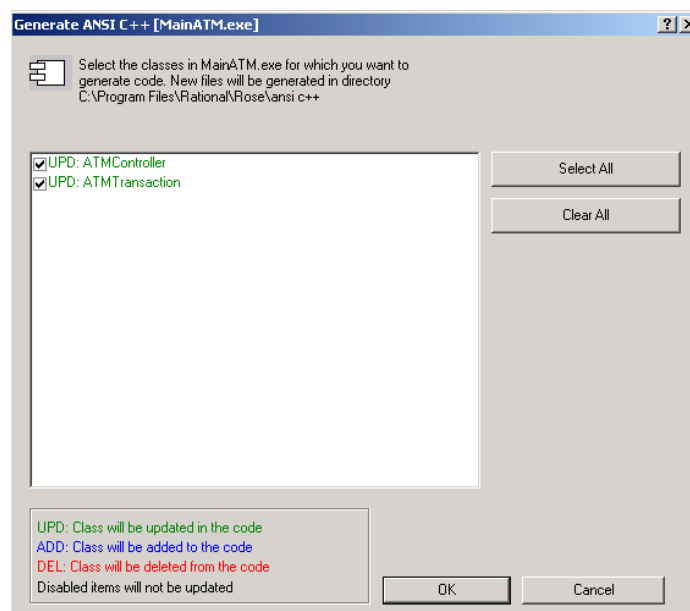
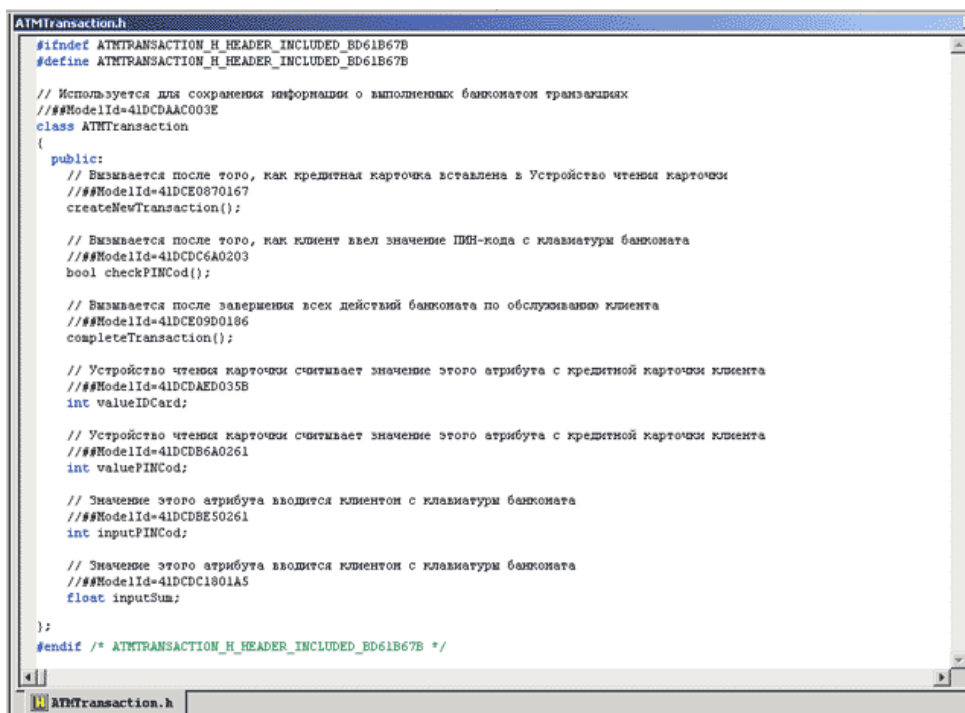


Рис. 2.7.6. Окно выбора классов для генерации программного кода

Для просмотра и редактирования созданных файлов с текстом программного кода на языке ANSI C++ предназначен встроенный текстовый редактор, который можно открыть с помощью операции контекстного меню: **ANSI C++**→**Browse Header\_** (Язык ANSI C++→Просмотреть заголовочный файл) или **ANSI C++**→**Browse Body\_** (Язык ANSI C++→Просмотреть файл реализации) для выбранного класса в браузере проекта.

После генерации программного кода для компонента MainATM.exe каждому классу, реализованному в данном компоненте, будет соответствовать 2 файла с текстом кода на языке ANSI C++. Так, например, для класса ATMTransaction будет сгенерирован заголовочный файл с расширением «h» (рис. 2.7.6) и файл реализации с расширением «cpp» (рис. 2.7.7).



```

ATMTransaction.h

#ifndef ATMTRANSACTION_H_HEADER_INCLUDED_E61B67B
#define ATMTRANSACTION_H_HEADER_INCLUDED_E61B67B

// Используется для сохранения информации о выполненных банкоматом транзакциях
// ##ModelId=41DCDAAC003E
class ATMTransaction
{
public:
    // Вызывается после того, как кредитная карточка вставлена в Устройство чтения карточек
    // ##ModelId=41DCE0870167
    createNewTransaction();

    // Вызывается после того, как клиент ввел значение ПИН-кода с клавиатуры банкомата
    // ##ModelId=41DCDC6A0203
    bool checkPINCod();

    // Вызывается после завершения всех действий банкомата по обслуживанию клиента
    // ##ModelId=41DCE09D0186
    completeTransaction();

    // Устройство чтения карточек считывает значение этого атрибута с кредитной карточкой клиента
    // ##ModelId=41DCDAED035B
    int valueIDCard;

    // Устройство чтения карточек считывает значение этого атрибута с кредитной карточкой клиента
    // ##ModelId=41DCDB6A0261
    int valuePINCod;

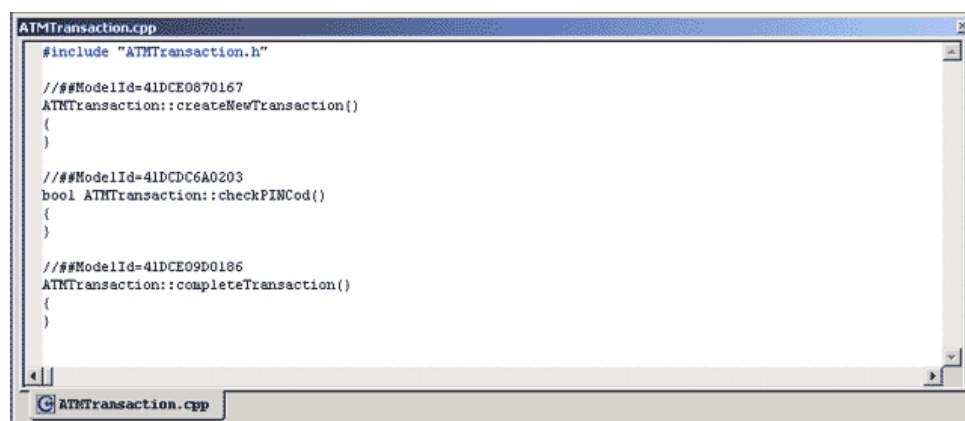
    // Значение этого атрибута вводится клиентом с клавиатуры банкомата
    // ##ModelId=41DCDBE50261
    int inputPINCod;

    // Значение этого атрибута вводится клиентом с клавиатуры банкомата
    // ##ModelId=41DCDC1801A5
    float inputSum;
};

#endif /* ATMTRANSACTION_H_HEADER_INCLUDED_E61B67B */

```

Рис. 2.7.6. Вид встроенного текстового редактора с загруженным в него заголовочным файлом ATMTransaction.h



```

ATMTransaction.cpp

#include "ATMTransaction.h"

// ##ModelId=41DCE0870167
ATMTransaction::createNewTransaction()
{
}

// ##ModelId=41DCDC6A0203
bool ATMTransaction::checkPINCod()
{
}

// ##ModelId=41DCE09D0186
ATMTransaction::completeTransaction()
{
}

```

Рис. 2.7.7. Вид встроенного текстового редактора с загруженным в него заголовочным файлом ATMTransaction.cpp

Как видно из рассмотрения полученного заголовочного файла, в нем содержится объявление в соответствии с правилами синтаксиса языка ANSI C++ всех операций и атрибутов класса ATMTransaction. При этом информация о документировании операций и атрибутов помещается в комментарии перед соответствующими элементами программы.

В файле реализации содержится заготовка для реализации всех операций класса ATMTransaction в соответствии с правилами синтаксиса языка ANSI C++. При этом каждая из операций имеет пустое тело реализации, которое следует написать дополнительно, исходя из функциональных требований модели и синтаксиса языка программирования ANSI C++.

Данную работу удобнее выполнять в выбранной интегрированной среде программирования, например, MS Visual C++ или Borland C++.

При использовании интегрированной среды кроме компиляции, отладки и тестирования исходных модулей программы разработчик получает возможность дополнить приложение графическим интерфейсом, необходимым для взаимодействия с пользователем.

Следует заметить, что при установленной на компьютер разработчика интегрированной среды сгенерированные файлы с текстом программного кода автоматически открываются в этой среде после двойного щелчка на пиктограмме этих файлов. Тем не менее, лучше копировать содержимое этих файлов в предварительно созданные программные проекты для полного контроля в этих средах процесса программирования и отладки приложений.

Сгенерированные программой IBM Rational Rose файлы с текстом программного кода содержат минимум информации. Для включения дополнительных элементов в программный код следует изменить свойства генерации программного кода, установленные по умолчанию.

### **2.7.2 Задания**

Сгенерировать код для нескольких классов системы, сравнить модель и полученный исходный код.

### **2.7.3 Контрольные вопросы**

1. Каковы преимущества автоматической кодогенерации?
2. Какие виды диаграмм используются для генерации кода?
3. Какие компоненты исходного кода генерирует Rational Rose?
4. Как в исходном коде отражаются атрибуты и операции класса?
5. Почему важно детально комментировать компоненты модели?

## Заключение

Следует отметить, что эффект от использования CASE-средства проявляется при разработке масштабных проектов в составе команды или проектной группы. Действительно, при рассмотрении модели системы управления банкоматом может сложиться впечатление того, что написать и отладить соответствующую программу гораздо проще непосредственно в той или иной интегрированной среде программирования.

Однако ситуация покажется не столь тривиальной, когда станет необходимо выполнить проект с несколькими десятками прецедентов и сотней классов. Именно для подобных проектов явно выявляется преимущество использования CASE-средств для документирования и реализации соответствующих моделей.

На стадии проектирования системы модели расширяются, уточняются и дополняются диаграммами, отражающими ее структуру.

Перечисленные модели в совокупности дают полное описание системы независимо от того, является ли она существующей или вновь разрабатываемой. Состав диаграмм в каждом конкретном случае зависит от необходимой полноты описания системы.

## Библиографический список

1. Амриш К. И. Разработка корпоративных Java-приложений с использованием J2EE и UML : пер. с англ. / К. И. Амриш, Х. З. Ахмед. Москва : Вильямс, 2002. 272 с.
2. Боггс У. UML и Rational Rose / У. Боггс, М. Боггс. Москва : ЛОРИ, 2000. 582 с.
3. Буч Г. Объектно-ориентированный анализ и проектирование с примерами приложений на C++ / Г. Буч. Санкт-Петербург : Невский диалект, 1999. 560 с.
4. Буч Г. Язык UML. Руководство пользователя / Г. Буч, Дж. Рамбо, А. Джекобсон. Москва : ДМК Пресс, 2000. 432 с.
5. Гома Х. UML. Проектирование систем реального времени, параллельных и распределенных приложений / Х. Гома. Москва : ДМК Пресс, 2002. 704 с.
6. Грекул В. И. Проектирование информационных систем / В. И. Грекул, Г. Н. Денищенко, Н. Л. Коровкина. Москва : Интернет-университет информационных технологий, 2005. 304 с.
7. Грехем И. Объектно-ориентированные методы. Принципы и практика / И. И. Грехем. Москва : Вильямс, 2004. 880 с.
8. Коналлен Дж. Разработка Web-приложений с использованием UML : пер. с англ. / Дж. Коналлен. Москва : Вильямс, 2001. 288 с.
9. Ларман К. Применение UML и шаблонов проектирования. Изд. 2 / К. Ларман. Москва : Вильямс, 2002. 624 с.
10. Леоненков А. В. Объектно-ориентированный анализ и проектирование с использованием UML и IBM Rational Rose / А. В. Леоненков. Москва : Интуит, 2006. 320 с.
11. Леоненков А. В. Самоучитель UML. Изд. 2 / А. В. Леоненков. Санкт-Петербург : БХВ-Петербург, 2004. 432 с.
12. Леффингуэлл Д. Принципы работы с требованиями к программному обеспечению. Унифицированный подход / Д. Леффингуэлл, Д. Уидриг. Москва : Вильямс, 2002. 448 с.
13. Марка Д. Методология структурного анализа и проектирования SADT (Structured Analysis & Design Technique) : пер. с англ. / Д. Марка, К. МакГоуэн. Москва : МетаТехнология, 1993. 240 с.
14. Маклаков С. В. Создание информационных систем с AllFusion Modeling Suite / С. В. Маклаков. Москва : Диалог-МИФИ, 2005. 432 с.
15. Нейбург Э. Дж. Проектирование баз данных с помощью UML / Э. Дж. Нейбург, Р. А. Максимчук. Москва : Вильямс, 2002. 288 с.
16. Рамбо Дж. UML: специальный справочник / Дж. Рамбо, А. Якобсон, Г. Буч. Санкт-Петербург : Питер, 2001. 656 с.

17. Овчинникова Е. В. Моделирование бизнес-процессов с помощью AllFusion Process Modeler : учебно-методическое пособие. Екатеринбург : Изд-во УрГУПС, 2007. 101 с.
18. Основы работы с AllFusion Process Modeler: методические указания по дисциплине «Автоматизированное проектирование информационных систем» для студентов направления 230100 «Информатика и вычислительная техника» / сост. В. А. Фролов. Ульяновск : Изд-во УлГТУ, 2014. 39 с.
19. Парфёнов А. Н. Визуальное моделирование в среде IBM Rational Rose / А. Н. Парфёнов. Улан-Удэ : Изд-во Бурятского госуниверситета, 2008. 134 с.
20. Розенберг Д. Применение объектного моделирования с использованием UML и анализ прецедентов / Д. Розенберг, К. Скотт. Москва : ДМК Пресс, 2002. 160 с.
21. Фаулер М. UML. Основы / М. Фаулер, К. Скотт. Санкт-Петербург : Символ-Плюс, 2002. 192 с.
22. Федотова Д. Э. CASE-технологии : практикум / Д. Э. Федотова, Ю. Д. Семенов, К. Н. Чижик. Москва : Горячая линия-Телеком, 2005. 160 с.
23. Шаллоуей А. Шаблоны проектирования. Новый подход к объектно-ориентированному анализу и проектированию / А. Шаллоуей, Дж. Р. Тротт. Москва : Вильямс, 2002. 288 с.
24. Шмуллер Д. Освой самостоятельно UML за 24 часа / Д. Шмуллер. Москва : Вильямс, 2002. 352 с.
25. Якобсон А. Унифицированный процесс разработки программного обеспечения / А. Якобсон, Г. Буч, Дж. Рамбо. Санкт-Петербург : Питер, 2002. 496 с.
26. Starr L. Executable UML. How to build class models / L. Starr. Prentice Hall PTR, 2002. 418 p.
27. Wampler B. E. The Essence Object-Oriented Programming with Java and UML / B. E. Wampler. Addison-Wesley, 2002, 290 p.

Учебное издание

Составитель

*Тумэн Гомбоцыренович Дармаев*

## **CASE-ТЕХНОЛОГИИ**

Учебное пособие

Подписано в печать 29.10. 2018. Формат 60х84 1/8.

Усл. печ. л. 16,39. Уч.-изд. л. 7,91.

Заказ № 179.

Издательство Бурятского госуниверситета

670000, г. Улан-Удэ, ул. Смолина, 24 а